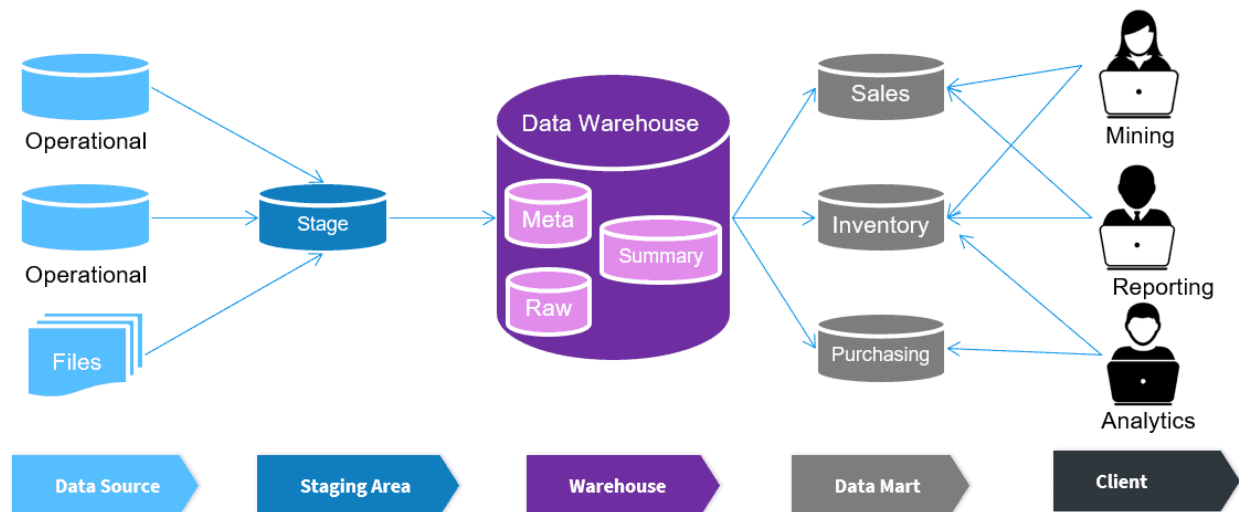


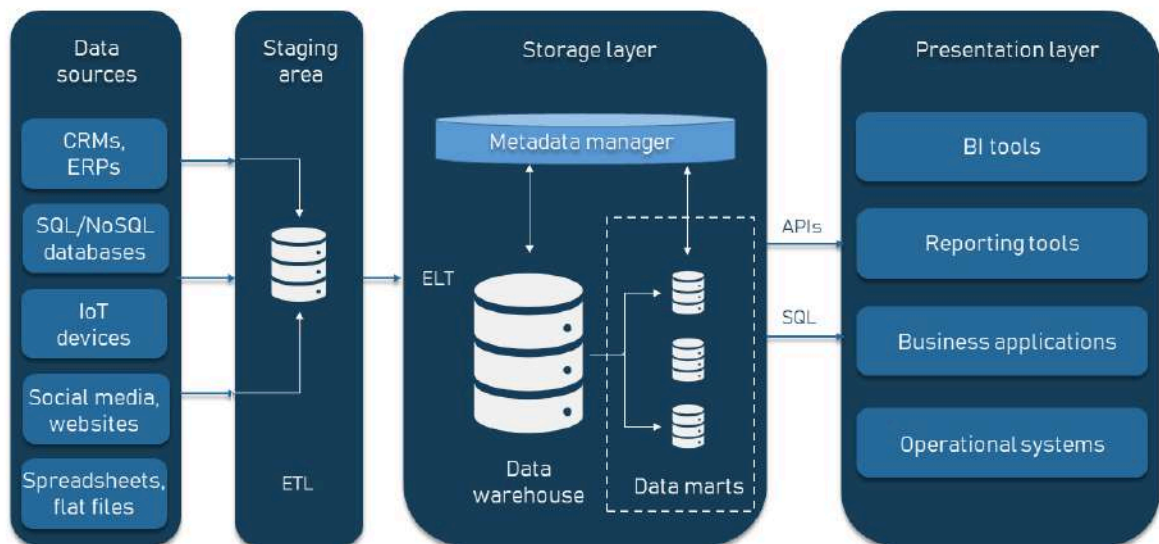
Evolution of Data Engineering

From brittle pipelines to unified, governed analytics platforms

1) Early Era — RDBMS & ETL (1990s–2008)



ENTERPRISE DATA WAREHOUSE COMPONENTS



How it worked

- Source systems → ETL tool → centralized RDBMS warehouse
- Schema designed upfront (schema-on-write)
- Batch loads, nightly or weekly

Core challenges

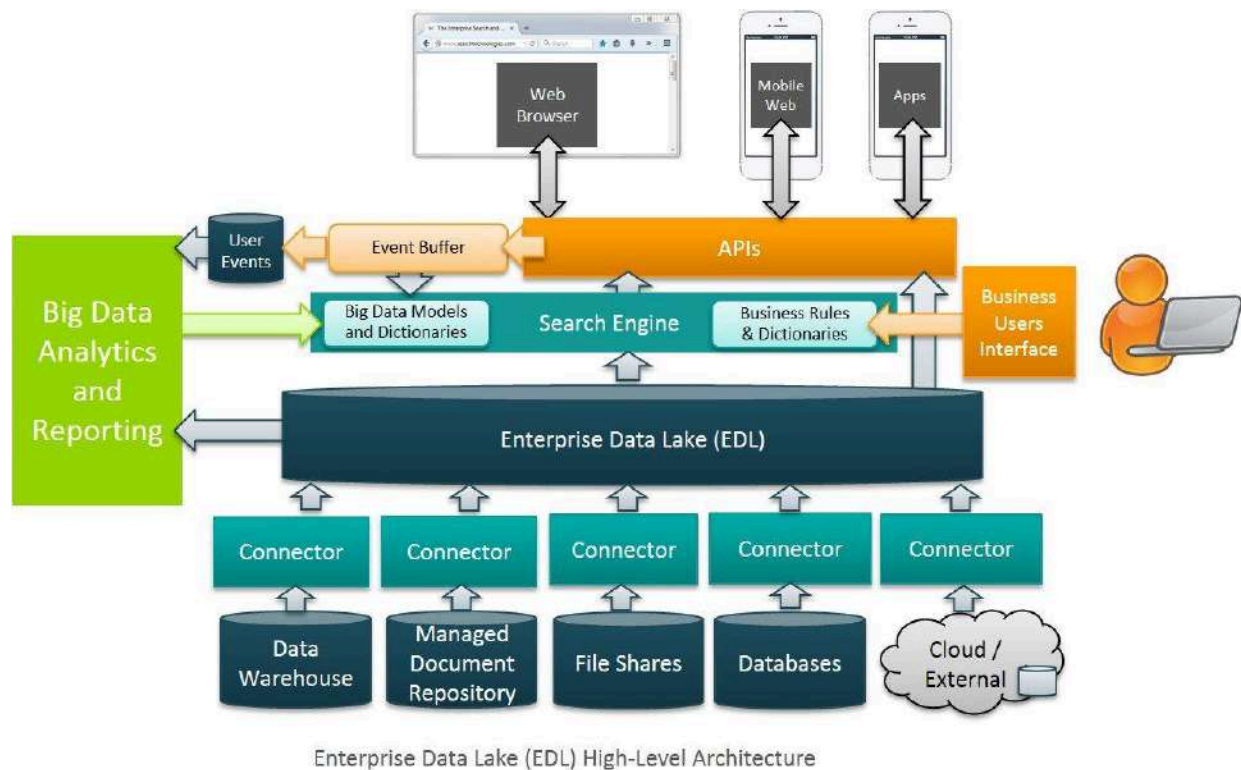
- **Scalability ceiling:** Vertical scaling only; hardware-bound.
- **Rigidity:** Any schema change broke pipelines.
- **Latency:** Batch windows; no near-real-time.
- **Cost:** Over-provisioned on-prem infrastructure.
- **Data loss:** Raw data discarded after transformation.

Outcome

- Stable for structured data
- Failed under data volume, velocity, and variety growth

2) Big Data Era — Hadoop & Distributed Storage (2008–2015)





Introduced

- **Apache Hadoop**
- HDFS + MapReduce
- Schema-on-read

Problems it solved

- Horizontal scalability
- Cheap storage for raw data
- Ability to store semi/unstructured data

New problems created

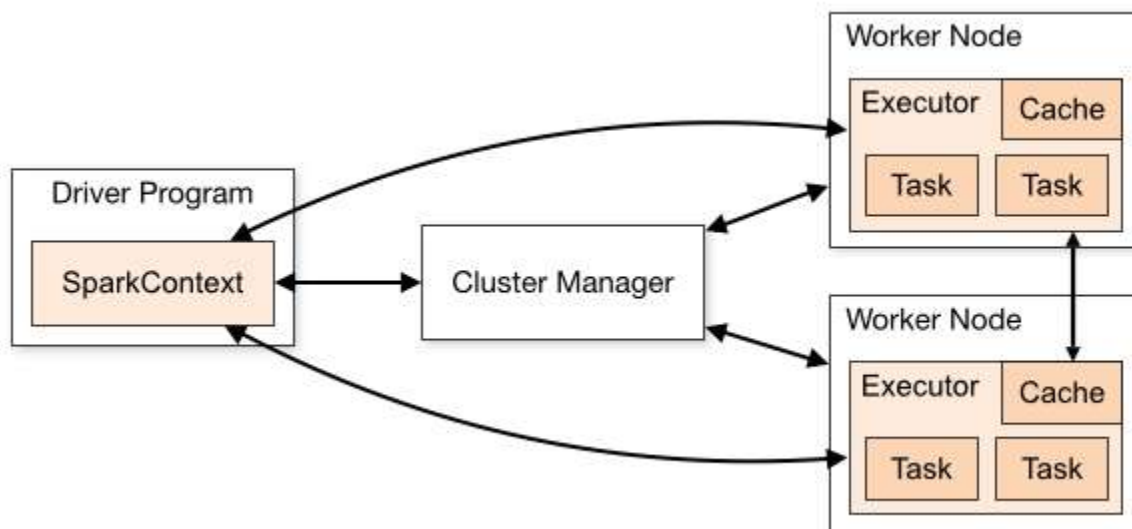
- **Operational complexity:** Cluster tuning, failures, jobs.
- **Slow processing:** Disk-heavy MapReduce.

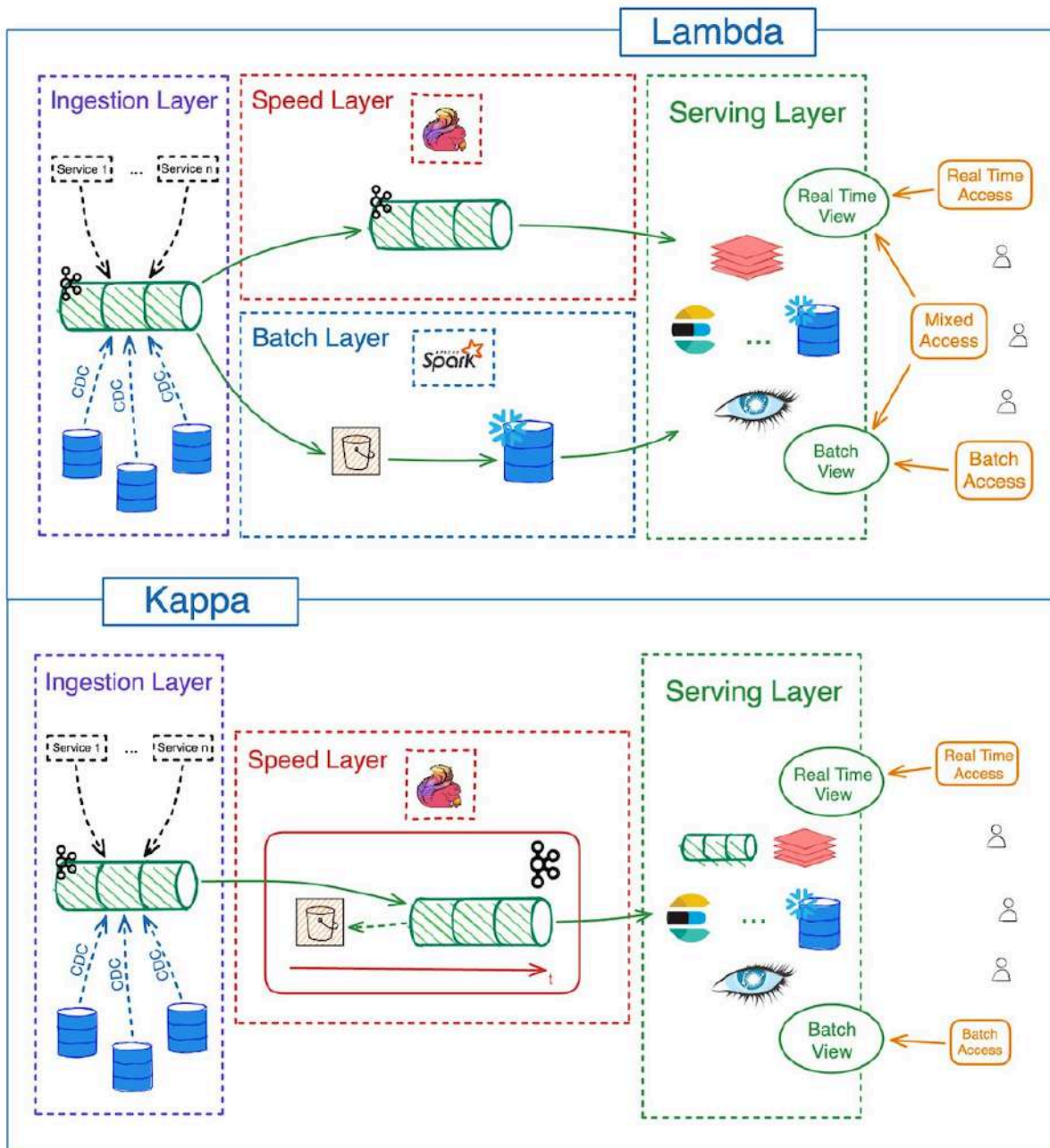
- **Poor BI usability:** SQL was an afterthought.
- **Data swamp:** No governance, no quality enforcement.

Outcome

- Storage solved
- Compute and usability became bottlenecks

3) Processing Evolution — Spark & Streaming (2015–2019)





Introduced

- **Apache Spark**
- In-memory compute
- Streaming + batch convergence

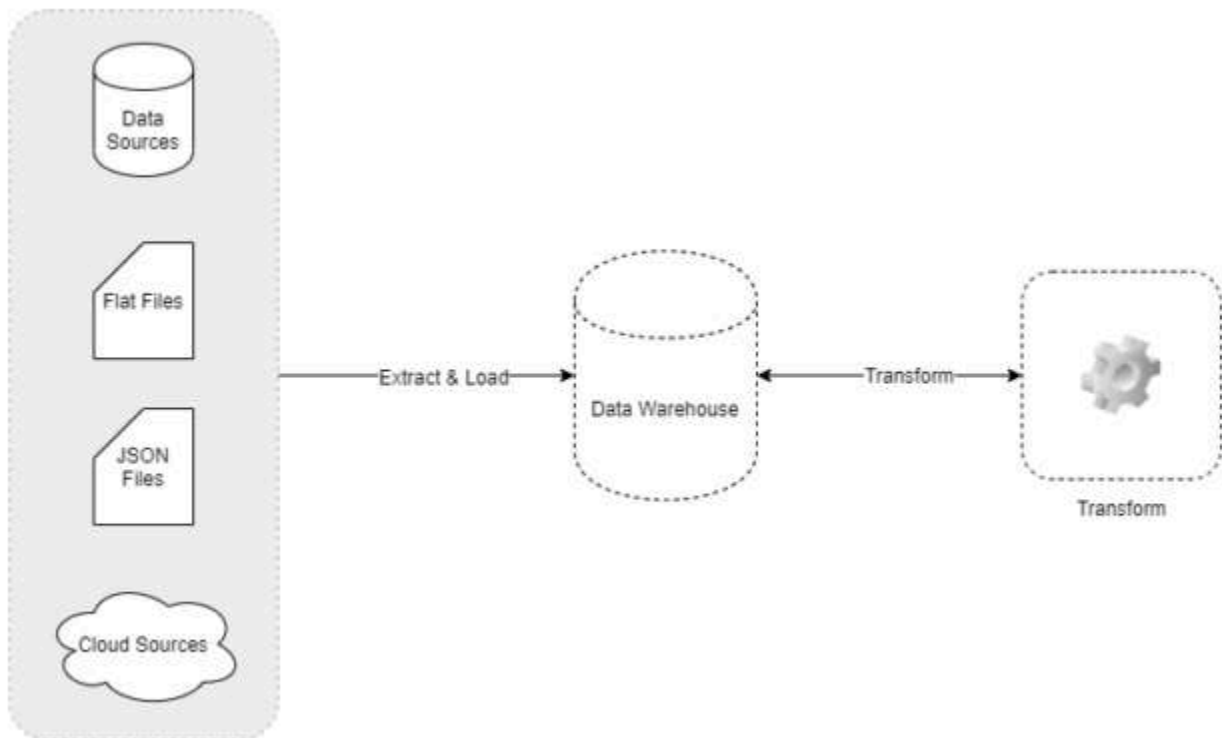
Problems it solved

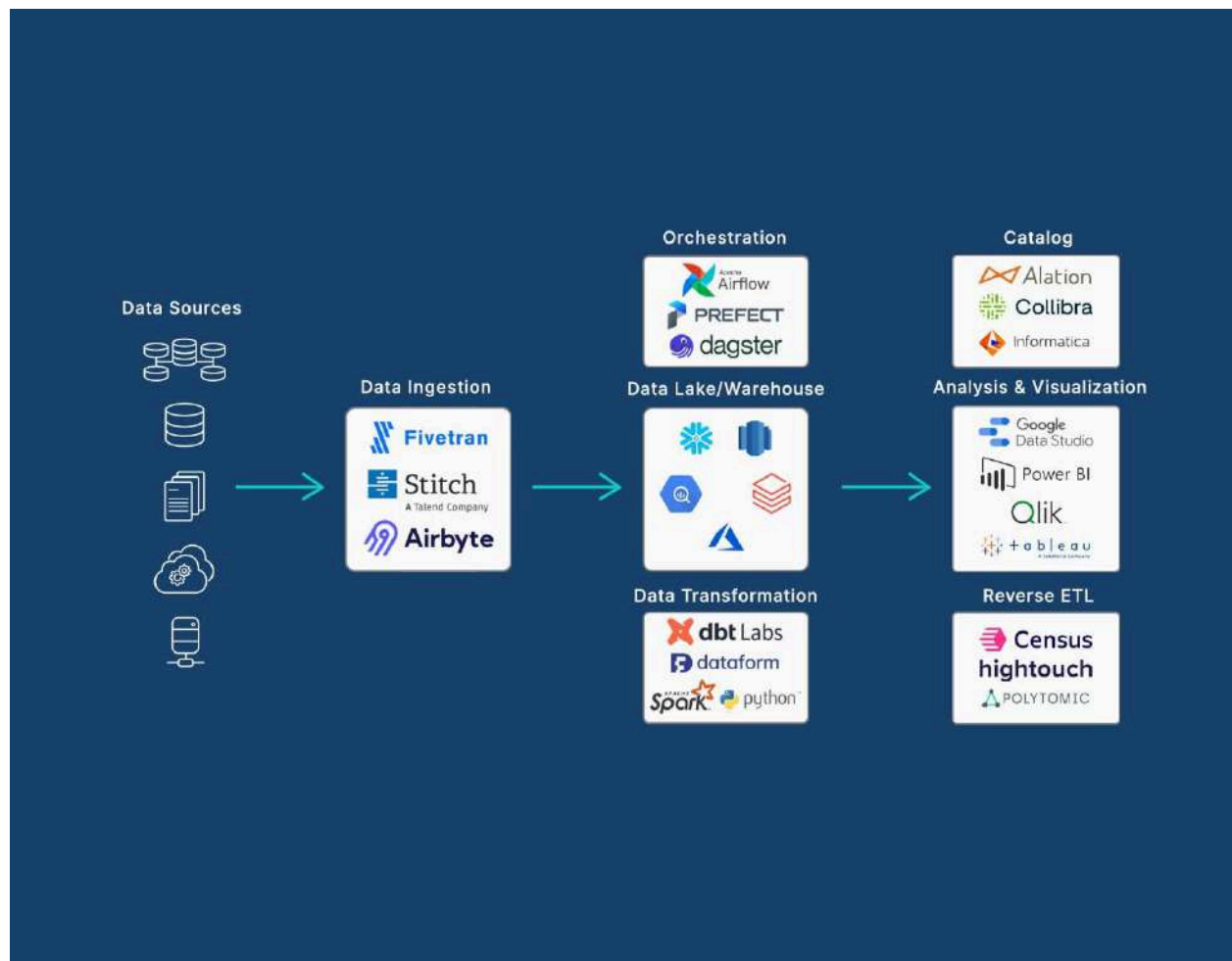
- Faster processing (memory-first)
- Unified batch and streaming logic
- Better SQL (Spark SQL)

Remaining gaps

- **Data reliability:** No native ACID on lakes
 - **Pipeline fragility:** Code-heavy transformations
 - **Governance gap:** Security and lineage bolted on
 - **BI friction:** Still engineering-heavy for analysts
-

4) Cloud Data Warehouses & ELT (2017–2021)





Introduced

- **Snowflake**
- Separation of storage and compute
- ELT instead of ETL

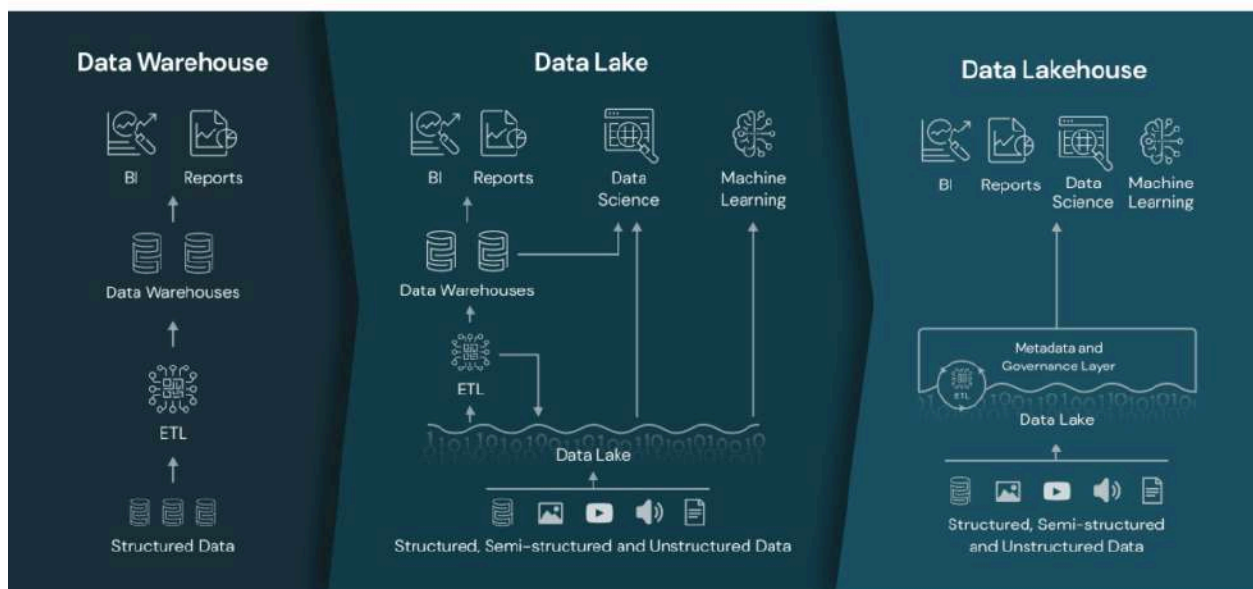
Problems it solved

- Elastic scaling
- SQL-first analytics
- Reduced infra management

New constraints

- Vendor lock-in
- Cost opacity
- Limited ML/advanced analytics
- Still batch-oriented pipelines

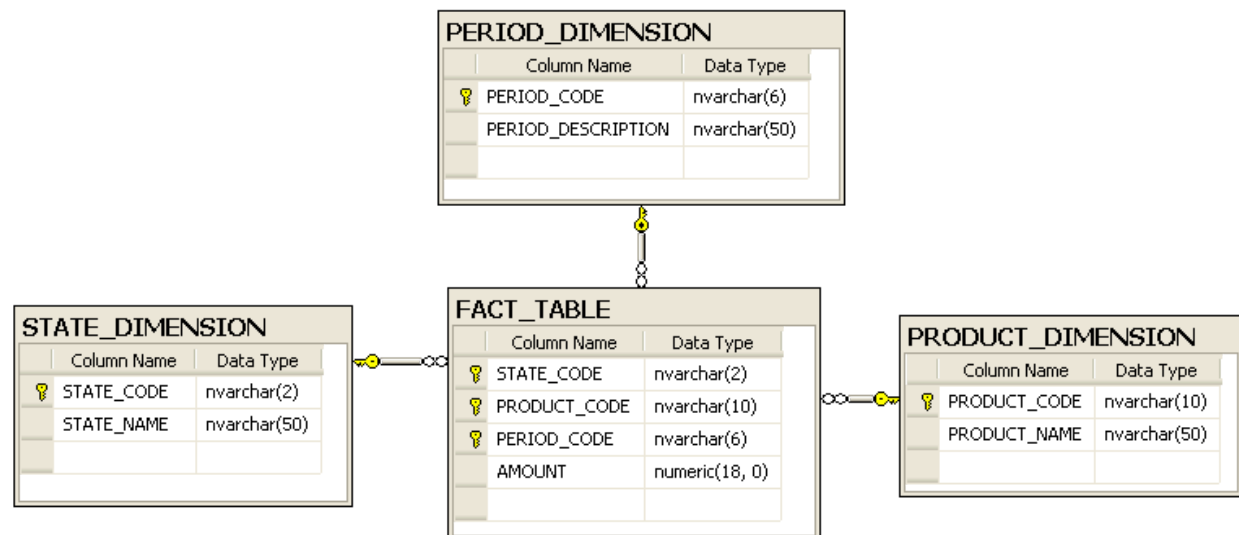
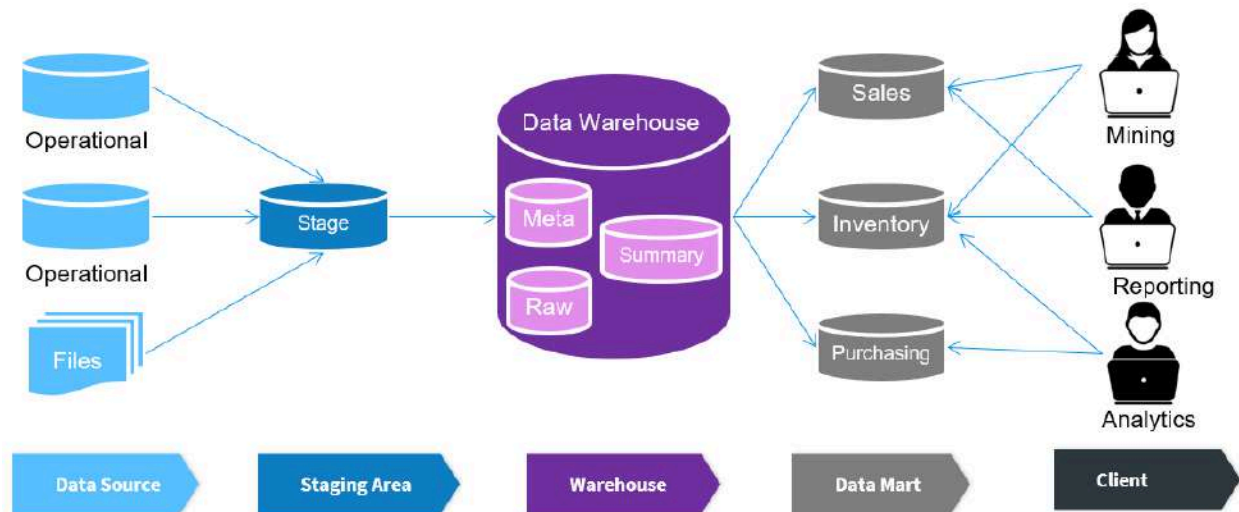
5) Lakehouse Era — ACID + Open Storage (2020–2023)



Evolution: Data Warehouse → Data Lake → Lakehouse

Why each appeared, what broke, what finally converged

1) Data Warehouse — The First Enterprise Standard



What it was

- Centralized analytical database
- Structured data only
- ETL + schema-on-write
- SQL-first analytics

Why it emerged

- Business reporting needed consistency and performance

- OLTP systems were not built for analytics

Pros

- Strong performance for BI
- Clean, trusted data
- Mature SQL ecosystem
- Clear governance model

Cons (structural limits)

- Rigid schemas: Change = pipeline redesign
- ETL bottleneck: Slow onboarding of new data
- Cost: Expensive storage + compute
- Data loss: Raw data discarded after transformation
- No ML readiness: Only curated aggregates

Typical platforms

- Teradata, Oracle, SQL Server
- Cloud variant: Snowflake

2) Data Lake — Reaction to Scale and Variety

- **Log data**
- **Clickstreams**
- **IoT**
- **Semi/unstructured formats**

What changed

- **Cheap object storage**
- **Schema-on-read**
- **Store everything, decide later**

Pros

- **Infinite scalability**
- **Low-cost storage**
- **Supports structured + unstructured data**
- **Ideal for data science experimentation**

Cons (new failures)

- **No ACID: Duplicates, partial writes**
- **Data swamp: No enforced quality**
- **Poor BI performance**
- **Complex tooling: Hadoop, Spark, Hive sprawl**
- **Governance bolted on, not native**

Typical platforms

- **Apache Hadoop**
- **S3 / ADLS + Spark**

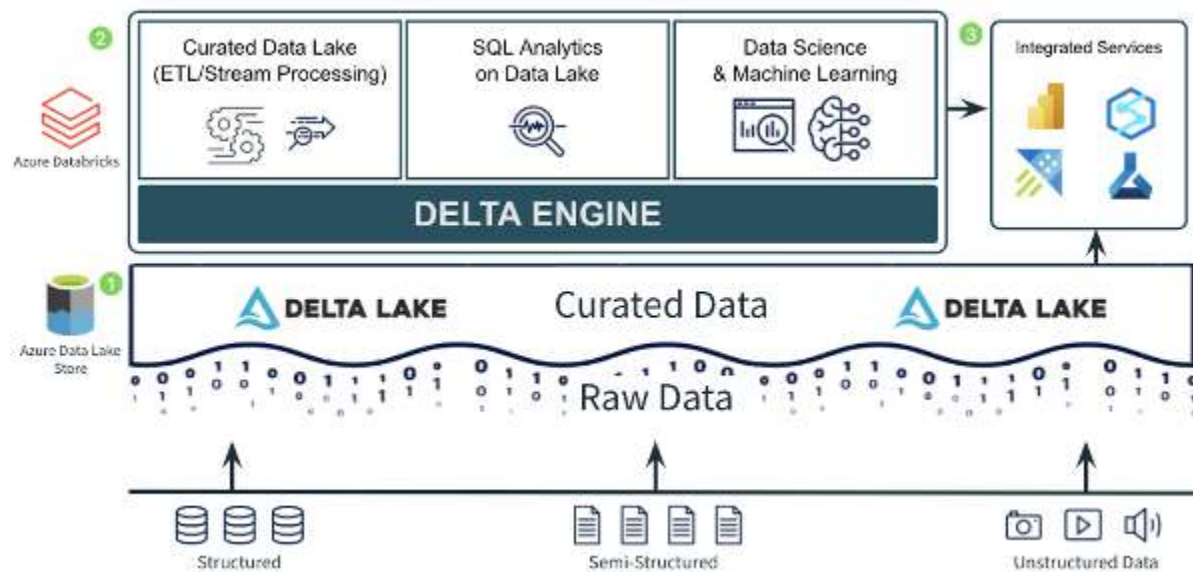
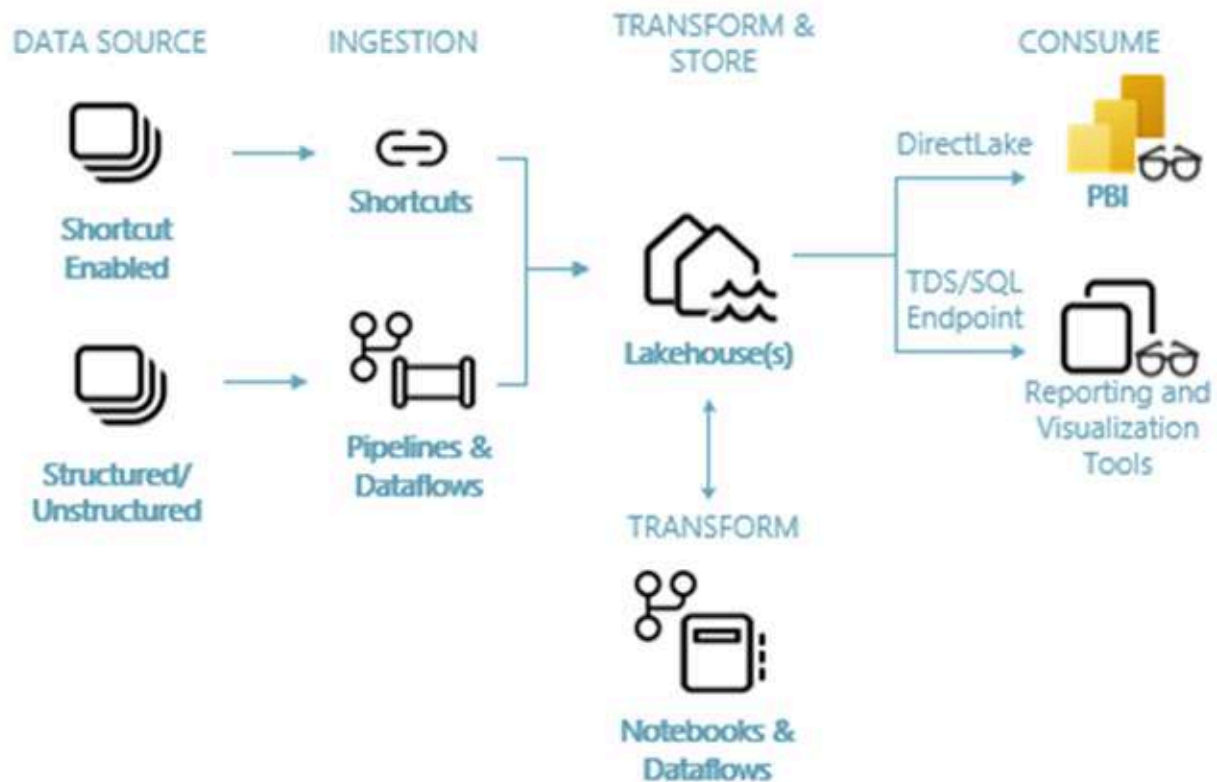
3) The Conflict (Warehouse vs Data Lake)

Dimension	Warehouse	Data Lake
Data type	Structured	Any
Schema	On write	On read
BI performance	Excellent	Poor
ML support	Weak	Strong
Cost	High	Low
Reliability	Strong	Weak
Governance	Native	Manual

Outcome:

Enterprises ran both, duplicating data, cost, and complexity.

4) Lakehouse — The Resolution



Why Lakehouse emerged

To combine:

- Warehouse reliability
- Lake scalability and openness

Core innovations

- ACID transactions on object storage
- Table formats (Delta / Iceberg / Hudi)
- Unified batch + streaming
- SQL + ML on same data

What it fixes (point-by-point)

Lake Problem	Warehouse Problem	Lakehouse Solution
No ACID	—	ACID on files
Poor BI	—	Optimized SQL engine
Data swamp	—	Enforced schemas + versioning
Rigid ETL	Rigid ETL	ELT + schema evolution
Duplicate systems	Duplicate systems	One data copy
ML isolation	ML isolation	Shared data for BI + ML

Typical platforms

- Databricks
- Microsoft Fabric

5) Architectural Comparison (Mental Model)

WAREHOUSE

Sources → ETL → Curated Tables → BI

DATA LAKE

Sources → Raw Files → (Many Tools) → BI / ML (fragile)

LAKEHOUSE

Sources → One Lake (ACID Tables)

→ BI

→ ML

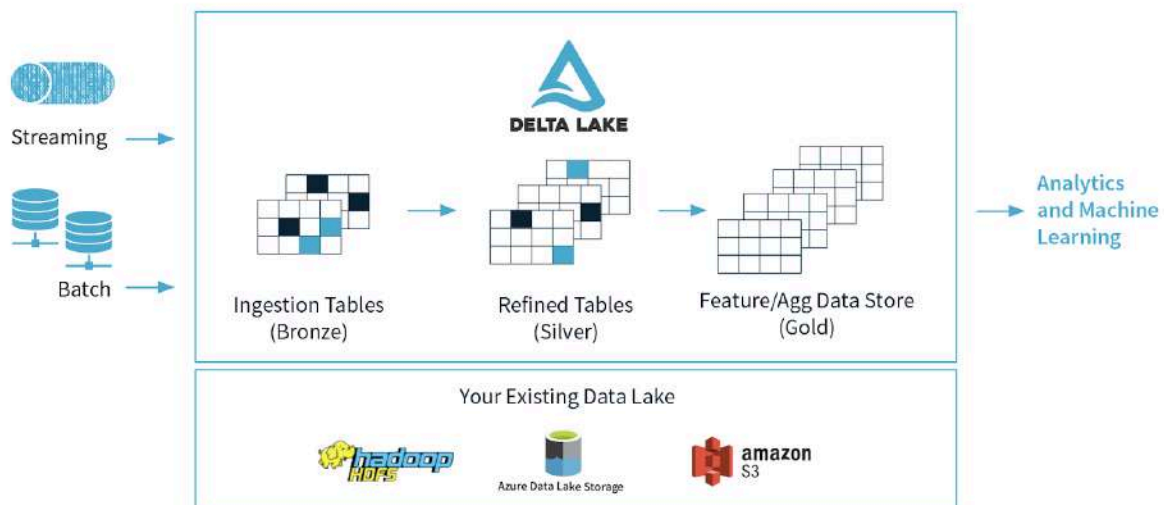
→ Streaming

6) DP-700 Exam Framing

- Warehouse failed due to rigidity and cost
 - Data Lake failed due to lack of trust and performance
 - Lakehouse exists to eliminate duplication
 - Fabric extends lakehouse with platform unification + governance
-

7) One-Glance Memory Anchors

- Warehouse = trust without scale
- Data Lake = scale without trust
- Lakehouse = trust + scale on one copy of data



Introduced

- Databricks

- Delta Lake / Iceberg / Hudi
- ACID on object storage

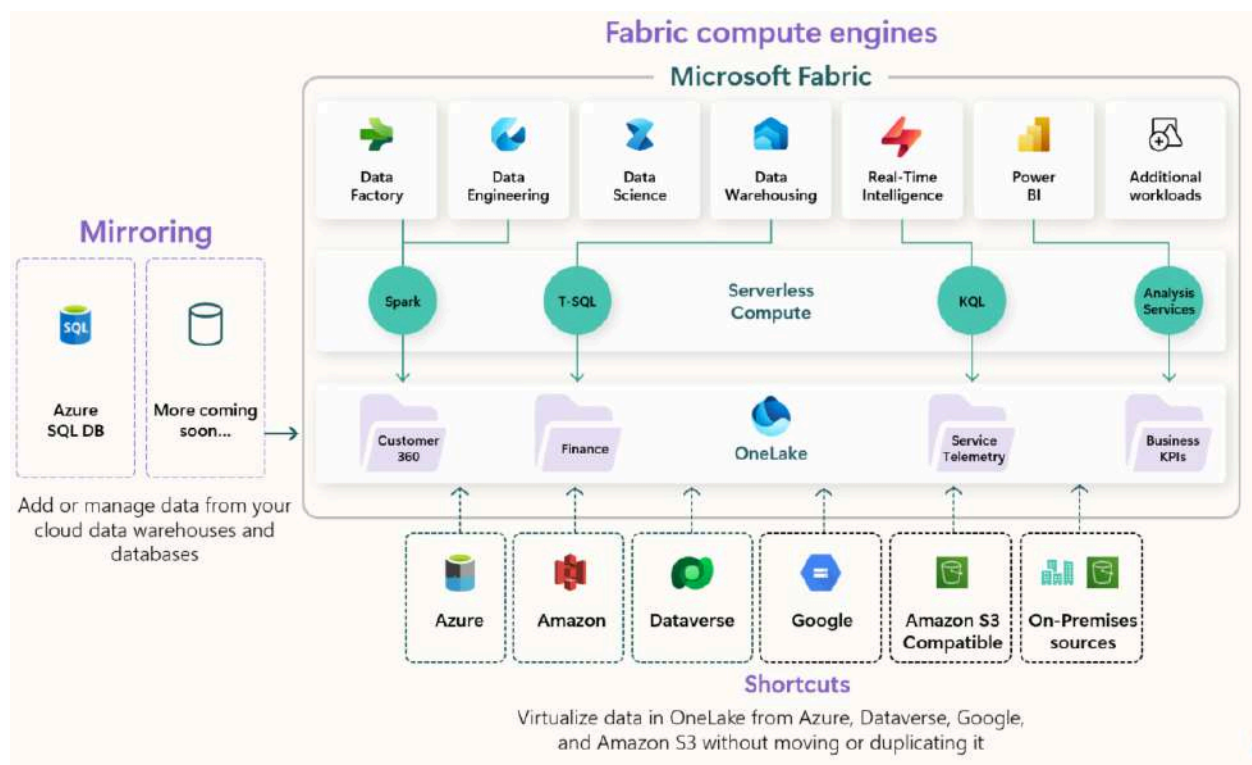
Problems it solved

- Reliability on data lakes
- Time travel, schema evolution
- Single source for BI + ML

Still unsolved

- Fragmented tools
- Governance complexity
- Skill silos (engineer vs analyst)

6) Unified Analytics Platforms — Fabric / Modern DP-700 Scope (2023–)



Introduced

- **Microsoft Fabric**
- OneLake
- Lakehouse + Warehouse + Real-time + BI
- Native governance

What finally got solved

- **Tool sprawl** → single platform
- **Data duplication** → OneLake
- **Governance lag** → built-in lineage, security
- **Engineer–Analyst gap** → shared artifacts
- **Batch vs streaming divide** → unified engine

Evolution Summary (Problem → Solution)

Old Challenge	Why It Failed	Modern Resolution
Fixed schemas	Change intolerance	Schema evolution
Vertical scaling	Cost + limits	Elastic compute
Batch-only	Latency	Streaming + micro-batch
Raw data loss	Reprocessing impossible	Immutable lake storage
Data swamp	No trust	ACID + governance
Tool silos	Slow delivery	Unified platforms

DP-700 Exam Angle

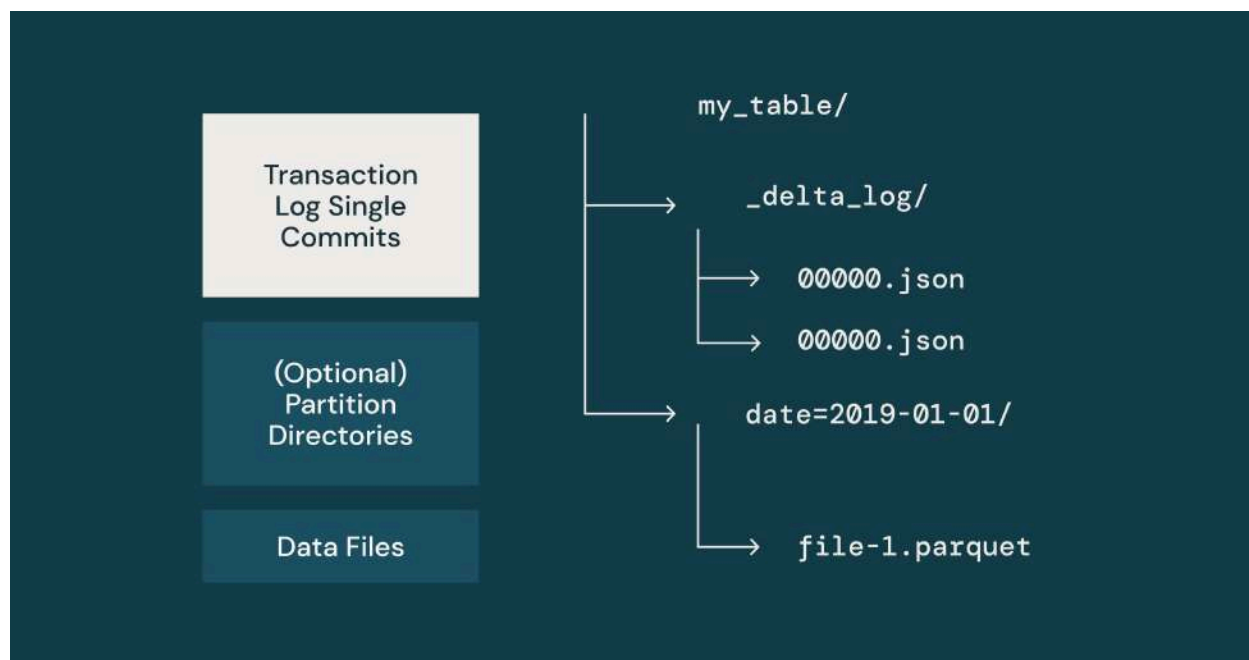
- Expect **why** shifts happened, not just **what**

- Focus on:
 - ETL → ELT reasoning
 - Data lake vs lakehouse
 - Governance as first-class concern
 - Unified analytics benefits over stitched stacks
-

One-Glance Memory Anchors

- RDBMS solved structure, not scale
- Hadoop solved storage, not usability
- Spark solved speed, not reliability
- Cloud DW solved SQL scale, not openness
- Lakehouse solved trust, not fragmentation
- Fabric solves **end-to-end cohesion**

Delta Log / Transaction Log — explained simply and clearly



1) What is Delta Log (Transaction Log)

Delta Log is a **special folder** inside a Delta table.

- Folder name: `_delta_log`
- It keeps the **history and rules** of the table
- It tells the system:

- what data exists
- what changed
- which version is correct

Used by **Delta Lake**.

One-line meaning:

Delta Log is the *brain + memory* of a Delta table.

2) Why Delta Log was needed (problem before)

In a normal data lake:

- Files are added directly
- No tracking
- If a job fails → half data written
- Two jobs overwrite each other
- No idea which data is correct

This causes:

- Duplicate data
 - Corrupt tables
 - Wrong BI results
-

3) What is stored inside Delta Log

Delta Log stores **metadata**, not actual data.

Main contents

1. Transaction JSON files

- One file per change
- Example: `000000000000000010.json`

2. Actions recorded

- `add` → new data file added
- `remove` → old file removed
- `metadata` → schema change
- `protocol` → reader/writer rules

3. Schema information

- Column names
- Data types
- Nullable rules

4. Table versions

- Version 0, 1, 2, 3...
- Each change creates a new version

5. Checkpoints (Parquet)

- Snapshot summary
- Speeds up reads

4) How Delta Log works (step-by-step)

WRITE REQUEST

↓

Validate schema + rules

↓

Create new data files

↓

Update delta log (new version)

↓

Commit succeeds

If any step fails:

- Log is NOT updated
- Old version remains valid

This is the key safety feature.

5) How Delta Log solves data lake problems

Data Lake Problem	How Delta Log fixes it
Partial writes	Commit happens only after success
Concurrent writes	Serialized transactions
Data corruption	Old version stays untouched
No history	Full version history
No rollback	Time travel

6) What is ACID (very important)

ACID are **rules for safe data updates**.

A — Atomicity

- All or nothing
- Either full data written or zero

Delta Log role:

Commit only after success.

C — Consistency

- Data follows schema and rules

Delta Log role:

Rejects bad schema or invalid writes.

I — Isolation

- Multiple jobs don't disturb each other

Delta Log role:

Uses versioning so jobs don't clash.

D — Durability

- Once written, data stays safe

Delta Log role:

Metadata stored reliably; versions immutable.

7) Example (simple)

Two jobs writing sales data at same time.

Without Delta Log

- Files overwrite
- Duplicates
- Broken table

With Delta Log

- Job A → version 5
- Job B → version 6
- Readers see only completed versions

No corruption.

8) Relationship recap (lock this)

Data files → actual data

Delta Log → truth, history, rules

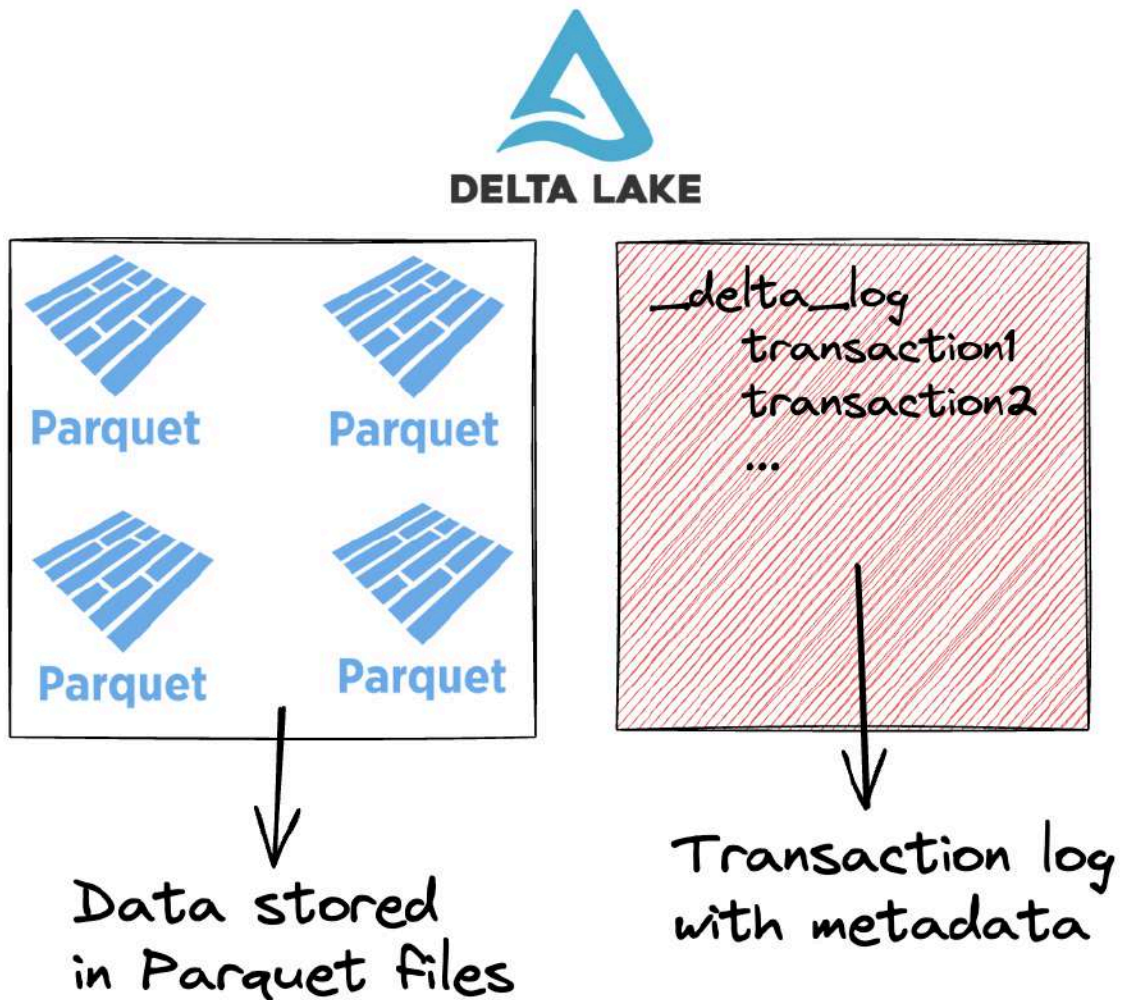
ACID → guarantees provided by Delta Log

9) One-line exam memory

Delta Log enables ACID by tracking every change as a versioned transaction.

Delta Lake, Delta Table — what they are, file formats, and components

Contents of a Delta table



1) What is Delta Lake

Delta Lake is a **storage layer** that sits on top of a data lake.

Plain meaning:

- Data stays as files in object storage

- Delta Lake adds **rules, safety, and history**

Purpose:

- Make data lakes reliable like databases
- Enable ACID transactions, schema control, and time travel

2) What is a Delta Table

A **Delta table** is **data stored in Delta Lake format**.

It is **not** a database table.

It is:

- A **folder** in storage
- With **data files + a transaction log**

Structure:

```
/sales_delta_table/  
├─ part-0001.parquet  
├─ part-0002.parquet  
├─ _delta_log/  
|   ├─ 00000000000000000000.json  
|   ├─ 00000000000000000001.json  
|   └─ 00000000000000000010.checkpoint.parquet
```

3) File formats used in Delta Lake

1) Parquet (data files)

- Columnar
- Compressed
- Fast for analytics
- Stores actual rows and columns

Used for:

- Table data
 - Checkpoints
-

2) JSON (transaction log)

- Human-readable
- Append-only
- Records every change

Used for:

- Adds
 - Removes
 - Schema changes
 - Version control
-

4) Core components of Delta Lake

1) Data Files

- Format: Parquet
- Contain actual table data

- Immutable (never updated in place)
-

2) Delta Log (`_delta_log`)

- Transaction history
- Version control
- Schema rules
- Commit protocol

This is what gives ACID.

3) Schema Enforcement

- Rejects invalid data
 - Prevents silent corruption
-

4) Schema Evolution

- Allows controlled schema changes
 - No pipeline breakage
-

5) Time Travel

- Query older versions
 - Rollback mistakes
-

6) Concurrency Control

- Multiple writers supported
 - Readers see only committed data
-

5) How Delta Lake works (write path)

Write request

↓

Validate schema

↓

Write new parquet files

↓

Update delta log (new version)

↓

Commit success

If failure occurs:

- Delta log not updated
 - Old version remains active
-

6) Difference: Delta Lake vs Delta Table

Term	Meaning
Delta Lake	Technology / storage layer

Delta Table One dataset stored using Delta Lake

7) Why Delta Lake exists (problem → fix)

Problem in Data Lake	Delta Lake Fix
Partial writes	Atomic commits
No schema control	Enforcement + evolution
No history	Versioned log
Concurrent writes	Isolation
Data corruption	Immutable files

8) Where Delta Lake is used

- Lakehouse architectures
 - BI + ML on same data
 - Platforms like **Databricks** and Fabric-style systems
-

9) One-line memory lock

Delta Lake = Parquet data + Delta Log = ACID data lake

End

Case 1 — Copy activity writes to Files (default)

Input: CSV

Action: Pipeline → Copy activity → Sink = **Lakehouse (Files)**

Result in OneLake:

`/Files/sales.csv`

- File remains **CSV**
- No Delta table
- SQL endpoint cannot query it
- Spark can read it as a file

This is just storage.

Case 2 — Copy activity writes to Tables

Input: CSV

Action: Pipeline → Copy activity → Sink = **Lakehouse (Tables)**

Result in OneLake:

`/Tables/Sales/`

`├ part-0000.parquet`

`├ part-0001.parquet`

`└ _delta_log/`

- CSV is **ingested**
- Stored as **Delta Lake table**

- Underlying format = Parquet + `_delta_log`
- SQL endpoint can query it
- Spark can query it

CSV does **not** exist anymore as CSV in Tables.

Case 3 — Notebook loads CSV and writes table

Input: CSV in Files

Action: Spark notebook

```
df = spark.read.csv("/Files/sales.csv", header=True)

df.write.format("delta").saveAsTable("Sales")
```

Result:

- Original CSV stays in **Files**
 - Delta table created in **Tables**
-

Visual mental model (OneLake)

Lakehouse

├ Files → raw files (csv, json, parquet)

└ Tables → Delta Lake only

- **Files** = data lake behavior
 - **Tables** = warehouse-like behavior
-

Key rule to never forget

Tables = Delta only

Files = original format

Lakehouse does **not auto-promote** CSV to Delta.

Exam lock-in rule

If question says:

- “Loaded CSV into Lakehouse using pipeline”
- And does **not** say *Tables*

→ **Answer: CSV stored as file**

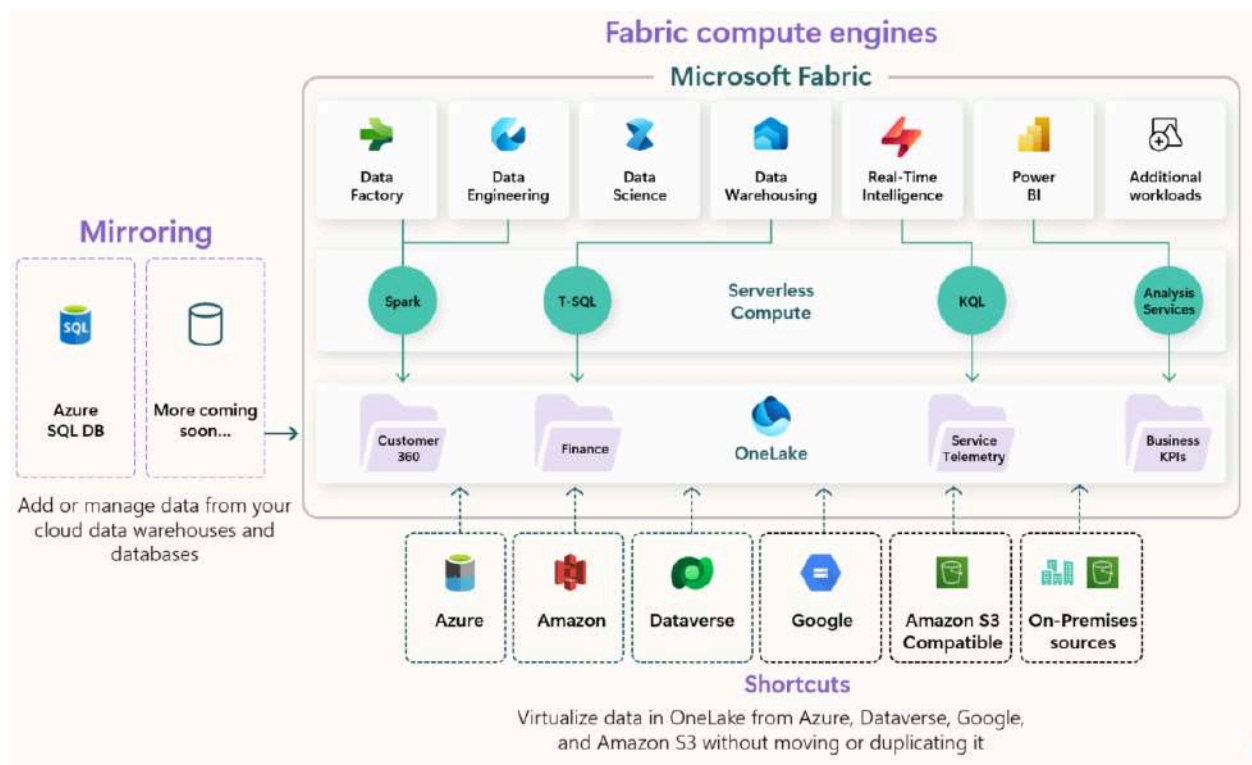
If it explicitly says:

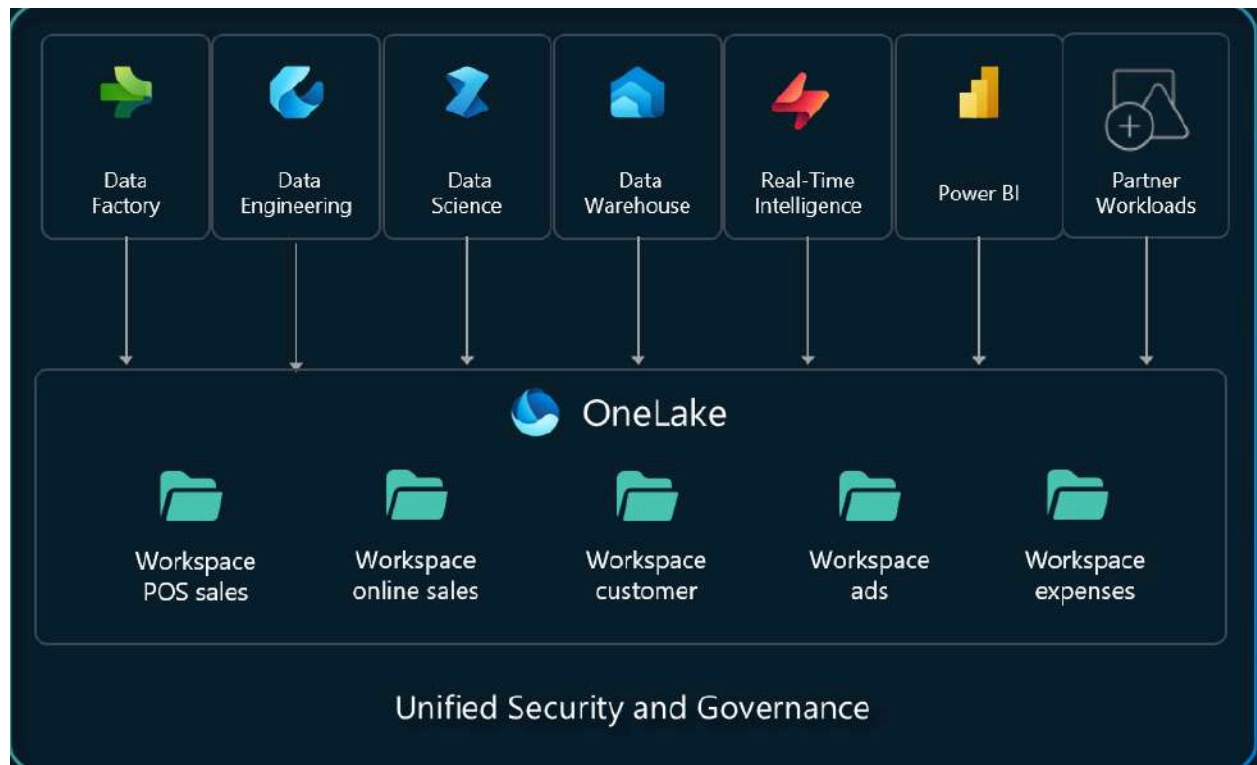
- Sink = Tables
- Or “created a table”

→ **Answer: Delta table**

That single distinction answers all such questions.

Why Microsoft Fabric exists — even though Azure already exists





1) First understand Azure (the problem context)

Microsoft Azure is a **cloud platform**, not a single analytics system.

Azure gives **separate services**:

- Data Factory → ingestion
- Synapse → SQL analytics
- Databricks → Spark
- Data Explorer → streaming
- Power BI → visualization
- ADLS → storage

Each is good **alone**.

The real problem

- Too many tools
- Separate setup, billing, security
- Data copied again and again
- Engineers needed everywhere
- Slow end-to-end delivery

Azure was **powerful**, but **fragmented**.

2) What Microsoft saw (industry reality)

Most customers wanted:

- One platform
- One storage
- One security model
- SQL + Spark + BI together
- Less engineering, more outcomes

Competitors already moving here:

- Databricks (Lakehouse)
- Snowflake (SQL-first analytics)

Azure analytics stack was **too complex** for most teams.

3) What is Microsoft Fabric

Microsoft Fabric is a **single SaaS analytics platform built on top of Azure**.

Key point:

Fabric does NOT replace Azure.

Fabric **uses Azure internally** but hides the complexity.

4) Why Fabric was created (core reasons)

Reason 1: Tool sprawl

Before:

ADF + ADLS + Synapse + Databricks + Power BI

After Fabric:

Fabric (everything inside)

Reason 2: Data duplication

Before:

- Same data copied to:
 - ADLS
 - Synapse
 - Power BI
 - ML stores

Fabric introduces:

- **OneLake** (single data copy)
-

Reason 3: Skill silos

Before:

- Data engineer
- Spark engineer
- SQL developer
- Power BI developer

Fabric:

- Shared artifacts
 - SQL, Spark, BI on same data
-

Reason 4: Governance pain

Before:

- Security configured per service
- Lineage fragmented

Fabric:

- Built-in governance
 - Automatic lineage
 - Unified security
-

5) What Fabric actually contains

Fabric is a **bundle of analytics experiences**:

- Data Engineering (Spark)
- Data Factory (ingestion)

- Data Warehouse (SQL)
- Lakehouse (Delta tables)
- Real-Time Analytics
- Power BI
- OneLake (storage)

All **pre-wired**.

6) Azure vs Fabric (clear difference)

Azure	Microsoft Fabric
Cloud platform	Analytics SaaS
Building blocks	Ready house
DIY architecture	Opinionated architecture
Infra control	Infra abstracted
Many services	One platform
For experts	For teams

Azure = Lego box

Fabric = Fully built smart house

7) Why Fabric is important for DP-700

DP-700 focuses on:

- Lakehouse
- OneLake
- Unified analytics
- Reduced data movement
- SQL + Spark together

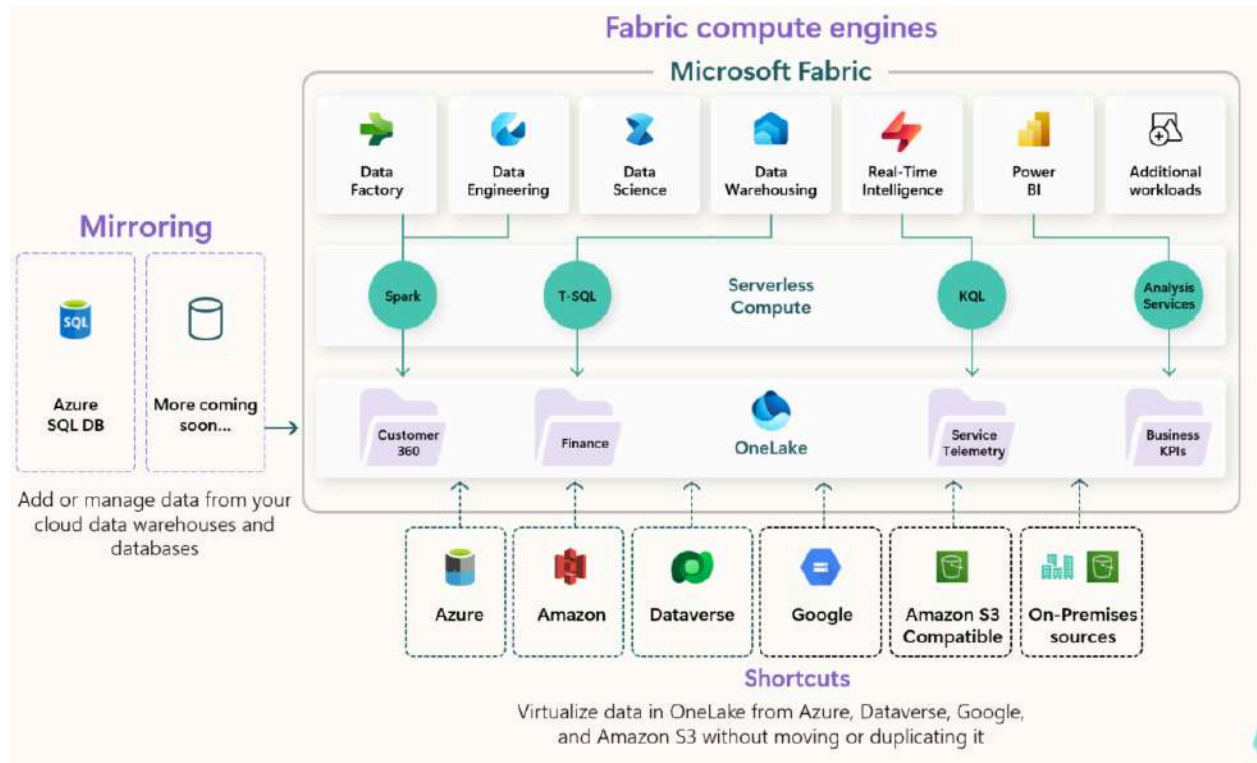
Fabric is Microsoft's **official answer** to modern data engineering.

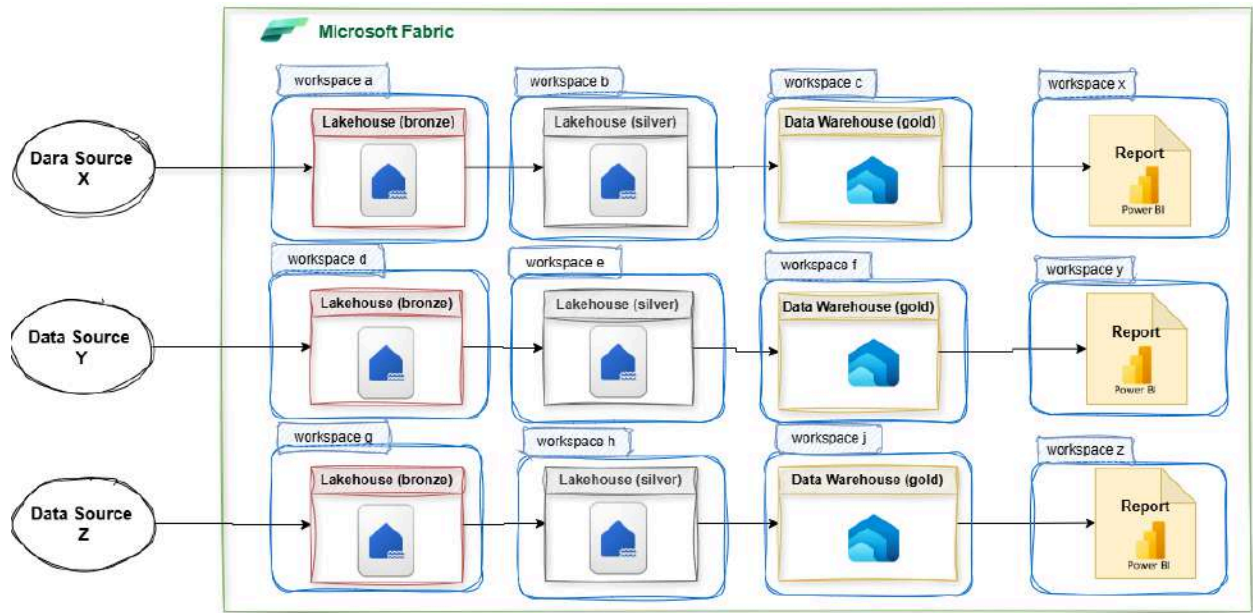
8) One-line memory lock

Azure gives tools; Fabric gives outcomes.

End.

Microsoft Fabric hierarchy — clear, strict, exam-ready





1) What “Fabric hierarchy” means

Hierarchy = how Fabric organizes everything
(from tenant → data → reports)

Fabric is owned and governed at the **tenant level**, but **work happens inside workspaces**, and **data lives in OneLake**.

2) Top → Bottom hierarchy

Level 1: Tenant

- Company-level boundary
- Created when Fabric is enabled in Microsoft 365
- Controls:
 - Licensing
 - Admin settings
 - Security policies

This is the **highest control layer**.

Level 2: Capacity

- Compute power
- Determines performance and cost
- Shared across workspaces

Think: **engine power**, not data.

Level 3: Workspace (MOST IMPORTANT)

- Main working container
- Everything lives inside a workspace
- Security is applied here

A workspace can contain:

- Lakehouse
 - Warehouse
 - Pipelines
 - Notebooks
 - Power BI reports
-

Level 4: OneLake (Storage layer)

OneLake

- Single logical data lake for entire tenant

- Automatically created
- No manual ADLS setup

OneLake structure:

OneLake

└─ Workspace

└─ Item (Lakehouse / Warehouse)

└─ Tables / Files

Level 5: Items (Artifacts)

Items are **analytics objects** inside a workspace.

Common items:

- Lakehouse
- Warehouse
- Data Pipeline
- Notebook
- Dataflow Gen2
- Semantic Model
- Power BI Report

Level 6: Data Objects

Inside items:

Lakehouse

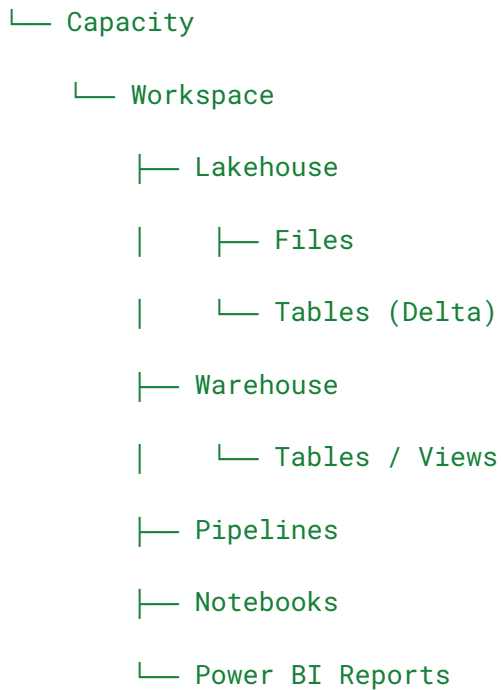
- Files (raw data)
- Tables (Delta tables)

Warehouse

- Tables
- Views
- Stored logic

3) Visual mental model (lock this)

Tenant



All data physically resides in **OneLake**.

4) Why this hierarchy exists (problem → fix)

Old Azure Problem

Fabric Hierarchy Fix

Multiple storages

OneLake

Tool-level security

Workspace security

Data duplication

Single copy

Fragmented lineage

Workspace-level lineage

Hard governance

Tenant-level control

5) Key exam facts (DP-700)

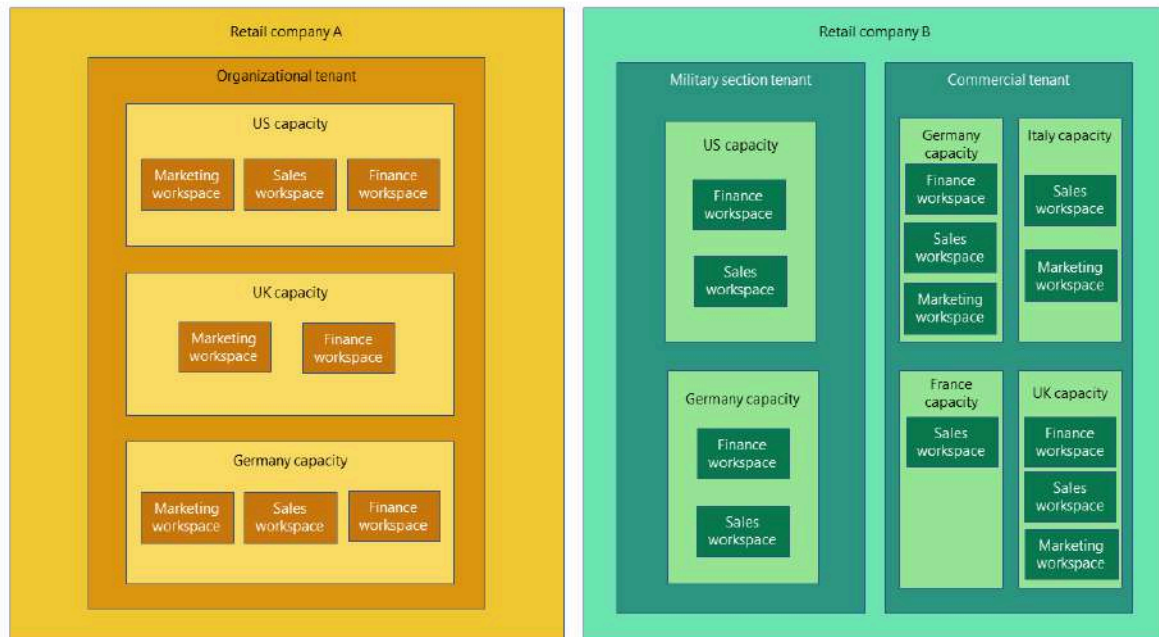
- **Workspace is the unit of collaboration**
- **OneLake is tenant-wide**
- **Items do not own storage — OneLake does**
- **Security flows downward** (tenant → workspace → item)
- **Capacity is shared, not tied to data**

6) One-line memory lock

Tenant owns Fabric, Workspace runs Fabric, OneLake stores Fabric data.

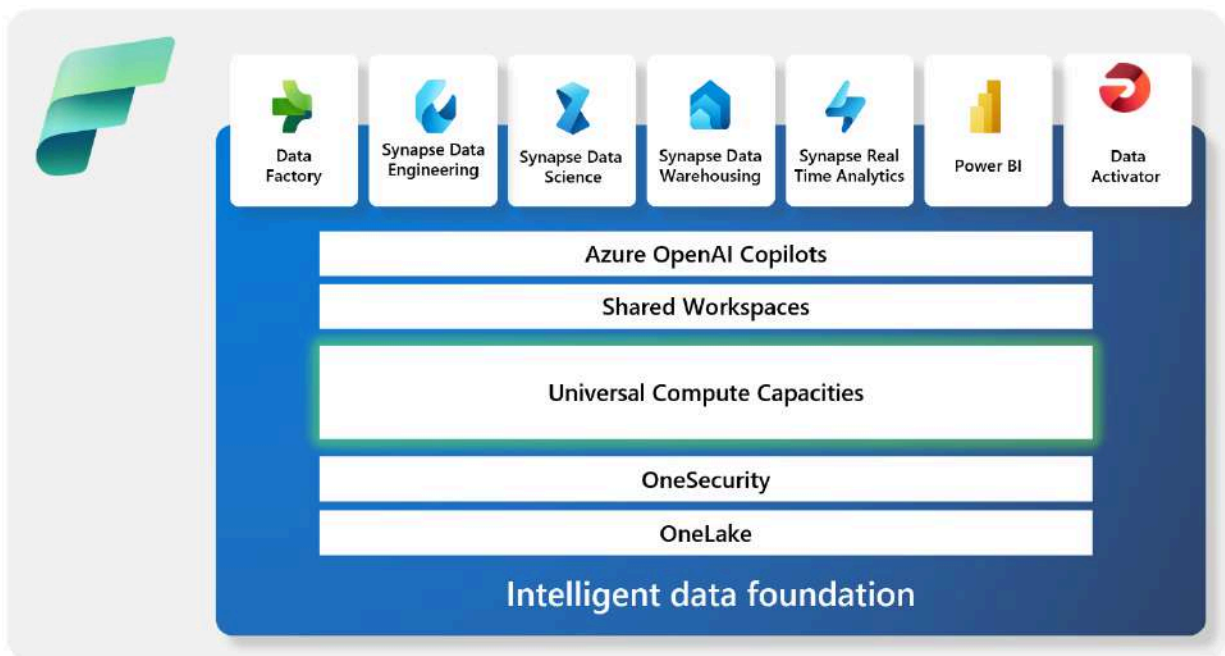
End.

Microsoft Fabric Capacity — what it is, why it exists, and types



Microsoft Fabric

The data platform for the era of AI



SKU	Capacity unit	Pay-as-you-go	Reservation
F2	2	\$262.80/month	\$156.334/month ~41% savings
F4	4	\$525.60/month	\$312.667/month ~41% savings
F8	8	\$1,051.20/month	\$625.334/month ~41% savings
F16	16	\$2,102.40/month	\$1,250.667/month ~41% savings
F32	32	\$4,204.80/month	\$2,501.334/month ~41% savings
F64	64	\$8,409.60/month	\$5,002.667/month ~41% savings
F128	128	\$16,819.20/month	\$10,005.334/month ~41% savings
F256	256	\$33,638.40/month	\$20,010.667/month ~41% savings
F512	512	\$67,276.80/month	\$40,021.334/month ~41% savings
F1024	1024	\$134,553.60/month	\$80,042.667/month ~41% savings
F2048	2048	\$269,107.20/month	\$160,085.334/month ~41% savings

1) What is Fabric Capacity

Microsoft Fabric capacity is **shared compute power**.

Plain meaning:

- Capacity = **CPU + memory** that runs everything in Fabric
- Workspaces **do not run by themselves**
- They run **on a capacity**

No capacity → Fabric items cannot execute.

2) Why Fabric Capacity exists

Before Fabric:

- Power BI had its own capacity
- Synapse had its own compute
- Data Factory had its own runtime

This caused:

- Separate billing
- Separate scaling
- Separate throttling

Fabric solved this by:

One shared engine for all analytics

Capacity is that shared engine.

3) What uses Fabric Capacity

All Fabric workloads consume capacity:

- Lakehouse (Spark)
- Warehouse (SQL)
- Data Pipelines
- Notebooks
- Dataflows Gen2
- Real-Time Analytics
- Power BI reports and refresh

If something runs → it **burns capacity units**.

4) How capacity works (simple flow)

User runs query / pipeline

↓

Workspace sends request

↓

Fabric Capacity provides compute

↓

Result returned

Multiple workspaces can share **one capacity**.

5) Why capacity is important

Capacity controls:

- Performance
- Concurrency
- Cost
- Throttling behavior

Low capacity:

- Slow queries
- Queued pipelines
- Report delays

High capacity:

- Parallel execution
 - Faster Spark jobs
 - Smooth BI experience
-

6) Types of Fabric Capacity

1) Fabric Capacity (F-SKU) — MAIN TYPE

These are modern Fabric SKUs.

Examples:

- F2
- F4
- F8
- F16
- F32
- F64
- F128
- F256
- F512
- F1024
- F2048

Rules:

- Higher F → more compute
- One F capacity can serve many workspaces
- Billed per hour

This is what DP-700 focuses on.

2) Power BI Premium Capacity (P-SKU) — Legacy

Older model:

- P1, P2, P3, etc.

Limitations:

- Designed mainly for Power BI
- Not full Fabric feature set

Microsoft direction:

- Move from P-SKU → F-SKU
-

3) Trial Capacity

- Free, temporary
- Limited compute
- Used for learning and demos

Not production-grade.

7) Capacity vs Workspace (exam trap)

Thing	What it is
Capacity	Compute engine
Workspace	Logical container
OneLake	Storage
Item	Analytics object

Workspace **uses** capacity.

Capacity **does not belong** to workspace.

8) Capacity sharing rule (critical)

- One capacity → many workspaces
 - One workspace → only one capacity
 - Heavy workload in one workspace can impact others on same capacity
-

9) One-line memory lock

Fabric Capacity is the shared compute engine that powers all Fabric workloads.

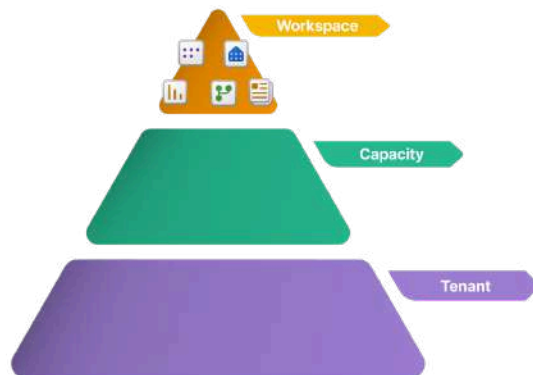
End.

Roles in Microsoft Fabric — clear, strict, exam-ready

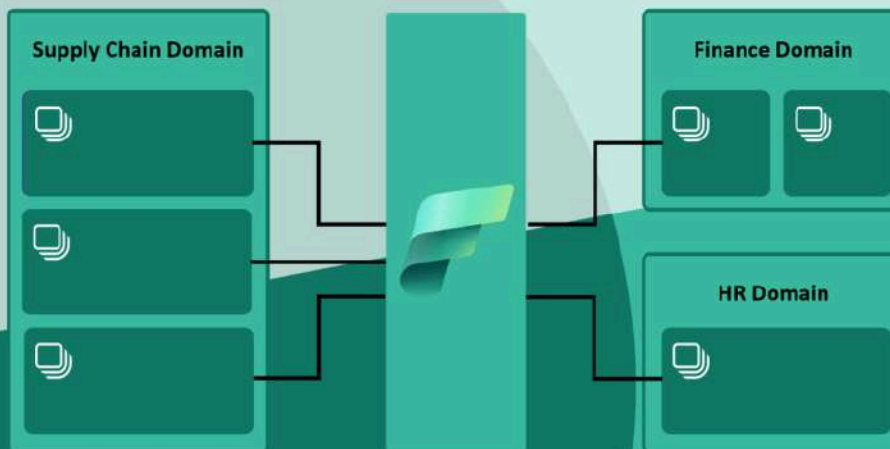
 **Workspace Admin**
Control workspace
resources and access

 **Capacity Admin**
Control capacity, manage
computational resources

 **Tenant Admin**
Set up tenant, create capacity,
enable global features



Governance in Microsoft Fabric: Understanding the Roles and Domains



1) Big picture: where roles apply

Roles exist at **two main levels**:

1. **Tenant level** → Admin control
2. **Workspace level** → Day-to-day work control

Fabric **does not use table-level IAM roles like databases**.
Access is controlled primarily through **workspace roles**.

2) Tenant-level role (highest)

Fabric / Power BI Admin

- Applies to entire tenant
- Controls:
 - Enable/disable Fabric features
 - Capacity creation & assignment
 - Tenant-wide security settings

Key point:

- Admins **do not automatically access data**
 - They control **platform behavior**, not content
-

3) Workspace-level roles (MOST IMPORTANT)

Workspace is the **main security boundary** in Fabric.

1) Admin

- Full control

- Can:
 - Add/remove users
 - Assign capacity
 - Delete workspace
 - Modify all items

Think: **workspace owner**

2) Member

- Create and edit content
- Can:
 - Create lakehouse, warehouse, pipelines
 - Run notebooks and pipelines
 - Publish reports

Cannot:

- Manage workspace permissions

Think: **data engineer / developer**

3) Contributor

- Edit existing items
- Can:
 - Modify notebooks, pipelines, datasets
- Cannot:
 - Change workspace settings

- Add users

Think: **developer without admin rights**

4) Viewer

- Read-only
- Can:
 - View reports
 - Query data (read)
- Cannot:
 - Create or modify anything

Think: **business user**

4) Role capability comparison

Action	Admin	Member	Contributor	Viewer
Manage users	Yes	No	No	No
Create items	Yes	Yes	No	No
Edit items	Yes	Yes	Yes	No
Run pipelines	Yes	Yes	Yes	No

View reports Yes Yes Yes Yes

5) Data access inside Fabric (important nuance)

Fabric roles control **access to items**, not row-level security by default.

For data-level control:

- Use **Power BI RLS**
- Use **SQL permissions** in Warehouse
- Use **Delta table permissions** (where supported)

Workspace role = **entry gate**

Data security = **inside the house**

6) Why Fabric uses this role model

Old Azure model:

- IAM per service
- Hard to manage
- Inconsistent security

Fabric model:

- Workspace = security boundary
- One role assignment works for:
 - Lakehouse
 - Warehouse
 - BI

- Pipelines

Result:

- Simpler governance
 - Faster collaboration
-

7) Exam memory locks (DP-700)

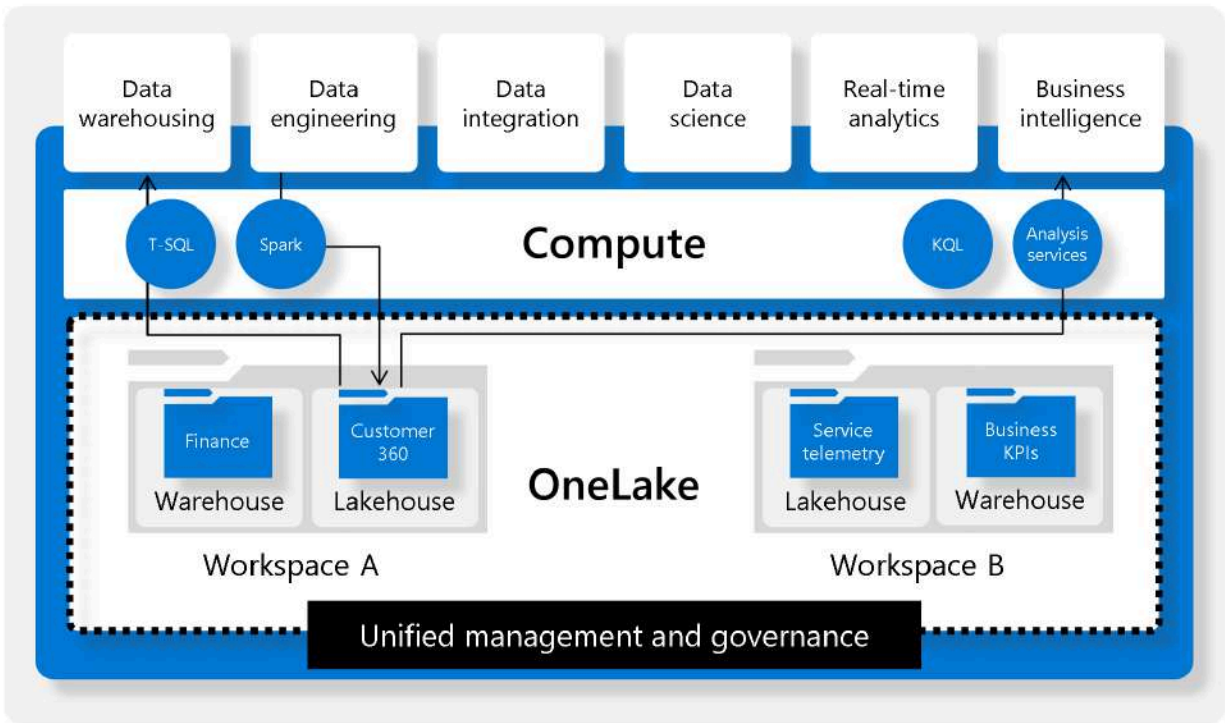
- Workspace is the **unit of security**
 - Roles are **Admin / Member / Contributor / Viewer**
 - Capacity is **not** a role
 - OneLake inherits workspace access
 - Tenant admin $\neq$ data owner
-

8) One-line memory lock

In Microsoft Fabric, security is managed at the workspace level using four roles: Admin, Member, Contributor, Viewer.

End.

What is OneLake in Microsoft Fabric





1) What OneLake is

OneLake is the **single, unified data lake** for **Microsoft Fabric**.

Plain meaning:

- OneLake = **one storage system**
- Shared by **all Fabric workloads**
- Automatically managed

You do not create it.
It exists once per tenant.

2) Why OneLake was created

Before OneLake:

- Every tool had its own storage
- Same data copied many times
- Different security models
- High cost and inconsistency

OneLake fixes this by enforcing:

One copy of data, many engines

3) What lives in OneLake

Everything stored by Fabric lives here:

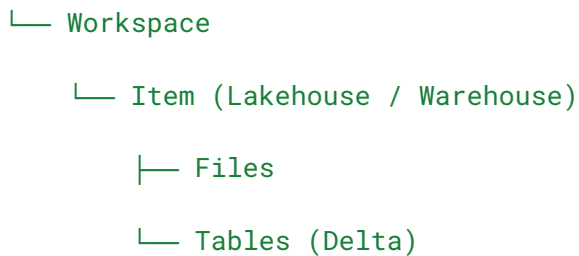
- Lakehouse files and Delta tables
- Warehouse data
- Dataflow Gen2 outputs
- Real-Time Analytics data
- Power BI semantic model storage

Physically: object storage

Logically: one global lake

4) OneLake hierarchy (important)

OneLake



Rules:

- Workspace maps to a folder
 - Items do not own storage
 - OneLake owns all storage
-

5) What problems OneLake solves

Old Problem

OneLake Resolution

Multiple data copies	Single logical copy
Tool-specific storage	Unified storage
Complex access control	Workspace-based security
Data movement	Zero-copy sharing
Fragmented lineage	End-to-end lineage

6) OneLake vs traditional Data Lake

Traditional Data Lake	OneLake
Manually created	Auto-provisioned
Tool-specific access	Platform-wide access
Separate governance	Built-in governance
Data copied across tools	Single shared copy

7) OneLake + Lakehouse relationship

- OneLake = **where data lives**

- Lakehouse = **how data is organized**
- Delta tables = **how data is made reliable**

All lakehouse tables physically sit inside OneLake.

8) Exam-critical facts (DP-700)

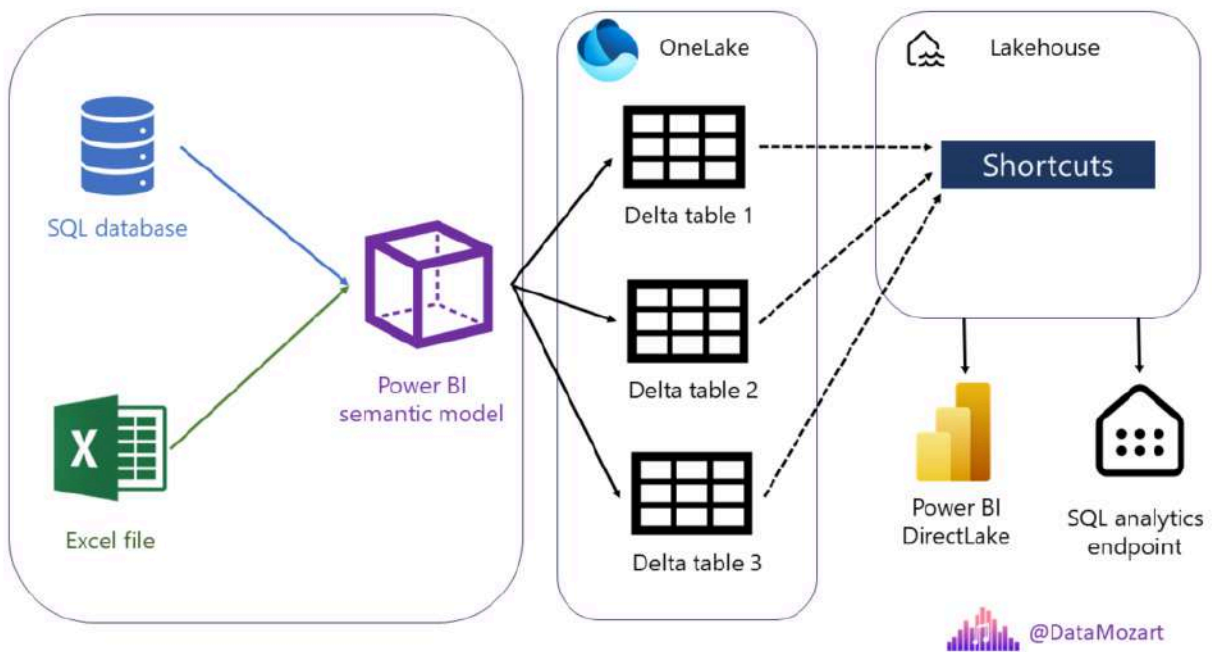
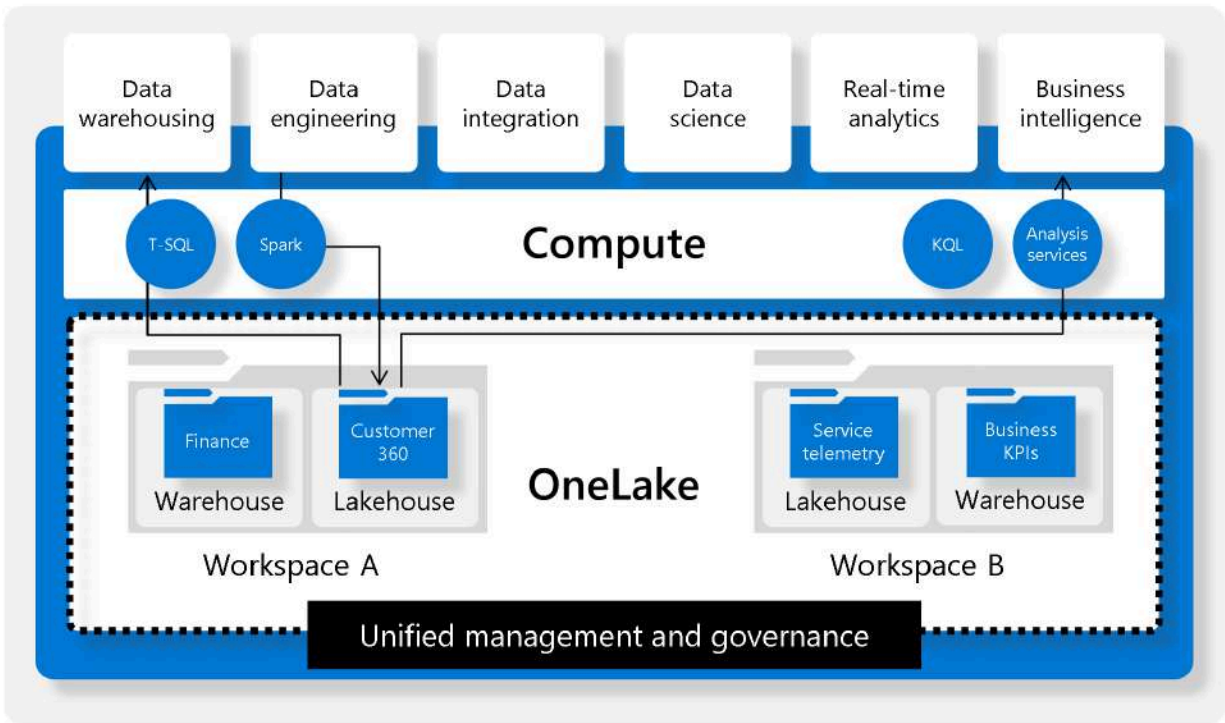
- OneLake is **tenant-wide**
 - OneLake is **logical**, not per workspace
 - Data is **not duplicated** between workloads
 - Security flows from **workspace roles**
 - Storage is abstracted from users
-

9) One-line memory lock

OneLake is Fabric's single, tenant-wide data lake that stores all analytics data with zero duplication.

End.

Why OneLake exists — why it was needed, and what it adds that did not exist before



1) The real problem before OneLake

Before OneLake, even in Azure, data looked like this:

ADF → ADLS (raw copy)

Synapse → ADLS (another copy)

Databricks → ADLS (another copy)

Power BI → its own storage

ML → another store

What was broken

- Same data copied **many times**
- High storage cost
- Sync delays
- Different security per tool
- No single source of truth
- Governance was fragmented

This was called **data sprawl**.

2) Why Microsoft created OneLake

OneLake was created to enforce one rule:

One copy of data for the entire analytics platform

Not per tool.

Not per team.

Not per engine.

3) What OneLake actually is

- A **single, tenant-wide data lake**
- Automatically created
- Used by all workloads in **Microsoft Fabric**
- Built on Azure storage, but **hidden from users**

You do not provision containers, accounts, or keys.

4) Features of OneLake (what did NOT exist before)

1) Single logical copy of data

Before:

- Data duplicated across services

With OneLake:

- One physical copy
- SQL, Spark, BI all read the same data

This never existed natively in Azure analytics.

2) Zero-copy sharing

Before:

- Move or copy data between systems

With OneLake:

- Same data visible to:

- Lakehouse
- Warehouse
- Power BI
- Real-time analytics

No pipelines. No duplication.

3) Automatic provisioning

Before:

- Create ADLS
- Configure networking
- Manage keys
- Set permissions

With OneLake:

- Created automatically
 - Fully managed
 - No infrastructure work
-

4) Workspace-based security

Before:

- IAM per service
- ACLs per storage account
- Hard to audit

With OneLake:

- Security inherited from workspace roles
 - One access model for all data
-

5) Built-in governance & lineage

Before:

- Separate governance tools
- Manual lineage stitching

With OneLake:

- End-to-end lineage across ingestion → storage → BI
 - Unified Purview integration
-

6) Open data formats

OneLake stores data as:

- Delta tables
- Parquet files

Not proprietary binaries.

This allows:

- Interoperability
 - Future portability
-

7) Shortcut capability (important)

OneLake can **virtually reference external data**:

- ADLS
- Amazon S3
- Other OneLake locations

No ingestion.
No duplication.

This did not exist in classic data lakes.

8) Performance optimization by design

Before:

- Storage unaware of analytics engines

With OneLake:

- Optimized for:
 - Spark
 - SQL
 - Power BI

Storage and compute are co-designed.

5) What OneLake is NOT (clarity)

- Not a database
- Not just ADLS renamed
- Not per workspace
- Not optional in Fabric

It is **the foundation**.

6) OneLake vs traditional data lake

Traditional Data Lake	OneLake
Many per team	One per tenant
Manual setup	Auto-managed
Data copied	Single copy
Tool-specific access	Platform-wide
Governance optional	Governance native

7) Why OneLake is critical for DP-700

DP-700 tests:

- Unified analytics
- Reduced data movement
- Lakehouse principles
- Governance-first design

OneLake is the **core enabler** of all four.

8) One-line memory lock

OneLake exists to eliminate data duplication and make one trusted copy of data usable by every analytics engine.

End.

OneLake File Explorer — what it is and how to access it on a local machine

1) What is OneLake File Explorer

OneLake File Explorer is a **Windows client** that lets **OneLake** appear like a **local drive** on your computer.

Meaning:

- OneLake looks like a normal folder
- You browse, open, copy files
- No Azure Storage account, keys, or containers

It is a **view**, not a copy.

2) Why it exists

Before:

- Data engineers needed portals, URLs, storage keys
- Analysts could not directly see lake files
- Debugging required cloud-only access

OneLake File Explorer fixes this:

- Direct visibility of lake data
- Faster debugging
- Easier file-level validation

- Familiar file-system experience
-

3) What you can do with it

- Browse OneLake folders
- View files (CSV, Parquet)
- Copy files **from local** → **OneLake**
- Copy files **from OneLake** → **local**
- Inspect Lakehouse **Files** area

All actions respect **workspace security**.

4) What you cannot do

- No bypass of Fabric security
- No arbitrary write into managed tables
- No execution of compute
- Not a database client

It is **storage access only**.

5) OneLake File Explorer hierarchy (what you see)

OneLake (virtual drive)

└─ Workspaces

└─ <Workspace Name>

└─ <Item Name>

└─ Files

└─ Tables

- **Files** → raw / landing data
 - **Tables** → managed Delta tables (read-only view)
-

6) How to implement (install) OneLake File Explorer

Step 1: Prerequisites

- Windows 10 or later
 - Fabric access enabled
 - Signed in with Microsoft account used for Fabric
-

Step 2: Download

- Download **OneLake File Explorer** from Microsoft Fabric documentation or Microsoft download center
-

Step 3: Install

- Run installer
 - Sign in using your Fabric account
 - Explorer mounts OneLake automatically
-

7) How to access OneLake on local machine

After installation:

1. Open **File Explorer**
2. You will see **OneLake** as a mounted location

Expand:

OneLake → Workspaces → Your Workspace → Lakehouse → Files

- 3.
4. Open or copy files like a normal folder

No credentials, keys, or URLs needed.

8) Security model (important)

- Access controlled by **Fabric workspace roles**
- Viewer → read-only
- Member / Admin → read + write (where allowed)
- Changes reflect instantly in OneLake

Local access ≠ elevated access.

9) Common use cases

- Validate ingestion outputs
- Upload reference files
- Inspect raw data
- Debug schema issues
- Share file-level data with analysts

10) Key differences vs ADLS Explorer

ADLS Explorer	OneLake File Explorer
Storage-account based	Tenant-wide
Requires keys / RBAC	Uses Fabric roles
Tool-specific	Platform-wide
Manual setup	Plug-and-play

11) One-line memory lock

OneLake File Explorer mounts OneLake as a local folder so users can securely browse and manage Fabric data from Windows.

End.

Short answer (lock this first)

OneLake is WHERE data lives.

Lakehouse and Warehouse are HOW you use that data.

Same storage. Different ways to work with it.

Step 1: What OneLake really is

OneLake is like **one big cupboard** for your whole company.

- All data files go here
- Only one physical copy
- Nobody works directly inside the cupboard

OneLake answers only one question:
“Where is data stored?”

Step 2: Why storage alone is not enough

If everyone just opens the cupboard:

- Some people want Excel-style tables
- Some want big raw files
- Some want fast reports
- Some want ML experiments

One storage cannot satisfy all **ways of working**.

So Fabric adds **interfaces**.

Step 3: What is a Lakehouse

Lakehouse = **flexible working style**

Think:

- Engineers
- Raw + processed data
- Files + tables
- Spark + SQL

Lakehouse is used when:

- Data is still evolving
- Schema changes
- ML or heavy transformation needed

It answers:

“I want freedom. I’ll shape the data myself.”

Step 4: What is a Warehouse

Warehouse = **disciplined working style**

Think:

- Business users
- Clean tables only
- Strong SQL rules
- High-performance reporting

Warehouse is used when:

- Data is final
- Schema is stable
- BI dashboards depend on it

It answers:

“I want fast, clean, trusted tables.”

Step 5: The critical point (this clears your confusion)

Lakehouse and Warehouse do **NOT** store separate data.

They both:

- Read from **the same OneLake**
- Write to **the same OneLake**
- Just **organize and expose data differently**

No duplication.

Simple analogy (10-year-old level)

- **OneLake** = water tank
- **Lakehouse** = tap for washing, cooking, experimenting
- **Warehouse** = filtered drinking-water tap

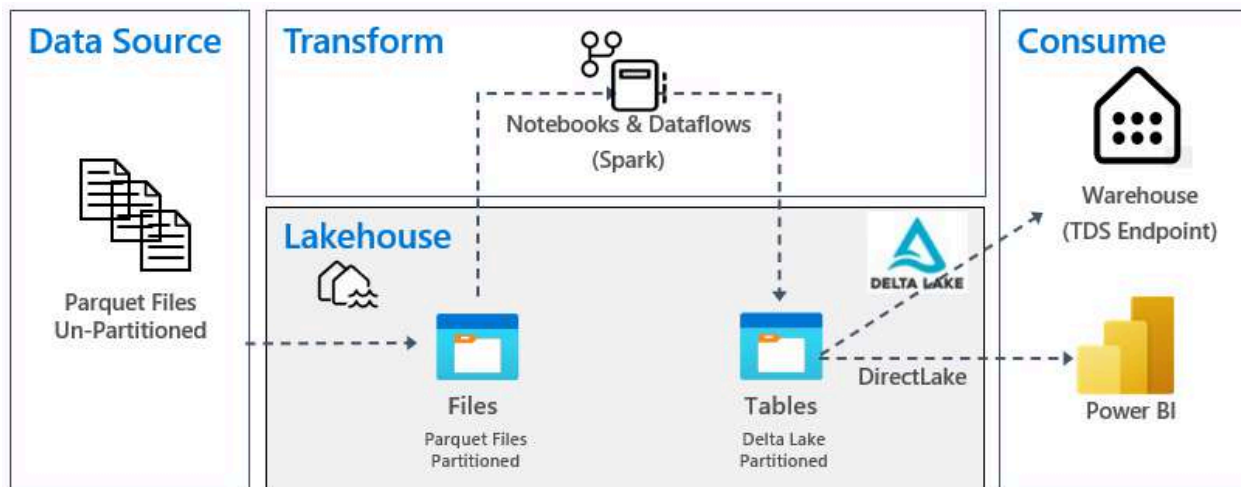
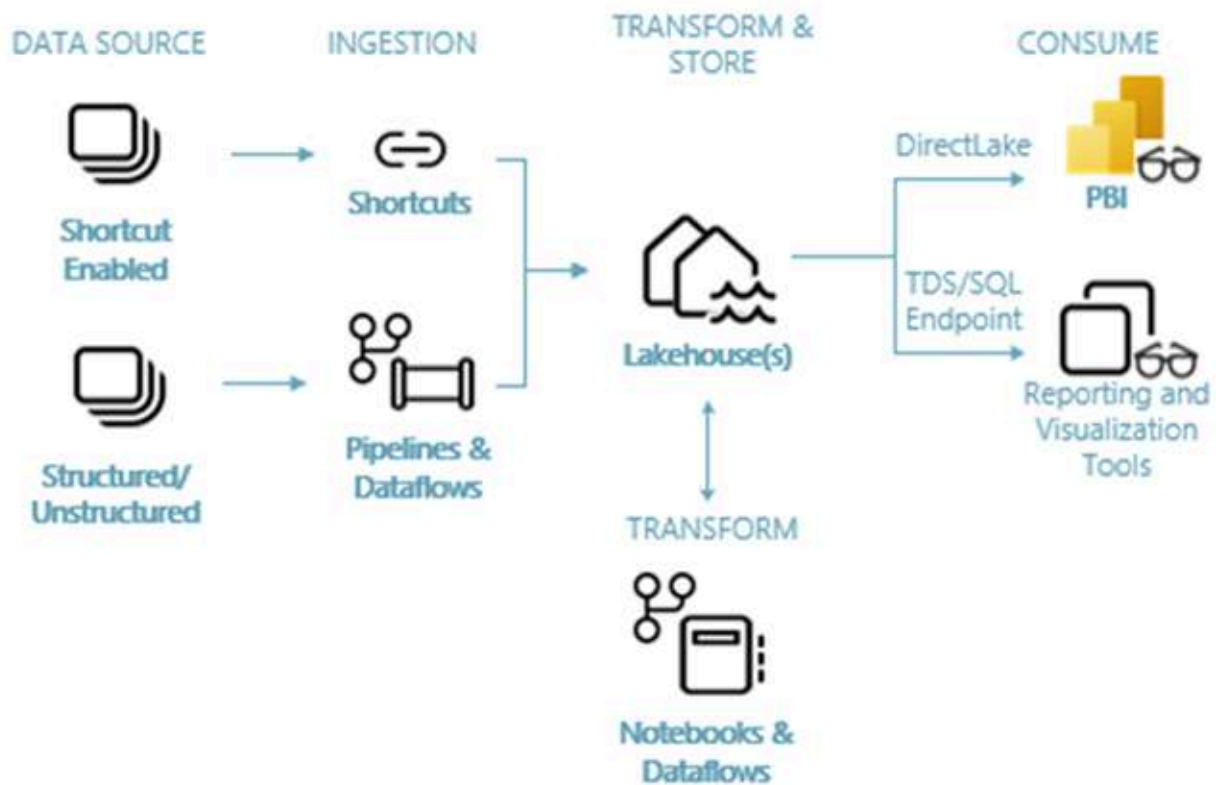
Same water.
Different usage.

Why Fabric gives both

Because teams are different:

Team	Uses
Data engineers	Lakehouse
Data scientists	Lakehouse
Business analysts	Warehouse
Executives	

Lakehouse — what it is, its components, how tables sit on files, and managed vs external tables (clear + technical)



1) What is a Lakehouse

A **lakehouse** is an **architecture** that combines:

- **Data lake storage** (cheap, scalable files)
- **Data warehouse behavior** (tables, SQL, reliability)

In **Microsoft Fabric**, a Lakehouse is an **item** that organizes data stored in **OneLake**.

Plain meaning:

Lakehouse lets you work with **files and tables together** on the **same data**.

2) Why Lakehouse exists (problem it solves)

Before:

- Data Lake → flexible but unreliable
- Data Warehouse → reliable but rigid
- Two systems → duplicated data

Lakehouse:

- One storage
 - One copy of data
 - BI + Spark + ML on same data
-

3) Core components of a Lakehouse

Component 1: Files

- Raw or semi-processed data
- Formats: CSV, JSON, Parquet
- Stored directly in OneLake
- No schema enforcement

Used for:

- Landing data
 - External ingestions
 - Exploration
-

Component 2: Tables

- Structured representation on top of files
- Format: **Delta tables**
- Schema enforced
- ACID compliant

Used for:

- Analytics
 - BI
 - Curated datasets
-

Component 3: Delta Engine

Provided by **Delta Lake**:

- Transaction log
 - Versioning
 - Concurrency control
-

Component 4: Compute engines

- Spark (engineering, ML)
- SQL endpoint (analytics)
- Power BI (reporting)

All read the same data.

4) How tables are created on top of files (important concept)

Step-by-step (technical but simple)

Raw file arrives (CSV / Parquet)

↓

Stored in Lakehouse Files area

↓

Spark reads file

↓

Writes data as Parquet + Delta Log

↓

Table metadata created

Result:

- Files still exist
- Table points to a **set of files**
- Delta Log tracks which files are valid

A table is **metadata + rules**, not separate storage.

5) What really happens under the hood

When you create a table:

- Data is stored as **Parquet files**
- Metadata stored in `_delta_log`
- Each change creates a **new version**

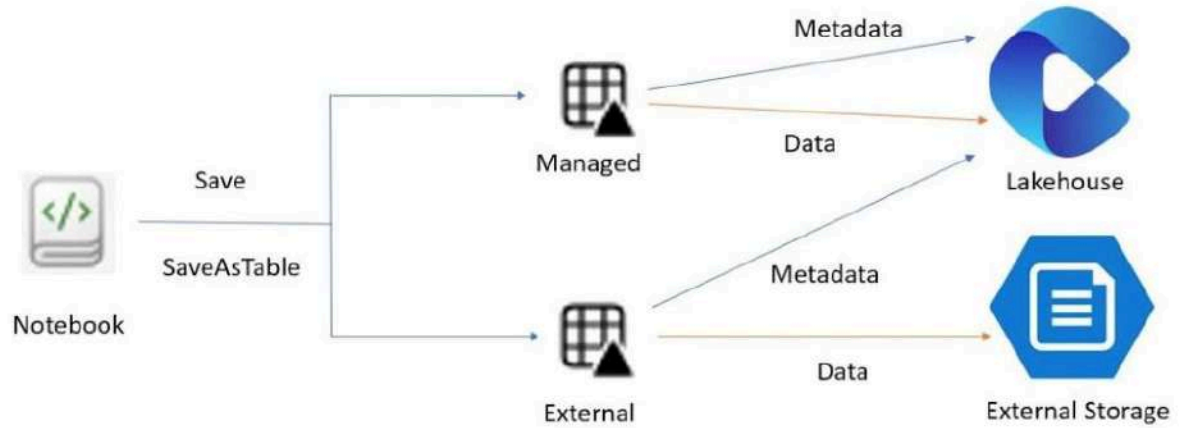
No row updates:

- Old files stay
 - New files are added
 - Delta Log decides which version is visible
-

6) Managed vs External tables (very important)



Mastering Managed and External Tables in Microsoft Fabric Delta Lake





Managed vs. External Tables



Managed Tables

- Full governance with Unity Catalog
- Optimized by default
- Data deleted on DROP TABLE
- Required for predictive optimization, liquid clustering
- Easier to manage in CI/CD, production



External Tables

- You define the storage location
- Data not deleted on DROP TABLE
- Unity Catalog can't enforce access control
- Use for specific migration or sharing scenarios

6.1 Managed Table

What it is

- Table **owned by the Lakehouse**
- Data stored in Lakehouse-managed path
- Fabric controls lifecycle

Characteristics

- Storage location hidden
- Drop table → data deleted
- Strong governance
- Simplest option

Technical behavior

- Fabric manages:
 - File paths
 - Permissions
 - Cleanup
- Uses Delta format internally

Use when

- Curated / final data
 - BI-ready datasets
 - You want safety
-

6.2 External Table

What it is

- Table **points to existing files**
- Data stored outside Lakehouse-managed area
- Metadata only inside Lakehouse

Characteristics

- Drop table → data NOT deleted
- Storage location explicit
- Shared across systems

Technical behavior

- Table metadata references:
 - External path (OneLake / ADLS / shortcut)
- Delta Log still used if Delta format

Use when

- Shared data
- External ingestion pipelines
- Multi-system access

7) Managed vs External — side-by-side

Aspect	Managed Table	External Table
--------	---------------	----------------

Data location	Fabric-managed	User-defined
Lifecycle	Auto-managed	User-managed
Drop behavior	Deletes data	Keeps data
Governance	Strong	Depends on storage
Complexity	Low	Higher
Best for	Gold layer	Bronze / shared

8) Why Lakehouse keeps both Files and Tables

- Files = flexibility
- Tables = trust

Removing either breaks:

- ML workflows
- BI workflows

Lakehouse exists to **avoid choosing one**.

9) DP-700 exam memory locks

- Lakehouse = **files + Delta tables**

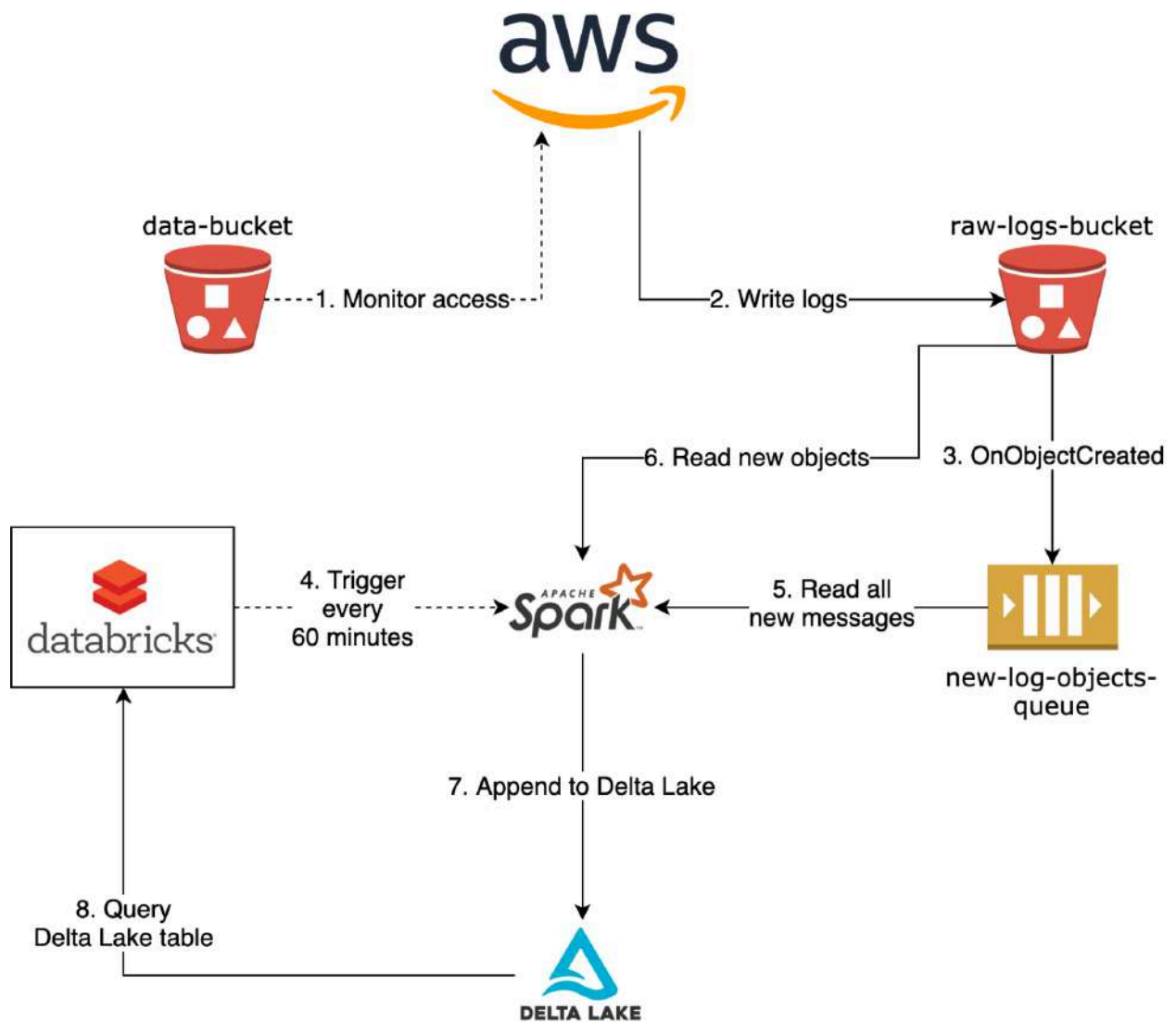
- Tables are **metadata over files**
 - Managed table = Fabric owns data
 - External table = Fabric owns metadata
 - OneLake stores everything once
-

10) One-line memory lock

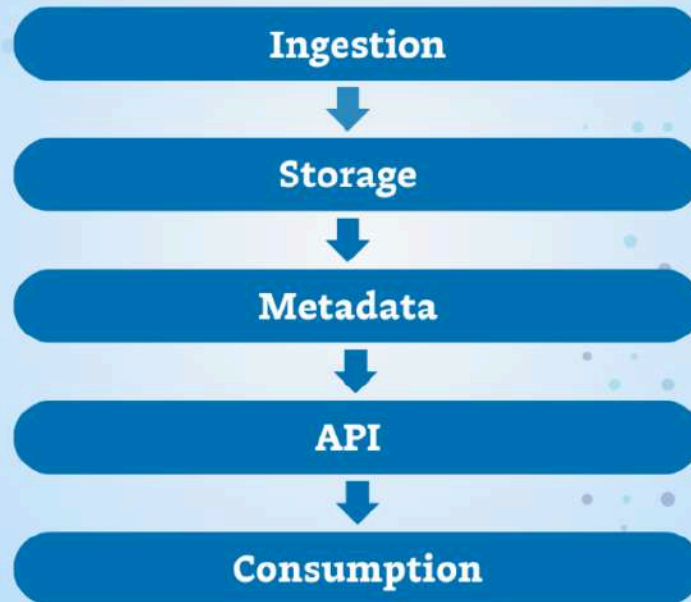
Lakehouse lets raw files and governed tables coexist on the same data without duplication.

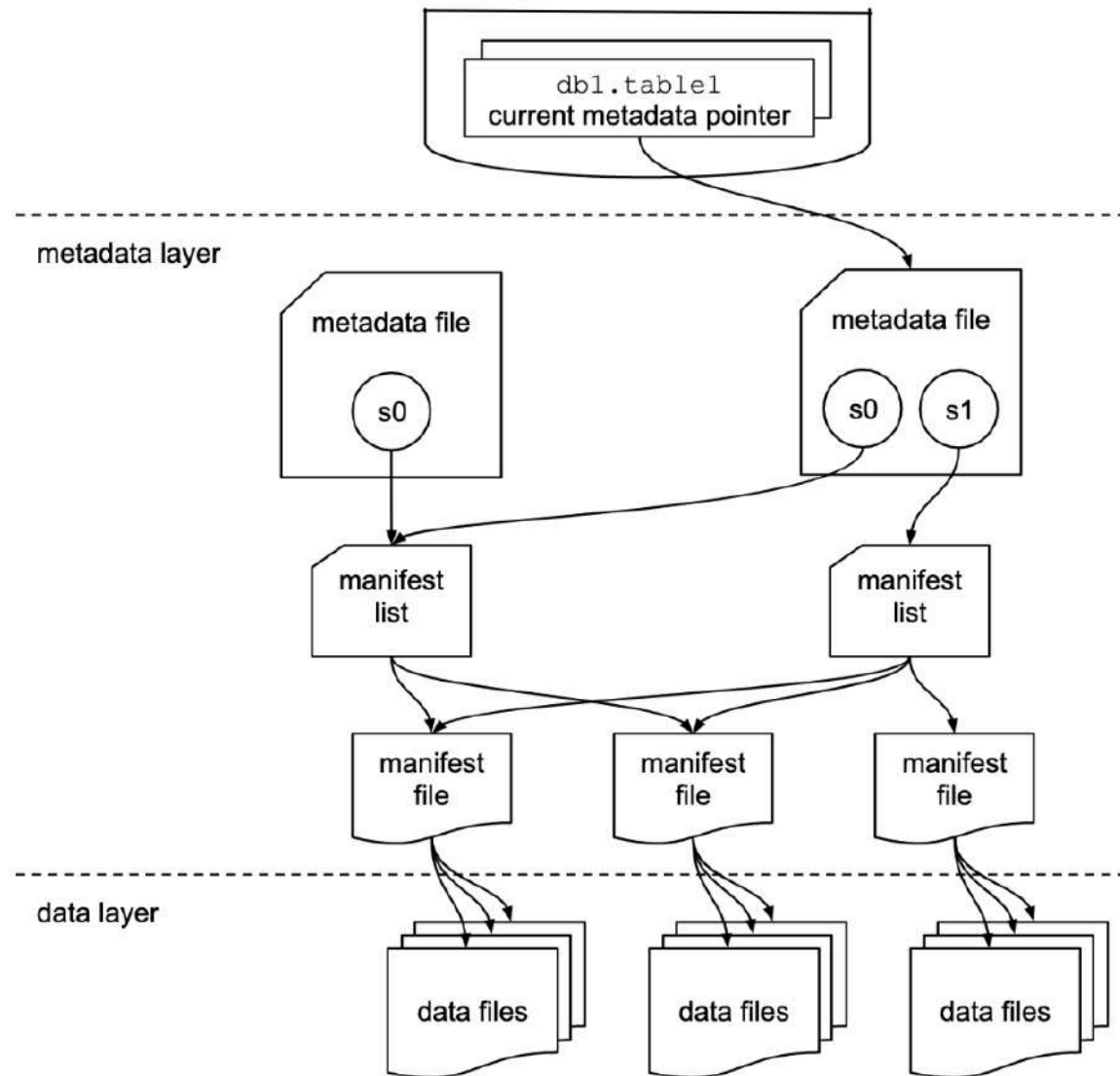
End.

This is the missing piece — Catalog objects (explained slowly, with a concrete example)



Data Lakehouse Architecture





First: your assumption is partly right, but incomplete

"If I load a CSV to create a table, Spark engine will create the table."

✓ **Yes — but Spark creates TWO different things**, not one.

Most confusion comes from mixing them.

The two things Spark creates (this is critical)

When you “create a table”, Spark does **both**:

1. **Data-level objects** → files + Delta Log
2. **Catalog-level objects** → table name, view name

These live in **different places** and have **different lifecycles**.

Let’s build this from zero (real example)

You have a CSV file

```
/Files/raw/emp.csv
```

```
id,name,salary
```

```
1,A,50000
```

```
2,B,60000
```

This is **just a file**.

No table. No metadata. No catalog.

Step 1: Spark reads the CSV

```
df = spark.read.csv("/Files/raw/emp.csv", header=True)
```

What exists now?

- Data in Spark memory
-  No table
-  No metadata
-  No Delta Log

Step 2: Spark writes data as Delta (THIS creates the Delta table)

```
df.write.format("delta").save("/Tables/emp")
```

Now Spark creates:

A) Data-level things (storage layer)

```
/Tables/emp/  
├─ part-0001.parquet  
├─ _delta_log/  
    └─ 00000000000000000000.json
```

👉 At storage level, this IS a Delta table

But Spark still does NOT know the **name** `emp`.

You can only read it by path:

```
spark.read.format("delta").load("/Tables/emp")
```

Step 3: Registering the table (THIS is catalog metadata)

Now you run:

```
CREATE TABLE emp  
  
USING DELTA  
  
LOCATION '/Tables/emp';
```

This does **NOT** touch data.

It only creates a **catalog object**.

What is a Catalog Object (finally explained)

A **catalog object** is just a **name + pointer**.

It stores:

- Table name: `emp`
- Object type: table / view
- Location: `/Tables/emp`
- Owner, permissions
- Schema snapshot (for discovery)

It does **NOT** store data.

Think of it as:

`emp`  `/Tables/emp`

That arrow is the catalog.

Now Spark can do this

```
SELECT * FROM emp;
```

Spark resolves it like this:

```
emp (catalog)
```

↓

```
location '/Tables/emp'
```

↓

```
_delta_log
```

↓

```
parquet files
```

What about a VIEW?

```
CREATE VIEW emp_high_salary AS
```

```
SELECT * FROM emp WHERE salary > 55000;
```

Spark creates **ONLY** a catalog object.

No files.

No Delta Log.

Just stored SQL text.

Now the deletion behavior makes sense

If you DROP VIEW

```
DROP VIEW emp_high_salary;
```

Deleted:

- View name
- SQL logic

Not deleted:

- Table
 - Delta Log
 - Parquet files
-

If you DROP TABLE (managed)

```
DROP TABLE emp;
```

Spark does **three deletes**:

1. Catalog object → table name removed
2. `_delta_log` → deleted
3. Parquet files → deleted

Because **managed table = Spark owns the data**.

If you DROP TABLE (external)

```
DROP TABLE emp;
```

Deleted:

- Catalog object only

Not deleted:

- Parquet
- `_delta_log`

Because Spark was **only pointing**, not owning.

Why Spark separates these two layers

Because:

- Same data can be registered with **multiple names**
- Data can exist without a table
- Tables can be recreated without rewriting data
- Governance and permissions live at catalog level

This separation is what makes Lakehouse powerful.

Final mental model (this removes all confusion)

CSV file

↓

Parquet files ← physical data

↓

Delta Log ← table behavior

↓

Catalog Object ← name, permissions, discovery

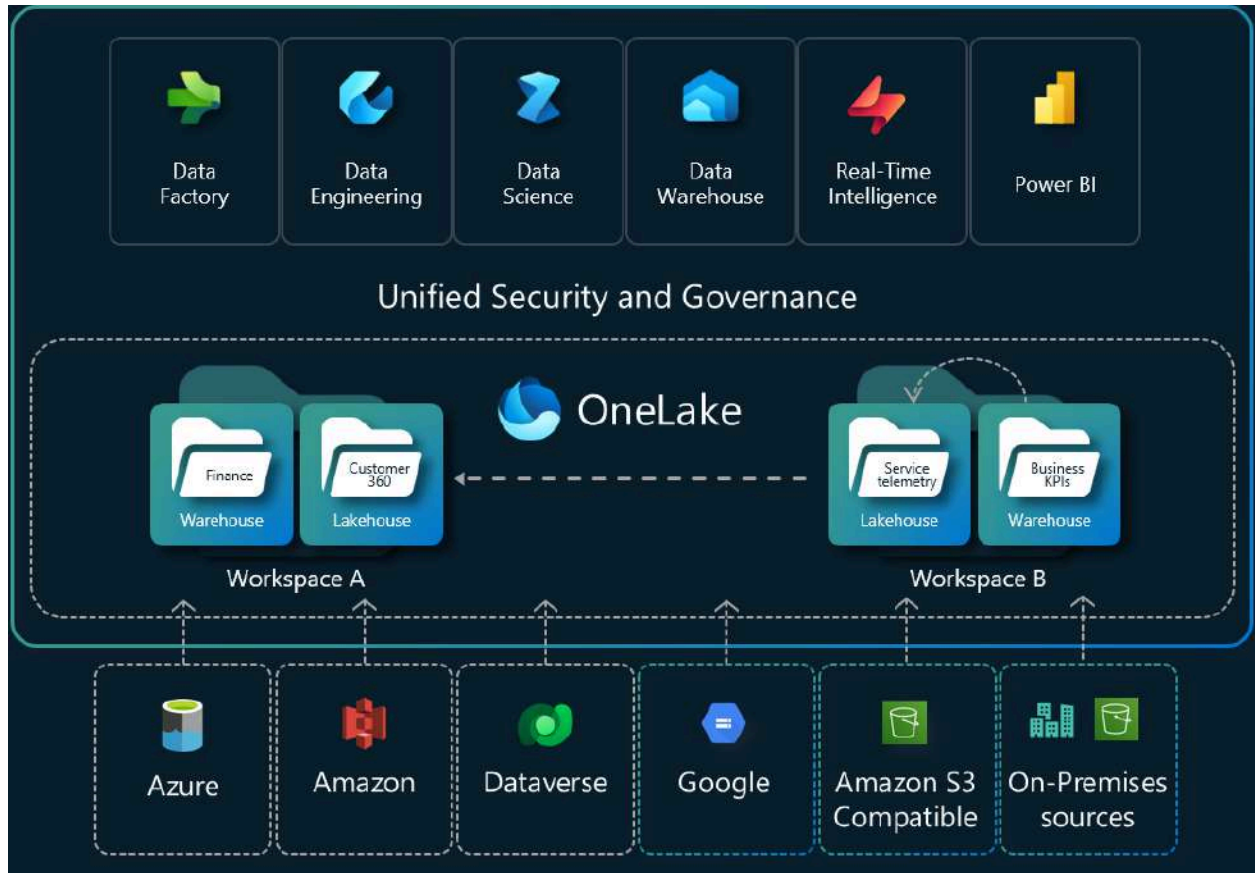
Deleting from the top does not always delete below.

One-line memory lock

Spark does not “create a table”; it writes data first, then optionally creates a catalog name that points to that data.

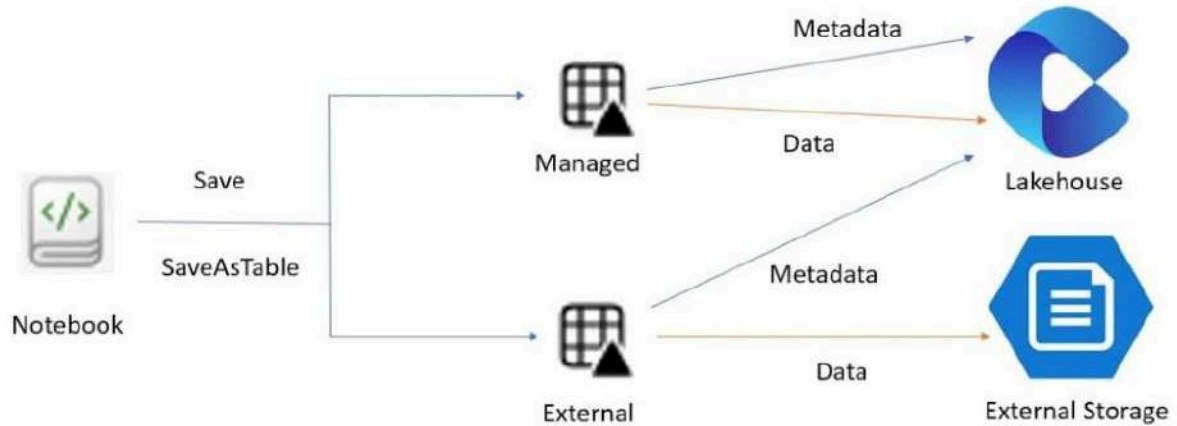
If you understand this, you understand Lakehouse completely.

OneLake table creation — easy, detailed notes

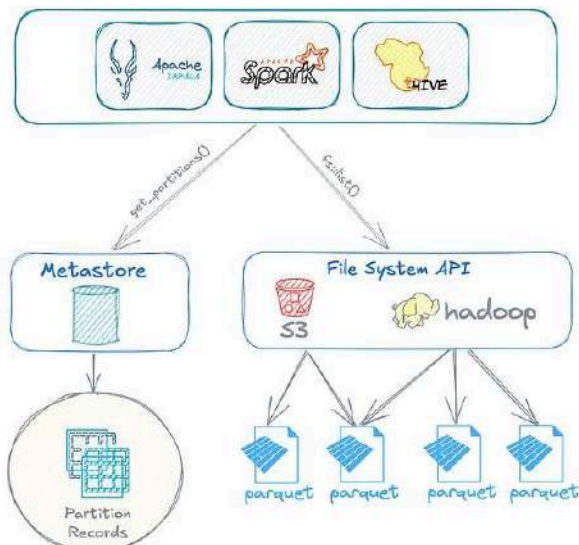




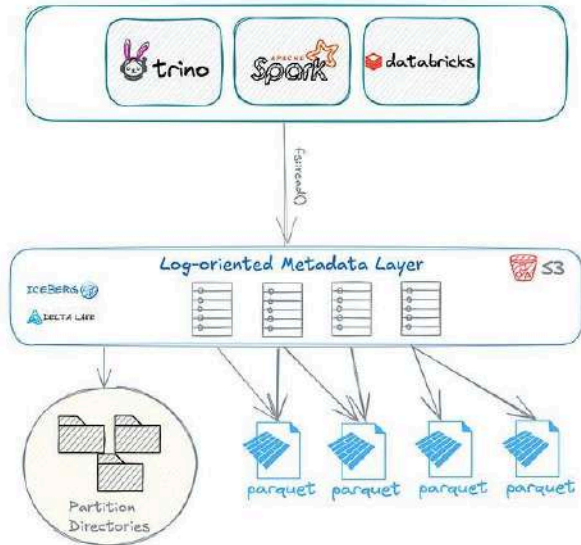
Mastering Managed and External Tables in Microsoft Fabric Delta Lake



Previous Generation Data Lake Table Management



New Generation Data Lake Table Management



4

1. First truth (fix this in mind)

Data is **ALWAYS** stored as files.
Tables **NEVER** store data.

Tables are **names that point to files**.

2. Two layers exist (this causes confusion)

Layer 1: Physical layer (REAL)

- Files in OneLake
- Formats:
 - Delta (Parquet + log)
 - Parquet
 - CSV / JSON
- This is the **only place data lives**

Layer 2: Catalog layer (LOGICAL)

- Table name
 - Schema
 - File path
 - Permissions
 - This is only **metadata**
-

3. What happens when you “create a table”

Scenario A — Create table FROM existing file

Example:

- CSV already exists in OneLake
- You say: *create table using this file*

What actually happens

-  No data copied
-  No new files created
-  Catalog object created
-  Table name points to file path

Result

- File = unchanged
- Table = pointer only

Analogy

- Phone contact saved
- Person already exists
- You did not create the person

Scenario B — Create table by LOADING data

Example:

- CTAS
- COPY INTO
- Spark `write.saveAsTable`

What actually happens

- ☒ New Delta files written
- ☒ Stored under `/Tables/<table_name>`
- ☒ Catalog object created

Result

- Files + table both exist

Analogy

- You built a house
- You also gave it a name

4. Very important comparison (memorize)

Action	Files created	Table created
Upload CSV only	☒ Yes	☒ No
Create table from file	☒ No	☒ Yes

COPY INTO table ☒ Yes ☒ Yes

CTAS ☒ Yes ☒ Yes

Spark saveAsTable ☒ Yes ☒ Yes

5. What happens when you DELETE a table

If table was created from file

- Table deleted ☒
- File remains ☒

If table wrote data

- Table deleted ☒
 - Files deleted ☒
-

6. Simple mental model (lock this)

- Files = data
- Table = name + pointer
- Catalog = index of pointers

Nothing else exists.

7. Final one-line answer

If you create a table **using an existing file**, only a catalog object is created.
Physical data already existed and is **not duplicated**.

8. Short exam-ready note

In OneLake, data is always stored as files.

A table is a catalog object that contains metadata and a pointer to file location.

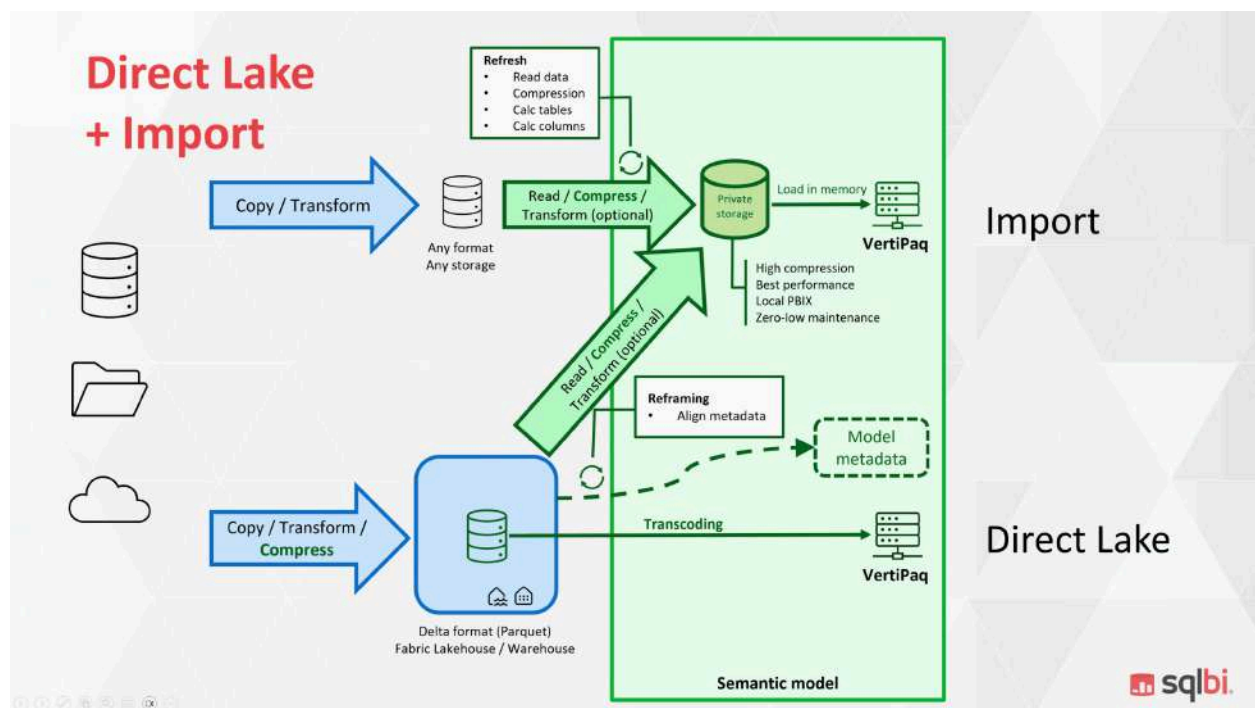
Creating a table from existing files creates only metadata, while loading data into a table creates both metadata and physical Delta files.

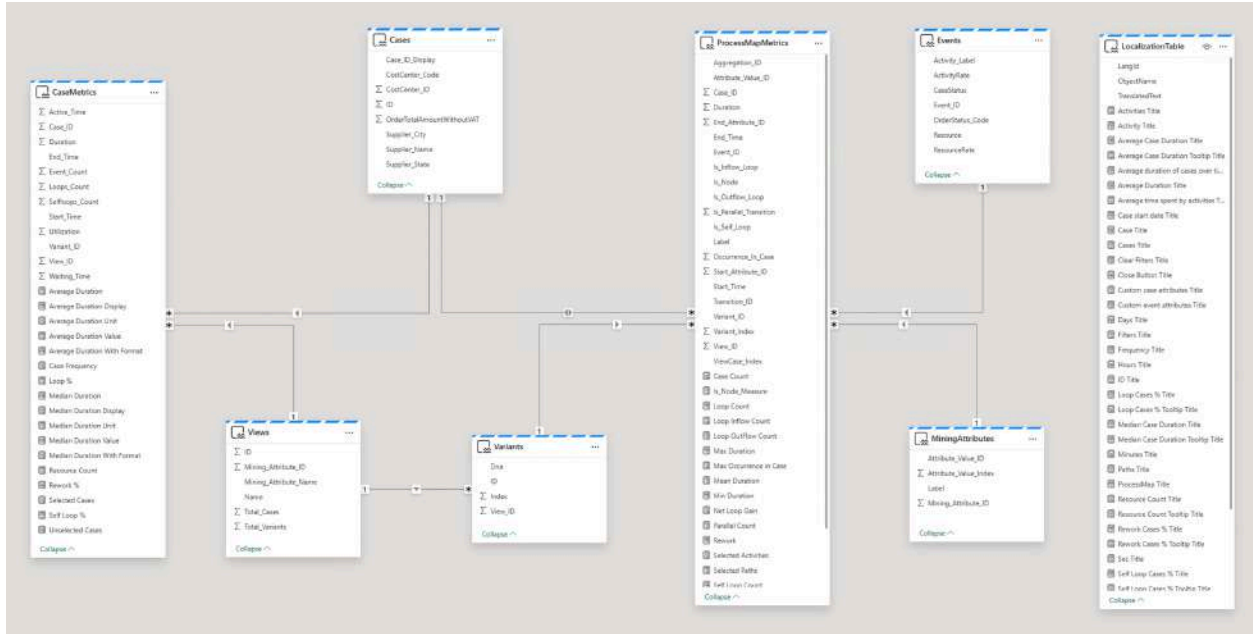
Fffffff

Fghogcvh

Default Semantic Model and SQL Endpoint in a Lakehouse

(clear, layered, exam-ready notes)





1) First lock the big picture

In a **Lakehouse**, when you create tables, **Fabric automatically creates two things for you**:

1. **SQL Endpoint** → for SQL access
2. **Default Semantic Model** → for Power BI analytics

Both sit on top of the same **Delta tables** stored in **OneLake**.

They **do not store data**.

They **expose data differently**.

2) What is the SQL Endpoint in a Lakehouse

Definition

The **SQL Endpoint** is an **automatically created, read-only SQL interface** for your Lakehouse tables.

It lets you query Delta tables using **T-SQL-like syntax**.

Why SQL Endpoint exists

Not everyone uses Spark.

SQL Endpoint exists for:

- SQL developers
- BI tools
- Lightweight analytics
- Fast reads without Spark startup

What SQL Endpoint can access

-  All **Lakehouse tables** (Delta tables)
-  Files area (CSV, JSON directly)
-  Write operations (mostly read-only)

What happens internally

SQL Query



SQL Endpoint



Delta Log



Parquet files (OneLake)

SQL Endpoint:

- Reads Delta Log

- Resolves latest version
- Reads Parquet files
- Returns results

No Spark job is started.

Key characteristics

Aspect	SQL Endpoint
Created automatically	Yes
Data storage	None
Read/Write	Mostly read-only
Uses Delta Log	Yes
Uses Spark	No
Best for	BI, SQL users

3) What is the Default Semantic Model

Definition

The **Default Semantic Model** is an **auto-generated Power BI dataset** created for your Lakehouse.

It exposes Lakehouse tables as:

- Measures
- Relationships
- Fields
- Analytics-ready objects

Why Semantic Model exists

Business users do not query tables directly.

They need:

- Metrics
- Aggregations
- Relationships
- Visuals

The semantic model provides this **analytics layer**.

What it includes by default

- All Lakehouse tables
- Column metadata
- Basic relationships (if detectable)
- Direct Lake connectivity

No manual dataset creation needed.

How it works internally

Power BI Report

↓

Default Semantic Model

↓

SQL Endpoint

↓

Delta Log

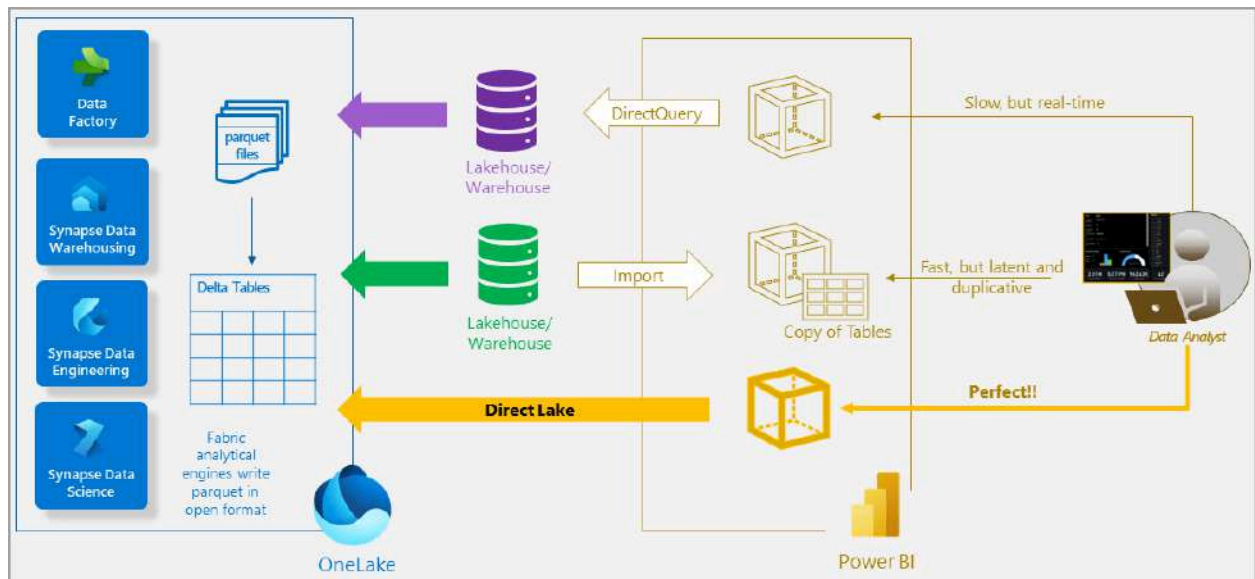
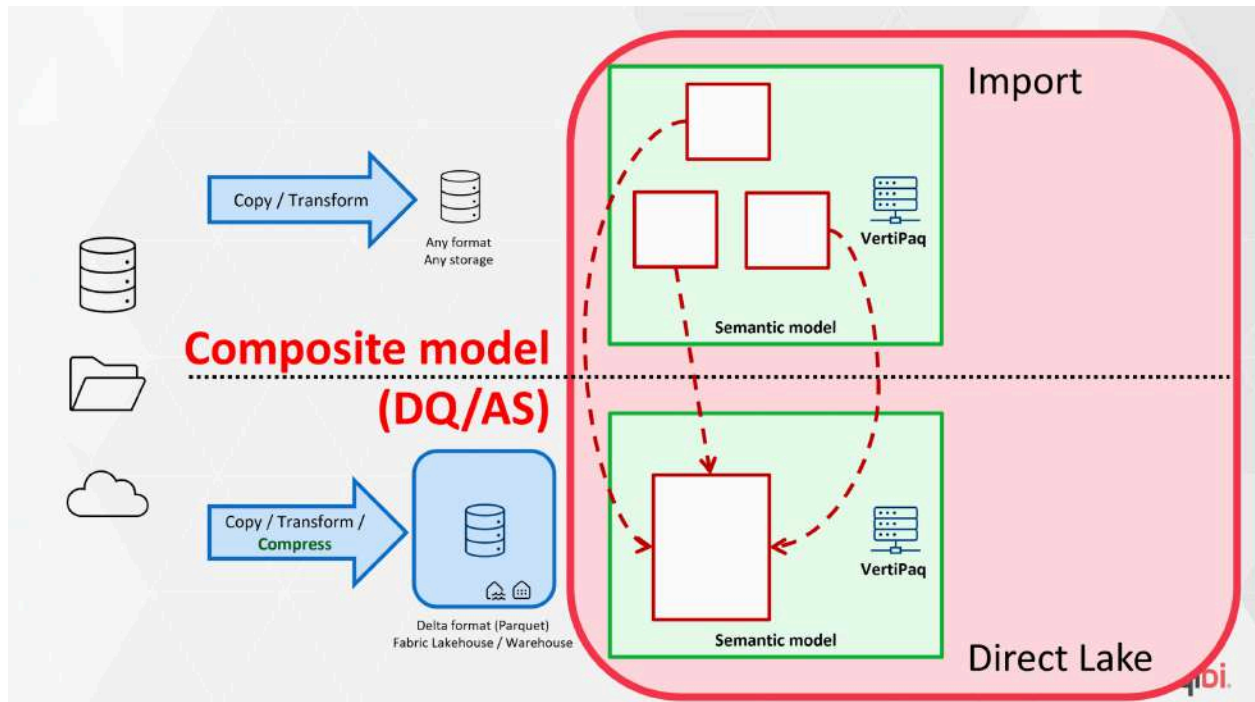
↓
Parquet files

This mode is called **Direct Lake**.

Key characteristics

Aspect	Default Semantic Model
Created automatically	Yes
Data storage	None
Query mode	Direct Lake
Uses SQL Endpoint	Yes
Used by	Power BI
Best for	Dashboards, KPIs

4) Relationship between Lakehouse, SQL Endpoint, Semantic Model



Lakehouse Tables (Delta)



SQL Endpoint



Default Semantic Model



Power BI Reports

Same data.
Different access layers.

5) What is Direct Lake (important term)

Direct Lake means:

- No data import
- No data duplication
- Queries read Delta tables directly from OneLake
- Uses SQL Endpoint + Delta Log

This is why Fabric is fast and cheap.

6) What happens when tables change

If you add a column

- Delta Log updated
- SQL Endpoint sees it immediately
- Semantic Model refreshes metadata

If you drop a table

- Table disappears from SQL Endpoint
- Table disappears from Semantic Model
- No cached data remains

Everything is **metadata-driven**.

7) What these are NOT (clear confusion)

Item	NOT a
SQL Endpoint	Not a data warehouse
Semantic Model	Not a copy of data
Lakehouse	Not only files
Direct Lake	Not import mode

8) DP-700 exam memory locks

- SQL Endpoint is **auto-created**
 - Semantic Model is **auto-created**
 - Both are **read layers**
 - Both rely on **Delta Log**
 - No data duplication
 - Direct Lake is default
-

9) One-line memory locks

- **SQL Endpoint** = *SQL window into Lakehouse tables*
 - **Semantic Model** = *Analytics layer for Power BI*
 - **Direct Lake** = *Query Delta tables without copying data*
-

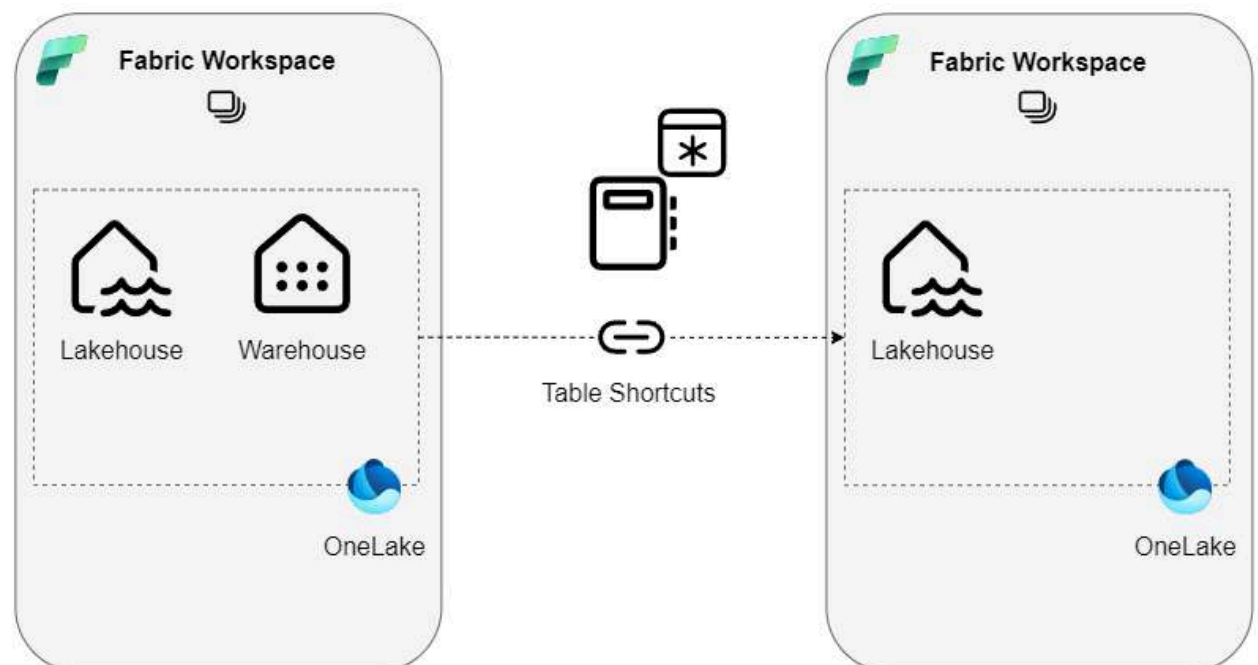
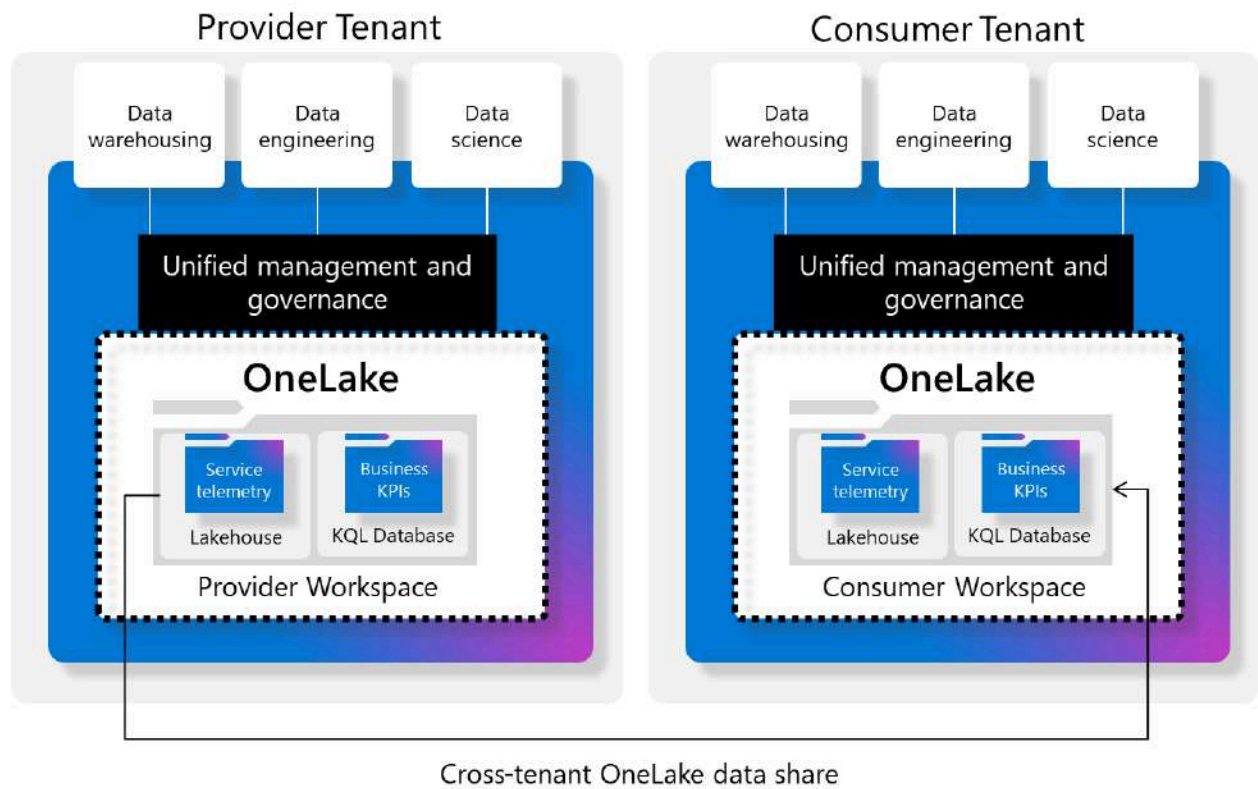
Final mental model (never forget)

Lakehouse stores data once; SQL Endpoint queries it; Semantic Model analyzes it.

End.

Shortcut feature in Microsoft Fabric (Lakehouse) — complete, clean notes





1) What is a Shortcut in Fabric

A **Shortcut** is a **virtual pointer** that lets OneLake **reference data stored somewhere else without copying it**.

Plain meaning:

Shortcut = *“Use external data as if it lives inside my Lakehouse.”*

No ingestion.

No duplication.

No extra storage cost.

2) Why Shortcut was needed (real problem)

Before Shortcuts:

- Data lived in:
 - ADLS
 - Amazon S3
 - Other Lakehouses
- To use it:
 - Copy data
 - Sync pipelines
 - Handle duplicates
 - Pay extra storage cost

This caused:

- Data sprawl
- Latency
- Governance issues

Shortcut solved this by virtualizing data access.

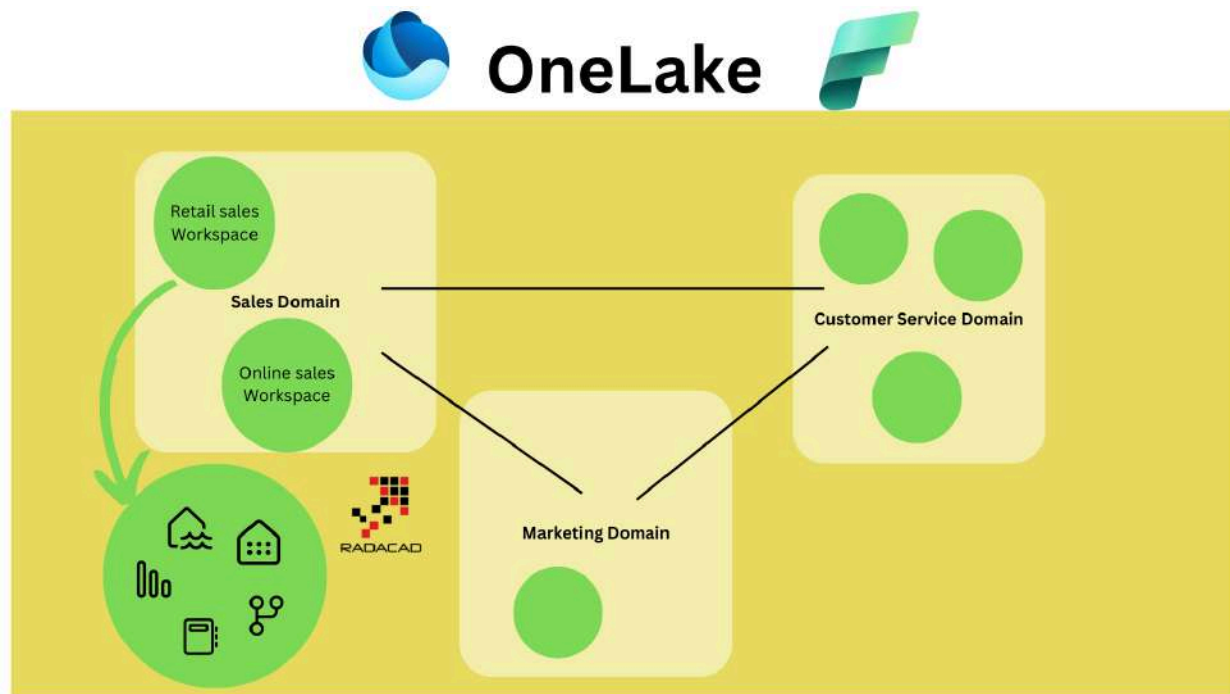
3) Where Shortcuts can point to

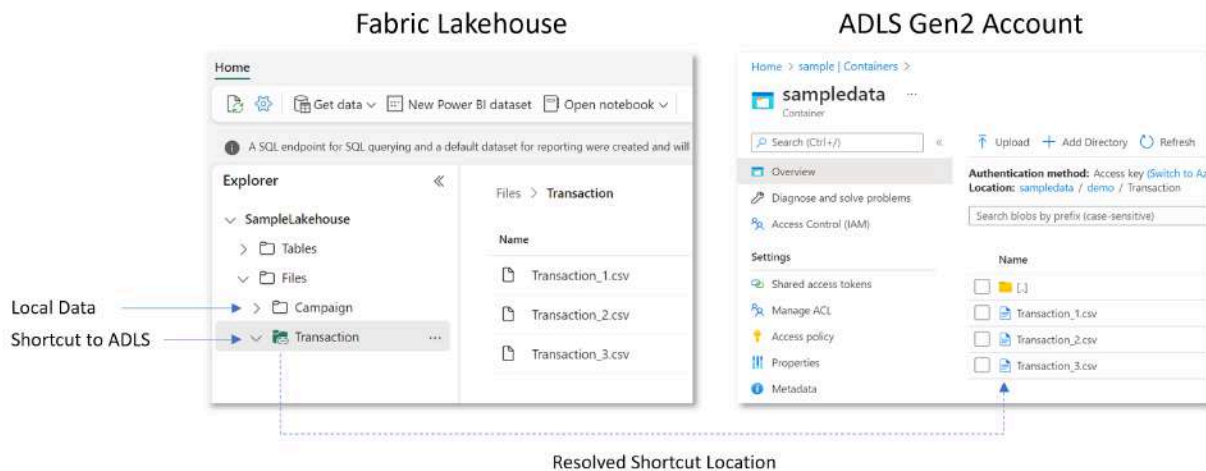
Fabric supports shortcuts to:

- OneLake (another workspace)
- Azure Data Lake Storage Gen2
- Amazon S3
- Other Fabric Lakehouses

All appear as **local folders** inside your Lakehouse.

4) How Shortcut works internally (important)





Lakehouse Files / Tables



Shortcut



External Storage (ADLS / S3 / OneLake)

Key rule:

- **Data never moves**
- Shortcut stores only:
 - Source location
 - Credentials
 - Access metadata

5) Shortcut vs Copy (critical difference)

Aspect	Shortcut	Copy
Data movement	No	Yes
Storage cost	Zero	High
Freshness	Always latest	Needs refresh
Performance	Depends on source	Faster
Ownership	External	Local
Governance	Shared	Duplicated

Shortcut = **reference**

Copy = **replication**

6) Shortcut types in Lakehouse

A) File Shortcut

- Points to folders/files
 - Used for:
 - Raw data
 - Semi-structured files
 - Appears under **Files**
-

B) Table Shortcut

- Points to Delta tables
- Used for:
 - Curated datasets
 - Cross-team sharing
- Appears under **Tables**

Table shortcut preserves:

- Schema
- Delta Log
- ACID behavior

7) How Spark reads data via Shortcut

Spark does **not know** it is external.

Flow:

Spark query

→ Lakehouse path

→ Shortcut resolution

→ External data

Delta Log (if present) is read **at source**.

This keeps:

- Versioning
- Schema enforcement

- Time travel
-

8) Security model of Shortcut

- Access controlled by:
 - Workspace permissions
 - Source storage credentials
- Users never see:
 - Keys
 - Secrets
 - Endpoints

Fabric enforces **least privilege**.

9) What Shortcut does NOT do

- Does NOT copy data
- Does NOT change data format
- Does NOT bypass security
- Does NOT convert CSV to Delta

Shortcut is **read access**, not ingestion.

10) When to use Shortcut (best practice)

Use Shortcut when:

- Data is owned by another team
- Data is large
- Data changes frequently
- You want single source of truth

Do NOT use Shortcut when:

- You need heavy transformations
- You need write access
- You want full ownership

11) Shortcut + Lakehouse (why this is powerful)

Shortcut enables:

- Cross-workspace lakehouse architecture
- Hub-and-spoke data sharing
- Enterprise-wide OneLake design

OneLake becomes:

One logical lake, many physical locations

12) DP-700 exam memory locks

- Shortcut = **virtual reference**
- No data duplication
- Supports ADLS, S3, OneLake
- Appears like local data

- Preserves Delta behavior
 - Governance stays centralized
-

13) One-line memory lock

Shortcut lets OneLake use external data as local data without copying it.

End.

Internal Shortcut

Definition

An **internal shortcut** points to data **already inside OneLake**.

Source

- Another Lakehouse
- Another workspace (Fabric)

Key points

- Data stays inside Fabric
- No data movement
- Fast access
- Same governance model

Use when

- Sharing data between teams
 - Hub-and-spoke Lakehouse design
 - Single source of truth inside Fabric
-

External Shortcut

Definition

An **external shortcut** points to data **outside OneLake**.

Source

- Azure Data Lake Storage Gen2
- Amazon S3

Key points

- Data remains outside Fabric
- No ingestion
- Slightly higher latency
- Requires credentials

Use when

- Data owned by external systems
- Large existing lakes
- Migration without copying

Quick comparison

Feature	Internal Shortcut	External Shortcut
Source location	OneLake	ADLS / S3
Data movement	No	No
Performance	Faster	Depends on source
Security	Fabric-native	Credential-based
Best use	Cross-workspace sharing	External data access

One-line memory lock

Internal shortcut = OneLake to OneLake.
External shortcut = OneLake to outside storage.

End.

You can create a *table shortcut* ONLY if the files are already in Delta format.

That means:

- Parquet files alone are NOT enough
- The folder **MUST** contain `_delta_log`

The hard requirement (this decides everything)

For Fabric to treat files as a **table**, the folder must look like this:

`/data/sales/`

├─ `part-0001.parquet`

├─ `part-0002.parquet`

└─ `_delta_log/`

 ├─ `00000000000000000000.json`

 ├─ `00000000000000000001.json`

 └─ `checkpoint.parquet`

If `_delta_log` exists →  **Table shortcut allowed**

If `_delta_log` does NOT exist →  **Only file shortcut possible**

Why Parquet-only files are not enough

Parquet files contain:

- Rows
- Columns

But they do NOT contain:

- Schema evolution rules
- Version history
- Transaction consistency
- File validity list

Fabric needs **Delta Log** to answer:

- Which files are part of the table?
- Which version is current?
- Which schema is correct?

Without `_delta_log`, Fabric **cannot guarantee correctness**, so it refuses to create a table shortcut.

What happens internally when you create a table shortcut

When you point a shortcut to a Delta folder:

1. Fabric checks for `_delta_log`
2. Reads the latest Delta version
3. Registers a **catalog object**
4. Shows it under **Tables**
5. Uses the **existing Delta Log** for reads

Important:

-  No new Parquet created
-  No new Delta Log created
-  No data movement

It is **zero-copy registration only**.

What if files are CSV or Parquet only?

Folder:

`/data/raw/`

`├─ emp.csv`

`└─ emp2.csv`

Result:

-  Table shortcut NOT possible
-  File shortcut only

To make it a table, you must run Spark:

```
df = spark.read.csv("/Files/raw/emp.csv", header=True)

df.write.format("delta").save("/Tables/emp")
```

Now `_delta_log` exists → table shortcut possible.

Mental model (lock this permanently)

Delta table = Parquet + `_delta_log`

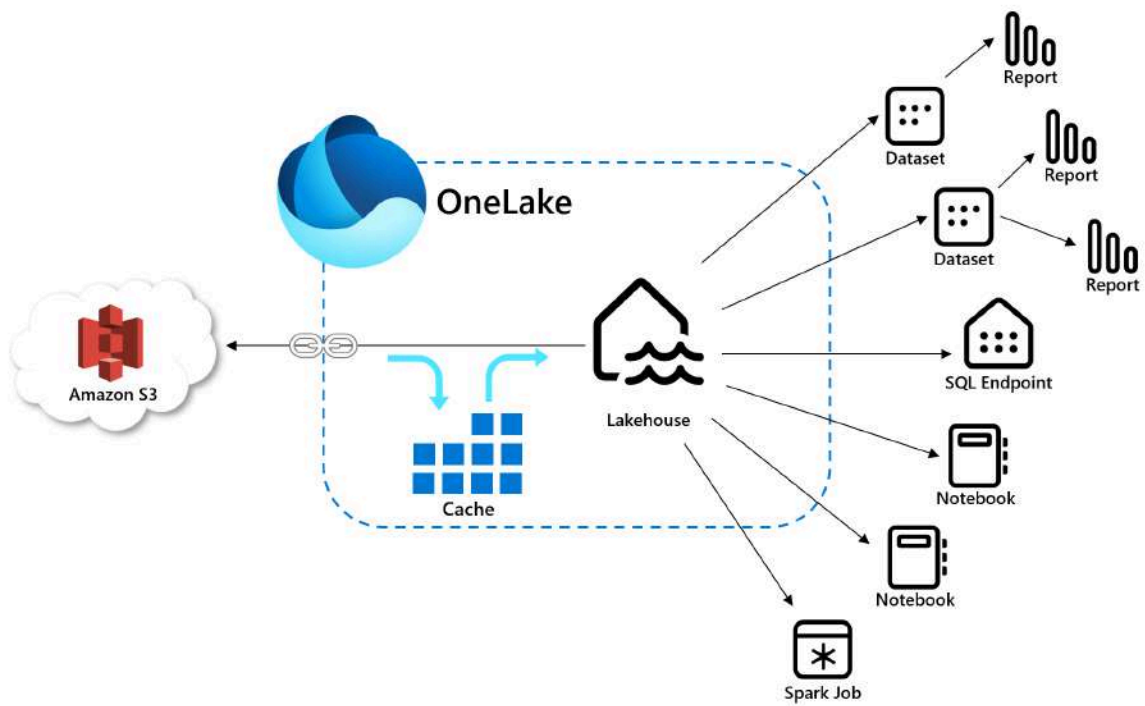
Table shortcut = Pointer to Delta table

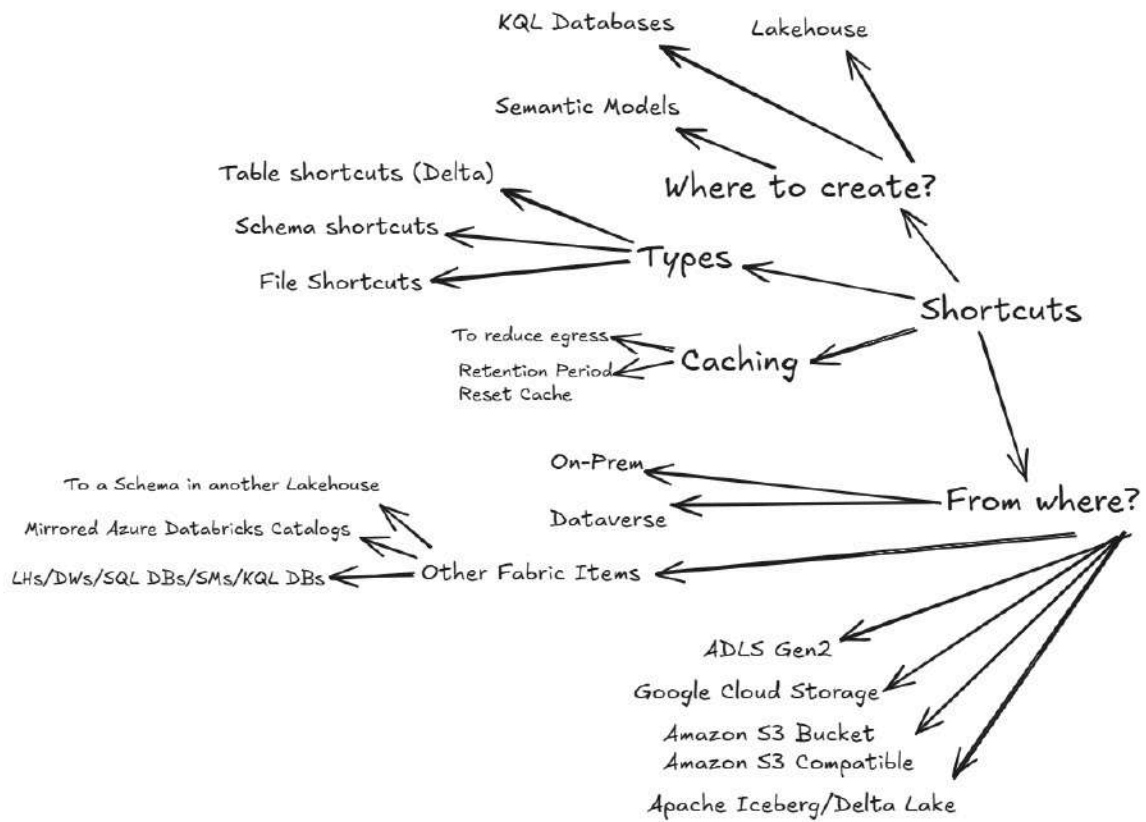
No Delta Log → no table shortcut.
No exception.

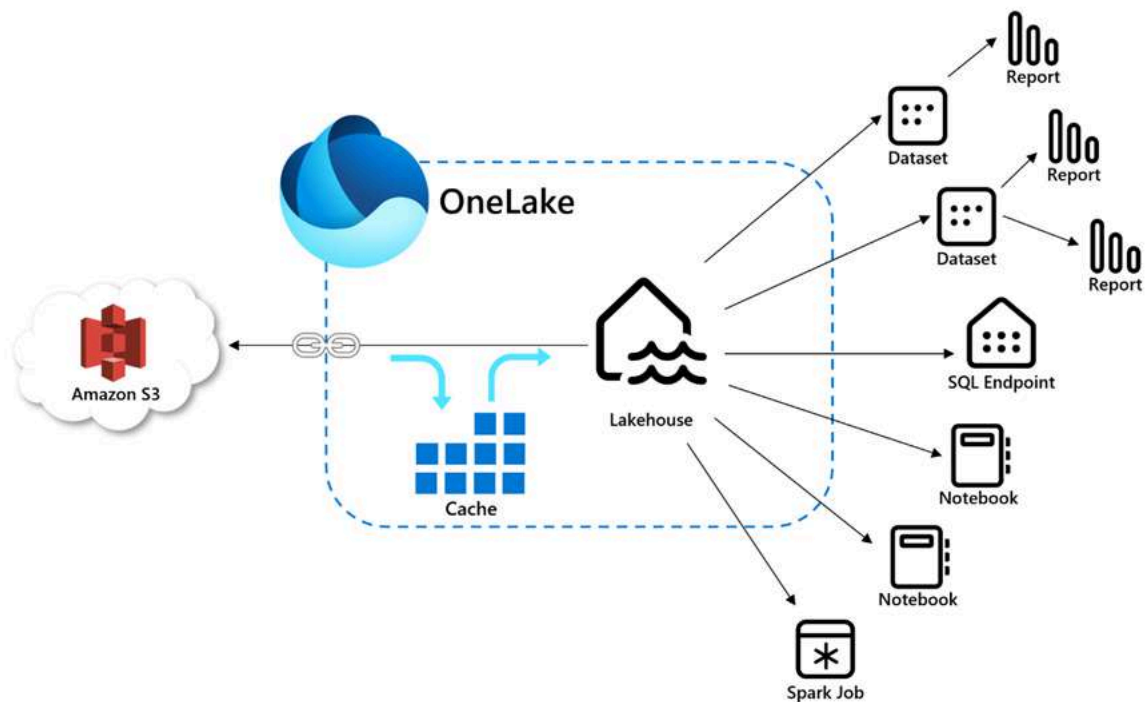
One-line memory lock

Fabric creates a table shortcut only when `_delta_log` exists; Parquet alone is never treated as a table.

What is caching in shortcuts (Fabric / OneLake) — explained cleanly







Short answer (lock this)

Caching in a shortcut means Fabric temporarily keeps a local copy of frequently accessed data to make reads faster — without changing the source data.(1-28 days and less than 1gb file)

Why caching is needed at all

Shortcuts often point to:

- ADLS
- Amazon S3
- Another OneLake location

These sources can be:

- Far away (network latency)
- Slower

- Shared with other systems

Reading them every time directly is expensive.

What Fabric does with caching

When you query data through a shortcut:

Query

↓

Check local cache

↓

If present → read from cache (fast)

↓

If not present → fetch from source → cache it → read

Caching is **automatic**.

What exactly is cached

- **Data blocks**, not entire datasets
- Mostly **Parquet data**
- Optimized for:
 - SQL queries
 - Spark scans
 - BI reads

Delta Log itself is **not duplicated** unless needed for query planning.

What caching does NOT do

- ❌ Does not create a Delta table
- ❌ Does not change source data
- ❌ Does not break freshness rules
- ❌ Does not bypass permissions

Cache is a **read optimization only**.

Cache + Delta table shortcuts (important)

When shortcut points to a **Delta table**:

- Fabric reads `_delta_log` from source
- Determines correct version
- Caches data files used by that version

If a new Delta version appears:

- Cache invalidates
- New files fetched

Consistency is preserved.

Cache + File shortcuts (CSV / Parquet)

- Cache accelerates reads
- No versioning

- If source file changes:
 - Cache refreshes automatically
 - Old cached blocks dropped

Still no history.

Where cache lives

- Inside Fabric-managed storage
- Transparent to user
- Not visible in OneLake File Explorer
- Managed by Fabric runtime

You cannot directly control files in cache.

Cache vs copy (very important)

Cache	Copy
Temporary	Permanent
Auto-managed	User-managed
Invisible	Visible
Read-optimized	Data movement

Can be invalidated Must be manually synced

Shortcut uses **cache**, not copy.

Why caching matters (practical)

Without cache:

- Every query hits external storage
- High latency
- High cost

With cache:

- Near-native performance
- External data feels local

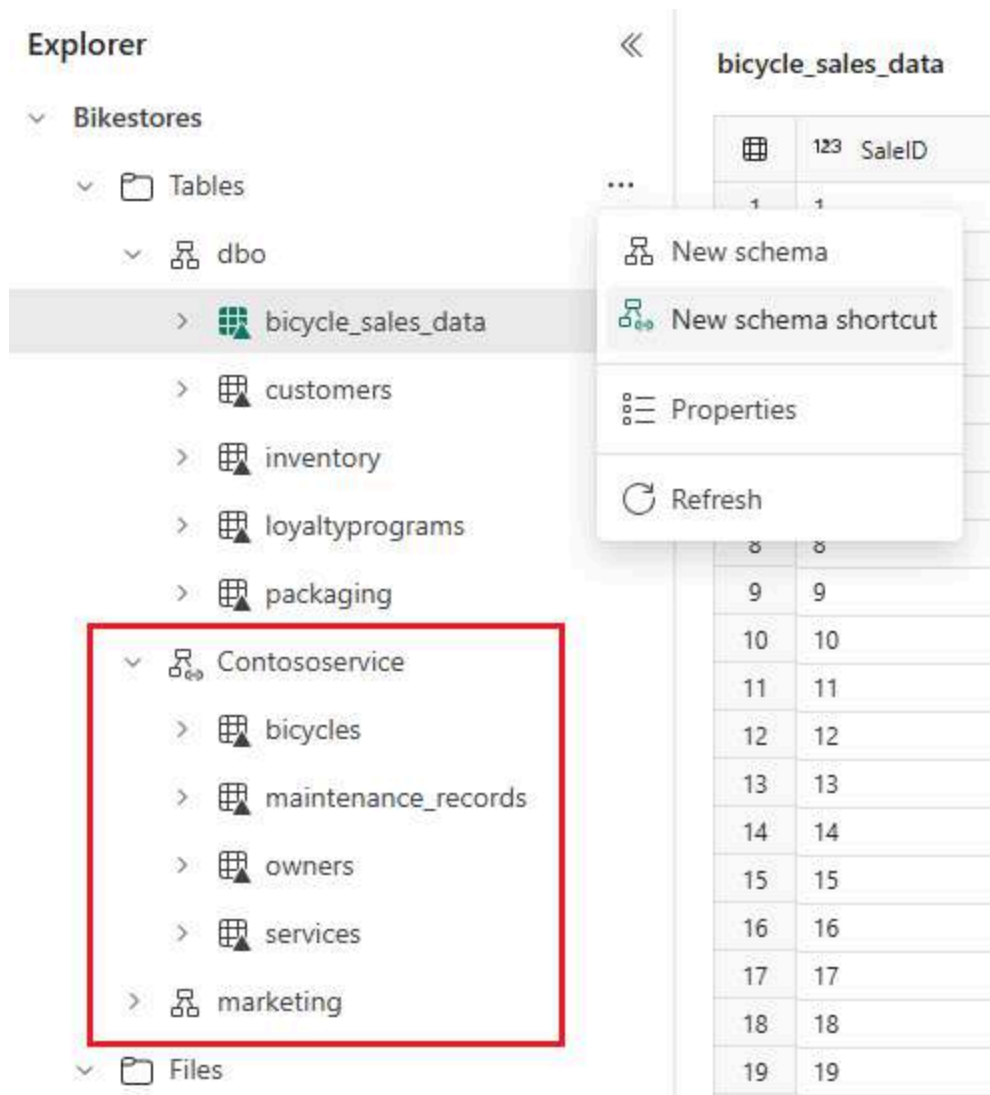
This is what makes shortcuts usable at scale.

One-line memory lock

Caching in shortcuts speeds up access by temporarily storing frequently read data locally without duplicating or owning the data.

Lakehouse with Schema feature — what it is and why it matters





What “Lakehouse with schema” means

A Lakehouse with schema means:

Data is stored like a data lake (files) but controlled by schema rules like a warehouse.

This capability is provided by Delta Lake and implemented in Microsoft Fabric Lakehouse.

Without schema vs with schema (core idea)

Data Lake (no schema)

- Any file structure allowed
- Columns can change silently
- Queries may break or give wrong results

Lakehouse with schema

- Schema is explicit
- Changes are tracked and controlled
- Queries remain stable

This is the key upgrade.

What schema features exist in a Lakehouse

1) Schema Enforcement

- Incoming data must match table schema
- Invalid writes are rejected

Example:

salary INT ❌ "50000" STRING

This prevents silent corruption.

2) Schema Evolution

- Schema can change intentionally
- New columns allowed

- Controlled data type changes

Example:

```
ALTER TABLE emp ADD COLUMN city STRING;
```

Old data remains valid.

3) Schema Versioning

- Every schema change creates a new version
- Old versions preserved
- Time travel supported

You can query:

```
SELECT * FROM emp VERSION AS OF 3;
```

4) Consistent Reads

- All users see the same schema
- No mixed column sets
- No partial data

Delta Log guarantees this.

Why schema feature is critical (problems it solves)

Problem (Data Lake)	Schema Feature Fix
Files change daily	Schema evolutio n
Wrong data types	Enforce ment
Broken reports	Consiste nt schema
No rollback	Versioni ng
Data swamp	Structur e + rules

Pros of Lakehouse with schema (exam + real world)

1) Flexibility without chaos

- **Accepts change**
- **But only when approved**

2) Warehouse-like trust

- Strong typing
 - Predictable BI results
-

3) No data duplication

- Same data for ML + BI
 - No separate warehouse load
-

4) Safe refactoring

- Add columns without reload
 - Roll back mistakes
-

5) Future-proof design

- Handles evolving data sources
 - Supports long-running analytics systems
-

Simple analogy (lock this)

- Data Lake → notebook with blank pages
- Warehouse → printed book
- Lakehouse with schema → digital document with track changes

Flexible, but controlled.

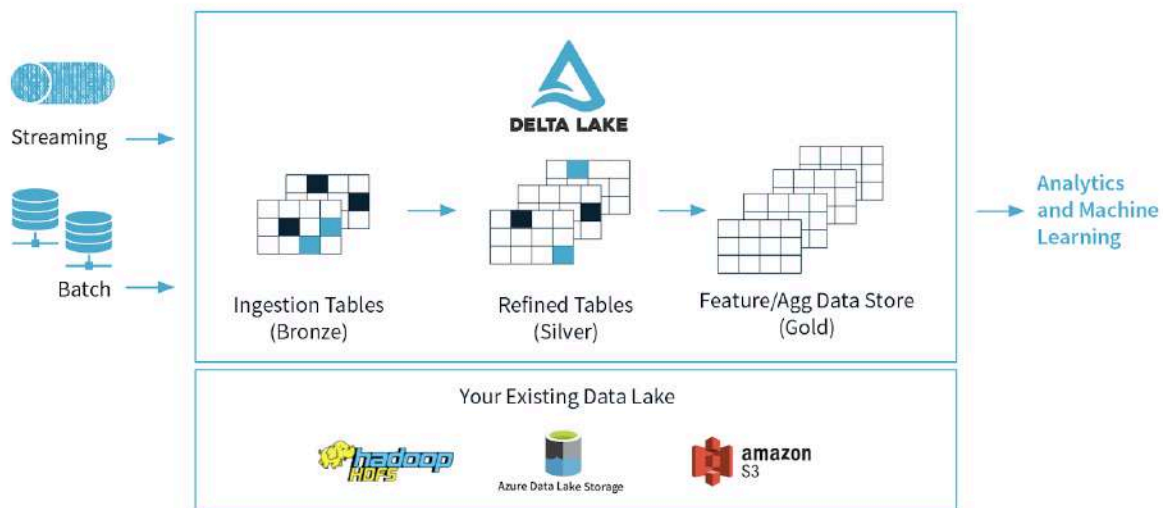
One-line memory lock

Lakehouse with schema gives data lake flexibility with warehouse-level safety and consistency.

Yes — if schema is enforced, a column is NOT removed automatically just because it disappears from the incoming file.

But you can remove it explicitly using a schema change command.

So it is restricted by default, not impossible.



Drop Columns from Schema

Operation	Variation	Merge Schema Compatible	Major Version Change	Minor Version Change	Patch Version Change
Drop Column	In flat Schema	Yes	No	Yes	Yes (if bug fix)
Drop Column	Inside a structure	Yes	No	Yes	Yes (if bug fix)
Drop Column	Inside a struct contained in array	Yes	No/Yes if catalog table	Yes	Yes (if bug fix)
Drop Column	Inside nested array	No	Yes	-	-

Created with Datawrapper

Why this restriction exists (important)

In a Lakehouse with schema:

- Schema represents a contract

- Incoming files are treated as data, not authority
- Files are allowed to be messy
- Tables must remain stable

If files were allowed to remove columns automatically:

- Old data would become invalid
- BI reports would break
- History would be corrupted

So Lakehouse protects the table, not the file.

What actually happens in your scenario

Initial table schema

`id, name, salary, address, city (10 columns total)`

New file arrives with only 9 columns (salary missing)

What Lakehouse does

- It does NOT drop `salary`
- It keeps the table schema unchanged

For new rows:

`salary = NULL`

-

Why

- Old data still has `salary`
- Schema consistency must be preserved

This is schema enforcement in action.

How to REALLY remove a column (explicit action)

If you *intentionally* want to remove a column, you must say so.

Example:

```
ALTER TABLE emp DROP COLUMN salary;
```

Now:

- Delta Log creates a new schema version
- Column is removed logically
- Old data is handled safely
- Change is auditable and reversible (via time travel)

This is schema evolution, not auto-detection.

Key distinction (this removes confusion)

Action	Allowed automatically	Requires explicit command
Add column	Yes (with evolution enabled)	No
Missing column in file	Yes (NULL filled)	No
Remove column from table	✗	✓
Change data type	✗	✓

Why Lakehouse behaves this way (design logic)

- Files are inputs
- Tables are systems of record
- Only humans (or controlled jobs) can change contracts

This prevents silent data loss.

Mental model (lock this)

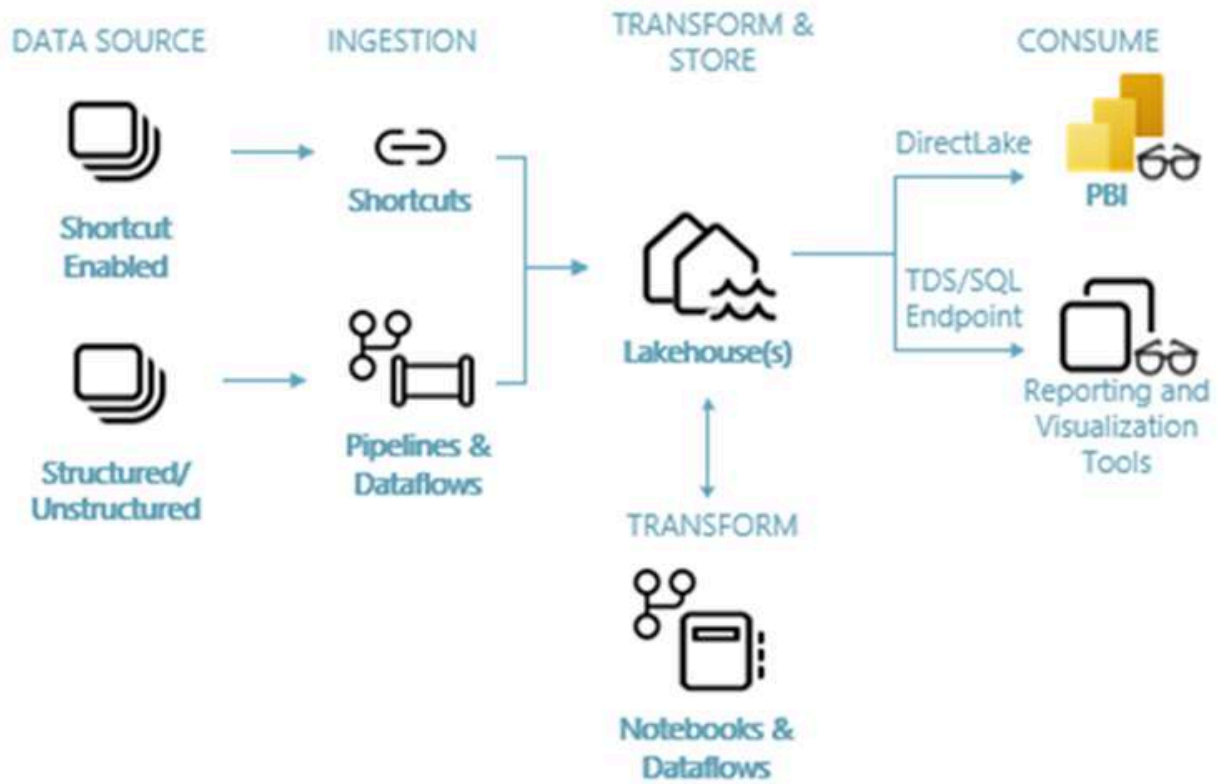
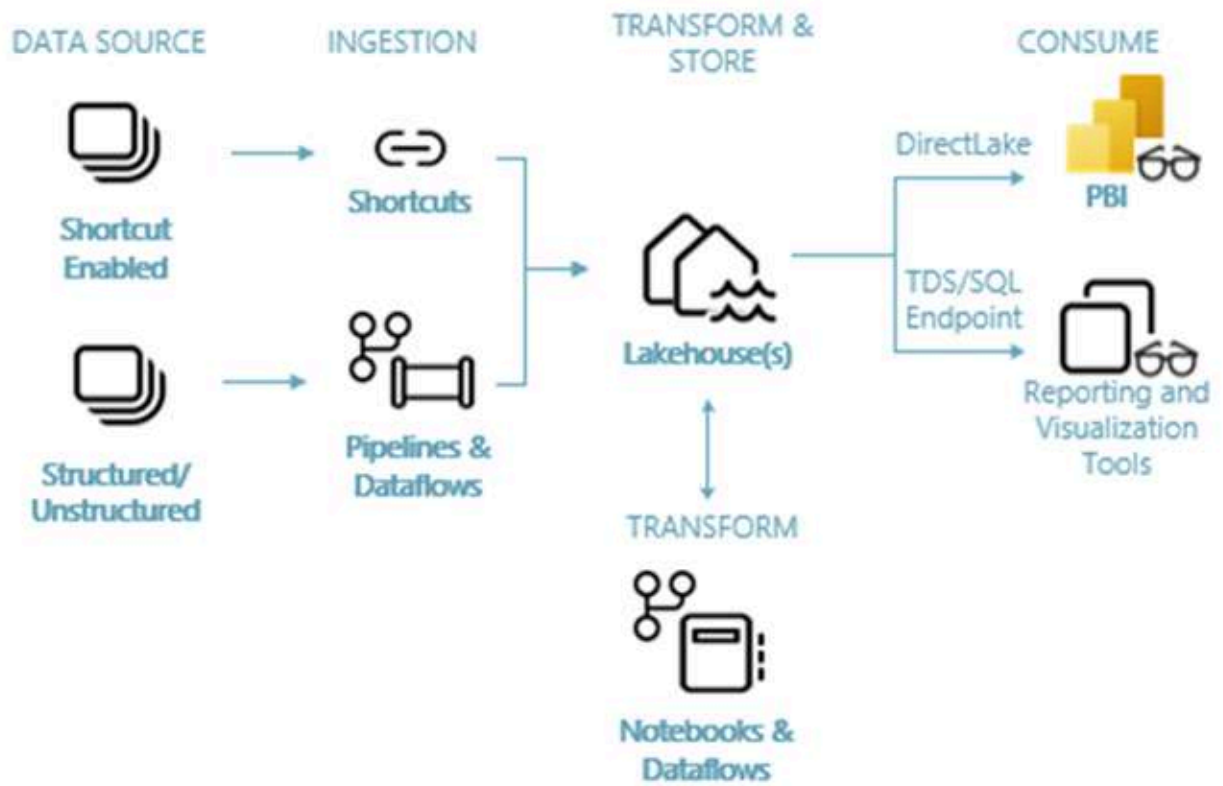
- Files can change freely
 - Tables change only on command
 - Schema = safety rail
-

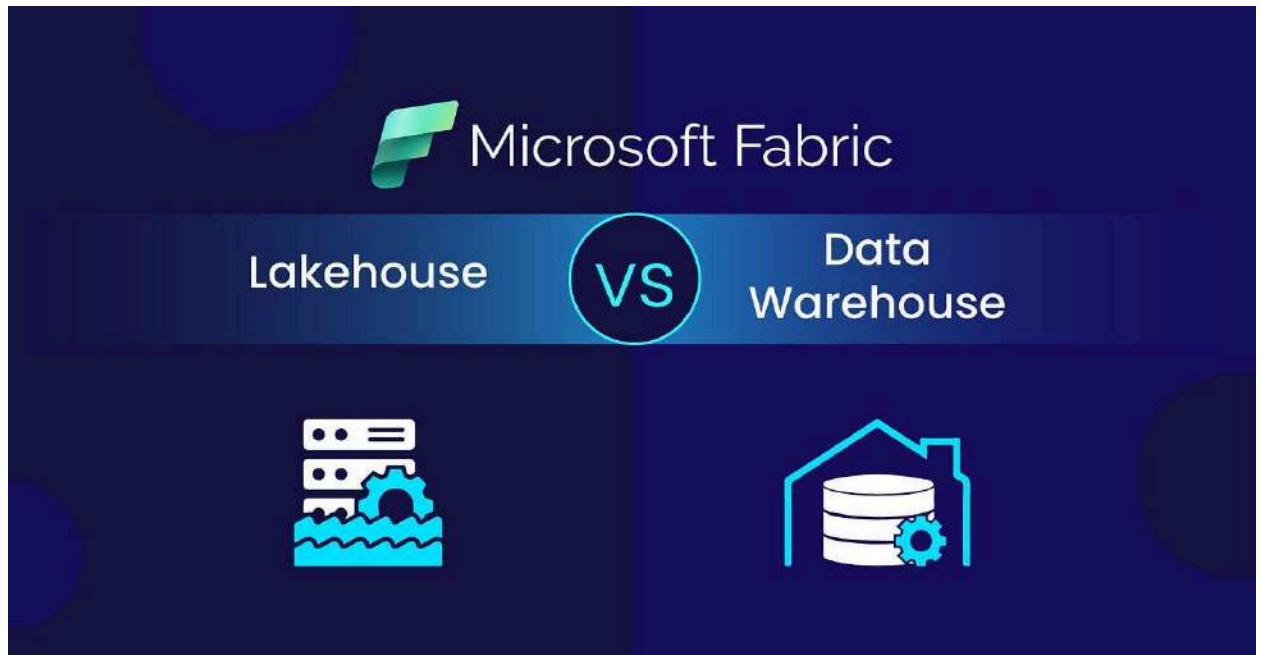
One-line memory lock

Schema enforcement stops accidental column removal; schema evolution allows intentional column removal.

That is the exact rule.

What is the SQL Endpoint in a Lakehouse (simple, exact, no ambiguity)





One-line definition (lock this)

The SQL Endpoint is a read-only SQL interface that lets you query Lakehouse Delta tables using pure SQL, without Spark.

Why SQL Endpoint exists

Lakehouse data is stored as:

- Parquet files
- Delta tables

But not everyone:

- Knows Spark
- Wants notebooks
- Needs engineering-level access

So Fabric provides a SQL Endpoint to make Lakehouse data feel like a traditional warehouse.

What SQL Endpoint connects to

The SQL Endpoint:

- Reads Delta tables from the Lakehouse
- Reads data stored in OneLake
- Uses Delta Log to ensure consistency

It does not read:

- Raw CSV files in **Files**
 - Non-Delta data
-

What you can do with SQL Endpoint

Allowed

- **SELECT** queries
- **JOINS**
- **WHERE, GROUP BY**
- Views
- Connect Power BI
- Run BI workloads

Not allowed

- **INSERT**

- **UPDATE**
- **DELETE**
- **MERGE**
- **File writes**
- **Schema changes**

SQL Endpoint = read-only by design

How SQL Endpoint works internally

SQL Query



SQL Endpoint Engine



Delta Log (latest version)



Parquet files



Results

- Delta Log ensures correct schema & version
 - Parquet files provide performance
-

Why Fabric keeps SQL Endpoint separate

- Prevents accidental data corruption
- Protects production datasets
- Gives analysts a safe interface
- Improves BI performance predictability

This is intentional restriction.

Relationship with Warehouse (important)

- Warehouse → SQL read + write
- Lakehouse SQL Endpoint → SQL read only

Both feel similar to users, but capabilities differ.

When to use SQL Endpoint

- Business analytics
 - Reporting
 - Ad-hoc querying
 - Power BI semantic models
-

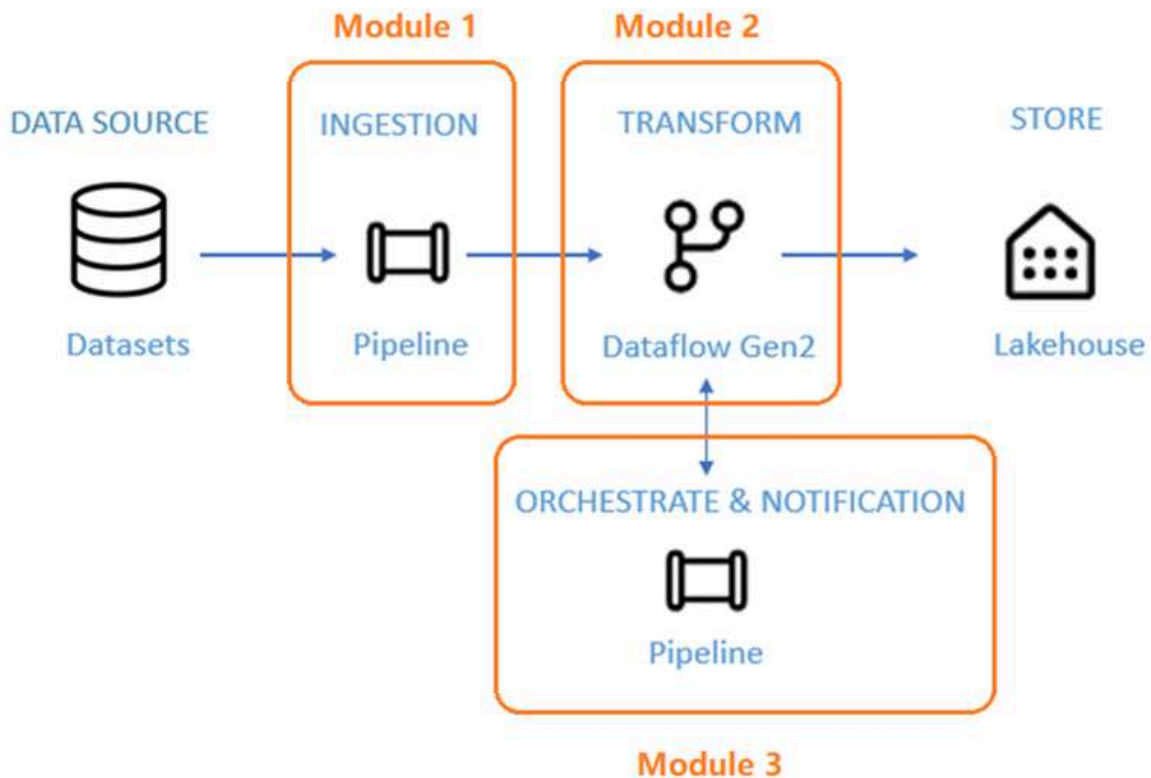
Mental model (never forget)

- Spark = build and change data
 - SQL Endpoint = safely read data
-

One-line memory lock

SQL Endpoint lets you query Lakehouse Delta tables using SQL only, without the risk of modifying data.

Fabric Data Factory — Notes (clear, structured, exam-ready)



Notebook

Notebook Double

Notebook

Another Notebook

Teams (Preview)

Send message to Teams

If Condition

Script Log failure

True

Send email

False

Script1

General

Settings

Workspace *

BugBashWorkspace

Refresh

Notebook *

NotebookDouble

Refresh

Open

New

Base parameters

New

Delete

	Name	Type	Value
☐	param1	Int	2

AZURE DATA FACTORY VS MICROSOSFT FABRIC



Deepak Goyal

ADF	Fabric
Dataset	No Dataset
Linked Service	Connection
Self-hosted IR	On-prem gateway
Dataflow (spark)	Dataflow gen 2 (power query)
Debug	No Debug
No Mail/ Team Activity	Mail & Team notification Acitvity
Better CI/CD	Limited CI/CD
More number of connectors	Less number of Connectors

1) What is Fabric Data Factory

Fabric Data Factory is the **data integration and orchestration layer** inside **Microsoft Fabric**.

Simple definition:

Fabric Data Factory is used to **ingest, move, and orchestrate data** into OneLake and other Fabric workloads using **pipelines**.

It is the **Fabric-native evolution of Azure Data Factory**, tightly integrated with:

- OneLake
- Lakehouse
- Warehouse
- Power BI
- Fabric security and governance

2) Why Fabric Data Factory was needed

Problem before (Azure era)

- Azure Data Factory existed **outside** analytics tools
- Separate service, setup, billing
- Data landed in ADLS → then copied again
- Weak lineage across analytics tools

Fabric goal

One platform, one storage, one governance model.

Fabric Data Factory was created to:

- Remove tool hopping
- Reduce data duplication
- Simplify ingestion into OneLake
- Provide end-to-end lineage

3) What Fabric Data Factory actually does

Fabric Data Factory handles **three core jobs**:

1 Data Ingestion

- Pull data from:
 - Azure SQL
 - Amazon S3
 - REST APIs
 - Files (CSV, JSON, Parquet)
 - Land data directly into **OneLake**
-

2 Data Movement

- Copy data between:
 - Files → Lakehouse
 - Lakehouse → Warehouse
 - External systems → Fabric

Uses **Copy Activity** (no-code / low-code).

3 Orchestration

- Control execution order
- Loop through files
- Add conditions
- Schedule pipelines

This makes ingestion **reliable and repeatable**.

4) Key components of Fabric Data Factory



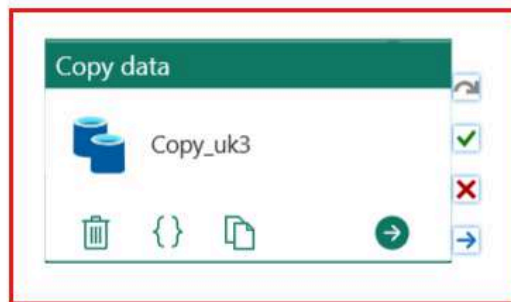
Power Query Dataflows



Azure Data Factory



**Data Factory
Microsoft Fabric**



General Source Destination Mapping Settings

Name * [Learn more](#) 

Description

Timeout ⓘ

Retry ⓘ

▼ Advanced

Retry interval (sec) ⓘ

Secure output ⓘ ☐

Secure input ⓘ ☐

◆ Pipeline

- A workflow
- Defines **what runs and in what order**

♦ **Activity**

- A single step in a pipeline
Examples:
- Copy Activity
- Notebook Activity
- Dataflow Gen2 Activity

♦ **Linked Service**

- Connection to a data source or target
Examples:
- S3
- Azure SQL
- OneLake

♦ **Dataset**

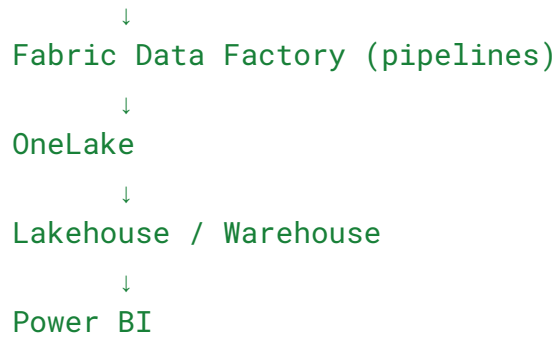
- Describes data structure and location
Examples:
- Folder
- File
- Table

♦ **Trigger**

- Controls **when** pipeline runs
Examples:
 - Schedule
 - Manual
 - Event-based
-

5) How Fabric Data Factory fits into Fabric

Source Systems



Fabric Data Factory is the **entry gate** of Fabric.

6) Fabric Data Factory vs Azure Data Factory

Aspect	Azure Data Factory	Fabric Data Factory
Platform	Separate Azure service	Built into Fabric
Storage target	ADLS	OneLake (native)
Governance	External	Unified
Lineage	Partial	End-to-end
Integration	Loose	Tight

Fabric Data Factory is **opinionated and integrated**, not generic.

7) What Fabric Data Factory is NOT

- Not a data warehouse
- Not a transformation engine (heavy logic)
- Not a reporting tool

It **moves and orchestrates data**, it does not analyze it.

8) When to use Fabric Data Factory

Use it when you need to:

- Ingest external data into Fabric
 - Automate daily data loads
 - Loop through files
 - Control data pipelines
 - Prepare data for Lakehouse / Warehouse
-

9) Why it is important for DP-700

DP-700 focuses on:

- End-to-end analytics
- OneLake-first design
- Reduced data movement
- Unified governance

Fabric Data Factory is the **ingestion backbone** of this model.

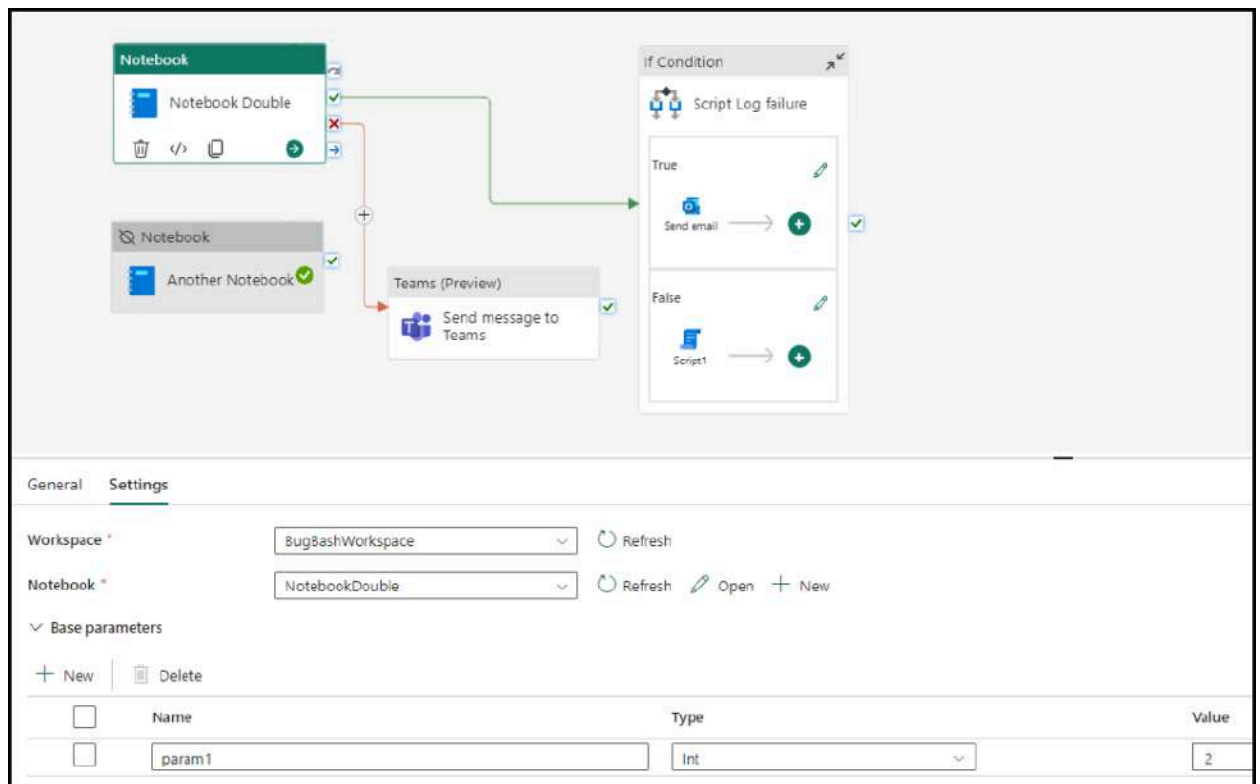
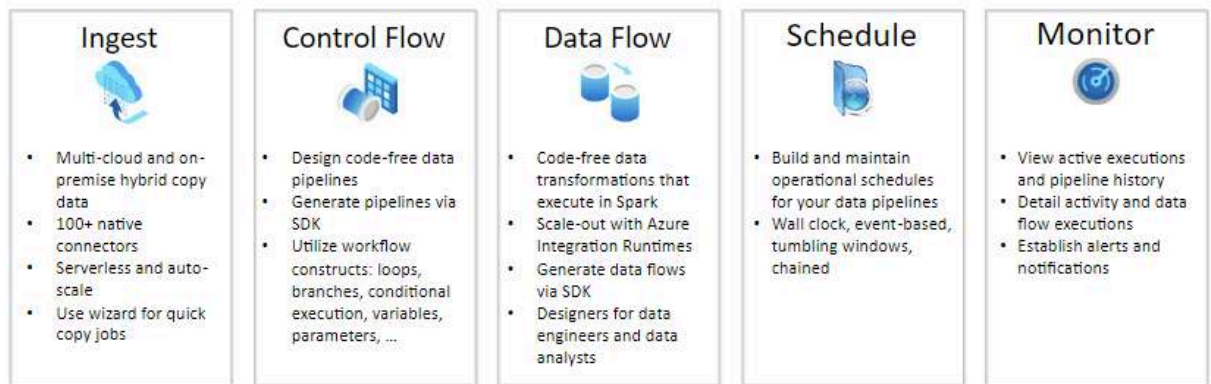
10) One-line memory lock

Fabric Data Factory is the built-in pipeline engine that brings data into OneLake and orchestrates analytics workflows inside Microsoft Fabric.

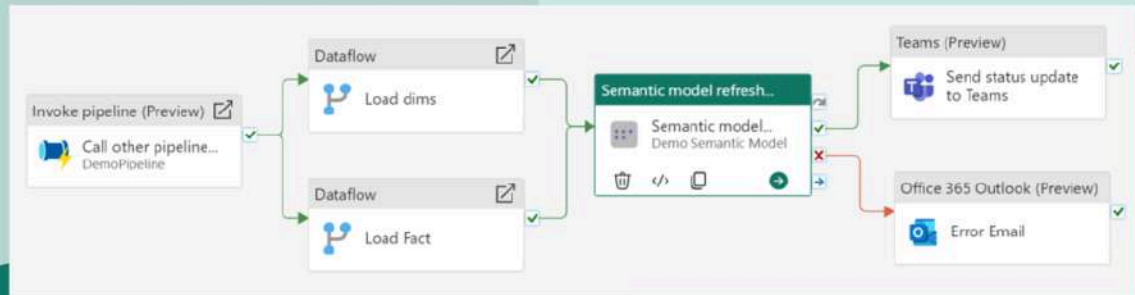
Important Activities in Fabric Data Factory and their uses

(clear notes, practical + exam-ready)

Code-Free ETL as a service



Orchestrate Power BI Semantic model refreshes through Fabric Pipelines



What “activities” mean (1 line)

In Microsoft Fabric Data Factory, an activity is a single executable step inside a pipeline.

Pipeline = workflow

Activity = action

1 Copy Activity (MOST IMPORTANT)

What it does

- Moves data from source → target

Typical use

- S3 → OneLake
- SQL → Lakehouse

- **Files → Warehouse**

Key points

- **No-code**
- **Fast**
- **Scalable**
- **No transformation logic**

When to use

Any time data needs to be ingested or moved

2 Dataflow Gen2 Activity

What it does

- **Performs data transformation**
- **Uses Power Query (low-code)**

Typical use

- **Clean columns**
- **Filter rows**
- **Type conversion**
- **Simple business rules**

Key points

- **Visual transformations**

- Writes output to OneLake
- Slower than Spark for big data

When to use

Light-to-medium transformations without coding

3 Notebook Activity

What it does

- Executes a Spark notebook

Typical use

- Complex transformations
- Delta table creation
- Schema evolution logic
- Advanced processing

Key points

- Uses PySpark / SQL
- Full control
- Most powerful activity

When to use

Heavy transformation or custom logic

4 Stored Procedure / SQL Script Activity

What it does

- Executes SQL logic on:
 - Warehouse
 - SQL endpoint

Typical use

- Data validation
- Post-load processing
- Aggregations

Key points

- SQL-only
- Fast for relational logic

When to use

Warehouse-side processing

5 Get Metadata Activity

What it does

- Reads file or folder metadata

Typical use

- List files
- Check file existence
- Get last modified date

Key points

- No data read
- Metadata only

When to use

Dynamic pipelines (file-driven logic)

ForEach Activity

What it does

- Loops over a list

Typical use

- Process files one by one
- Run copy per file
- Parallel execution

Key points

- Works with arrays
- Supports parallelism

When to use

Multiple files or dynamic inputs

7 Filter Activity

What it does

- Filters items in an array

Typical use

- Process only **.csv** files
- Ignore temp or unwanted files

Key points

- Used before ForEach
- Expression-based

When to use

Conditional execution inside loops

8 If Condition Activity

What it does

- Runs logic based on true / false

Typical use

- File exists → run pipeline

- No data → skip load

Key points

- Branching logic

When to use

Decision-based workflows

9 Wait Activity

What it does

- Pauses pipeline execution

Typical use

- Wait for downstream system
- Controlled delays

When to use

Time-based orchestration

10 Web Activity

What it does

- Calls REST APIs

Typical use

- Trigger external jobs
- Fetch tokens
- Notify systems

When to use

API integration

Activity grouping (easy memory)

Category	Activities
----------	------------

Data movement	Copy
---------------	------

Transformation	Dataflow Gen2, Notebook
----------------	-------------------------

Orchestration	ForEach, If, Filter, Wait
---------------	---------------------------

Me tad ata	Get Metadata
Int egr ati on	Web
SQ L log ic	SQL Script

Typical real pipeline (example)

Get Metadata



Filter (.csv only)



ForEach



Copy Activity



Notebook Activity

This is the most common Fabric pipeline pattern.

DP-700 exam focus

- Copy Activity → ingestion
- Notebook Activity → transformations
- Dataflow Gen2 → low-code prep
- ForEach + Get Metadata → dynamic pipelines

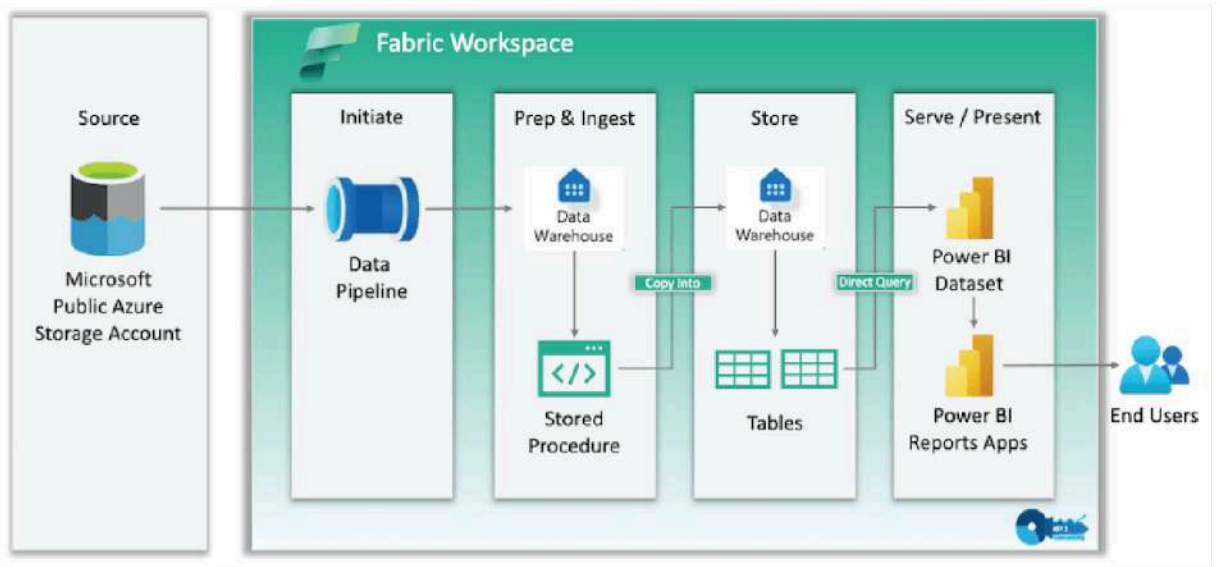
One-line memory lock

Copy moves data, Notebook transforms data, orchestration activities control flow.

End.

Fabric Data Factory — Pending / Advanced Topics Notes

(beyond basic pipelines & activities)



Connect

Configure trigger

Review and create

PreviewGet events

Configure events type and source

Select event type(s)

Event type(s) *

Select event type(s)

☒ Microsoft.Storage.BlobCreated

☒ Microsoft.Storage.BlobDeleted

☒ Microsoft.Storage.BlobRenamed

☒ Microsoft.Storage.BlobTierChanged

☒ Microsoft.Storage.DirectoryCreated

☒ Microsoft.Storage.DirectoryRenamed

☒ Microsoft.Storage.AsyncOperationInitiated

☒ Microsoft.Storage.DirectoryDeleted

☒ Microsoft.Storage.BlobInventoryPolicyCompleted

☒ Microsoft.Storage.LifecyclePolicyCompleted

Clear all

Back



1) Triggers in Fabric Data Factory

What they are

Triggers define when a pipeline runs.

Types

- Schedule trigger → time-based (hourly, daily)
- Manual trigger → on-demand
- Event-based (limited) → source-driven

Why important

- Enables automation
 - Eliminates manual runs
 - Production readiness
-

2) Parameters & Variables

Pipeline Parameters

- Passed from outside
- Used for dynamic behavior

Example:

```
fileName, folderPath, loadDate
```

Variables

- Used inside pipeline
- Temporary values (string, array, boolean)

Why needed

- Reusable pipelines
 - One pipeline → many datasets
 - Dynamic file processing
-

3) Dynamic Content & Expressions

What it is

- Expression language used inside activities

Examples:

```
@pipeline().parameters.fileName
```

```
@item().name
```

```
@utcNow()
```

Use cases

- Dynamic paths
 - Conditional logic
 - File-driven pipelines
-

4) Linked Services (Fabric context)

What they represent

- **Connection definitions**

Supported targets

- **OneLake (native)**
- **Azure SQL**
- **Amazon S3**
- **REST APIs**

Key difference vs Azure Data Factory

- **Tightly integrated with Fabric security**
 - **No manual storage key handling for OneLake**
-

5) Datasets (Logical Data Descriptors)

What they do

- **Describe where and how data exists**
- **Folder, file, or table abstraction**

Important behavior

- **Dataset $\neq$ data**
 - **Dataset = metadata only**
-

6) Incremental Load Patterns

Why needed

- Avoid full reloads
- Improve performance
- Reduce cost

Common patterns

- Timestamp-based filtering
- Watermark tables
- File-date based ingestion

Typical flow

Get last load date

→ Filter source

→ Copy new data only

7) Error Handling & Retry Logic

Built-in features

- Retry count
- Retry interval
- Timeout settings

Control-flow handling

- If Condition
- Failure paths
- Pipeline status checks

Why critical

- Production pipelines fail
 - Controlled recovery needed
-

8) Monitoring & Observability

What Fabric provides

- Pipeline run history
- Activity-level status
- Error messages
- Execution duration

Why important

- Debug failures
 - SLA tracking
 - Operational visibility
-

9) Lineage Integration (Fabric advantage)

What happens automatically

- **Source → Pipeline → OneLake → Lakehouse → Power BI**

Why powerful

- **End-to-end visibility**
 - **Governance-ready**
 - **No manual stitching**
-

10) Security Model

Access control

- **Workspace-level roles**
- **Inherited permissions**

Key rule

- **Pipeline access ≠ data access**
 - **Data access controlled by OneLake + workspace roles**
-

11) Performance & Optimization

Best practices

- **Use Copy Activity for ingestion**
- **Avoid row-by-row processing**
- **Prefer parallel ForEach when safe**

- Push transformations to Spark if heavy

12) Fabric Data Factory vs Azure Data Factory (Advanced View)

Area	Azure Data Factory	Fabric Data Factory
Platform	Standalone	Embedded in Fabric
Storage	ADLS	OneLake
Governance	Separate	Unified
Lineage	Partial	End-to-end
Analytics hand off	Manual	Native

13) Typical Enterprise Pipeline Pattern

Trigger

- Get Metadata
- Filter
- ForEach
- Copy Activity
- Notebook Activity
- Validation

This pattern covers 90% real-world use cases.

DP-700 / Interview Memory Locks

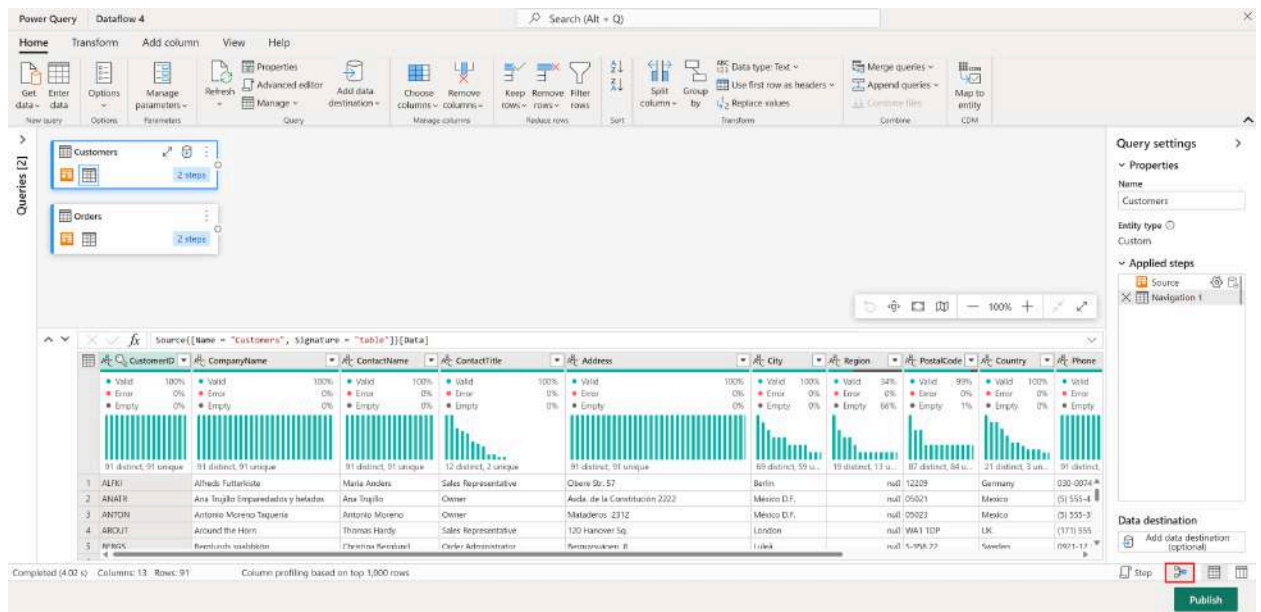
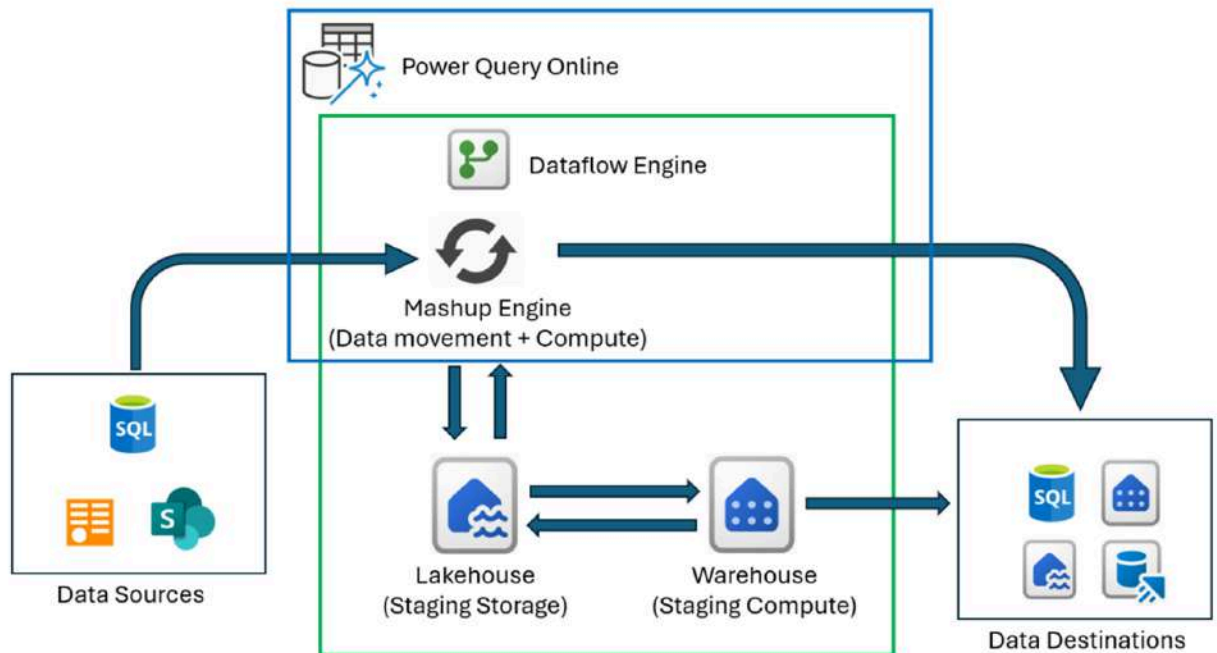
- Pipelines orchestrate, they don't analyze
 - Copy = ingestion
 - Notebook = transformation
 - OneLake = default landing zone
 - Fabric Data Factory = entry point of Fabric
-

One-line Final Lock

Fabric Data Factory handles ingestion, orchestration, and automation so analytics can start cleanly inside OneLake.

End.

Dataflow Gen2 — what it is, why it's different from ADF, and when to use which



1) What is Dataflow Gen2

Dataflow Gen2 is a low-code data transformation tool in Microsoft Fabric based on Power Query.

Simple definition:

Dataflow Gen2 is used to clean, shape, and prepare data before analytics, using a visual, Power Query-based experience.

It runs inside Fabric, writes data to OneLake, and is tightly integrated with:

- Lakehouse
 - Warehouse
 - Power BI
-

2) What problem Dataflow Gen2 solves

Not all data work needs:

- Pipelines
- Spark
- Engineering skills

Many teams just need:

- Column cleanup
- Type fixes
- Joins
- Filters
- Business rules

Dataflow Gen2 exists for this middle layer.

3) How Dataflow Gen2 works (conceptually)

Source

- Power Query transformations
- Output to OneLake (Lakehouse / Warehouse)

- No servers
 - No Spark code
 - Visual transformations
 - Repeatable and governed
-

4) Why Dataflow Gen2 is different from ADF

This is the key confusion area.
They solve different problems.

ADF (Azure / Fabric Data Factory)

Purpose:

- Move data
- Orchestrate workflows

Core strengths:

- Copy data between systems
- Scheduling
- Looping, conditions
- Large-scale ingestion

ADF is about control flow.

Dataflow Gen2

Purpose:

- Transform data
- Prepare analytics-ready datasets

Core strengths:

- Data cleaning
- Shaping
- Business logic
- Reusable transformation logic

Dataflow Gen2 is about data shaping.

5) Side-by-side comparison (lock this)

Aspect

Dataflow
Gen2

ADF / Fabric
Data Factory

Main role	Transformation	Ingestion & orchestration
Interface	Low-code (Power Query)	Pipeline-based
Coding	No	No / Low
Best for	Data prep	Data movement
Handles control flow	✗	✓
Handles heavy logic	✗	✗ (use Spark)
Writes to OneLake	✓	✓
Governance	Native	Native

6) Where Dataflow Gen2 should be used

Use Dataflow Gen2 when:

- **Data is already accessible**
- **Transformations are light to medium**
- **Business rules are needed**
- **Analysts or BI developers are involved**
- **You want reusable, governed logic**

Typical examples:

- **Clean CSV files**
 - **Join reference tables**
 - **Standardize columns**
 - **Prepare Gold-layer datasets**
-

7) Where ADF / Fabric Data Factory should be used

Use ADF when:

- **Data is coming from external systems**
- **You need scheduling and retries**
- **You must loop through files**
- **You need orchestration logic**
- **Volume is high**

Typical examples:

- S3 → OneLake ingestion
 - API → Lakehouse loads
 - Daily batch pipelines
 - File-based automation
-

8) Where Spark / Notebook fits (important context)

Neither Dataflow Gen2 nor ADF is for:

- Heavy transformations
- Complex joins at scale
- ML logic

That belongs to:

- Spark notebooks
 - Lakehouse transformations
-

9) Typical enterprise pattern (recommended)

ADF / Fabric Data Factory

- ingest data
- land in Files (OneLake)

Dataflow Gen2

- clean & shape data

→ write to Lakehouse tables

Spark Notebook (optional)

→ heavy logic / optimization

Power BI

Each tool does one job well.

10) Mental model (never forget)

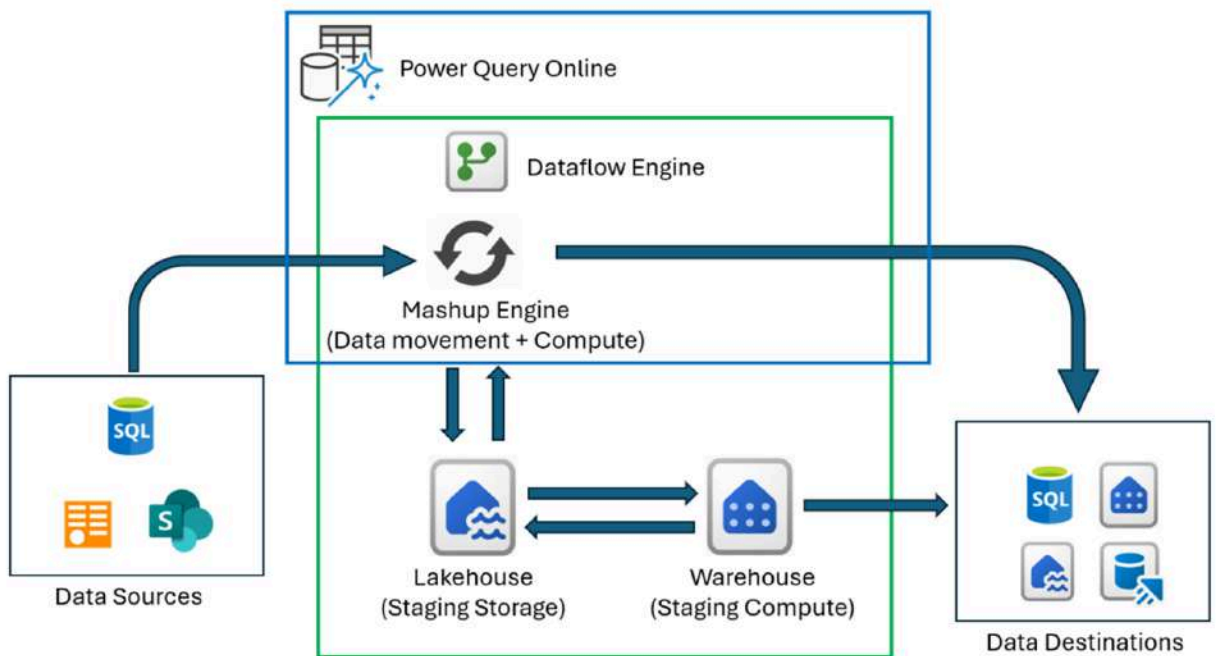
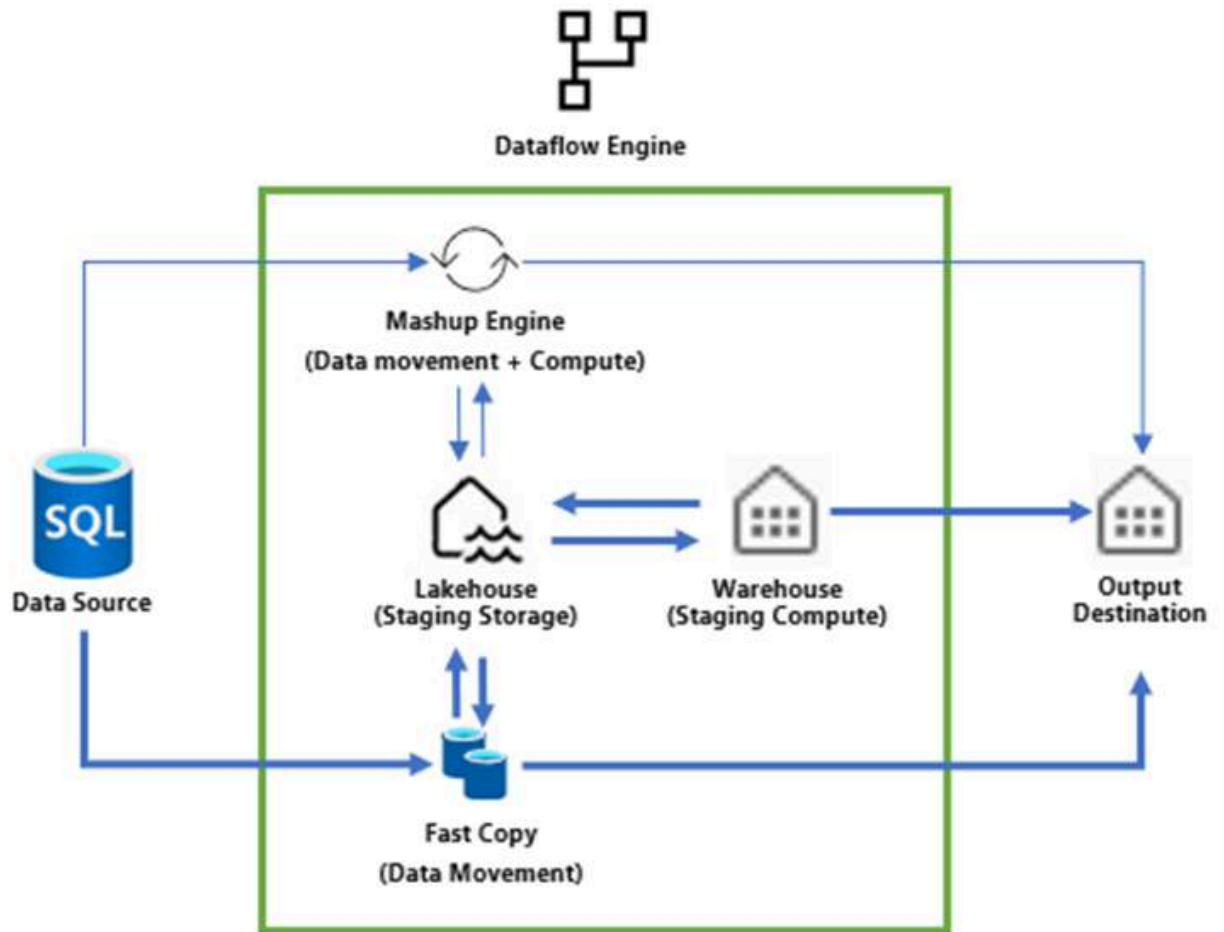
- ADF = delivery truck (moves data)
 - Dataflow Gen2 = kitchen (prepares data)
 - Spark = factory (heavy processing)
-

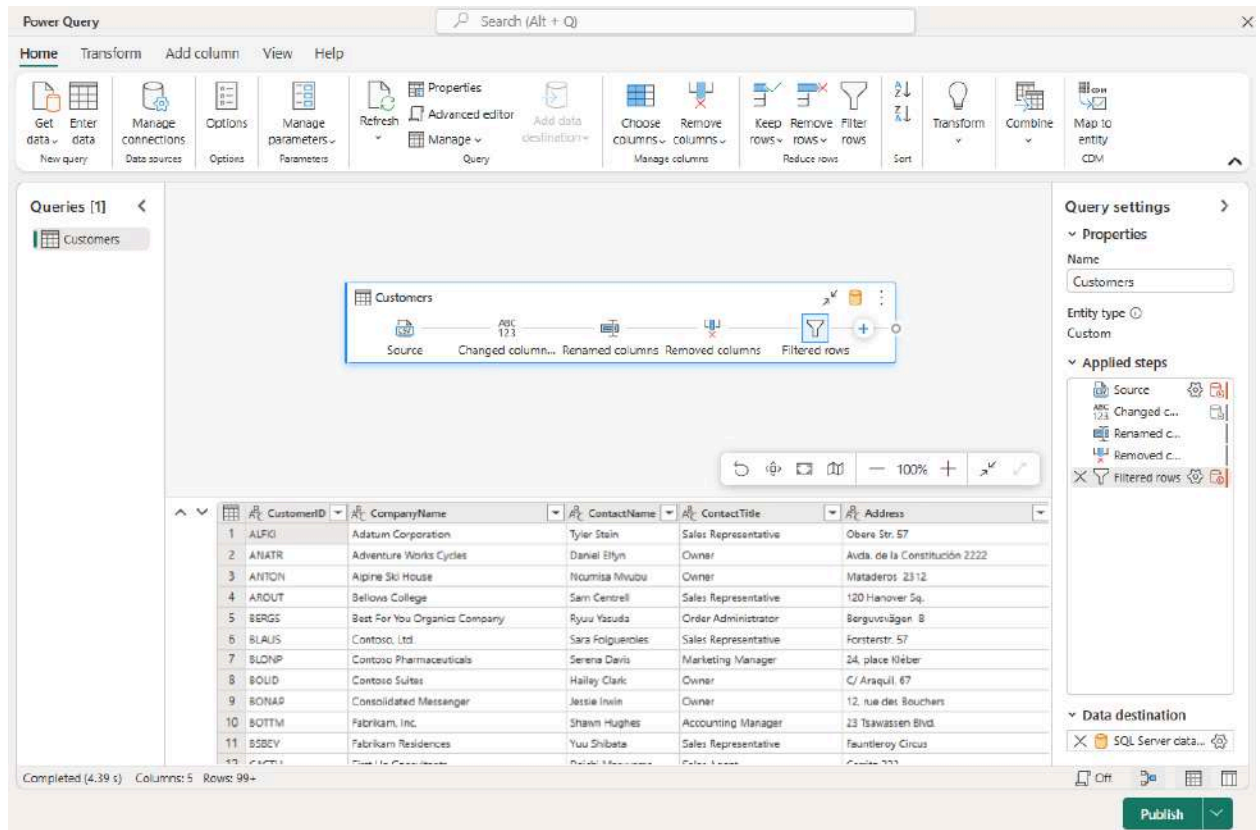
One-line memory lock

Use ADF to move data, Dataflow Gen2 to prepare data, and Spark when logic becomes heavy.

How to integrate Dataflow Gen2 with Fabric Data Factory

(what connects to what, and the exact patterns to use)





Core idea (lock this first)

Fabric Data Factory orchestrates.
Dataflow Gen2 transforms.

Integration = calling Dataflow Gen2 from a pipeline or feeding it data the pipeline ingests.

Pattern 1 — Call Dataflow Gen2 from a Pipeline (MOST COMMON)

When to use

- You want scheduling, retries, dependencies
- Dataflow Gen2 should run after ingestion

- You need end-to-end automation

Steps

1. Create Dataflow Gen2
 - Define Power Query transformations
 - Set output to Lakehouse / Warehouse
2. Create a Pipeline (Fabric Data Factory)
3. Add Dataflow Gen2 activity to the pipeline
4. Configure:
 - Workspace
 - Dataflow Gen2 name
5. Add Trigger (schedule/manual)

Flow

Pipeline Trigger



Copy Activity (optional)



Dataflow Gen2 Activity



Lakehouse / Warehouse

What happens

- Pipeline controls execution
 - Dataflow Gen2 runs transformations
 - Results land in OneLake
-

Pattern 2 — Pipeline Ingests → Dataflow Gen2 Transforms (Decoupled)

When to use

- Clear separation of duties
- Reusable Dataflow Gen2
- Multiple pipelines feed the same transformations

Steps

1. Pipeline uses Copy Activity to land data in Files
2. Dataflow Gen2 reads from Files
3. Dataflow Gen2 outputs curated tables

Flow

External Source

↓ (Copy Activity)

OneLake / Files

↓ (Dataflow Gen2)

Lakehouse Tables

Key benefit

- Dataflow Gen2 logic reusable across pipelines
-

Pattern 3 — Dataflow Gen2 First, Pipeline Later (Less Common)

When to use

- Transform data on-demand
- Orchestrate downstream tasks after transformation

Steps

1. Run Dataflow Gen2 (manual or scheduled)
2. Pipeline runs after to:
 - Validate
 - Move
 - Notify
 - Trigger notebooks

Flow

Dataflow Gen2



Pipeline (SQL / Notebook / Validation)

What integrates directly vs indirectly

Component	Direct Integratio n
Pipeline → Dataflow Gen2	✓
Dataflow Gen2 → Pipeline	✗ (pipeline triggers it)
Pipeline parameters → Dataflow Gen2	✗
Shared storage (OneLake)	✓

What is shared between them

- OneLake storage
- Workspace security
- Lineage
- Monitoring (run history)

No data copying between the two.

What NOT to expect (important)

- Dataflow Gen2 does not replace pipelines
- Pipelines do not do transformations
- Dataflow Gen2 does not handle looping/conditions
- Pipelines do not do row-level shaping

They are complementary by design.

Recommended enterprise pattern (best practice)

Fabric Data Factory (Pipeline)

- Ingest (Copy)
- Orchestrate (ForEach / If)
- Call Dataflow Gen2

Dataflow Gen2

- Clean
- Shape
- Load curated tables

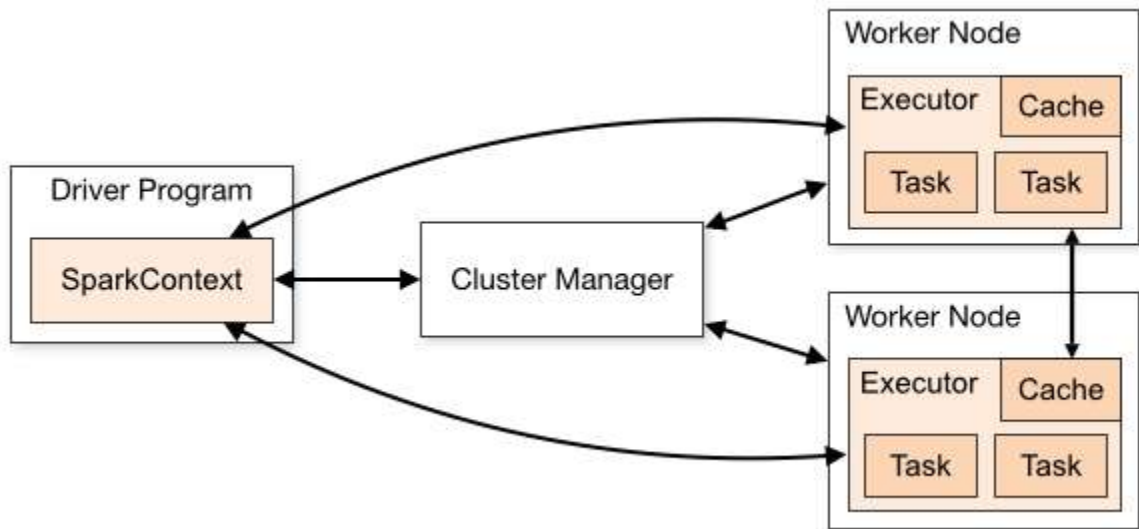
Spark (only if needed)

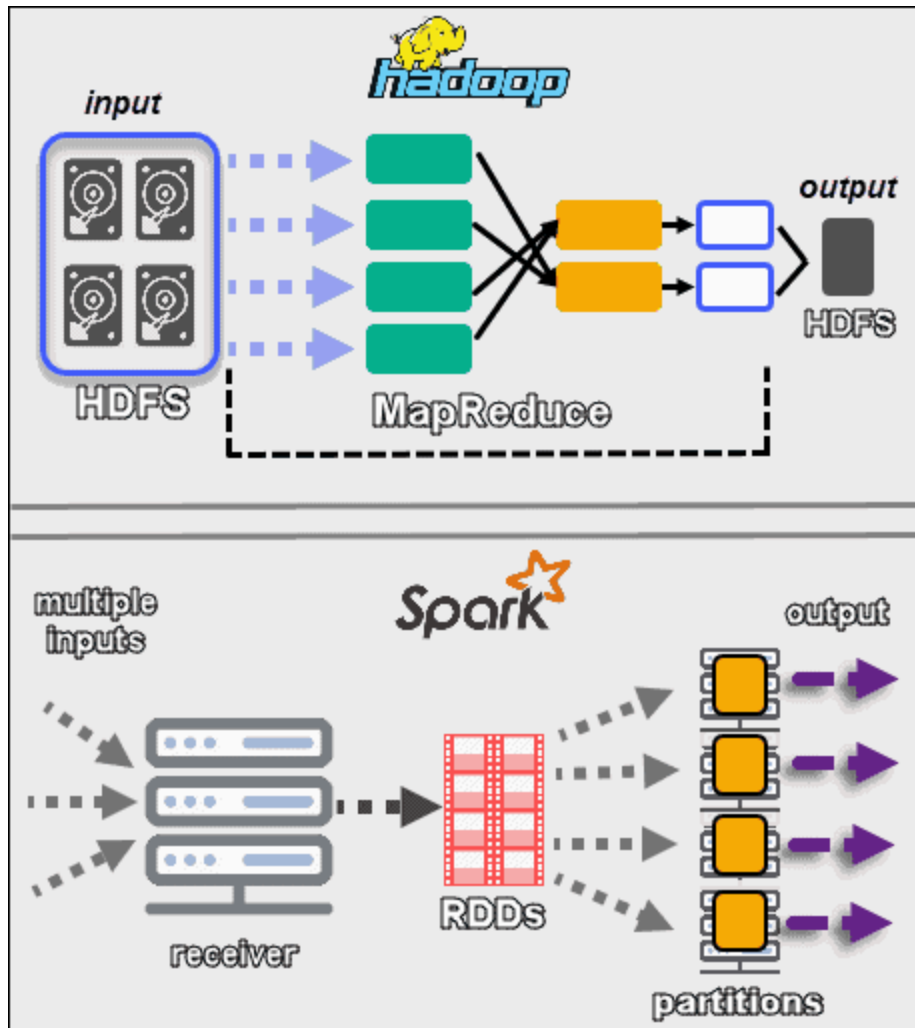
- Heavy logic
-

One-line memory lock

Use Fabric Data Factory to control WHEN things run, and Dataflow Gen2 to control HOW data is shaped.

Apache Spark — Core Notes (Big Data fundamentals, clear & complete)





1) What is Apache Spark

Apache Spark is a distributed data processing engine designed to process large-scale data fast.

Simple definition:

Spark splits big data into parts, processes them in parallel across many machines, and combines the result.

It is used for:

- Big data processing

- ETL
 - Streaming
 - Machine learning
 - SQL analytics
-

2) Why Spark is important in Big Data

Big data has 3 problems:

1. Data is too big for one machine
2. Data processing is slow
3. Data formats keep changing

Spark solves this by:

- Distributed processing
- In-memory computation
- Unified engine for batch, streaming, SQL, ML

Result:

- Faster analytics
 - Less infrastructure complexity
 - Real-time and batch together
-

3) What is a Node

A node is one machine (VM or physical server) in a cluster.

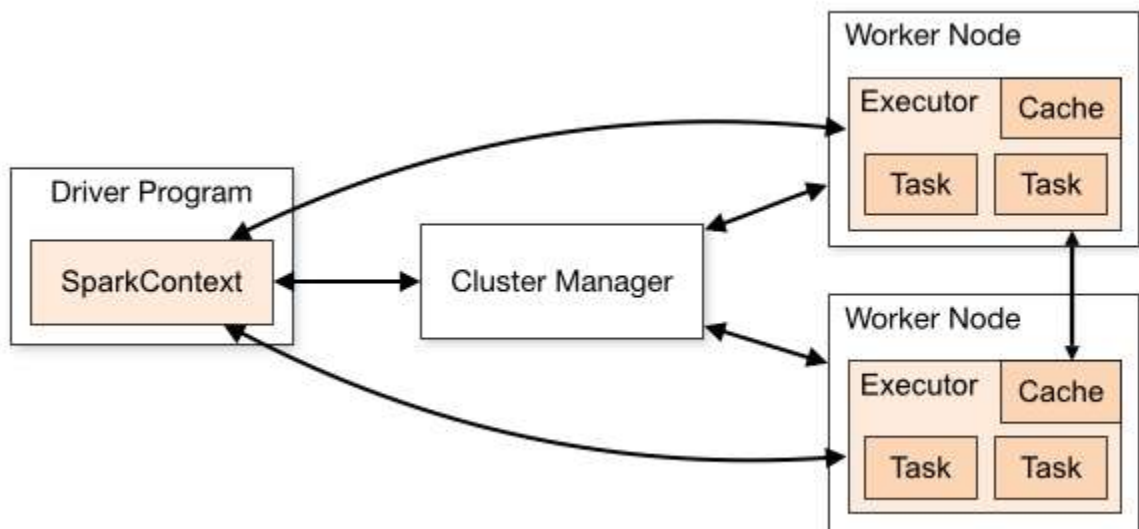
Types:

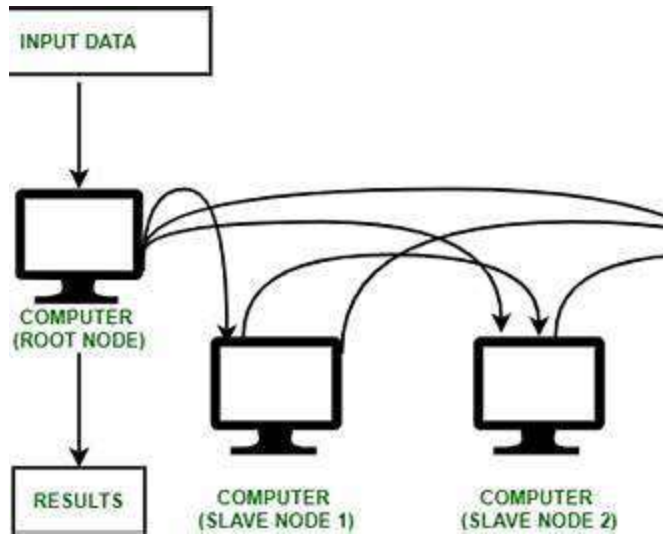
- Driver node → brain
- Worker node → does actual work

Each node has:

- CPU
- Memory
- Disk
- Network

4) What is a Cluster





A cluster is a group of nodes working together.

Purpose:

- Share workload
- Process data in parallel
- Handle large datasets

Example:

1 Driver node

10 Worker nodes

= 1 Spark Cluster

5) What is a Core

A core is a CPU processing unit.

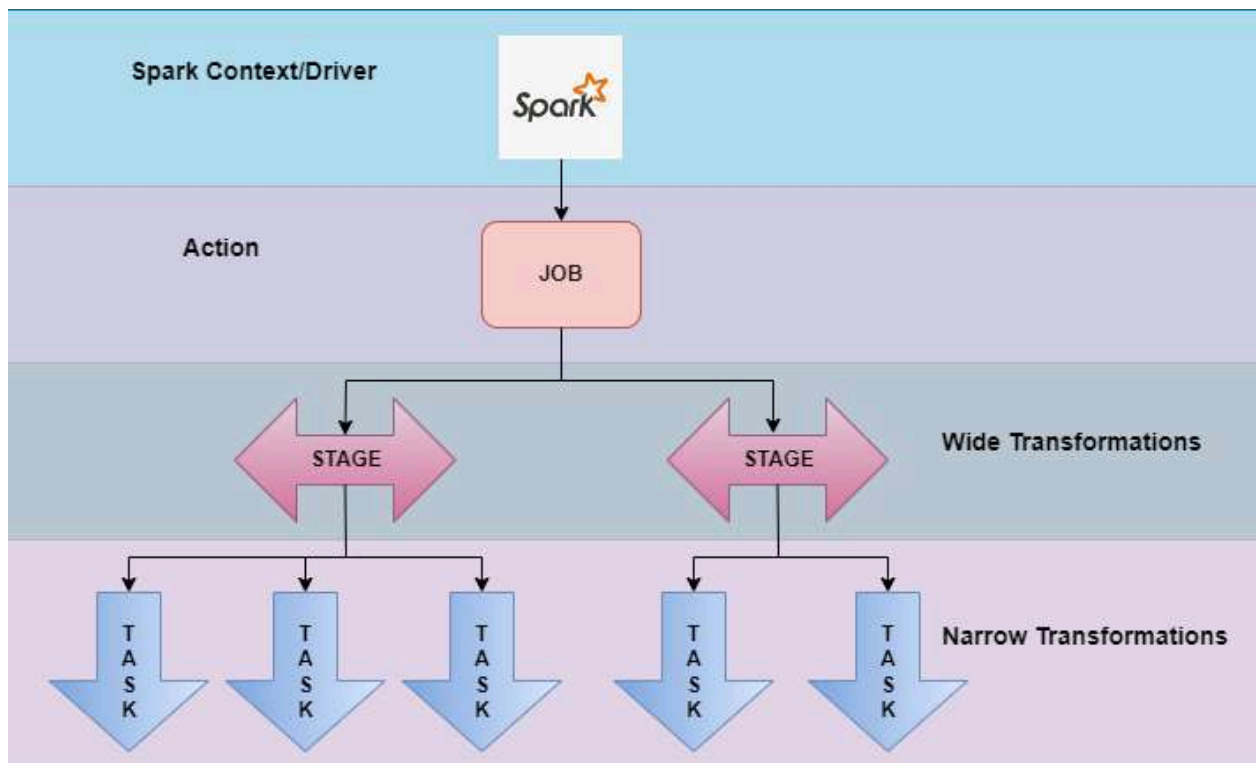
- One machine can have multiple cores

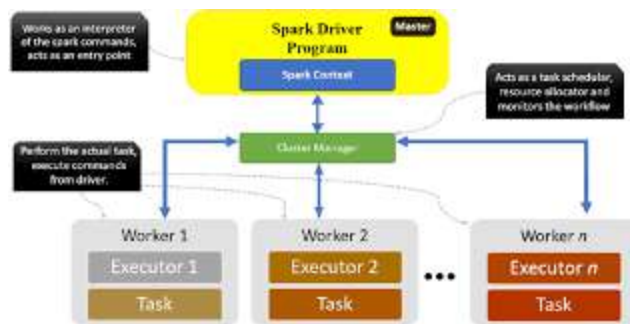
- Spark assigns tasks to cores
- More cores = more parallel work

Example:

- 1 node with 8 cores
- Spark can run 8 tasks at the same time on that node

6) Spark Architecture (how it works)





Driver

- Runs your Spark application
- Creates execution plan (DAG)
- Coordinates work

Executor

- Runs tasks
- Processes data
- Returns results

Flow

Code

- Driver
 - Tasks
 - Executors
 - Results
-

7) How Spark processes data (simple)

1. Data is split into partitions
2. Each partition processed in parallel
3. Intermediate results kept in memory
4. Final result aggregated

This avoids slow disk I/O.

8) Spark vs Hadoop (WHY Spark is better)

Hadoop (MapReduce era)

Apache Hadoop

Problems:

- Disk-based processing
 - Too slow for iterative jobs
 - Separate tools for SQL, ML, streaming
 - Complex development
-

Spark keeps data in RAM, Hadoop writes to disk after each step.

9) Why Spark replaced Hadoop MapReduce

- Faster execution
- Unified analytics engine

- Less code
- Better developer productivity

Hadoop today is mainly used for:

- Storage (HDFS)
- Legacy systems

Spark is used for processing.

10) Where Spark is used today

- Lakehouse platforms
 - Microsoft Fabric
 - Databricks
 - ETL pipelines
 - Streaming analytics
 - Machine learning pipelines
-

11) Simple analogy (lock this)

- Hadoop MapReduce → writing homework after every sentence
- Spark → writing everything in rough copy, finalizing at the end

Memory vs disk.

12) One-line memory locks

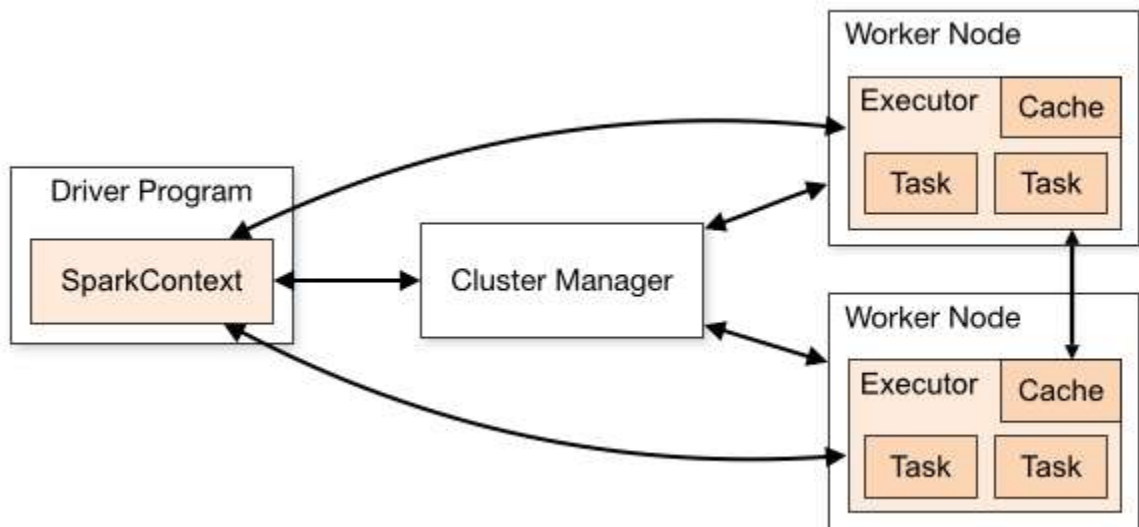
- Spark = distributed processing engine
 - Node = machine
 - Cluster = group of machines
 - Core = CPU execution unit
 - Spark is faster because it processes data in memory
-

Final one-liner

Apache Spark is the engine that made big data fast, flexible, and usable at scale.

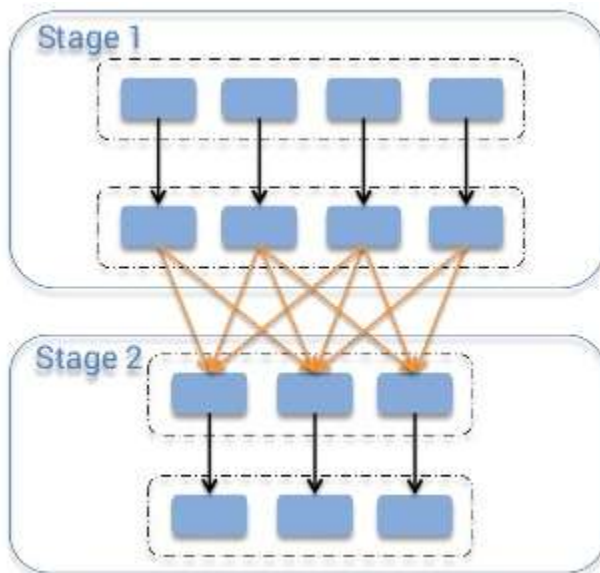
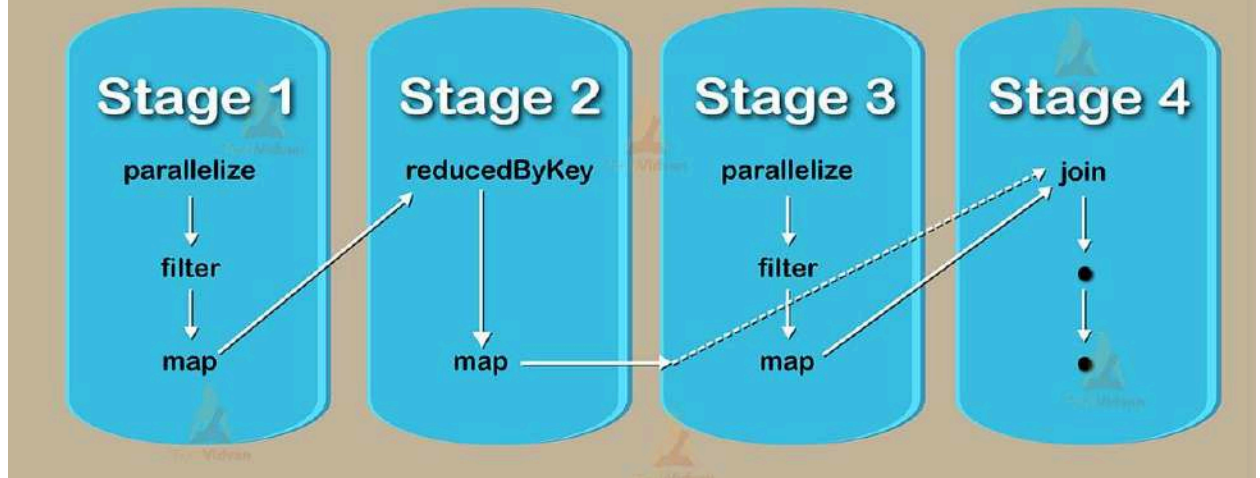
Apache Spark — Deep Concepts for Tough Interviews

(architecture, execution, memory, shuffles, tuning, internals)





DAG Visualisation



1) What Spark really is (interview-level definition)

Apache Spark is not a database and not storage.

It is:

A distributed compute engine that builds an execution plan (DAG) and runs it in parallel across a cluster using memory-first processing.

Spark's power comes from:

- **Lazy evaluation**
 - **DAG optimization**
 - **In-memory execution**
 - **Fault tolerance via lineage (not replication)**
-

2) Spark Core Abstractions (VERY IMPORTANT)

2.1 RDD (Resilient Distributed Dataset)

Low-level abstraction.

Properties:

- **Immutable**
- **Distributed**
- **Fault-tolerant via lineage**

Why RDD exists:

- **Foundation of Spark**
- **Used when fine-grained control is needed**

Limitation:

- **No optimizer**
 - **More code**
-

2.2 DataFrame

Most-used abstraction.

Properties:

- **Distributed table with schema**
- **Optimized by Catalyst Optimizer**
- **Backed by RDD internally**

Why DataFrame is preferred:

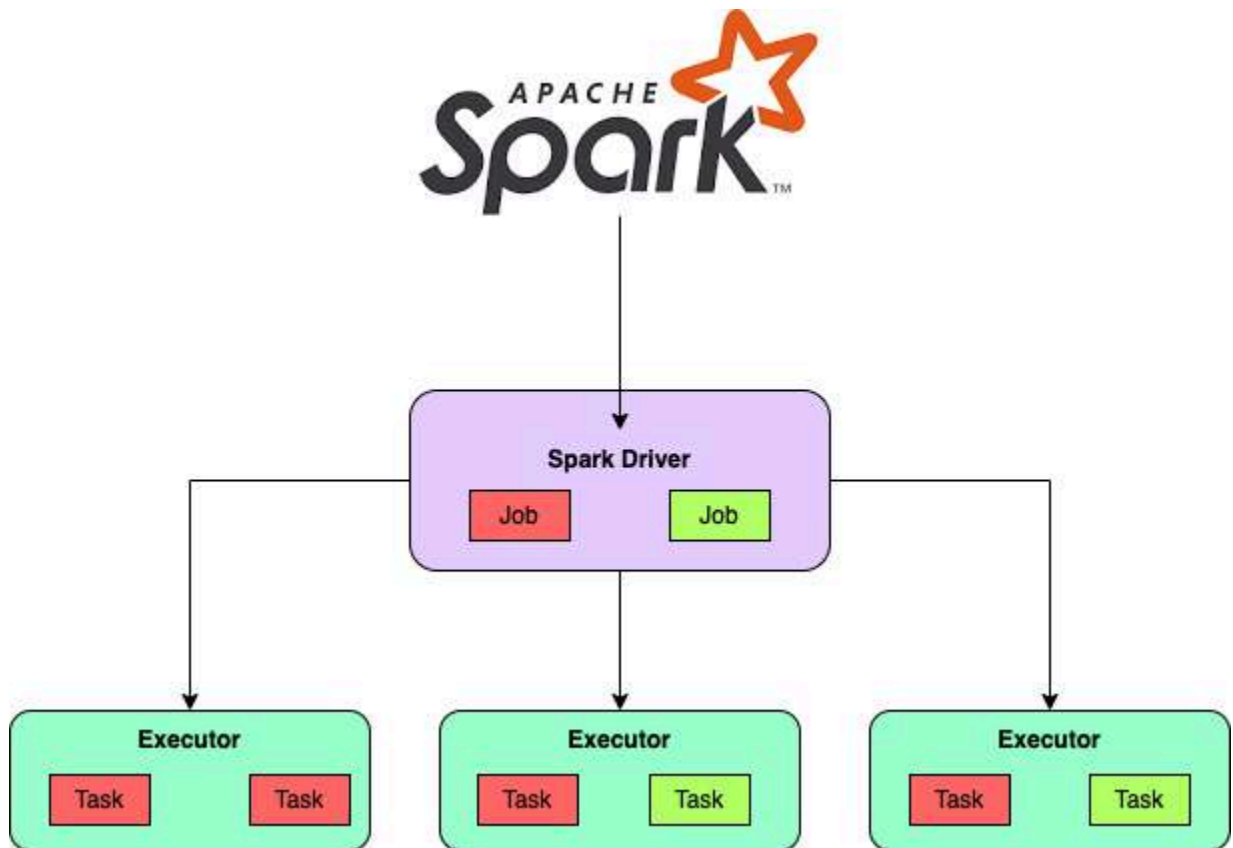
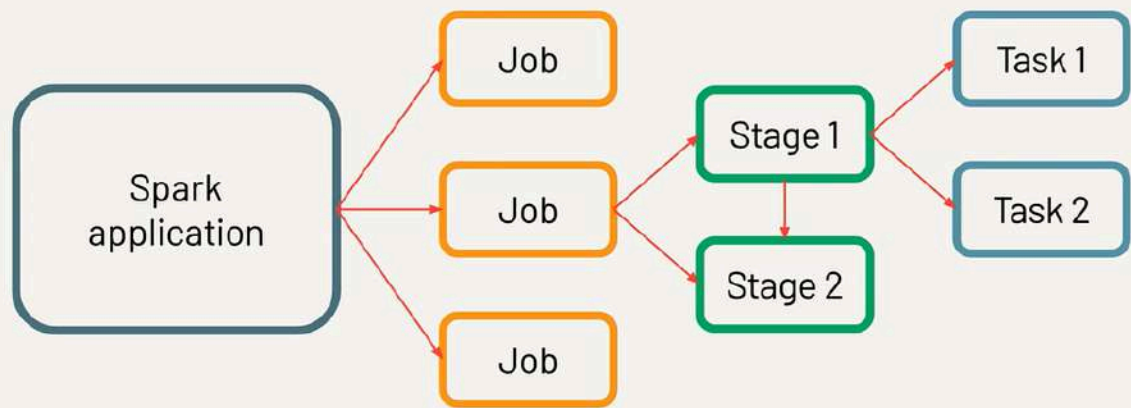
- **Less code**
 - **Faster**
 - **Optimized execution**
-

2.3 Dataset (Scala/Java)

- **Type-safe DataFrame**
 - **Compile-time checks**
 - **Rare in real projects**
-

3) Spark Execution Model (THIS IS CRITICAL)

Spark Execution



Step-by-step execution

1. Action triggered

- `count()`, `show()`, `write()`

2. DAG created

- Logical execution plan
- Based on transformations

3. DAG split into stages

- Based on shuffle boundaries

4. Stages split into tasks

- One task per partition

5. Tasks executed on executors

Terminology clarity (interview trap)

T e r m	Meaning
J o b	Triggered by an action
S t a g e	Group of tasks without shuffle

**T
a
s
k** **Smallest unit of
work**

**P
a
r
t
i
t
i
o
n** **Chunk of data**

4) Lazy Evaluation (why Spark is fast)

Spark does nothing until an action is called.

Example:

```
df.filter(col("age") > 30).select("name")
```

No execution yet.

Execution happens only when:

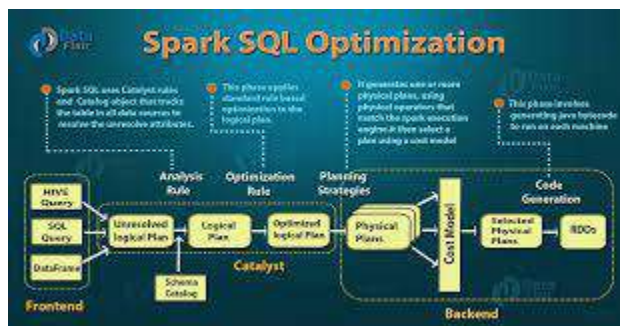
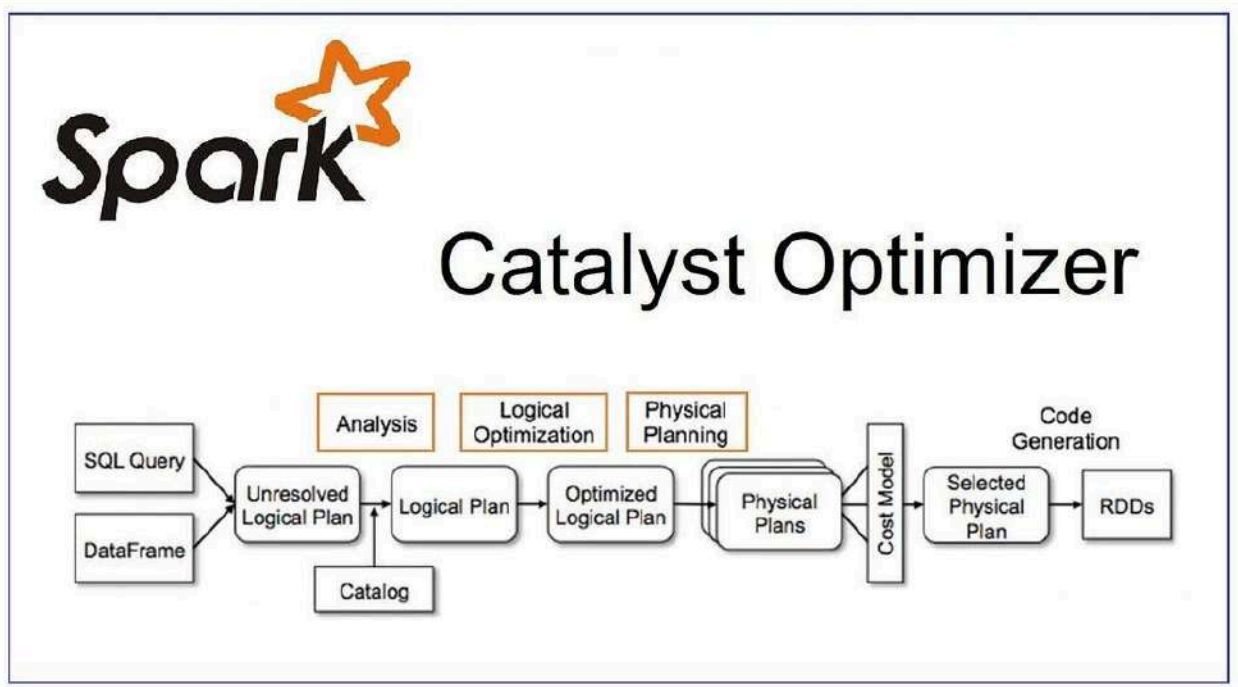
```
df.count()
```

Why this matters:

- **Spark can optimize the whole plan**
- **Removes unnecessary steps**

- Combines operations

5) Catalyst Optimizer (Spark's brain)



Catalyst does:

1. Logical plan creation
2. Rule-based optimization
3. Cost-based optimization

4. Physical plan selection

Optimizations include:

- **Predicate pushdown**
- **Column pruning**
- **Join reordering**
- **Constant folding**

This is why DataFrames beat RDDs.

6) Tungsten Engine (performance layer)

Tungsten improves:

- **Memory management**
- **CPU efficiency**
- **Cache utilization**

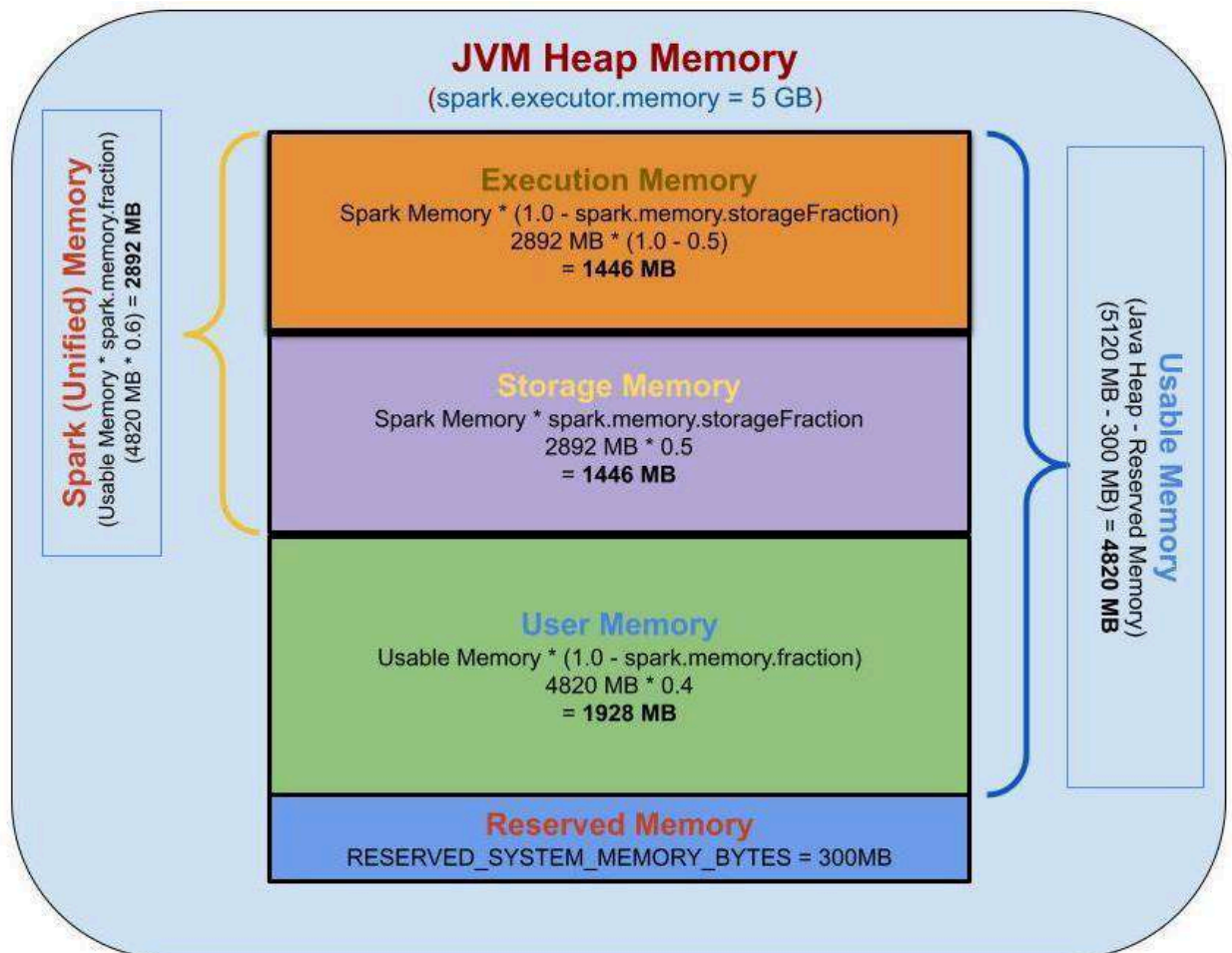
Key features:

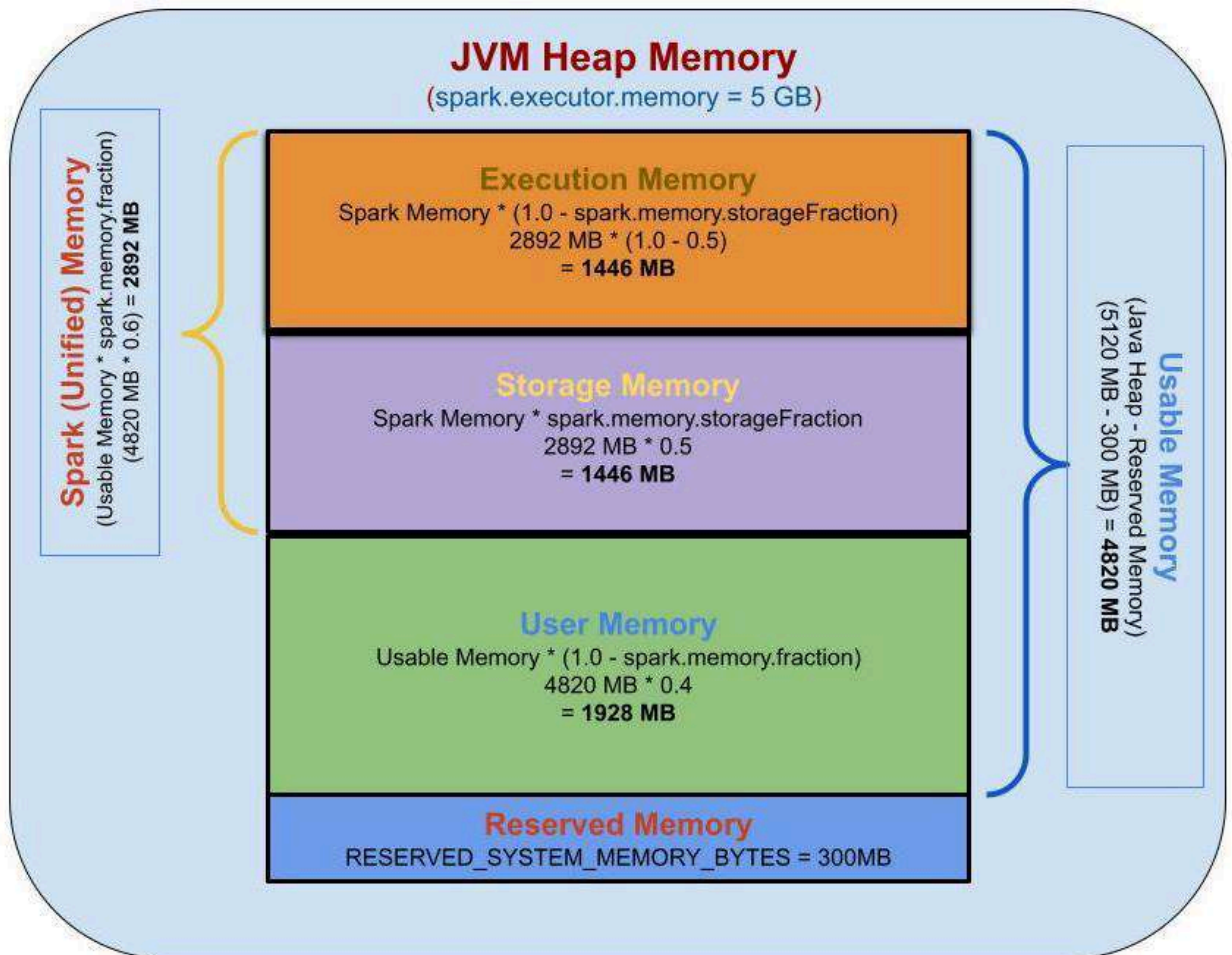
- **Off-heap memory**
- **Binary processing**
- **Whole-stage code generation**

Result:

- **Faster execution**
- **Less GC overhead**

7) Memory Management (INTERVIEW FAVORITE)





Executor memory is split into:

- Execution memory → shuffles, joins, aggregations
- Storage memory → cached data

Spark dynamically shares memory between them.

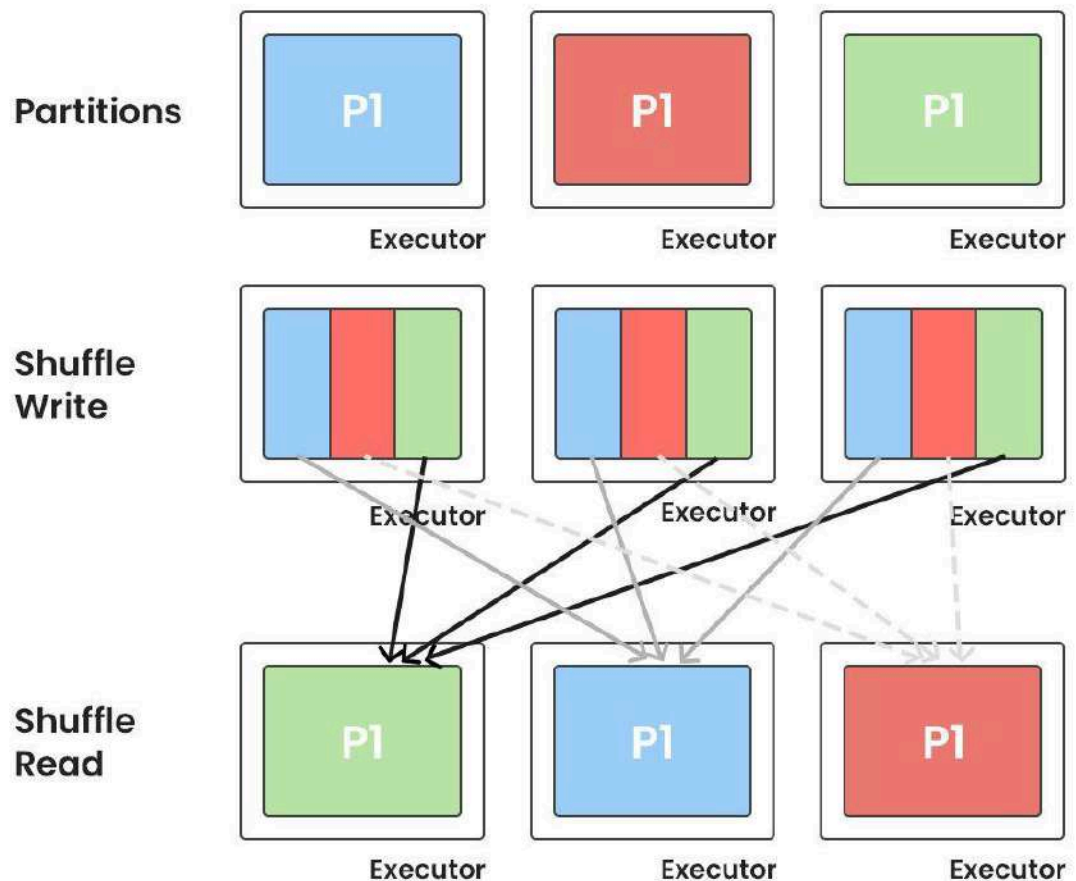
Important points:

- Caching is not guaranteed
 - Storage can evict data if execution needs memory
 - MEMORY_ONLY vs MEMORY_AND_DISK
-

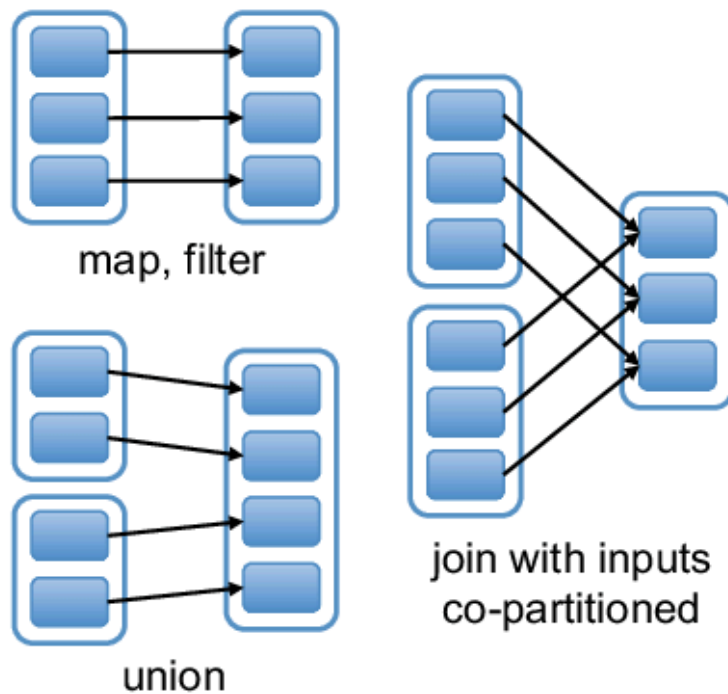
8) Shuffle (MOST EXPENSIVE OPERATION)



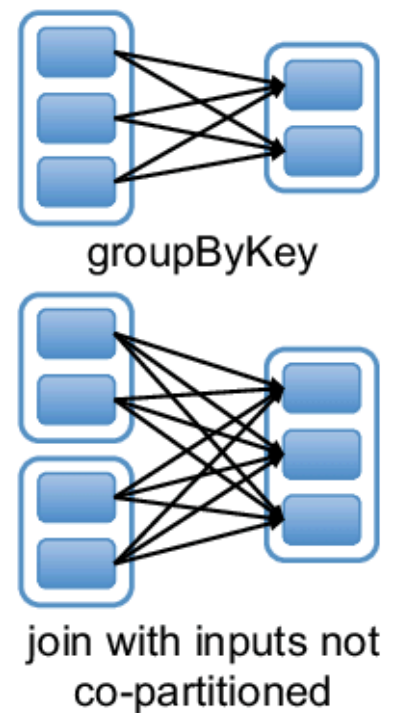
How Spark Architecture Shuffle Works ?



Narrow Dependencies:



Wide Dependencies:



Shuffle happens when data must move across nodes.

Caused by:

- `groupBy`
- `join`
- `distinct`
- `orderBy`

Why shuffle is expensive:

- Network I/O
- Disk I/O
- Serialization

- Memory pressure

Interview rule:

Most Spark performance issues are shuffle issues.

9) Narrow vs Wide Transformations

T y p e	E x a m p l e	S h u f f l e
N a r r o w	m a p , f i l t e r	✗
W i d e	g r o u p B y , j o	✓

i
n

Spark creates a new stage only at wide transformations.

10) Partitioning (performance control)

- Partition = unit of parallelism
- Too few → underutilization
- Too many → overhead

Golden rule:

Partitions $\approx$ 2-4 × total cores

Use:

- **repartition()** → increases partitions (shuffle)
 - **coalesce()** → reduces partitions (no shuffle)
-

11) Fault Tolerance (why Spark is resilient)

Spark does NOT replicate data.

Instead:

- Tracks lineage
- Recomputes lost partitions

This is cheaper than replication.

12) Spark vs Hadoop MapReduce (deep reason)

Apache Hadoop

A
s
p
e
c
t

Hadoop
MR

S
p
a
r
k

P
r
o
c
e
s
s
i
n
g

Disk
after
each
step

I
n
-
m
e
m
o
r
y

E
x
e
c
u
t
i
o
n

Map →
Reduce
only

D
A
G

I
t
e
r

Very
slow

F
a

a
t
t
i
v
e
j
o
b
s

s
t

A
P
I
s

Low-lev
el

H
i
g
h
-
l
e
v
e
l

O
p
t
i
m
i
z
a
t
i
o
n

None

C
a
t
a
l
y
s
t

Spark keeps intermediate data in memory → massive speedup.

13) Real Interview Scenarios

Q: Why Spark job is slow?

Answer:

- Too many shuffles
 - Skewed data
 - Bad partitioning
 - No predicate pushdown
 - Improper caching
-

Q: When to cache?

Answer:

- When data reused multiple times
 - Not for one-time actions
-

Q: Difference between repartition and coalesce?

Answer:

- repartition → full shuffle
 - coalesce → minimizes shuffle
-

14) Spark in Lakehouse (modern context)

Spark is used to:

- Convert raw files → Delta tables

- Handle schema evolution
- Perform heavy transformations
- Build ML pipelines

Fabric and Databricks use Spark as core engine.

Final Mental Model (lock this)

Spark =

DAG engine

+ Optimizer (Catalyst)

+ In-memory execution

+ Distributed fault tolerance

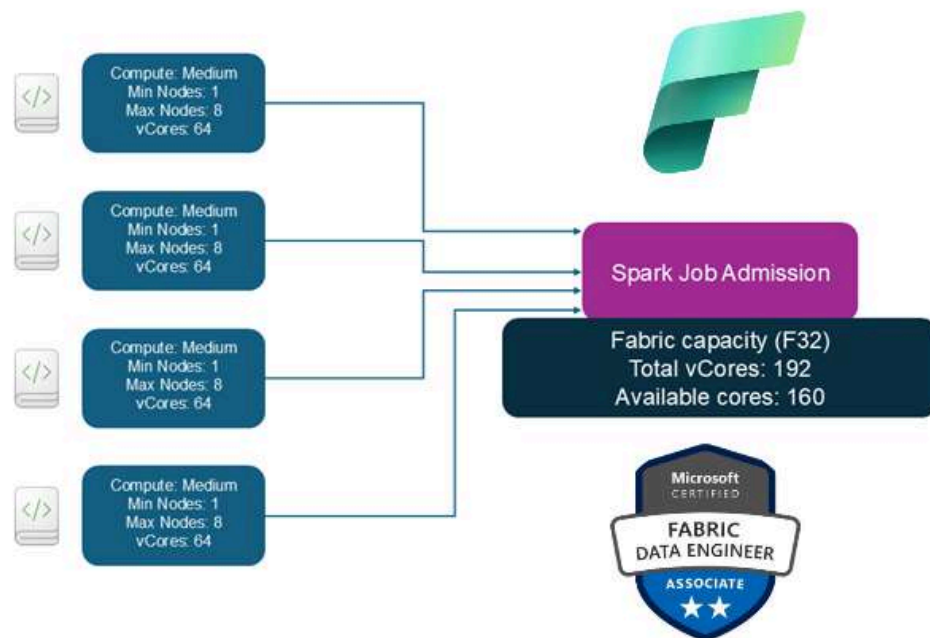
Ultimate one-line interview answer

Spark is a DAG-based, in-memory distributed compute engine optimized for large-scale data processing using lazy evaluation and cost-based optimization.

This level is enough to crack any Spark interview.

What are Node Sizes in Microsoft Fabric (clear, practical explanation)





Short definition (lock this)

Node size in Fabric means how much compute power (CPU + memory) is available to run workloads.

In Fabric, this is expressed as Capacity (F-SKU), not as individual VMs you manage.

How Fabric thinks about “nodes”

Fabric abstracts infrastructure. You do not create or size VMs.

Instead:

- You buy a capacity (F-SKU)
- Fabric internally provisions worker nodes for:
 - Spark (Lakehouse, notebooks)
 - SQL (Warehouse, SQL endpoint)
 - Pipelines, Dataflows, BI

So when people say “node size” in Fabric, they usually mean:

How big is the Fabric capacity backing my workloads?

Fabric Capacity (F-SKU) = Node Size Equivalent

Fabric uses F-SKUs to define compute size.

Common sizes:

C	R	Typic
a	e	al Use
p	l	
a	a	
c	ti	
i	v	
	e	
	S	

**t
y**

**i
z
e**

**F
2
/
F
4**

**V
e
r
y
S
m
a
ll**

**Dev,
POC**

**F
8
/
F
1
6**

**S
m
a
ll**

**Light
workl
oads**

**F
3
2**

**M
e
d
i
u
m**

**Regul
ar
produ
ction**

**F
6
4**

**L
a
r
g
e**

**Heavy
Spark
+ BI**

**F
1
2**

**V
e
r**

**Enter
prise
scale**

8 y
+ L
a
r
g
e

Bigger F-SKU ⇒

- **More CPU cores available**
 - **More memory**
 - **Higher concurrency**
 - **Faster Spark jobs**
-

How this maps to Spark (important)

When you run Spark in Fabric:

- **Fabric allocates executors behind the scenes**
- **Executor count and size depend on:**
 - **Capacity size**
 - **Current load**
 - **Workload type**

You cannot:

- **Choose VM type**
- **Manually set node RAM/CPU like Databricks**

Fabric does it automatically.

Conceptual mapping (for understanding only)

Fabric Capacity (F64)



Multiple internal worker nodes



Spark executors + SQL compute



Your jobs run

You control capacity size, not individual nodes.

Node size vs Workspace (important distinction)

Concept	Meaning
Capacity (F-SKU)	Compute power
Workspace	Logical container

Item (Lakehouse, Pipeline)	Uses comp ute
---	--

Node size	Intern al to capac ity
------------------	---

Many workspaces can share one capacity.

Why Fabric hides node sizes

Because Fabric is:

- SaaS-first
- Opinionated
- Simplified

Microsoft wants you to think in terms of:

- Performance
- Cost
- Concurrency

Not VM tuning.

How to choose the right “node size”

Choose based on:

- Spark job complexity
- Data volume
- Concurrency
- BI usage

Rules of thumb:

- ETL + Spark → at least F16–F32
 - Heavy Spark + BI → F64+
 - Learning / testing → F2–F4
-

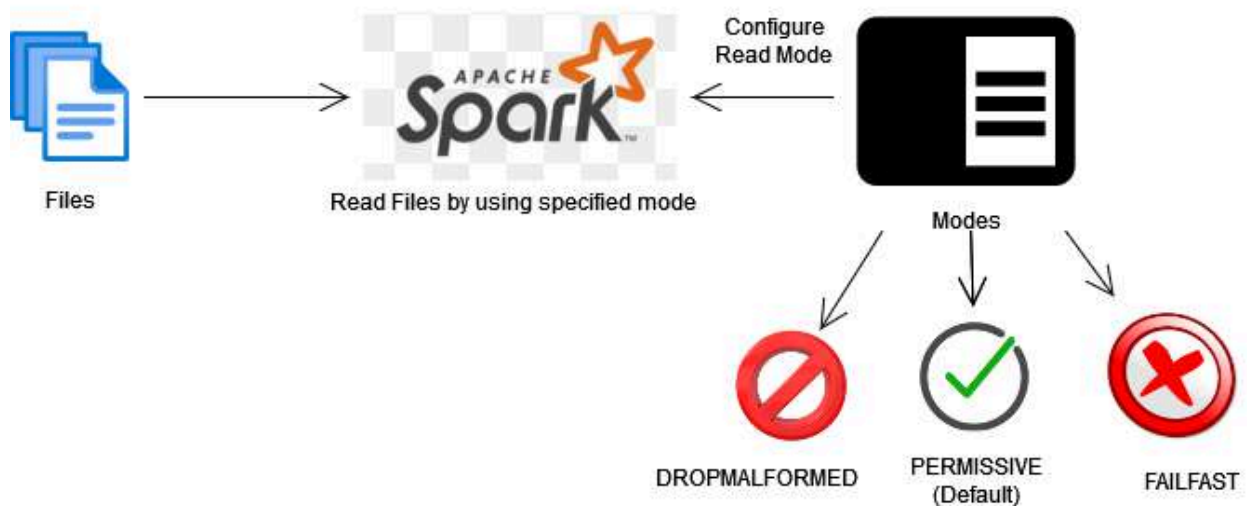
Key exam / interview points

- Fabric does not expose VM/node sizing
 - Compute is controlled via capacity
 - Node size is abstracted
 - Scaling = change capacity SKU
-

One-line memory lock

In Microsoft Fabric, “node size” is abstracted and controlled by capacity (F-SKU), not by individual machines you configure.

PySpark Data Write Modes — Complete Notes (with clear scenarios)



Overview

What “write mode” means (1 line)

A write mode tells Spark what to do if data already exists at the target location or table.

Syntax:

```
df.write.mode("<mode>").format("<format>").save("<path>")
```

The 4 Write Modes in PySpark

1 overwrite (MOST USED)

What it does

- Removes existing data
- Writes new data
- For Delta, creates a new table version

```
df.write \

.mode("overwrite") \

.format("delta") \

.save("/Tables/orders")
```

Behavior by target

Target	Result
Files (CSV/Parquet)	Old files deleted
Delta table	Old files logically removed, new version created

When to use

- Full refresh
- Rebuild table
- Initial loads

 Risky if used accidentally.

2 append

What it does

- Adds new data
- Keeps existing data

```
df.write \

.mode("append") \

.format("delta") \

.save("/Tables/orders")
```

Behavior by target

Targ et	Result
Files	New files added
Delt a tabl e	New files added, new version

When to use

- Incremental loads
- Daily batches
- Streaming sinks

⚠ Schema must match (unless evolution enabled).

3 errorIfExists (default)

What it does

- Fails if data already exists
- Prevents accidental overwrite

```
df.write \  
  .mode("errorIfExists") \  
  .format("parquet") \  
  .save("/Files/raw/orders")
```

Behavior

- If path/table exists → job fails
- If not → data written

When to use

- Safety-first pipelines
 - One-time loads
-

4 ignore

What it does

- Skips write if data exists
- No error

- No overwrite

```
df.write \

.mode("ignore") \

.format("parquet") \

.save("/Files/raw/orders")
```

Behavior

- Existing data remains unchanged
- Spark silently exits

When to use

- Idempotent pipelines
- Avoid duplicate loads

Mode behavior summary (lock this)

Mode	If target exists	Result
overwrite	Yes	Delete & rewrite
append	Yes	Add data

errorIf Exists	Yes	Fail
ignore	Yes	Do nothing

Important Delta-specific behavior

Overwrite + Delta ≠ physical delete

- Delta marks old files as removed in `_delta_log`
- Enables time travel

```
SELECT * FROM orders VERSION AS OF 1;
```

Append + Delta

- Adds files
 - Creates new version
 - Keeps full history
-

Common interview traps (read carefully)

Trap 1: overwrite on Delta ≠ truncate

- It is versioned
- Old data still accessible

Trap 2: append with schema mismatch

- Fails unless schema evolution enabled

```
.option("mergeSchema", "true")
```

Trap 3: ignore ≠ success

- Pipeline may “succeed” but write nothing

Typical real-world patterns

Full refresh (Gold layer)

```
.mode("overwrite")
```

Incremental load (Silver layer)

```
.mode("append")
```

Safe one-time ingestion

```
.mode("errorIfExists")
```

Idempotent landing zone

```
.mode("ignore")
```

Mental model (never forget)

- `overwrite` → replace truth
- `append` → extend truth
- `errorIfExists` → protect truth
- `ignore` → skip truth

One-line memory lock

Write mode controls how Spark behaves when data already exists—`overwrite` replaces, `append` adds, `errorIfExists` fails, `ignore` skips.

End.

Writing Data in Managed vs External Tables (PySpark) — Explained using your exact examples

MANAGED vs EXTERNAL TABLE

MANAGED TABLES



- Data and metadata managed by the system
- Dropping table deletes the data
- Stored in default warehouse/data lake path

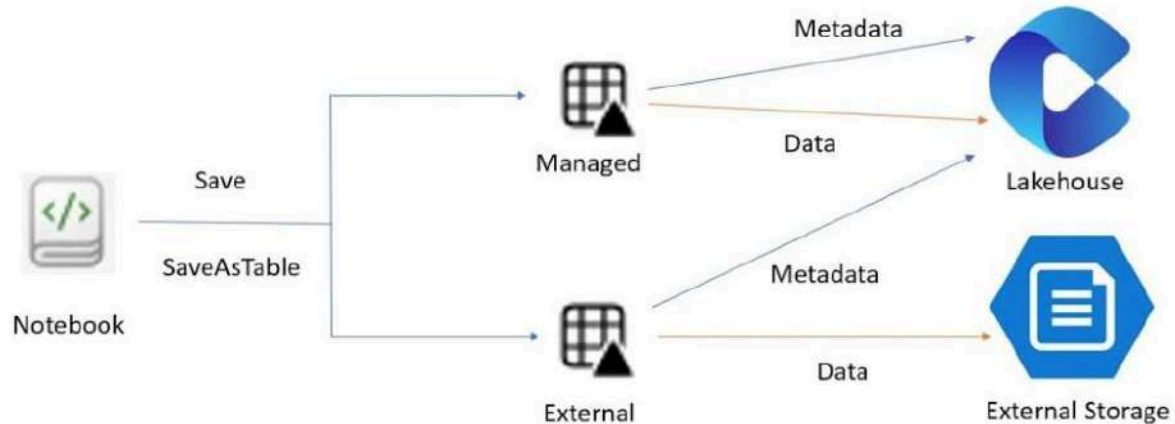
EXTERNAL TABLES



- Data location managed by you
- Dropping table keeps the data
- Can reside in ADLS, S3, etc.



Mastering Managed and External Tables in Microsoft Fabric Delta Lake



insertInto	saveAsTable
Table must exist before writing dataframe	Table will be created if not exist
ignores the column names and just uses position-based resolution	Uses column-name based resolution
It requires that the schema of the DataFrame is the same as the schema of the table	If saveMode is append dataframe schema should match with table but if saveMode is overwrite it's not required
insertInto does not support bucketBy	saveAsTable supports bucketBy
saveMode overwrite will truncate and inserts the data	saveMode overwrite drop and recreate the table to insert the data
Because it inserts data to an existing table, format or options will be ignored	If you try to change the format while writing dataframe to table in append mode, it will result in errors
If the table is partitioned, while writing dataframe partitionBy method should not be used	partitionBy method must be used while writing dataframe to partitioned table

First fix the mental model (VERY IMPORTANT)

Spark data writing has two independent decisions:

1. Where is data written physically?
→ Path (Files / Tables / external storage)
2. Who owns the data lifecycle?
→ Managed table vs External table

The method you use (**save** vs **saveAsTable**) decides ownership.

Your 3 examples — explained one by one

◆ Example 1 — Writing CSV to Files (NO table)

```
df_review.write \  
    .mode("overwrite") \  
    .format("csv") \  
    .option("path", "Files/raw_source") \  
    .save()
```

What this does

- Writes CSV files to Files/raw_source
- Creates NO table
- Creates NO catalog object

Result on storage

```
Files/  
└─ raw_source/  
    └─ part-0000.csv  
    └─ part-0001.csv
```

Key characteristics

- ❌ Not a table

- ❌ No schema enforcement
- ❌ No versioning
- ❌ Not queryable via SQL Endpoint

Use case

- Raw ingestion
- Landing zone
- Temporary outputs

♦ Example 2 — Writing CSV as a Managed Table (NOT recommended, but valid)

```
df_review.write \  
    .format("csv") \  
    .mode("overwrite") \  
    .saveAsTable("csv_man_tbl")
```

What Spark does internally

1. Chooses a system-managed location
2. Writes CSV files
3. Registers table in catalog

Result

```
Tables/  
└─ csv_man_tbl/
```


|— part-0000.csv

|— part-0001.csv

Catalog:

csv_man_tbl → managed location

Why this is a managed table

- No path specified
- Spark owns the data
- Table lifecycle = data lifecycle

Drop behavior

```
DROP TABLE csv_man_tbl;
```

- ✗ Data deleted
- ✗ Files deleted
- ✗ Metadata deleted

Important warning ⚠

- Managed tables should not be CSV
- CSV managed tables are fragile
- Delta is preferred

◆ Example 3 — Writing Delta External Table (BEST PRACTICE)

```
df_review.write \
```

```
.format("delta") \  
  
.mode("overwrite") \  
  
.option("path", "Files/ext_table_delta") \  
  
.saveAsTable("ext_delta_tbl")
```

What Spark does internally

1. Writes Delta files to specified path
2. Creates `_delta_log`
3. Registers table pointing to that path

Result on storage

`Files/`

```
└─ ext_table_delta/  
    ├── part-0000.parquet  
    ├── part-0001.parquet  
    └─ _delta_log/
```

Catalog:

`ext_delta_tbl → Files/ext_table_delta`

Why this is an external table

- Explicit `path` provided
- Spark does NOT own the data
- Table is just a pointer

Drop behavior

```
DROP TABLE ext_delta_tbl;
```

- ✓ Data remains
- ✓ Delta log remains
- ✗ Only catalog metadata deleted

Summary table (LOCK THIS)

Code Pattern	Table Type	Format	Ownership	Drop Deletes Data
<code>.save(path)</code>	No table	CSV	User	✗
<code>.saveAsTable()</code> (no path)	Managed table	CSV	Spark	✓
<code>.saveAsTable()</code> + <code>path</code>	External table	Delta	User	✗

Correct usage guidance (INTERVIEW + REAL LIFE)

- ✗ Avoid
- Managed tables with CSV

- CSV for analytics tables

✓ Recommended

- Files + CSV/Parquet → raw layer
 - External Delta tables → curated layer
 - Managed Delta tables → serving layer (optional)
-

Correct enterprise pattern

Raw Ingestion

→ Files (CSV / Parquet)

Transform

→ External Delta Tables

Serve

→ Managed Delta Tables (if needed)

Mental model (never forget)

- `.save()` → writes files only
- `.saveAsTable()` → creates table
- `path + saveAsTable` → external table
- `no path + saveAsTable` → managed table

One-line memory lock

Ownership is decided by `saveAsTable` and `path`, not by file format.

End.

`notebookutils` in PySpark (Microsoft Fabric) — Complete Notes with Practical Examples

What is `notebookutils`

`notebookutils` is a Fabric-provided utility library available inside PySpark notebooks.

Simple definition:

`notebookutils` helps a notebook interact with the Fabric environment — parameters, files, pipelines, secrets, and notebook control.

It is not a Spark feature.

It is Fabric runtime-specific.

Why `notebookutils` is needed

Spark alone can:

- Read/write data
- Transform data

Spark cannot:

- Read pipeline parameters
- Control notebook execution
- Exit notebooks with status

- Access Fabric-linked resources cleanly

`notebookutils` fills this gap.

Main namespaces inside `notebookutils`

`notebookutils`

└─ `notebook`

└─ `fs`

└─ `widgets`

└─ `runtime`

└─ `credentials`

Each serves a different operational purpose.

1 `notebookutils.notebook` (MOST IMPORTANT)

Used to control notebook execution.

✓ `exit()` — return value to pipeline

`notebookutils.notebook.exit("SUCCESS")`

Use case

- Return status to Fabric Data Factory pipeline
- Signal success/failure

Pipeline can read this output.

✓ `run()` — run another notebook

```
result = notebookutils.notebook.run(  
    "Child_Notebook",  
    timeout_seconds=300,  
    arguments={"date": "2024-01-01"}  
)
```

Use case

- Modular pipelines
 - Parent → child notebook execution
 - Parameterized workflows
-

2 `notebookutils.widgets` (parameters inside notebook)

Used to accept inputs from pipelines or users.

Create widget

```
notebookutils.widgets.text("run_date", "2024-01-01")
```

Read widget value

```
run_date = notebookutils.widgets.get("run_date")
```

```
print(run_date)
```

Use case

- Dynamic file paths
 - Date-driven processing
 - Reusable notebooks
-

3 `notebookutils.fs` (file system utilities)

Used for file-level operations in OneLake.

List files

```
notebookutils.fs.ls("Files/raw")
```

Check if path exists

```
notebookutils.fs.exists("Files/raw/orders")
```

Remove files or folders

```
notebookutils.fs.rm("Files/temp", recurse=True)
```

Create directories


```
notebookutils.fs.mkdirs("Files/archive/2024")
```

Use case

- Cleanup temporary files
 - Archive processed data
 - Validate input availability
-

4 `notebookutils.runtime` (execution context)

Used to get runtime information.

Get workspace details

```
notebookutils.runtime.context
```

Provides:

- Workspace ID
- Item name
- Environment details

Use case

- Logging
 - Environment-specific logic
 - Debugging
-

5 `notebookutils.credentials` (secure access)

Used to access managed credentials securely.

Get credential

```
cred = notebookutils.credentials.get("MyCredential")
```

Use case

- Secure access to external systems
 - Avoid hardcoding secrets
-

Typical real-world pipeline pattern

Fabric Data Factory

↓

Notebook (uses widgets)

↓

Process data

↓

```
notebookutils.notebook.exit("SUCCESS")
```

What `notebookutils` is NOT

- Not a Spark API

- Not portable to Databricks / EMR
- Not available outside Fabric

Comparison: Spark vs notebookutils

Capability	S p a r k	noteboo kutils
Data processing	✓	✗
File I/O	✓	✓ (basic ops)
Pipeline interaction	✗	✓
Notebook orchestration	✗	✓
Environment metadata	✗	✓

Interview memory locks

- **notebookutils** = environment control

- Spark = data processing
 - Pipelines talk to notebooks via widgets + exit
 - `run()` enables modular notebook design
-

One-line final lock

`notebookutils` connects Spark notebooks to the Fabric platform for orchestration, parameters, and environment control.

End.

`notebookutils.notebook.run` — Focused Notes (Fabric PySpark)

What is `notebookutils.notebook.run`

`notebookutils.notebook.run` is used to execute one notebook from another notebook in Microsoft Fabric.

Simple definition:

It enables parent–child notebook orchestration with parameters and return values.

Why it exists (problem it solves)

Without `run()`:

- One huge notebook
- Hard to reuse logic
- Difficult error handling
- No modular pipelines

With `run()`:

- Small, reusable notebooks
 - Clear execution order
 - Parameterized execution
 - Controlled success/failure
-

Basic syntax (must remember)

```
result = notebookutils.notebook.run(  
    "ChildNotebookName",  
    timeout_seconds=300,  
    arguments={  
        "run_date": "2024-01-01",  
        "env": "prod"  
    }  
)
```

Parameters explained (VERY IMPORTANT)

1 Notebook name

`"ChildNotebookName"`

- Must exist in the same workspace

- Case-sensitive
-

2 timeout_seconds

```
timeout_seconds=300
```

- Max execution time
 - If exceeded → notebook fails
 - Prevents hung pipelines
-

3 arguments

```
arguments={"key": "value"}
```

- Passed to child notebook
 - Always string key-value pairs
 - Received using widgets
-

How child notebook receives parameters

Child notebook code

```
notebookutils.widgets.text("run_date", "")  
  
run_date = notebookutils.widgets.get("run_date")
```

This value comes from the parent's **arguments**.

How return value works

Child notebook

```
notebookutils.notebook.exit("SUCCESS")
```

Parent notebook

```
status = notebookutils.notebook.run("ChildNotebook", 300, {})
```

```
print(status)    # SUCCESS
```

Returned value is always a string.

Execution flow (mental model)

Parent Notebook

↓ `run()`

Child Notebook

↓ `process`

↓ `exit("value")`

Parent receives value

No shared variables.

Only arguments in, string out.

Error handling behavior

If child fails:

- Parent `run()` throws exception
- Pipeline fails (unless handled)

Typical safe pattern

`try:`

```
    result = notebookutils.notebook.run("ChildNotebook", 300, {})
```

`except Exception as e:`

```
    notebookutils.notebook.exit("FAILED")
```

Common real-world use cases

1 Modular ETL

- Ingest notebook
- Transform notebook
- Validation notebook

2 Reusable logic

- Same child notebook
- Different parameters
- Different datasets

3 Pipeline orchestration

Fabric Data Factory

- Notebook (parent)
 - `notebookutils.notebook.run()`
 - Child notebooks
-

What `run()` does NOT do (important)

- ❌ Does not share Spark session state
- ❌ Does not share DataFrames
- ❌ Does not bypass security
- ❌ Does not run in parallel (single call)

Each notebook runs independently.

Best practices (interview-ready)

- Keep child notebooks small
 - Use `run()` for orchestration, not heavy logic
 - Always use `exit()` explicitly
 - Pass only required parameters
 - Avoid deep nesting (max 2–3 levels)
-

Comparison (quick clarity)

Feature	<code>run()</code>
---------	--------------------

Cross-notebook execution	✓
Parameter passing	✓
Return value	✓ (string only)
Shared variables	✗
Parallel execution	✗

One-line memory lock

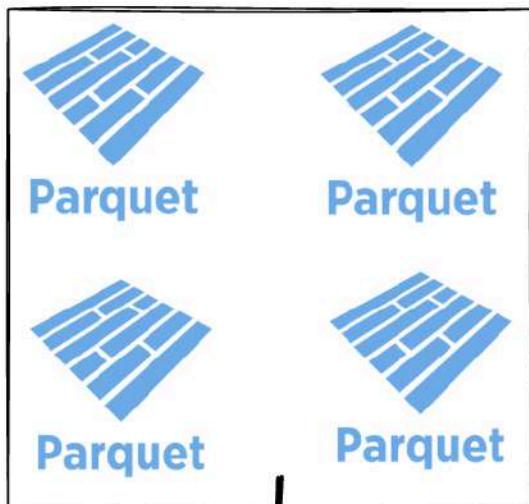
`notebookutils.notebook.run` is used to orchestrate reusable notebooks by passing parameters in and receiving a string result out.

End.

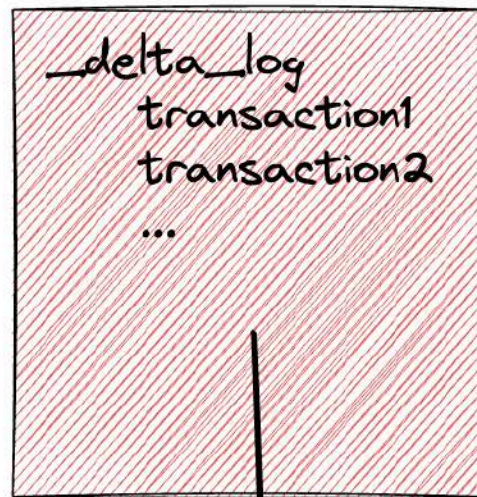
Time Travel in Delta Tables — Complete Notes (concept → internals → examples)



Contents of a Delta table



↓
Data stored
in Parquet files



Transaction log
with metadata

What is Time Travel in Delta Tables

Time Travel is the ability to query a Delta table as it existed at a specific point in time or version.

Simple definition:

Time Travel lets you go back in time and read old versions of a Delta table without restoring backups.

This feature is provided by Delta Lake.

Why Time Travel exists (problem it solves)

Without Time Travel:

- Overwrites permanently delete data
- Debugging data issues is hard
- Accidental updates are costly

With Time Travel:

- Safe overwrites
 - Auditing
 - Debugging
 - Easy rollback
-

How Time Travel works (core idea)

Delta tables never update files in place.

Instead:

- Every write creates new Parquet files
- Delta Log records a new version
- Old files are kept (until cleanup)

Time Travel simply tells Spark:

“Read the table using an older Delta Log version.”

Delta Log's role (critical)

Each write creates a new file in:

_delta_log/

```
|— 000000000000000000.json ← version 0
```

```
|— 000000000000000001.json    ← version 1
```

```
|— 000000000000000002.json    ← version 2
```

Each version records:

- Which files were added
- Which files were removed
- Schema changes (if any)

Ways to use Time Travel

1 Time Travel by VERSION

```
SELECT * FROM orders VERSION AS OF 2;
```

Reads the table exactly as it was in version 2.

2 Time Travel by TIMESTAMP

```
SELECT * FROM orders
```

```

TIMESTAMP AS OF '2024-01-10 10:30:00';

```

Reads the latest version before this time.

③ PySpark example

```
df_old = spark.read \  
    .format("delta") \  
    .option("versionAsOf", 2) \  
    .load("/Tables/orders")
```

Practical example (step-by-step)

Step 1 — Initial load (Version 0)

```
df.write.format("delta").mode("overwrite").save("/Tables/orders")
```

Table data:

id	amount
1	100
2	200

Step 2 — Append data (Version 1)

```
df_new.write.format("delta").mode("append").save("/Tables/orders")
```

Data now:

1	100
2	200
3	300

Step 3 — Overwrite data (Version 2)

```
df_fix.write.format("delta").mode("overwrite").save("/Tables/orders")
```

Current data:

```
1 | 150
```

```
2 | 250
```

Step 4 — Time Travel queries

```
-- Before overwrite
```

```
SELECT * FROM orders VERSION AS OF 1;
```

Returns:

```
1 | 100
```

```
2 | 200
```

```
3 | 300
```

Time Travel for debugging (real use case)

Scenario

- Report shows wrong numbers today

- Yesterday was correct

Solution

```
SELECT * FROM orders VERSION AS OF 10;
```

Compare:

- Yesterday's data
- Today's data

Find exactly what changed.

Time Travel for rollback (safe recovery)

```
CREATE OR REPLACE TABLE orders AS
```

```
SELECT * FROM orders VERSION AS OF 5;
```

This:

- Restores old data
 - Creates a new version
 - Does NOT delete history
-

Retention & cleanup (important)

Delta does NOT keep history forever by default.

Retention settings

- Default data retention: 7 days

- Default log retention: 30 days

Old files are removed using:

VACUUM orders;

 After VACUUM:

- Time Travel to old versions is not possible

What Time Travel does NOT do

-  It is not backup
-  It is not replication
-  It does not protect after VACUUM
-  It does not work on CSV/Parquet-only files

Time Travel works only for Delta tables.

Comparison: Delta vs non-Delta

Feature	Delta Table	CSV / Parquet
Versioning		
Time Travel		

ACID  

Rollback  

Interview memory locks

- Time Travel = read old Delta Log versions
 - Overwrite ≠ delete
 - VACUUM removes history
 - Only Delta tables support Time Travel
-

One-line final lock

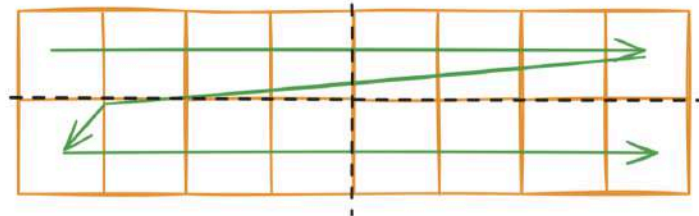
Time Travel in Delta tables lets you query, audit, and restore previous versions of data using the Delta transaction log.

OPTIMIZE command in PySpark / Delta Lake — deep but simple notes

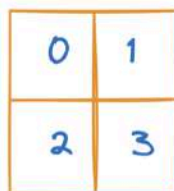




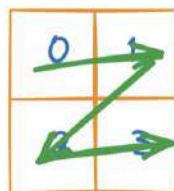
Instead of querying the data linearly ...



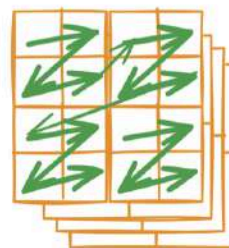
Organize the data via ZOrder ala ...



Organize data
in quadrant
pattern (0,..., 3)

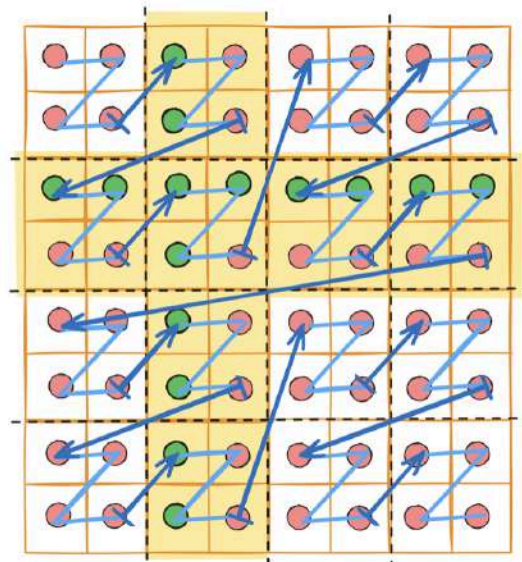
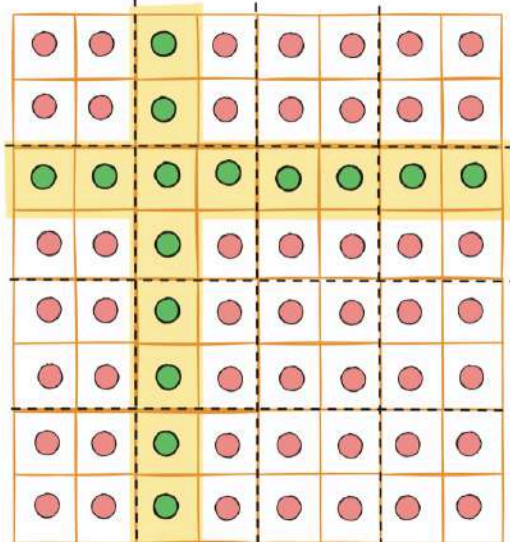


Visit quadrants
in Z-pattern order



...recursively

select * from points where x = 2 or y = 2



What is OPTIMIZE

OPTIMIZE is a Delta Lake command used to improve query performance by rewriting many small Parquet files into fewer large files.

Simple definition:

OPTIMIZE reduces the small-files problem in Delta tables by compacting data files.

It is provided by Delta Lake, not by core Spark.

Why OPTIMIZE is needed (the real problem)

How Delta tables grow normally

Every write operation creates files:

- Append
- Merge
- Streaming
- Incremental loads

Result after many writes:

1000s of small Parquet files

This causes:

- Too many files to open
- High metadata overhead
- Slow query planning
- Poor scan performance

This is called the small files problem.

What OPTIMIZE actually does

Before OPTIMIZE

orders/

└─ part-00001.parquet (5 MB)

└─ part-00002.parquet (6 MB)

└─ part-00003.parquet (4 MB)

└─ part-00004.parquet (7 MB)

After OPTIMIZE

orders/

└─ part-10001.parquet (512 MB)

Same data.
Fewer files.

Syntax

SQL

```
OPTIMIZE orders;
```

With Z-Ordering

```
OPTIMIZE orders
```

```
ZORDER BY (customer_id);
```

How OPTIMIZE works internally (backend)

Step-by-step backend flow

1. Scan Delta Log
 - Identify active Parquet files
 - Ignore removed files
2. Select candidate files
 - Small files below target size (e.g. < 128MB)
3. Read data
 - Spark reads rows from selected files
4. Rewrite data
 - Writes fewer, larger Parquet files
5. Update Delta Log
 - Old files marked as **removed**
 - New compacted files marked as **added**
 - New table version created

 Old files are NOT deleted immediately (time travel still works).

What OPTIMIZE does NOT do

-  Does not change data values
-  Does not change schema
-  Does not delete history
-  Does not work on CSV/Parquet-only tables

It only reorganizes files.

How OPTIMIZE improves performance

1 Fewer files → faster scans

- Spark opens fewer file handles
- Less metadata load

2 Better parallelism

- Each task processes larger chunks
- Reduced task scheduling overhead

3 Faster query planning

- Smaller file list in Delta Log
 - Faster execution plans
-

OPTIMIZE + ZORDER (advanced performance)

What is Z-Ordering

Z-Ordering physically clusters related data together.

Example:

```
OPTIMIZE orders ZORDER BY (order_date);
```

Result:

- Similar `order_date` values stored together

- Queries with **WHERE order_date = ...** scan less data

Backend behavior

- Rows sorted using a space-filling curve
 - Improves data skipping
-

OPTIMIZE vs VACUUM (common confusion)

Com mand	Purpose
OPTI MIZE	Improve performance
VAC UUM	Remove old files

They solve different problems.

When to run OPTIMIZE (best practice)

Run OPTIMIZE when:

- Table is heavily appended
- Streaming writes are used
- MERGE operations are frequent
- Query performance degrades

Do NOT run:

- After every write
- On tiny tables

Typical enterprise pattern

Daily ingestion

- Delta writes
- OPTIMIZE (weekly / daily)
- VACUUM (monthly)

Interview-ready Q&A

Q: Does OPTIMIZE delete data?

No. It rewrites files and keeps history.

Q: Is OPTIMIZE mandatory?

No. It is a performance tuning step.

Q: Does OPTIMIZE work with non-Delta tables?

No.

Mental model (lock this)

Many small files → OPTIMIZE → Few large files

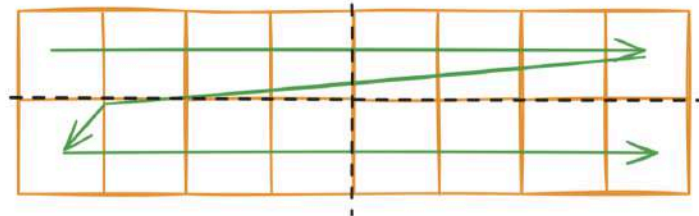
One-line final lock

OPTIMIZE improves Delta table performance by compacting small Parquet files and updating the Delta Log with fewer, larger files.

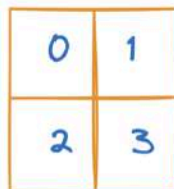
Z-ORDER vs V-ORDER in Delta tables — what they are, how they work, why performance improves



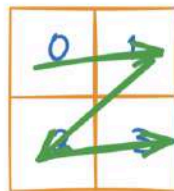
Instead of querying the data linearly ...



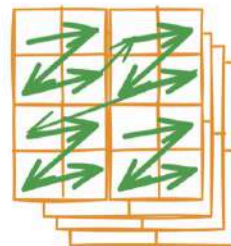
Organize the data via ZOrder ala ...



Organize data
in quadrant
pattern (0,..., 3)

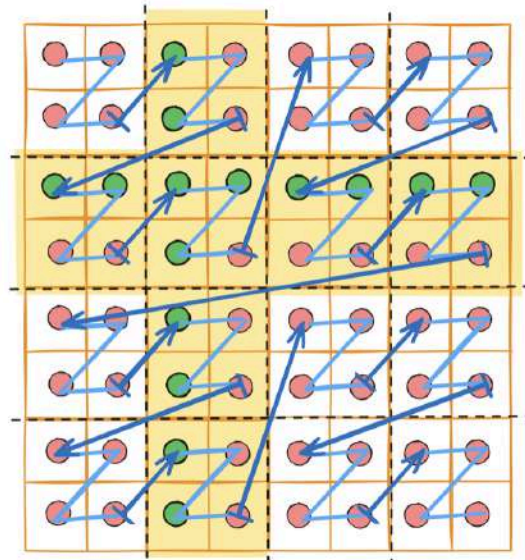
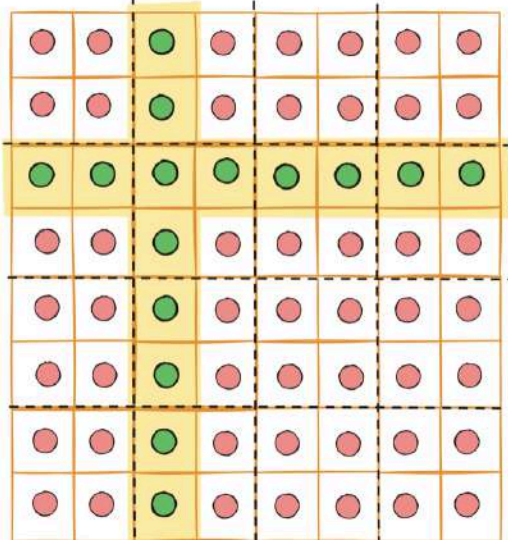


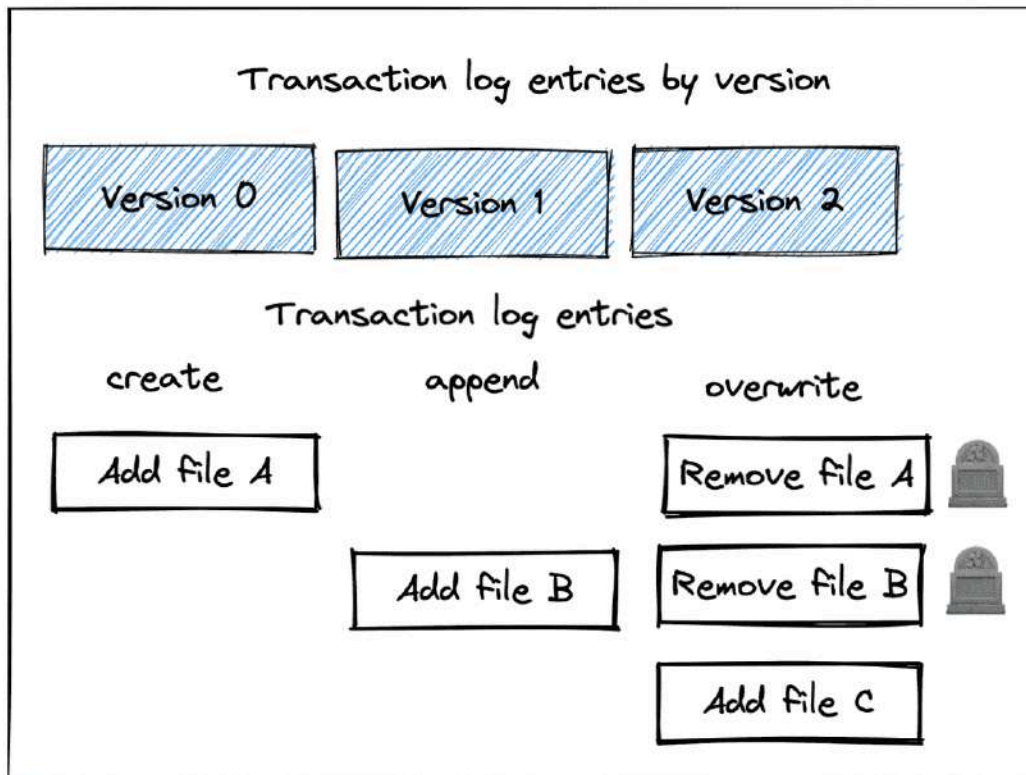
Visit quadrants
in Z-pattern order



...recursively

select * from points where x = 2 or y = 2





1) Z-ORDER — query-time pruning via data clustering

What Z-ORDER is

Z-ORDER is a data-clustering technique in Delta Lake that physically colocates rows with similar values across one or more columns.

Definition:

Z-ORDER rearranges rows so that filters on chosen columns scan less data.

Command

OPTIMIZE orders

```
ZORDER BY (order_date, customer_id);
```

How Z-ORDER works (backend)

1. Reads active Parquet files selected by **OPTIMIZE**
2. Rewrites rows using a space-filling (Z-order) curve
3. Packs related values close together
4. Writes new Parquet files
5. Updates **_delta_log** with a new version

No values change. Only row placement changes.

Why performance improves

- Data skipping: Spark skips entire file blocks when min/max stats don't match filters
- Fewer bytes scanned for selective predicates

Works best with:

- Equality or range filters
 - Columns frequently used in **WHERE**
-

When to use Z-ORDER

- Tables queried by specific columns
- Large fact tables

- BI-style filters (date, id, status)

Avoid when:

- Queries scan most rows anyway
 - Columns are highly random with no selectivity
-

2) V-ORDER — storage layout for fast reads (engine-level)

What V-ORDER is

V-ORDER is a columnar storage layout optimization that improves how Parquet data is written for faster scan performance.

Definition:

V-ORDER optimizes how columns are encoded and laid out inside Parquet files for faster vectorized reads.

Command (write-time)

```
df.write \  
    .format("delta") \  
    .option("delta.enableVOrder", "true") \  
    .mode("overwrite") \  
    .save("/Tables/orders")
```

Or during optimize (platform-managed):

```
OPTIMIZE orders;
```

(In Fabric, V-ORDER is often applied automatically for supported engines.)

How V-ORDER works (backend)

- Writes Parquet with optimized column ordering
- Improves I/O patterns and CPU cache locality
- Enhances vectorized execution

No re-clustering by values; it's about how columns are stored, not where rows go.

Why performance improves

- Faster column scans
 - Better compression
 - More efficient CPU usage
 - Especially beneficial for wide tables
-

When to use V-ORDER

- Default choice for Delta writes in modern engines
- Wide tables
- Analytical scans

No downside for most workloads.

3) Z-ORDER vs V-ORDER — clear difference

Aspect	Z-ORDER	V-ORDER
Purpose	Reduce data scanned by filters	Speed up columnar reads
Acts on	Row clustering by column values	Column storage layout
Applied via	<code>OPTIMIZE ... ZORDER BY</code>	Write/engine optimization
Helps with	Predicate pruning	Scan & CPU efficiency
Needs column choice	Yes	No
Use case	Targeted queries	General performance

4) How they work together

Best practice:

```
OPTIMIZE orders ZORDER BY (order_date);
```

Result:

- `OPTIMIZE` → fewer files
- `Z-ORDER` → rows clustered for filters

- V-ORDER → faster column scans

This combination yields maximum read performance.

5) What they do NOT do

- Do not change data values
 - Do not change schema
 - Do not delete history
 - Do not replace indexing (they complement it)
-

Mental model (lock this)

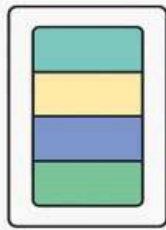
- OPTIMIZE = fewer files
 - Z-ORDER = smarter row placement for filters
 - V-ORDER = faster column reads inside files
-

One-line memory lock

Z-ORDER reduces how much data is read; V-ORDER reduces how fast the data is read.

OPTIMIZE vs OPTIMIZE WRITE vs VACUUM (Delta Lake / Fabric) — deep, clear, backend-level notes

PySpark

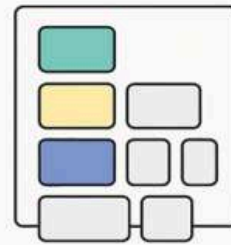


DELTA LAKE

OPTIMIZE

Compacts small files into larger ones

```
spark.sql("OPTIMIZE  
delta.'/path/to/table'")
```



VACUUM

Removes old, unused files

```
spark.sql("VACUUM  
delta.'/path/to/table'")
```



Should VACUUM be used to clean or preserve data in Delta Lake?

Preserve Data

Retaining data ensures historical analysis and recovery options are available.



Clean Data

Using VACUUM can reduce storage costs and improve query performance by removing outdated data.

First, lock the big picture

These three commands solve three different problems:

Command	Solves	When it runs
OPTIMIZE	Too many small files (after data is written)	Post-write
OPTIMIZE WRITE	Prevents small files while writing	During write
VACUUM	Removes old, unused files	Cleanup

They are not interchangeable.

1 OPTIMIZE — *Post-write file compaction*

What OPTIMIZE is

OPTIMIZE is a Delta Lake maintenance command that rewrites many small Parquet files into fewer large files.

Definition:

OPTIMIZE improves read performance by compacting files after data is already written.

Command

OPTIMIZE orders;

With clustering:

```
OPTIMIZE orders ZORDER BY (order_date);
```

What happens at backend (step-by-step)

1. Reads Delta Log
2. Finds active Parquet files
3. Selects small files (< target size, e.g. 128MB)
4. Reads their data using Spark
5. Writes fewer, larger Parquet files
6. Updates Delta Log:
 - Old files → **removed**
 - New files → **added**
7. Creates a new table version

⚠ Old files still exist physically (for time travel).

What OPTIMIZE does NOT do

- ❌ Does not delete history
 - ❌ Does not remove old files
 - ❌ Does not change data values
 - ❌ Does not change schema
-

Why OPTIMIZE improves performance

- Fewer files → fewer file opens
 - Smaller Delta Log metadata
 - Faster query planning
 - Better scan throughput
-

2 OPTIMIZE WRITE — *Prevent small files at write time*

What OPTIMIZE WRITE is

Optimize Write is a write-time optimization, not a SQL command.

Definition:

Optimize Write tries to write well-sized files from the beginning, avoiding the small-files problem.

How it is enabled

In PySpark:

```
df.write \  
    .format("delta") \  
    .option("optimizeWrite", "true") \  
    .mode("append") \  
    .save("/Tables/orders")
```

(In Fabric, this is often enabled by default.)

What happens at backend

During the write:

1. Spark buffers data across partitions
2. Combines small output partitions
3. Writes fewer, larger Parquet files
4. Updates Delta Log normally

Key difference:

- No rewrite
 - No second pass
 - Happens once, during write
-

What OPTIMIZE WRITE does NOT do

- ✗ Does not reorganize existing files
- ✗ Does not Z-ORDER data
- ✗ Does not clean old files

It only affects new writes.

When OPTIMIZE WRITE is useful

- Streaming ingestion
 - Frequent appends
 - Incremental pipelines
-

3 VACUUM — *Physical cleanup of old files*

What VACUUM is

VACUUM is a cleanup command that permanently deletes old Parquet files that are no longer referenced by the Delta Log.

Definition:

VACUUM frees storage by deleting obsolete files after retention period.

Command

VACUUM orders;

With custom retention (DANGEROUS):

VACUUM orders RETAIN 0 HOURS;

What happens at backend

1. Reads Delta Log
2. Identifies files marked as **removed**
3. Checks retention threshold
4. Deletes files physically from storage

After VACUUM:

- Files are gone forever
 - Time travel beyond retention is impossible
-

Default retention rules

Item	Default
------	---------

Data file	7 days
-----------	--------

Delta log	30 days
-----------	---------

These protect against accidental data loss.

What VACUUM does NOT do

- ❌ Does not improve query speed directly
- ❌ Does not compact files
- ❌ Does not change Delta versions

It only reclaims storage.

4 OPTIMIZE vs OPTIMIZE WRITE vs VACUUM (side-by-side)

Feature	OPTIMIZE	OPTIMIZE WRITE	VACUUM
When	After write	During write	After write
Purpose	File compacti on	Avoid small files	Delete old files
Reads data again	✓	✗	✗
Rewrites files	✓	✗	✗
Deletes files	✗	✗	✓
Affects old data	✓	✗	✓ (cleanup)
Improves query speed	✓	Indirect	✗

5 How they work together (REAL production pattern)

Data ingestion

→ Optimize Write (during append)

→ OPTIMIZE + Z-ORDER (periodic)

→ VACUUM (scheduled cleanup)

Each step has one responsibility.

6 Common interview traps (important)

Trap 1

“OPTIMIZE deletes data”

✗ False — only VACUUM deletes data

Trap 2

“Optimize Write replaces OPTIMIZE”

✗ False — Optimize Write only affects new data

Trap 3

“VACUUM improves performance”

✗ False — it saves storage, not query time

Mental model (lock this permanently)

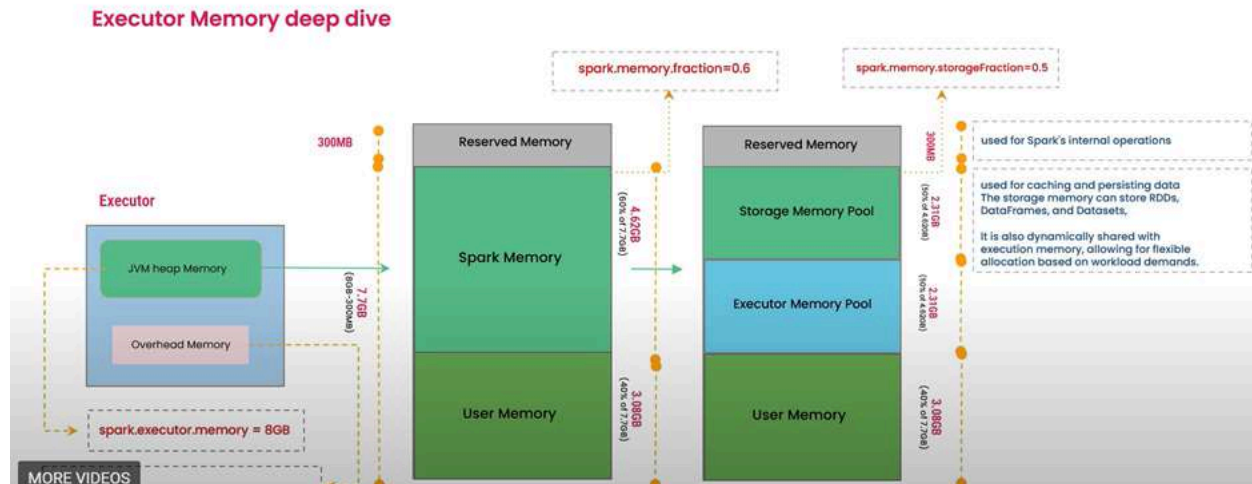
- Optimize Write → *write better*
 - OPTIMIZE → *fix past writes*
 - VACUUM → *clean garbage*
-

One-line final memory lock

Optimize Write prevents small files, OPTIMIZE fixes existing small files, and VACUUM permanently deletes old files no longer needed.

End.

Virtual Environment (Virtual Env) in Spark — Microsoft Fabric



Workspace settings

- General
- License info
- Azure connections
- System storage
- Git integration
- OneLake
- Workspace identity
- Network security

- Power BI
- Data Engineering/Science
- Spark settings**
- Data Factory

Spark settings

Configure and manage settings for Spark workloads and the default environment for the workspace.

Pool **Environment** Jobs High concurrency Automatic log

Set default environment

☐ Off

The default environment will provide Spark properties, libraries, and developer settings for notebooks and Spark job definitions in this workspace when users don't select a different environment.

[Learn more about Set default environment](#)

Runtime

Runtime Version

Runtime version defines which version of Spark your Spark pool will use.

[Learn more about Runtime Version](#)

1.2 (Spark 3.4, Delta 2.4)

Save

Discard

Workspace settings

Search

About

Premium

Azure connections

System storage

Git integration

Other

Power BI

Use the automatically created starter pool or create custom pools for workspaces and items in the capacity. If the setting Customize compute configurations for items is turned off, this pool will be used for all environments in this workspace.

StarterPool

Pool details

Node family	Node size	Number of nodes
Memory optimized	Medium	1 - 1

Customize compute configurations for items

When turned on, users can adjust compute configuration for individual items such as notebooks and Spark job definitions.

Learn more about Customize compute configurations for items

First, the straight answer

A Virtual Environment in Spark (Fabric) is a library-isolated Python environment that lets you control which Python packages and versions your Spark jobs use.

Even though a workspace has a default environment, virtual environments exist to solve isolation, reproducibility, and dependency conflicts.

What the default workspace environment is

When you create a Fabric workspace, Microsoft Fabric provides:

- A shared default Python/Spark environment
- Preinstalled libraries (pyspark, pandas, numpy, etc.)
- Same environment for all notebooks (by default)

Limitation of default environment

- Shared by everyone

- Library upgrades affect others
 - Version conflicts are common
 - Not production-safe
-

What a Virtual Environment is (in Fabric Spark)

A virtual environment is:

- A custom Python runtime
- With your chosen libraries
- Attached to specific notebooks or jobs

Think of it as:

“This notebook runs in *its own Python world*.”

Why virtual environments are needed (core reasons)

1 Dependency conflicts

Example:

- Notebook A needs `pandas==1.5`
- Notebook B needs `pandas==2.0`

Default environment 
Virtual environments 

2 Production stability

Without virtual env:

- Someone upgrades a library
- Your production notebook breaks

With virtual env:

- Your job runs with locked versions
 - No surprise failures
-

3 Reproducibility

You want:

- Same results today
- Same results next month

Virtual env ensures:

- Same Python
 - Same library versions
 - Same behavior
-

4 Multi-team workspace reality

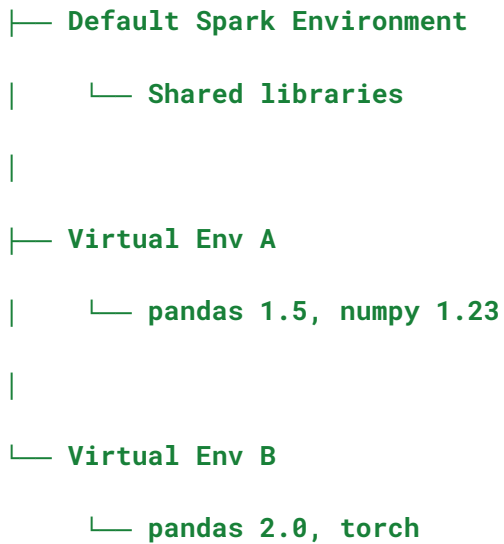
In real enterprises:

- One workspace
- Many teams
- Different stacks

Virtual envs isolate teams without creating new workspaces.

How virtual env works internally (simple)

Workspace



Notebook → attached to one environment only.

What virtual env controls

Item	Controlled by Virtual Env
Python packages	✓
Package versions	✓
Isolation	✓

Spark engine 

Cluster size 

Capacity 

Spark runtime stays same.
Only Python layer changes.

What happens if you don't use virtual env

- Library conflicts
- Unplanned breaking changes
- Difficult debugging
- Unsafe production jobs

Default env is fine only for learning / exploration.

When to use what (clear rule)

Use default environment when

- Learning Fabric
- Quick experiments
- POCs
- Single-user workspace

Use virtual environment when

- Production pipelines
 - Scheduled jobs
 - Multiple notebooks share logic
 - External libraries required
 - Version control matters
-

Common interview question

Q: If workspace already has an environment, why virtual env?

A: Because the workspace environment is shared and mutable, while virtual environments provide isolation, stability, and reproducibility.

Mental model (lock this)

- Workspace env → shared kitchen
 - Virtual env → private kitchen
-

One-line memory lock

Virtual environments in Fabric Spark exist to isolate Python dependencies and ensure stable, reproducible Spark workloads beyond the shared default workspace environment.

Monitoring Spark Jobs & Scheduling Notebooks in Microsoft Fabric — Complete Notes

MI 🔍 🗨 ⋮ 🗑

```

1 //Spark
2 import org.apache.spark.sql.functions.{ countDistinct, col, count, when }
3 val df = spark.range(0, 100000000)
4 df.select(df.columns.map(c => countDistinct(col(c)).alias(c)): _*).collect
5

```

[1] ✓ 19 sec - Apache Spark session started in 10 sec 336 ms. Command executed in 9 sec 378 ms 2:29:17 PM, 7/22/22 Scala ▾

✓ Spark jobs Succeeded 🔍 ⋮

Job runs

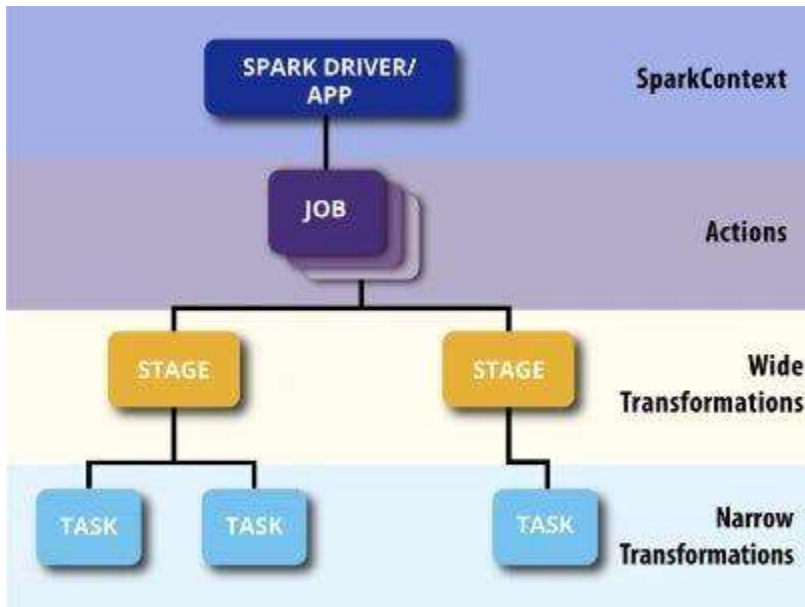
ID	Description	Status	Stages	Tasks	Duration
✓ Job 0	collect at <console>:31	✓ Succeeded	1/1		4 sec
Stage 0	collect at <console>:31	✓ Succeeded	-		4 sec
✓ Job 1	collect at <console>:31	✓ Succeeded	1/1		< 1 ms
Stage 1	collect at <console>:31	⚠ Skipped	-		-
Stage 2	collect at <console>:31	✓ Succeeded	-		< 1 ms

⋮

```

import org.apache.spark.sql.functions.{countDistinct, col, count, when}
df: org.apache.spark.sql.Dataset[Long] = [id: bigint]
res34: Array[org.apache.spark.sql.Row] = Array([100000000])

```





Run Notebook ▾

Trial:
59 days left



Home

Activities

Run

View



Validate



Run



Schedule



View



Notebook



Run Notebook
Sales



General

Settings



Workspace *

A-F2



Refresh

Notebook *

NB - Loading Sales



Refresh



Open



New

> Base parameters

PART A — Monitoring Spark Jobs in Fabric

1) What is a Spark Job (context)

A Spark job is created when an action runs:

- `show()`
- `count()`
- `write()`

Each job is internally broken into:

Job → **Stages** → **Tasks**

Monitoring means tracking how this execution behaves.

2) Where to monitor Spark jobs in Fabric

2.1 Notebook → Spark Monitoring (primary)

When a notebook is running:

- Open the Spark session
- Click Spark monitoring / Spark UI

You can see:

- **Jobs**
- **Stages**
- **Tasks**
- **Executors**

- Storage
-

2.2 What you see inside Spark Monitoring

Jobs tab

- One job per action
- Status: Running / Succeeded / Failed
- Duration

Stages tab

- Shuffle boundaries
- Slow stages indicate performance issues

Tasks tab

- Parallel execution
- Skewed tasks visible here

Executors tab

- Memory usage
 - CPU usage
 - Executor failures
-

3) How to read Spark Monitoring (interview + real use)

Identify slow jobs

- Long-running stages

- High shuffle read/write
- Many failed tasks

Identify skew

- One task much slower than others
- Usually caused by uneven data

Identify memory pressure

- Executor memory near limit
- Frequent recomputation

4) Common Spark job failures (what monitoring shows)

Issue	Visible Symptom
Out of memory	Executor lost
Shuffle issue	Stage retry
Bad schema	Job fails early
Path issue	Job fails at write

Monitoring helps pinpoint exact stage.

5) Logs & error details

Fabric provides:

- Driver logs
- Executor logs
- Stack traces

Always check:

- Which stage failed
 - Which transformation caused it
-

PART B — Scheduling Notebooks in Fabric

Big rule (lock this)

Notebooks are NOT scheduled directly.
They are scheduled via Fabric Data Factory pipelines.

6) Why notebooks are not scheduled directly

Notebooks:

- Do computation
- Should not manage time or retries

Pipelines:

- Handle scheduling

- Handle retries
- Handle dependencies

This separation is intentional.

7) Scheduling a Notebook using Fabric Data Factory

Step-by-step

Step 1 — Create a Pipeline

- Go to Fabric Data Factory
 - Create a new Pipeline
-

Step 2 — Add Notebook Activity

- Drag Notebook activity
 - Select:
 - Workspace
 - Notebook name
-

Step 3 — (Optional) Pass parameters

Notebook:

```
notebookutils.widgets.text("run_date", "")
```

Pipeline:

```
{ "run_date": "2024-01-01" }
```

Step 4 — Add Trigger (Schedule)

Trigger options:

- Daily
- Hourly
- Weekly
- Custom cron

Step 5 — Publish

Once published:

- Notebook runs automatically
- Spark job is created
- Monitoring available

8) Monitoring Scheduled Notebook Runs

Where to monitor

- Pipeline run history
- Notebook activity status
- Spark monitoring for each run

You can trace:

[Trigger](#) → [Pipeline](#) → [Notebook](#) → [Spark Job](#)

9) Retry & failure handling (important)

Pipeline controls:

- Retry count
- Retry interval
- Timeout

Notebook should:

```
notebookutils.notebook.exit("SUCCESS")
```

or fail explicitly.

10) Typical production pattern (best practice)

Trigger (schedule)

↓

Pipeline

↓

Notebook

↓

Spark Job

↓

Delta Write

↓

OPTIMIZE / VACUUM (optional)

Monitoring happens at two levels:

- Pipeline (orchestration)
 - Spark (execution)
-

11) What NOT to do

- ❌ Infinite loops in notebooks
 - ❌ Scheduling notebooks manually
 - ❌ Ignoring Spark UI
 - ❌ Putting orchestration logic inside Spark code
-

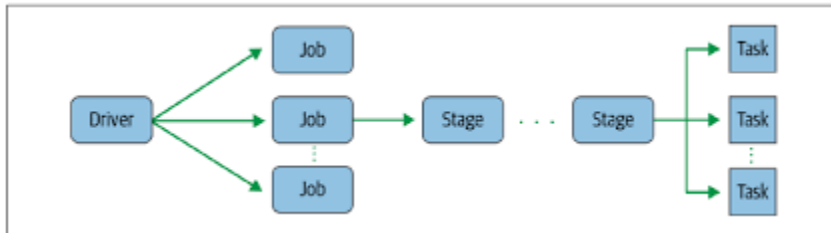
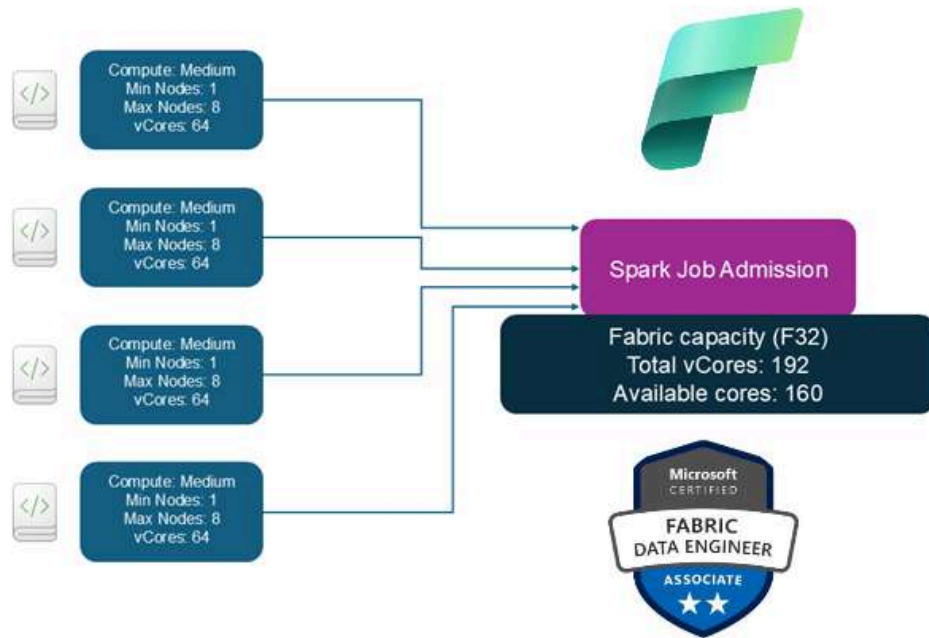
Mental model (lock this)

- Spark Monitoring → *How the job ran*
 - Pipeline Monitoring → *When and why it ran*
-

One-line final memory lock

In Fabric, Spark jobs are monitored through Spark UI inside notebooks, and notebooks are scheduled and controlled using Fabric Data Factory pipelines.

Spark Job Definition in Microsoft Fabric — clear, exact meaning



Short definition (lock this)

A Spark Job Definition in Fabric is a saved, reusable configuration that tells Fabric how to run Spark code without opening a notebook.

It defines:

- What code to run
 - Which Spark environment to use
 - With what parameters
 - How it should execute and be scheduled
-

Why Spark Job Definition exists (problem it solves)

Notebooks are:

- Interactive
- User-driven
- UI-dependent

Production needs:

- Repeatable execution
- Automation
- Scheduling
- No UI dependency

Spark Job Definition = production wrapper around Spark execution.

What a Spark Job Definition contains

A Spark Job Definition is metadata + configuration, not data.

Core components

1. Entry point
 - Python file (`.py`)
 - Notebook
 2. Arguments
 - Runtime parameters
 3. Spark environment
 - Default or Virtual Environment
 4. Compute binding
 - Fabric capacity
 5. Execution settings
 - Timeout
 - Retry
 6. Logging & monitoring hooks
-

What it looks like conceptually

Spark Job Definition

- └─ Code reference (`.py` / notebook)
- └─ Parameters
- └─ Environment (libraries)
- └─ Spark runtime
- └─ Execution rules

How it is different from a Notebook

Aspect	Notebook	Spark Job Definition
Purpose	Interactive analysis	Automated execution
UI required	Yes	No
Scheduling	Indirect (via pipeline)	Direct or via pipeline
Reusability	Medium	High
Production readiness	Medium	High

How a Spark Job Definition runs internally

1. Job definition is triggered (manual / pipeline / schedule)
2. Fabric provisions Spark compute
3. Spark driver starts
4. Code is executed
5. Job status recorded
6. Logs stored for monitoring

Trigger

→ Job Definition

→ Spark Driver

→ Executors

→ Delta / Files

What happens at Spark level (important clarity)

Inside Fabric, a Spark Job Definition still runs standard Spark jobs:

- Actions create Spark jobs
- Jobs split into stages
- Stages split into tasks
- Tasks run on executors

Job Definition ≠ Spark Job (internal)

It is a wrapper that runs Spark jobs.

When to use Spark Job Definition

Use it when:

- Running scheduled ETL
- Running batch processing
- No human interaction needed
- Code is stable and versioned
- CI/CD style execution

When NOT to use it

Avoid when:

- Exploratory analysis
- Debugging logic
- Ad-hoc transformations

Use notebooks instead.

Spark Job Definition vs Pipeline Notebook Activity

Feature	Job Definition	Pipeline Notebook
Execution	Direct Spark	Spark via notebook
UI dependency	None	Notebook
Modularity	High	Medium
Best for	Batch jobs	Orchestration

Monitoring Spark Job Definition

You can monitor:

- Run history
- Status (Success / Failed)
- Duration
- Logs
- Spark UI per run

Same Spark monitoring as notebooks, but headless.

Mental model (do not forget)

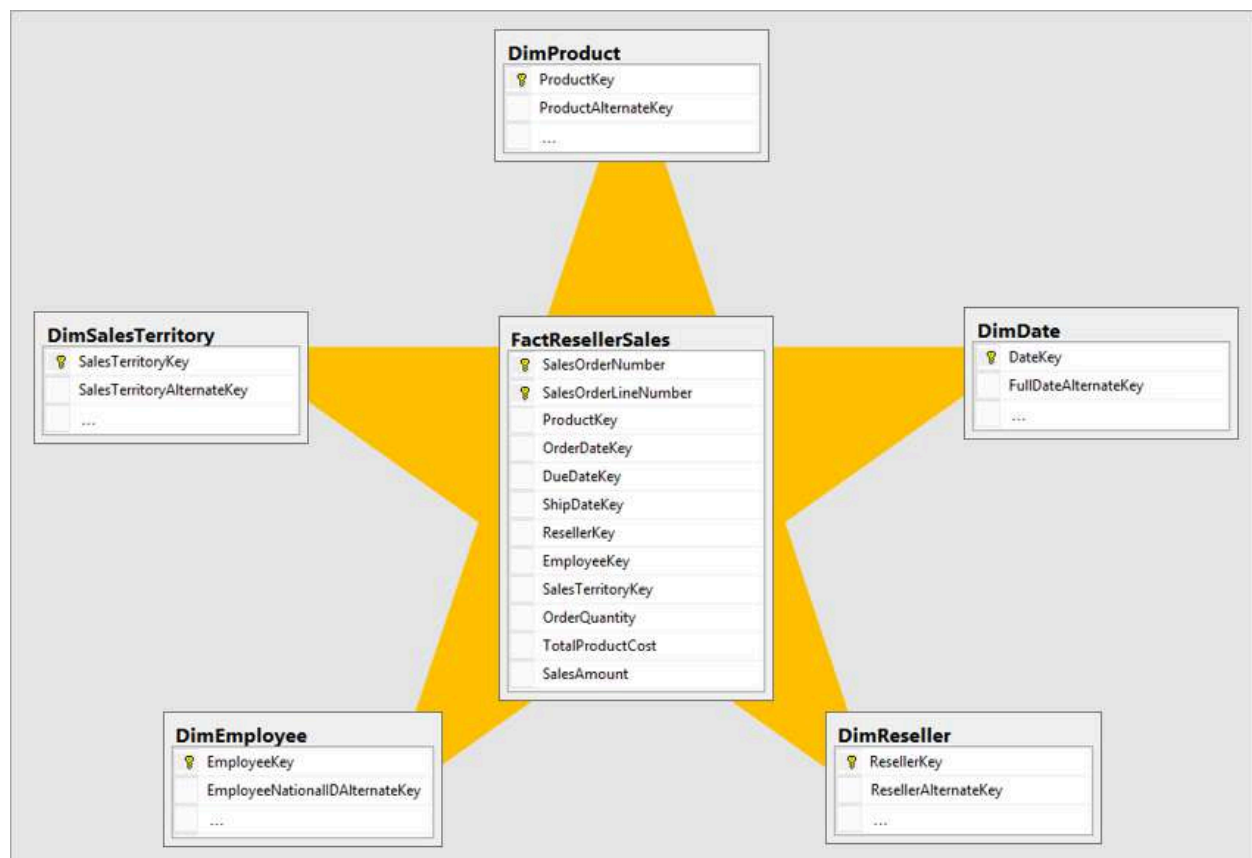
- Notebook = interactive Spark
 - Spark Job Definition = headless Spark
 - Spark Job (internal) = action-triggered execution unit
-

One-line memory lock

A Spark Job Definition in Fabric is a production-grade, headless way to run Spark code with fixed configuration and scheduling.

What is a Data Warehouse, why it exists, and what breaks if you don't have one





1) What is a Data Warehouse

A Data Warehouse is a centralized, structured, SQL-optimized system designed for fast analytics, reporting, and business queries.

Simple definition:

A data warehouse stores clean, structured, business-ready data optimized for SQL queries and BI tools.

In Microsoft Fabric, a Warehouse is:

- SQL-first
 - Schema-enforced
 - BI-ready
 - Optimized for reporting workloads
-

2) Why a Data Warehouse was created (historical reason)

Originally, companies had:

- OLTP systems (ERP, CRM)
- Highly normalized tables
- Transaction-focused design

Problems:

- Reports were slow
- Queries locked production systems
- Business logic was inconsistent
- Complex joins everywhere

Warehouse solved this by separating analytics from operations.

3) What a Warehouse is optimized for

A warehouse is optimized for:

- Read-heavy workloads
- Complex aggregations
- Joins across large tables
- Concurrent BI users
- Consistent business logic

Key characteristics:

- Star / Snowflake schema
 - Fact and dimension tables
 - Predefined schema
 - SQL as first-class citizen
-

4) Why we need a Warehouse (core reasons)

1 Performance for BI

BI tools (Power BI, Tableau) need:

- Low latency
- Predictable response time
- High concurrency

Warehouse provides:

- Query-optimized storage
 - Indexing
 - Caching
 - MPP execution
-

2 Single Source of Truth

Warehouse enforces:

- One definition of revenue
- One definition of customer
- One definition of KPIs

Without it:

- Each report calculates differently
 - Numbers don't match
 - Trust breaks
-

3 Simplicity for analysts

Analysts want:

```
SELECT SUM(sales)

FROM fact_sales

WHERE year = 2024;
```

Not:

- JSON parsing
- Schema-on-read
- File handling
- Spark jobs

Warehouse abstracts complexity.

4 Governance & security

Warehouse provides:

- Column-level security
- Role-based access
- Auditing
- Data masking

Critical for enterprise compliance.

5) What happens if you don't have a Warehouse

This is the most important part.

✗ Issue 1 — BI directly on raw data

If BI hits:

- CSV files
- Raw Parquet
- Semi-structured data

Results:

- Slow dashboards
 - Timeouts
 - Inconsistent metrics
 - Poor user experience
-

✗ Issue 2 — Everyone defines logic differently

Without a warehouse:

- One report calculates revenue one way
- Another uses different filters
- Numbers don't match

This creates:

- Business arguments
 - No data trust
 - Decision paralysis
-

✗ Issue 3 — Heavy load on processing engines

If you query:

- Spark
 - Lakehouse files
- for every dashboard refresh:

You get:

- High compute cost
- Unstable performance
- Resource contention

Warehouse offloads this.

✗ Issue 4 — Poor concurrency

Lake-based engines are great for:

- Batch processing

They are not ideal for:

- 100s of BI users
- Concurrent ad-hoc SQL

Warehouse is built for concurrency.

6) Warehouse vs Lakehouse (clear boundary)

Aspect	Warehouse	Lakehouse
Primary use	BI & reporting	Processing & analytics
Interface	SQL-first	Spark-first
Schema	Strict	Flexible
Concurrency	Very high	Medium
Data type	Structured	Structured + semi
Users	Analysts, BI	Engineers, DS

They complement, not replace, each other.

7) Why Fabric still has a Warehouse (even with Lakehouse)

Even though Lakehouse is powerful:

- It is processing-oriented
- It is schema-flexible
- It is engineer-centric

Warehouse exists to:

- Serve BI at scale
- Provide governed SQL access
- Simplify analytics for business users

That's why Fabric has both.

8) Typical modern architecture (Fabric)

Sources

- Files (raw)
- Lakehouse (processing / Delta)
- Warehouse (serving / BI)
- Power BI

Each layer has a clear responsibility.

9) Mental model (lock this)

- Lakehouse = kitchen (prepare data)
- Warehouse = dining table (consume data)
- BI = customer

You don't cook on the dining table.

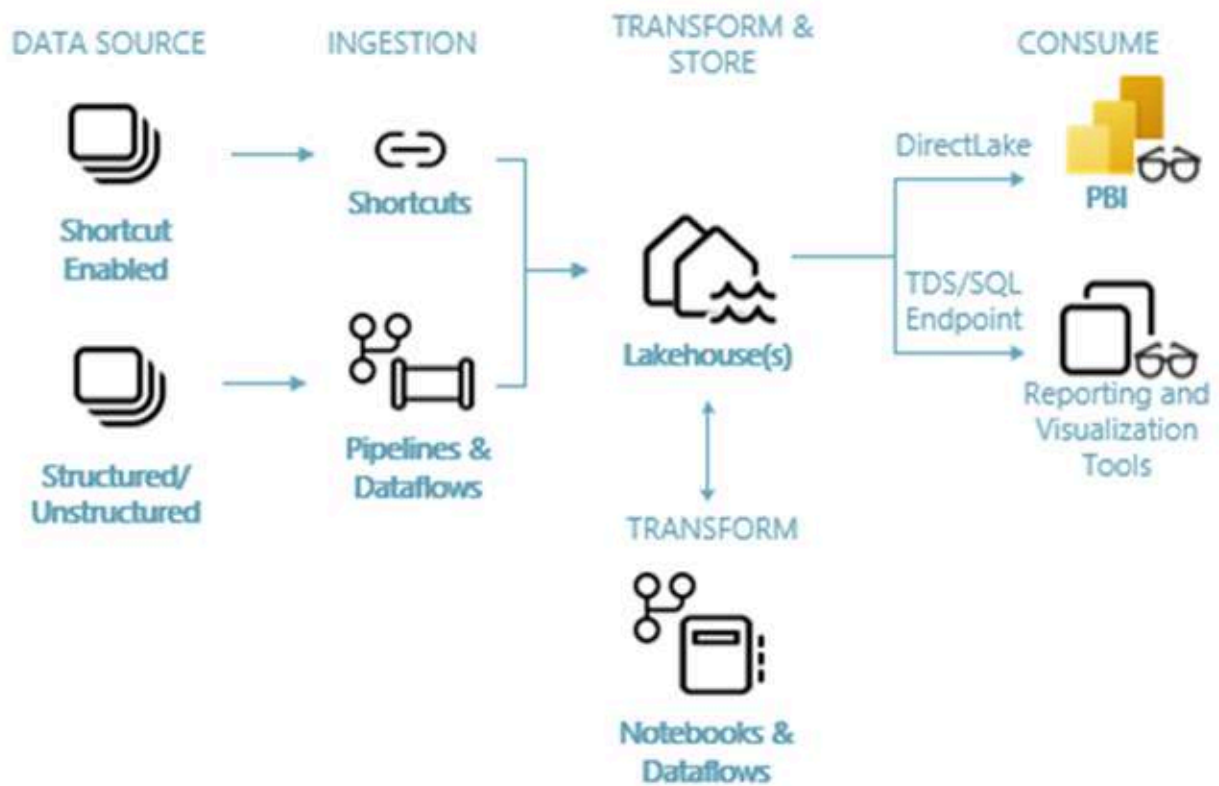
One-line final memory lock

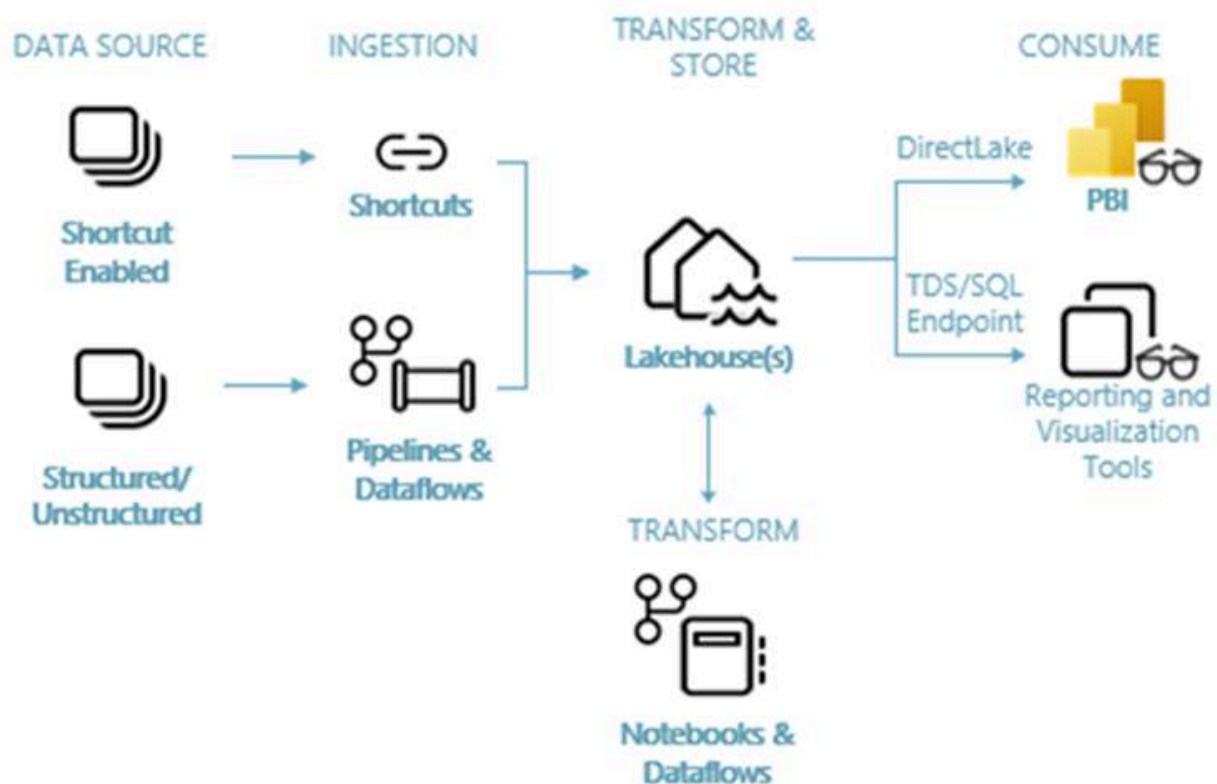
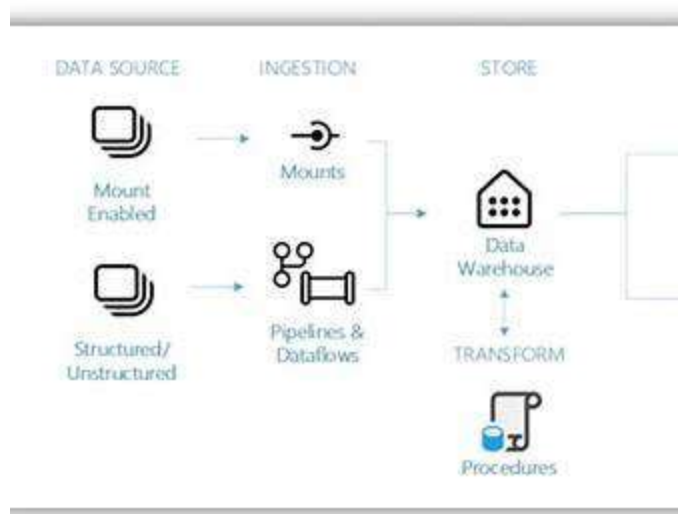
A data warehouse exists to provide fast, trusted, SQL-based analytics; without it, BI becomes slow, inconsistent, and unreliable.

docu

D

Import Lakehouse SQL Endpoint into Warehouse (Microsoft Fabric)





4

1. What is happening conceptually (simple mental model)

- Lakehouse stores data physically in OneLake as Delta/Parquet files.
- Lakehouse SQL endpoint is a read-only SQL layer automatically created on top of Lakehouse tables.

- Warehouse is a T-SQL–first analytical engine optimized for reporting, BI tools, and SQL users.
- Importing Lakehouse SQL endpoint into Warehouse does not copy data.
- Warehouse creates metadata references (shortcuts-like behavior) to Lakehouse tables via the SQL endpoint.

Result:

One physical data copy → two compute engines → shared analytics.

2. Why Fabric allows this

Problem before Fabric:

- Data Lake (files) and Data Warehouse (tables) were isolated.
- Data had to be duplicated and synced.
- Governance, performance tuning, and costs increased.

Fabric solution:

- OneLake + Lakehouse + Warehouse
 - Separate storage from compute
 - Same data accessed via Spark SQL and T-SQL.
-

3. What “import” really means (important)

Term	Reality
Import	Metadata registration
Data movement	✗ No
Storage duplication	✗ No
Performance	Optimized for SQL engine

Consistency	Always reflects Lakehouse data
--------------------	---

Warehouse reads Lakehouse tables through the SQL endpoint layer, not directly from files.

4. Step-by-step: Import Lakehouse SQL endpoint into Warehouse

Step 1: Preconditions

- Lakehouse exists
- Tables exist in Lakehouse (Delta tables)
- Warehouse exists in same Fabric workspace

Step 2: Open Warehouse

- Go to Warehouse
- Click Get data → Lakehouse

Step 3: Select Lakehouse SQL endpoint

- Fabric auto-detects available Lakehouses
- Select the required Lakehouse
- Choose tables or schemas

Step 4: Complete import

- Tables appear inside Warehouse
 - Tables are read-only references
-

5. What gets created in Warehouse

- Table names
- Column schema
- Data types
- Statistics (logical)
- SQL metadata

What does not get created:

- No Parquet files
- No Delta log
- No physical storage

6. How queries work internally

When you run SQL in Warehouse:

1. SQL engine parses T-SQL query
 2. Query planner maps tables to Lakehouse SQL endpoint
 3. Endpoint reads Delta files from OneLake
 4. Results returned to Warehouse engine
 5. BI tools (Power BI) consume results
-

7. Example

Lakehouse table

```
SELECT * FROM sales_lh.customer;
```

Stored as:

- Files: `/Tables/customer/`
- Format: Delta (Parquet + `_delta_log`)

Warehouse query

```
SELECT city, SUM(amount)
```

```
FROM customer
```

```
GROUP BY city;
```

Warehouse:

- Executes T-SQL
 - Reads same Delta data
 - No copy, no sync job
-

8. Why not query Lakehouse SQL endpoint directly?

Lakehouse SQL Endpoint	Warehouse
Read-only	Read + write
Limited SQL features	Full T-SQL
Not ideal for BI governance	Designed for BI
Spark-first	SQL-first
No stored procedures	Stored procedures supported

Warehouse exists to:

- Serve SQL-heavy teams
- Support Power BI Direct Lake
- Provide enterprise SQL semantics

9. Common misconceptions (cleared)

- **✗** Warehouse stores separate data
→ False
 - **✗** Import means copy
→ False
 - **✗** Performance is worse
→ False (SQL engine optimized)
 - **✗** Delta logs duplicated
→ False (single Delta log in Lakehouse)
-

10. When to use what

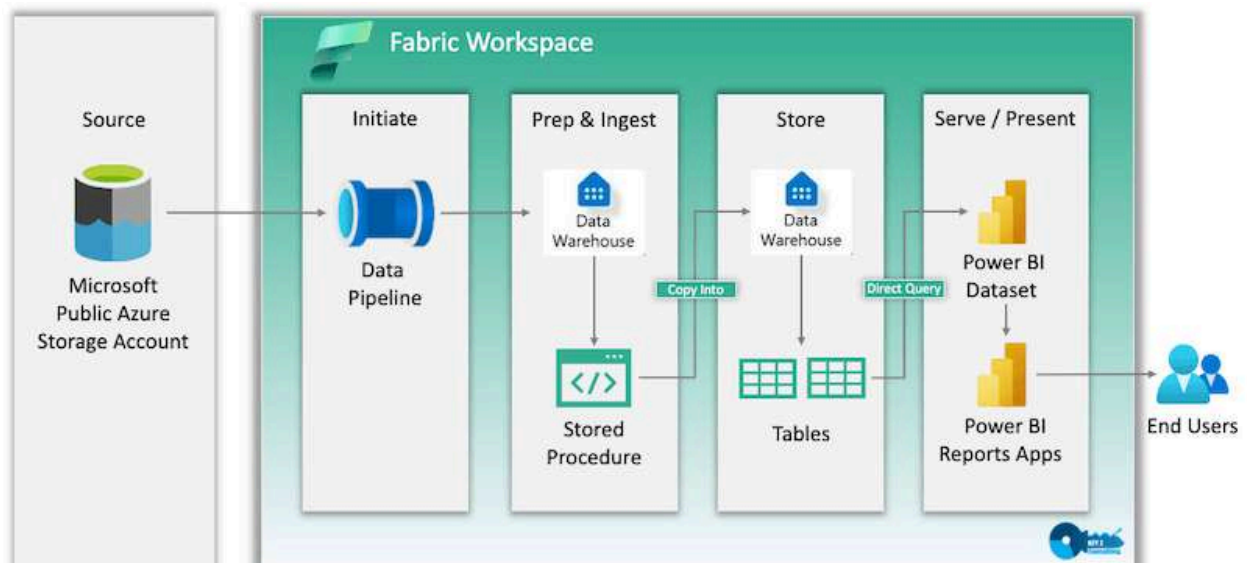
- **Use Lakehouse**
 - Ingestion
 - Transformations
 - Spark workloads
 - Data engineering
- **Use Warehouse**
 - Reporting
 - SQL analytics
 - Power BI
 - Business users
- **Use both together**
 - Engineering in Lakehouse
 - Consumption in Warehouse

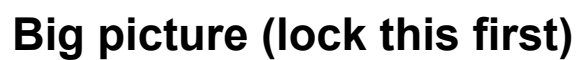
Final mental model (lock this)

- Lakehouse = Data truth (files + Delta log)
- SQL endpoint = SQL window over Lakehouse
- Warehouse = Enterprise SQL engine
- OneLake = Single storage backbone

One dataset. Multiple engines. Zero duplication.

Loading Data into a Warehouse — COPY INTO vs ETL vs CTAS (complete notes)





In Microsoft Fabric Warehouse, data can be loaded using three main methods, each solving a different problem:

1. COPY INTO → fast ingestion from files
2. ETL (Transform + Load) → business logic-driven loading
3. CTAS (Create Table As Select) → SQL-based transformation + load

These are not alternatives; they are complementary.

1 COPY INTO — *Fastest way to load files into Warehouse*

What COPY INTO is

COPY INTO is a high-performance bulk loading command used to load data from external storage or OneLake files directly into Warehouse tables.

Definition:

COPY INTO moves data from files → warehouse tables with minimal transformation.

Syntax example

```
COPY INTO dbo.orders  
FROM 'Files/raw/orders/'  
WITH (  
    FILE_TYPE = 'CSV',  
    FIRSTROW = 2  
);
```

How COPY INTO works (backend)

1. Warehouse engine scans file location

2. Reads data in parallel
3. Applies basic parsing (CSV/Parquet)
4. Loads data into warehouse storage
5. Commits transaction

No Spark.

No complex transformations.

Supported sources

- OneLake (Files)
 - ADLS Gen2
 - Azure Blob
 - S3 (via shortcut)
-

When to use COPY INTO

- Raw ingestion
 - Initial loads
 - Large file-based datasets
 - Minimal transformation needed
-

Limitations

-  No joins
 -  No complex logic
 -  Limited transformations
 -  Schema must mostly match
-

ETL Method — *Transform first, then load*

What ETL means in Fabric Warehouse context

ETL = Extract → Transform → Load, where:

- Transformations happen before data reaches Warehouse
- Warehouse stores clean, curated data

Transformations can be done using:

- Spark (Lakehouse notebooks)
 - Dataflow Gen2
 - Fabric Data Factory
-

Typical ETL flow

Source

- Files (raw)
 - Spark / Dataflow Gen2 (transform)
 - Warehouse (load)
-

Load example (after ETL)

```
INSERT INTO dbo.orders  
  
SELECT *  
  
FROM lakehouse_clean.orders;
```

Why ETL is important

- Business logic enforcement
 - Data quality checks
 - Deduplication
 - Standardization
-

When to use ETL

- Complex transformations
- Multiple sources
- Business rules
- Slowly changing dimensions
- Production-grade pipelines

Trade-offs

- Slower than COPY INTO
- More compute cost
- More moving parts

But much more reliable.

3 CTAS — *Create Table As Select*

What CTAS is

CTAS creates a new warehouse table using the result of a SELECT query.

Definition:

CTAS = create + transform + load in one SQL statement.

Syntax example

```
CREATE TABLE sales_summary
AS
SELECT
    customer_id,
    SUM(amount) AS total_sales
FROM dbo.orders
GROUP BY customer_id;
```

How CTAS works internally

1. SQL engine executes SELECT

2. Results are materialized
3. New table is created
4. Data is written in warehouse storage

No Spark required.

When CTAS is used

- Building aggregated tables
 - Creating fact/dimension tables
 - Gold-layer modeling
 - Fast SQL-based transformations
-

Limitations

- ✗ Cannot update existing table
 - ✗ Table is recreated
 - ✗ Not ideal for incremental loads
-

4 Side-by-side comparison (EXAM + INTERVIEW READY)

Aspect	COPY INTO	ETL	CTAS
Purpose	Fast ingestion	Business transformation	SQL transformation
Source	Files	Any	Tables/views
Transform capability	Low	High	Medium

Performance	Very high	Medium	High
Incremental load	Limited	Yes	No
Complexity	Low	High	Medium
Best layer	Raw → Bronze	Silver	Gold

5 Choosing the right method (decision guide)

Use COPY INTO when

- Data is file-based
 - Schema is stable
 - You want speed
 - No heavy logic required
-

Use ETL when

- Data needs cleaning
 - Business rules apply
 - Multiple joins required
 - Data quality matters
-

Use CTAS when

- Creating new analytical tables
 - Aggregations required
 - SQL-only teams
 - One-time or batch modeling
-

6 Real-world Fabric pattern (best practice)

Files (OneLake)

→ COPY INTO (landing tables)

Lakehouse / ETL

→ Clean + transform

Warehouse

→ CTAS (facts / dimensions)

→ BI consumption

Mental model (lock this)

- COPY INTO → move fast
- ETL → make it right
- CTAS → shape it for analytics

One-line final memory lock

COPY INTO ingests fast, ETL transforms deeply, and CTAS builds analytics-ready tables using SQL.

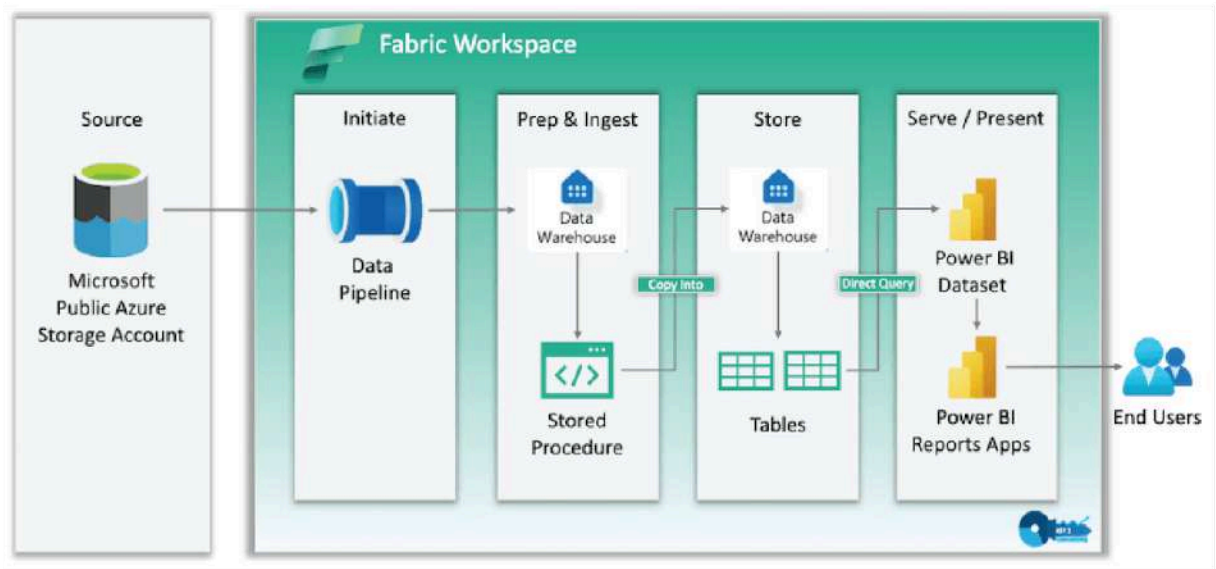
Df

Ff

Zdfghjkb

ZdfghjkbZdfghjkb

Information Views, System Views, and Query Insights Views — Warehouse (Microsoft Fabric)



www.MyTecBits.com.sql - localh...rs (sa) 6 X

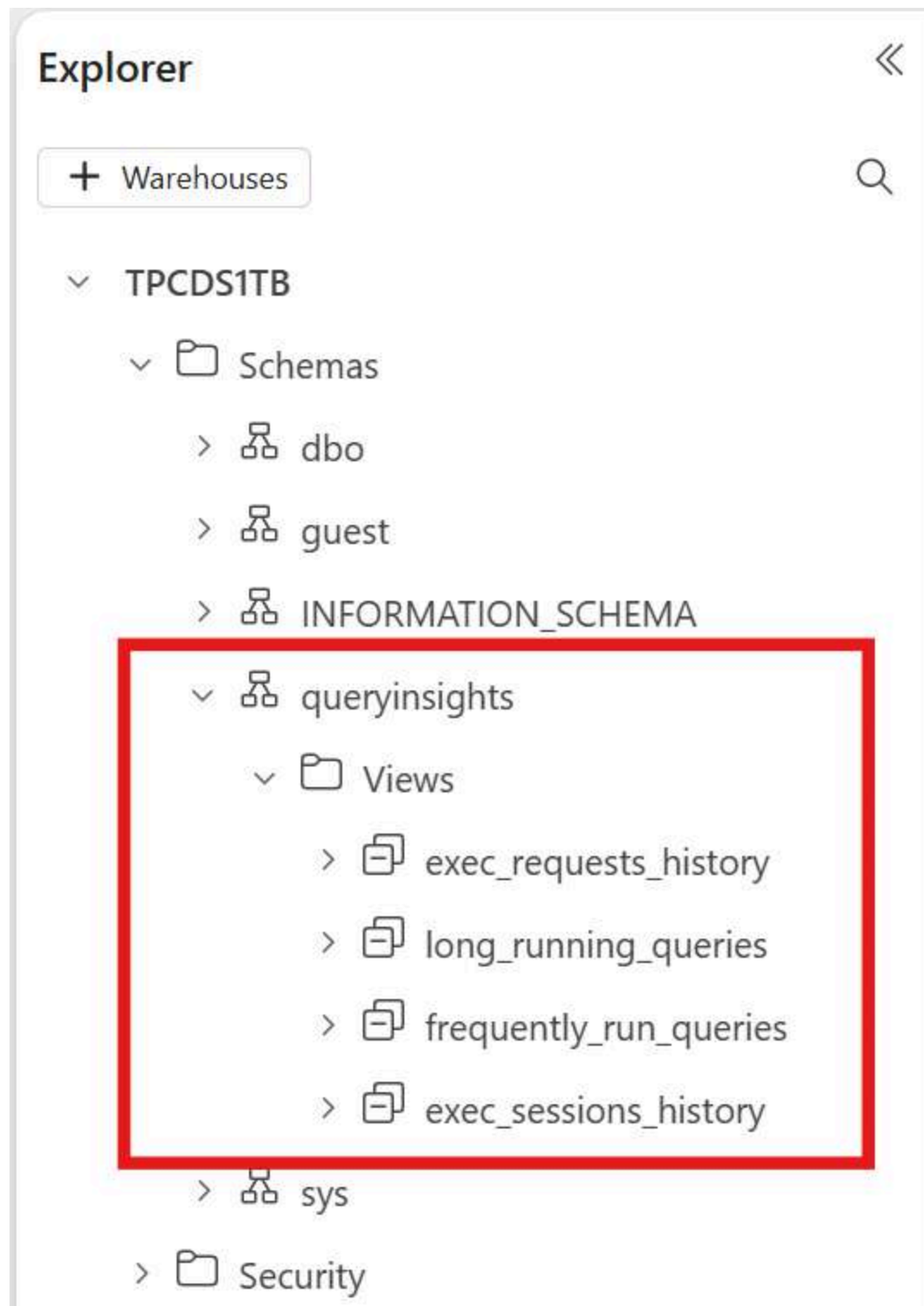
Users > BeaulinTwinkle > Downloads > www.MyTecBits.com.sql

Run Cancel Disconnect Change Connection WideWorldImporters

```
162
163 SELECT * FROM INFORMATION_SCHEMA.TABLES
164 GO
```

Results Messages

	TABLE_CATALOG	TABLE_SCHEMA	TABLE_NAME	TABLE_TYPE
1	WideWorldImporters	Warehouse	Colors	BASE TABLE
2	WideWorldImporters	Warehouse	Colors_Archive	BASE TABLE
3	WideWorldImporters	Sales	OrderLines	BASE TABLE
4	WideWorldImporters	Warehouse	PackageTypes	BASE TABLE
5	WideWorldImporters	Warehouse	PackageTypes_Archive	BASE TABLE
6	WideWorldImporters	Warehouse	StockGroups	BASE TABLE



Big picture (lock this)

In a **Warehouse**, these three view types answer **three different questions**:

- **Information Views** → *What objects exist?*
- **System (SYS) Views** → *How is the warehouse built and behaving internally?*
- **Query Insights Views** → *How are queries performing and being used?*

All are **read-only metadata views**.

1 Information Views (**INFORMATION_SCHEMA**)

What they are

Standard SQL metadata views that describe **logical objects** in the warehouse.

They answer:

What tables, columns, schemas, views exist?

Examples

```
SELECT * FROM INFORMATION_SCHEMA.TABLES;
```

```
SELECT * FROM INFORMATION_SCHEMA.COLUMNS  
WHERE TABLE_NAME = 'orders';
```

What information they expose

- Tables
 - Views
 - Columns
 - Data types
 - Schemas
 - Constraints
-

Characteristics

- ANSI SQL standard

- Portable across SQL engines
 - Business / developer friendly
 - Logical-level metadata
-

When to use

- Schema discovery
 - Data modeling validation
 - Tool integration
 - BI metadata inspection
-

2 System Views (**sys**)

What they are

Engine-specific internal metadata views.

They answer:

How is the warehouse physically and logically implemented?

Examples

```
SELECT * FROM sys.tables;
```

```
SELECT * FROM sys.columns;
```

```
SELECT * FROM sys.schemas;
```

What system views expose

- Internal object IDs
- Storage metadata
- Indexes
- Distribution info
- Dependencies

- Permissions
-

Characteristics

- Vendor-specific
 - More detailed than INFORMATION_SCHEMA
 - Used by DBAs and engineers
 - Reflect engine internals
-

When to use

- Deep troubleshooting
 - Dependency analysis
 - Security audits
 - Advanced optimization
-

3 Query Insights Views (**queryinsights**)

What they are

Performance and usage monitoring views provided by Fabric.

They answer:

Who is running queries, how often, and how expensive they are.

Examples

```
SELECT * FROM queryinsights.exec_requests;
```

```
SELECT * FROM queryinsights.query_texts;
```

```
SELECT * FROM queryinsights.query_stats;
```

What Query Insights expose

- Query text
 - Execution duration
 - CPU usage
 - Data scanned
 - Frequency
 - Success / failure
-

Characteristics

- Observability-focused
 - Performance-centric
 - Used for monitoring and tuning
 - Fabric-specific
-

When to use

- Performance troubleshooting
 - Cost optimization
 - Identifying slow queries
 - Understanding BI workload patterns
-

4 Side-by-side comparison (EXAM READY)

Aspect	INFORMATION_SCHEMA	sys	queryinsights
Purpose	Object metadata	Engine metadata	Query performance
Scope	Logical	Physical + logical	Runtime
SQL Standard	Yes	No	No
Used by	Analysts / Devs	DBAs / Engineers	Platform / Performance teams

Read-only

Yes

Yes

Yes

5 What they do NOT do

-  Do not store data
-  Do not modify warehouse state
-  Do not affect performance directly

They are **observation tools only**.

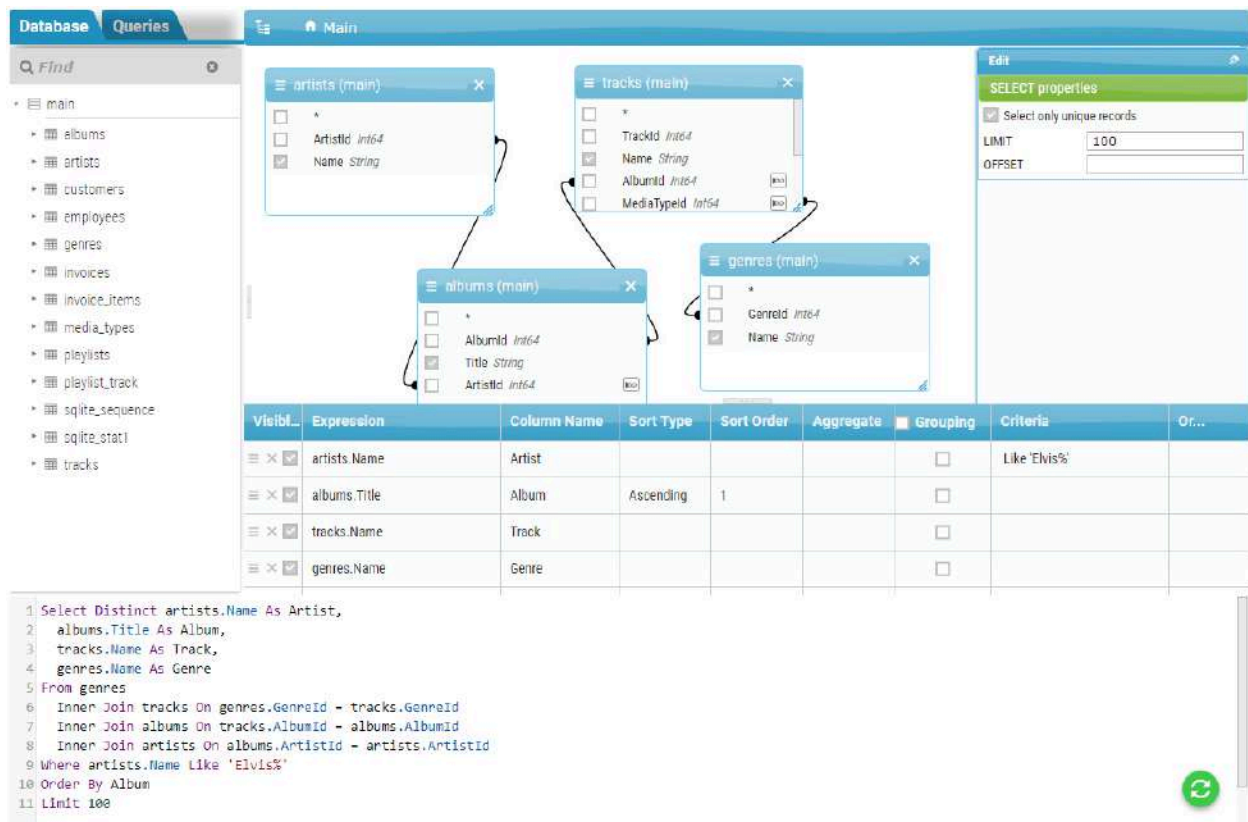
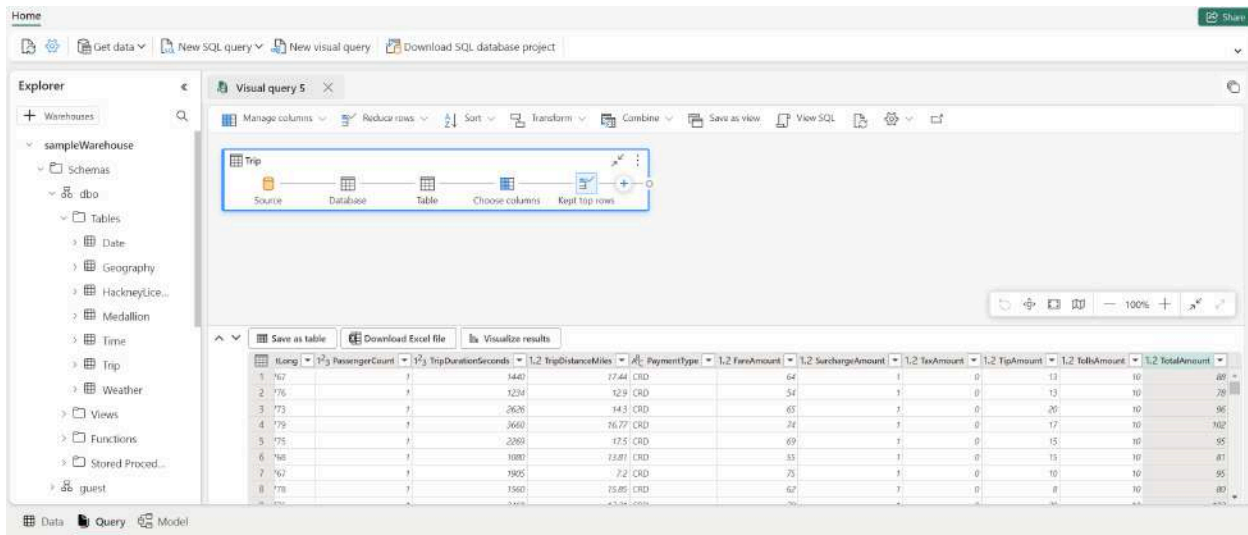
Mental model (lock this permanently)

- **INFORMATION_SCHEMA** → *What exists?*
 - **sys** → *How it exists internally?*
 - **queryinsights** → *How it is being used?*
-

One-line final memory lock

Information views describe objects, system views expose internals, and query insights views reveal query behavior and performance in a Fabric Warehouse.

Visual Query — what it is, why it exists, how it works (Warehouse / Fabric context)



What is a Visual Query

A **Visual Query** is a no-code / low-code way to build SQL queries using a graphical interface instead of writing SQL manually.

Simple definition:

Visual Query lets you create SELECT, JOIN, FILTER, GROUP BY queries by clicking, not typing SQL.

The system **automatically generates SQL** in the background.

Why Visual Query exists (core reason)

Not everyone:

- Knows SQL well
- Wants to write long joins
- Is comfortable with query syntax

Visual Query solves:

- SQL learning barrier
 - Query syntax errors
 - Slow development for simple analytics
-

What Visual Query is built on

Internally:

- You design logic visually
- The engine converts it to **standard SQL**
- SQL runs on the **Warehouse engine**

There is **no separate execution engine**.

What you can do with Visual Query

Using clicks and selections, you can:

- Select tables
- Choose columns
- Apply filters
- Create joins
- Aggregate data (SUM, COUNT, AVG)
- Sort results

Equivalent to writing:

```
SELECT city, SUM(sales)
FROM orders
GROUP BY city;
```

But without typing it.

What Visual Query generates (important)

Visual Query **always produces SQL**.

You can:

- View the generated SQL
- Copy it
- Modify it manually later

This means:

Visual Query ≠ replacement for SQL
Visual Query = SQL generator

What Visual Query is best for

- Simple analytics
 - Ad-hoc exploration
 - Business users
 - Analysts new to SQL
 - Quick validations
-

What Visual Query is NOT good for

- Complex ETL logic
- Stored procedures
- Advanced window functions
- Performance tuning
- Production pipelines

Those require **manual SQL**.

Visual Query vs Writing SQL (clear difference)

Aspect	Visual Query	Manual SQL
Skill needed	Low	Medium–High

Speed (simple queries)	Very fast	Medium
Flexibility	Limited	Full
Debugging	Limited	Full
Production use	Low	High

Visual Query in Warehouse workflow

Typical usage:

Warehouse

- Visual Query (explore)
- Generated SQL
- Save as View
- Use in Power BI

Visual first, SQL later.

Common misconception (important)

- ✗ Visual Query is not a separate storage
- ✗ Visual Query is not a separate engine
- ✗ Visual Query does not bypass SQL

It is **only a UI layer**.

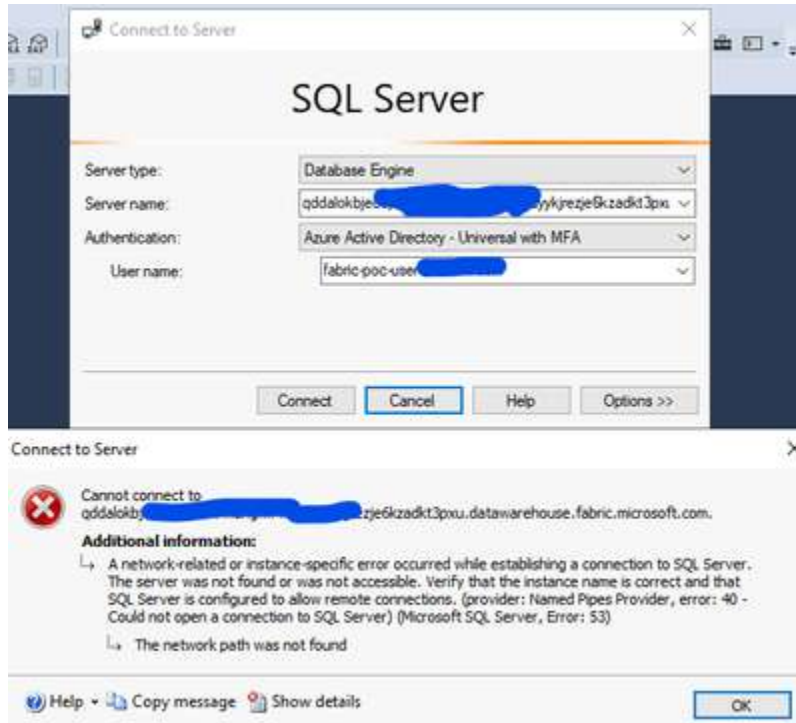
Mental model (lock this)

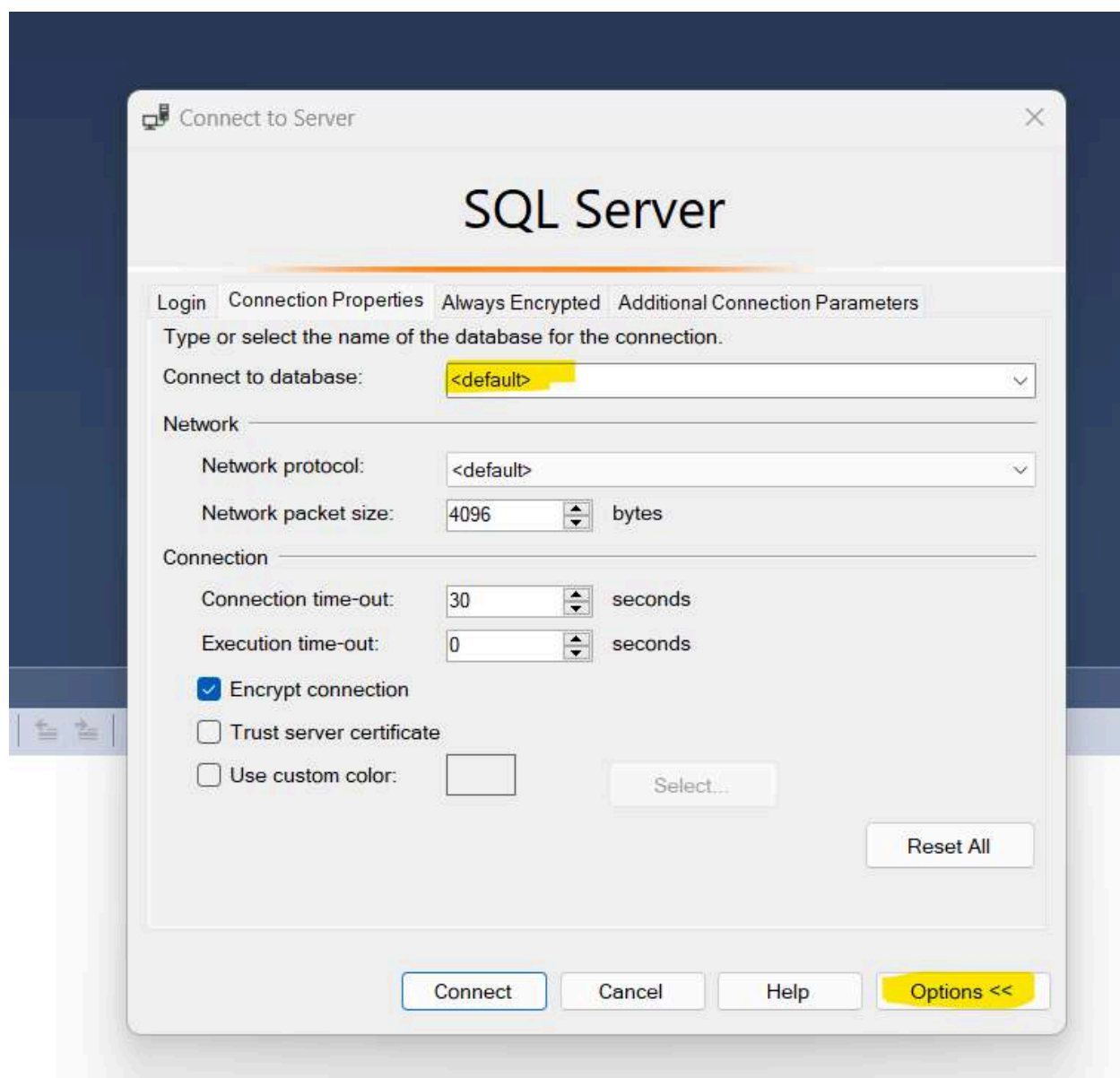
- Visual Query = drag-and-drop SQL builder
 - SQL = actual execution logic
-

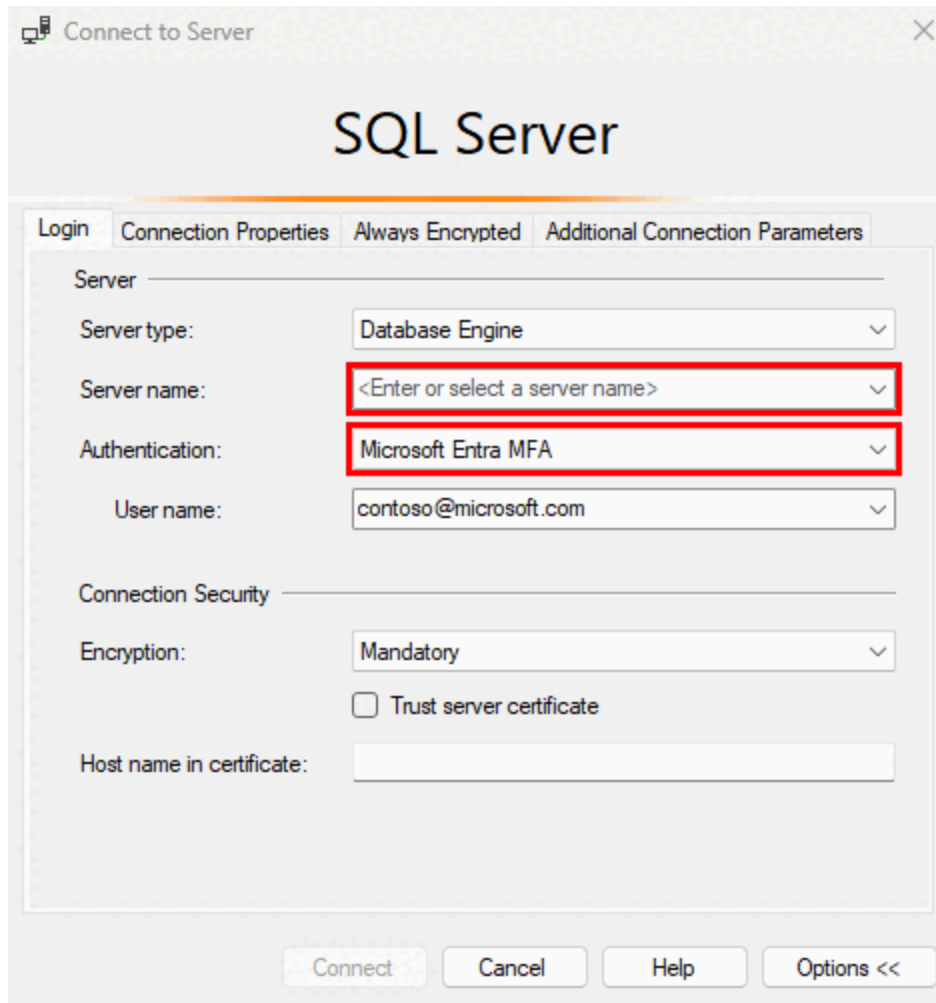
One-line memory lock

Visual Query is a graphical interface that builds SQL queries automatically for you, running them on the warehouse engine.

Integrating Microsoft Fabric Warehouse with SSMS (step-by-step notes)







1) What you are actually connecting to

SSMS does **not** connect to OneLake or files.

SSMS connects to the **SQL endpoint of the Fabric Warehouse**.

Mental model:

SSMS

- T-SQL
 - Fabric Warehouse SQL engine
 - Data stored in OneLake
-

2) Prerequisites (must be true)

- Fabric Warehouse already created
 - You have **access** to the workspace
 - SSMS **v19 or later** installed
 - Microsoft Entra ID (Azure AD) login available
-

3) Get the SQL connection details from Fabric

Steps

1. Open **Fabric Workspace**
2. Open your **Warehouse**
3. Click **Settings / Connection info**
4. Copy:
 - **Server name**
 - **Database name**

It looks like:

`<workspace>.sql.fabric.microsoft.com`

4) Connect from SSMS (exact steps)

Step 1 — Open SSMS

- Click **Connect**
 - Choose **Database Engine**
-

Step 2 — Fill connection details

Field	Value
Server name	<code><workspace>.sql.fabric.microsoft.com</code>
Authentication	Azure Active Directory – Universal with MFA

User	Your Fabric account
Database	Warehouse name (optional)

Step 3 — Sign in

- Complete Microsoft login
- MFA if enabled

Connection succeeds → warehouse appears in Object Explorer.

5) What you can do after connecting

You can:

- Run **SELECT** queries
- Create **views**
- Create **functions**
- Create **stored procedures**
- Use **CTAS**
- Use **COPY INTO**
- Query **sys**, **INFORMATION_SCHEMA**, **queryinsights**

Example:

```
SELECT * FROM sys.tables;
```

6) What you CANNOT do from SSMS

- ❌ Manage Spark
- ❌ Access Files/Lakehouse directly
- ❌ Run Spark SQL
- ❌ Modify OneLake files

SSMS is **SQL-only**.

7) Security & access model

- Authentication → Entra ID (Azure AD)
- Authorization → Fabric workspace roles
- No SQL logins or passwords

If you can't connect:

- Check workspace role
 - Check warehouse permissions
-

8) Common connection errors & causes

Error	Cause
Login failed	No workspace access
Cannot open database	Wrong warehouse name
Timeout	Network / firewall
MFA loop	Wrong auth mode

Always use **AAD Universal with MFA**.

9) When SSMS is useful vs Fabric UI

Use SSMS when:

- Heavy T-SQL development
- Stored procedures
- DBA-style work
- Familiar SQL tooling

Use Fabric UI when:

- Lakehouse work
- Spark notebooks
- Data engineering

- Visual query

They complement each other.

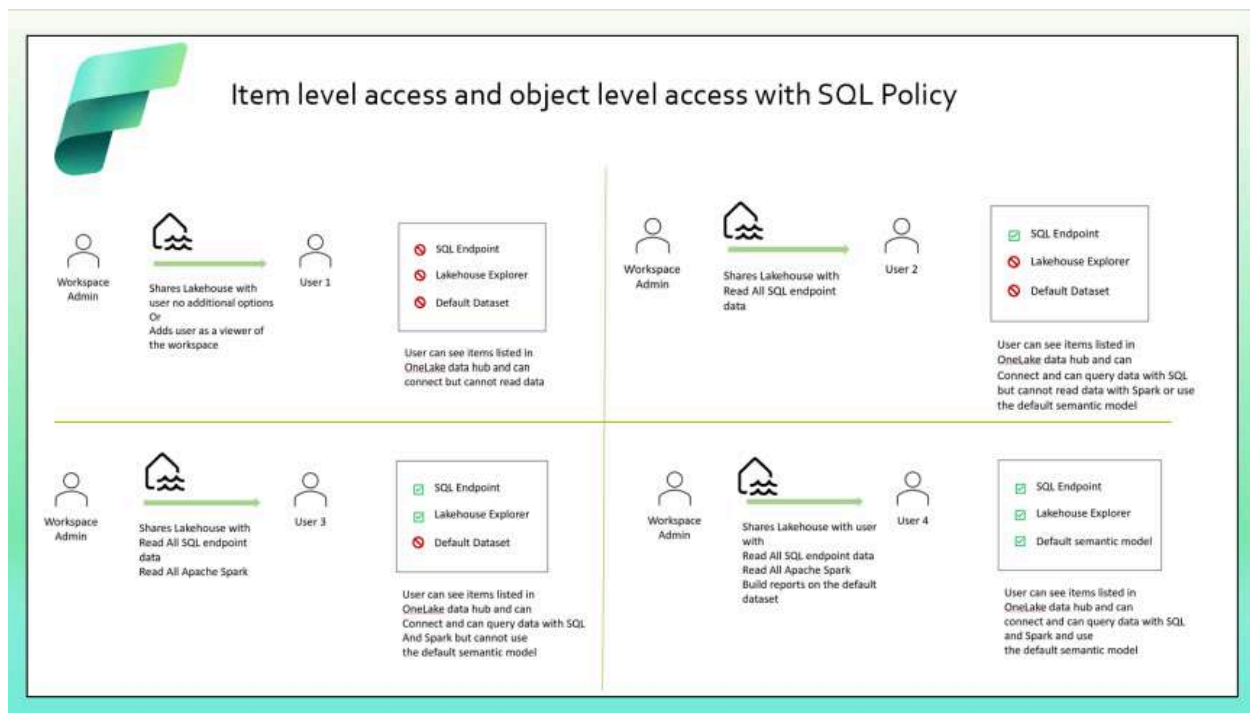
Final mental model (lock this)

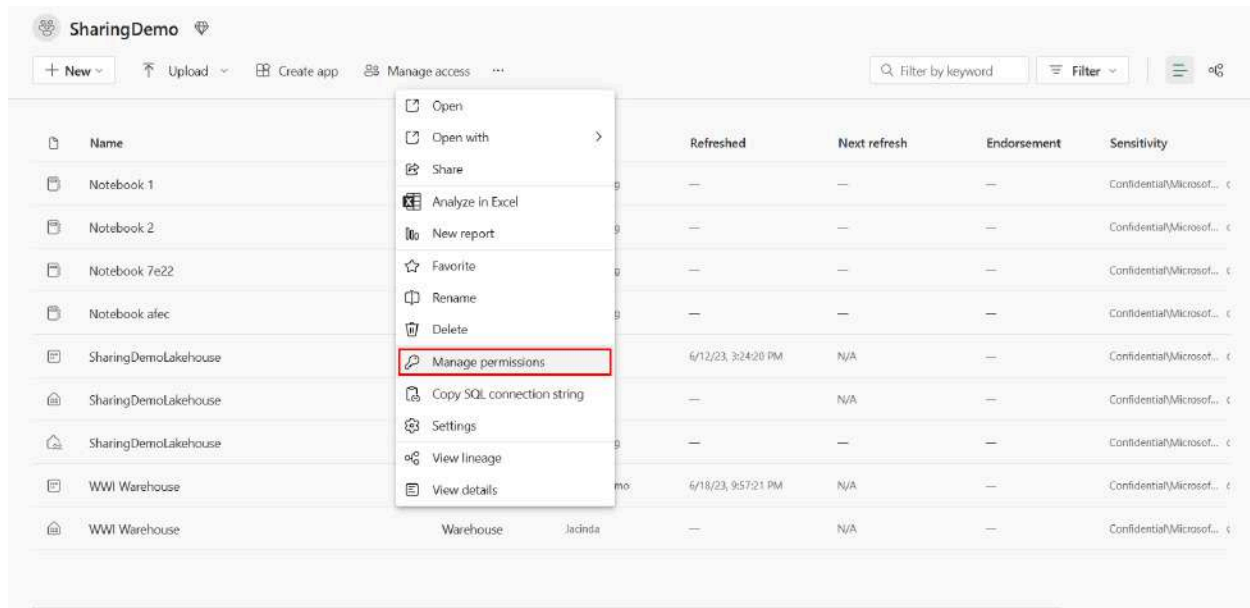
- Fabric Warehouse = cloud SQL engine
- SSMS = remote SQL client
- Connection = Entra ID–secured SQL endpoint

One-line memory lock

SSMS connects to Fabric Warehouse through its SQL endpoint using Azure AD authentication, enabling full T-SQL development without accessing OneLake directly.

Access Control in Fabric Warehouse (when sharing with users) — clear, practical notes

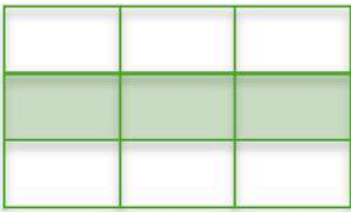






Microsoft Fabric Warehouse and SQL Endpoint

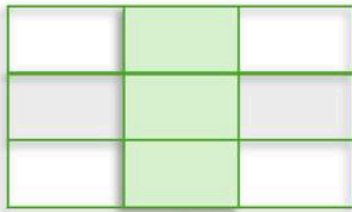
Row Level Security





```
CREATE SECURITY POLICY [policy_name]
ADD FILTER PREDICATE [sch].[ivtf_fn_name](...)
ON [schema].[tablename] WITH (STATE = ON)
```

Column-Level security





```
GRANT SELECT ON schema.TABLE(col1,col2) TO User
```

First, lock the core idea

Access control in a **Warehouse** is handled at **two levels**:

1. **Fabric workspace access** → *Can the user see the warehouse?*
2. **Warehouse SQL permissions** → *What can the user do inside the warehouse?*

Both are required.

1 Workspace-level access (Fabric security)

This is the **first gate**.

Workspace roles

Role	What user can do
Admin	Full control
Member	Create & manage items
Contributor	Modify items
Viewer	Read-only access

Rule

- If a user has **no workspace access**, they **cannot access the warehouse at all**
- Even if SQL permissions exist

Workspace access = **visibility + entry**

2 Warehouse-level SQL permissions (T-SQL security)

Once inside the warehouse, **SQL permissions apply**.

These are classic SQL-style permissions.

Grant database access

```
CREATE USER [user@company.com] FROM EXTERNAL PROVIDER;
```

This maps Entra ID user to the warehouse.

Grant read-only access

```
GRANT SELECT ON SCHEMA::dbo TO [user@company.com];
```

User can:

- Query tables
- Read views

Cannot:

- Insert
 - Update
 - Delete
-

Grant write access

```
GRANT INSERT, UPDATE, DELETE ON SCHEMA::dbo TO [user@company.com];
```

User can modify data.

Grant full control (not recommended widely)

```
GRANT CONTROL ON DATABASE::[WarehouseName] TO [user@company.com];
```

3 Role-based access (recommended approach)

Instead of granting permissions to users directly, use **roles**.

Create role

```
CREATE ROLE reporting_role;
```

Grant permissions to role

```
GRANT SELECT ON SCHEMA::dbo TO reporting_role;
```

Add users to role

```
ALTER ROLE reporting_role ADD MEMBER [user@company.com];
```

Why roles

- Easier management
 - Cleaner audits
 - Scales better
-

4 Object-level security (fine-grained)

Table-level

```
GRANT SELECT ON dbo.orders TO reporting_role;
```

Column-level (masking alternative)

```
GRANT SELECT (order_id, amount) ON dbo.orders TO reporting_role;
```

User cannot see restricted columns.

5 Row-Level Security (RLS)

Why RLS

- Same table
 - Different users
 - Different rows
-

Example


```
CREATE FUNCTION fn_rls_region(@region AS VARCHAR(50))
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN
    SELECT 1 AS fn_result
    WHERE @region = USER_NAME();

CREATE SECURITY POLICY region_policy
ADD FILTER PREDICATE dbo.fn_rls_region(region)
ON dbo.orders
WITH (STATE = ON);
```

Now:

- User sees only allowed rows
 - Enforced automatically
-

6 Dynamic Data Masking (DDM)

Why masking

- Hide sensitive data
 - No need for separate tables
-

Example

```
ALTER TABLE dbo.customers
ALTER COLUMN email
ADD MASKED WITH (FUNCTION = 'email()');
```

Non-privileged users see:

```
xxxx@xxxx.com
```

Admins see real value.

7 How sharing a warehouse actually works (step-by-step)

Step 1

Add user to **Fabric workspace**
(Role: Viewer / Contributor)

Step 2

Create user in warehouse

```
CREATE USER [user@company.com] FROM EXTERNAL PROVIDER;
```

Step 3

Assign role / permissions

```
ALTER ROLE reporting_role ADD MEMBER [user@company.com];
```

Done.

8 What happens if you skip steps (common mistakes)

Missed step	Result
No workspace access	Cannot see warehouse
No CREATE USER	Login fails
No SQL permission	Can connect, but cannot query
Direct user grants only	Hard to manage

9 Security model summary (LOCK THIS)

Fabric Workspace Access

↓
Warehouse User (Entra ID)
↓
SQL Role
↓
Schema / Table / Row / Column

Layered security.

10 Interview-ready one-liners

- Workspace role controls **visibility**
- SQL permissions control **capability**
- Roles are preferred over user grants
- RLS controls **rows**
- Masking controls **columns**

One-line final memory lock

In Fabric Warehouse, users need workspace access to enter and SQL permissions to operate, with roles, RLS, and masking providing fine-grained security.

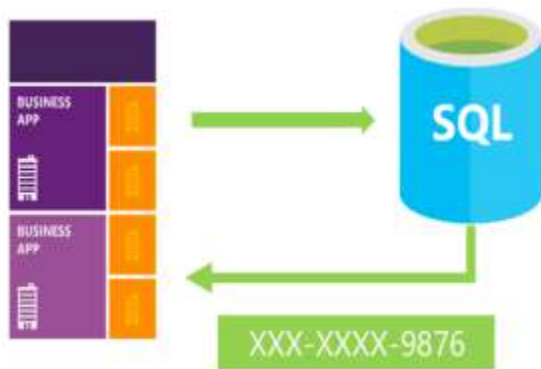
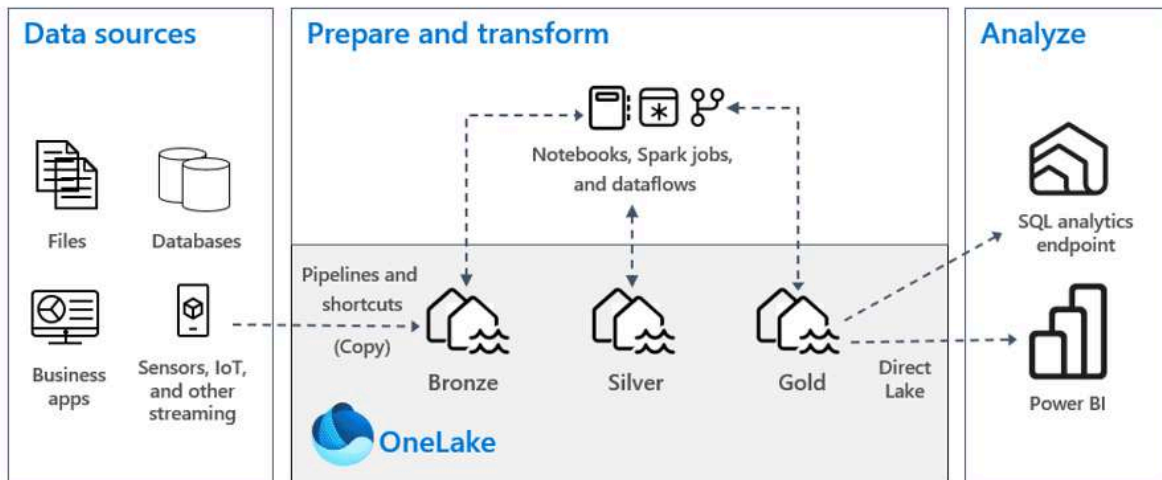
Dynamic Data Masking (DDM) in Fabric Warehouse — complete, practical notes

The screenshot shows a SQL query window with the following code:

```
24 EXECUTE AS USER = 'MaskUser'  
25 SELECT * FROM DynamicDataMaskExample  
26 REVERT  
27 GO
```

Below the query window, the 'Results' tab is active, displaying a table with 4 columns: ID, Name, Phone, and Email. The 'Name' column is highlighted with a red box, and a red arrow points to it. The table contains two rows of data:

ID	Name	Phone	Email
1	xxxx	9876543210	john.doe@yahoo.com
2	xxxx	5879612546	bob.marley@google.com



1) What is Dynamic Data Masking

Dynamic Data Masking (DDM) is a runtime security feature that hides sensitive data from unauthorized users while keeping the actual data unchanged in storage.

Simple definition:

DDM masks data **at query time**, not at rest.

- Data is stored normally
- Masking is applied **only when data is queried**
- Authorized users see real values
- Others see masked values

2) Why DDM is needed

Without DDM:

- Sensitive columns exposed to all readers
- Need duplicate tables or views
- Complex logic in every query

With DDM:

- One table
 - Centralized security
 - Zero data duplication
 - No query changes needed
-

3) What DDM does and does NOT do

✓ What it does

- Masks column values dynamically
- Applies automatically
- Works with SELECT queries
- Honors user permissions

✗ What it does NOT do

- Does NOT encrypt data
- Does NOT change stored values
- Does NOT replace Row-Level Security
- Does NOT stop privileged users

DDM is **obfuscation**, not encryption.

4) How DDM works internally (Fabric Warehouse)

User runs SELECT

↓

SQL engine checks permissions

↓

If user has UNMASK permission → show real data
Else → apply mask function

↓
Result returned

Storage remains unchanged in OneLake.

5) Masking functions available

Fabric Warehouse supports standard SQL masking functions:

Mask Type	Use case	Example output
<code>default()</code>	General text/number	XXXX
<code>email()</code>	Email addresses	aXXX@XXX.co m
<code>partial()</code>	Show part of value	XXX-XX-1234
<code>random()</code>	Numeric data	83921

6) How to implement DDM (step-by-step)

Step 1 — Create / identify table

```
CREATE TABLE dbo.customers (  
    customer_id INT,  
    name VARCHAR(100),  
    email VARCHAR(100),  
    salary INT  
);
```

Step 2 — Apply masking to column

Mask email

```
ALTER TABLE dbo.customers  
ALTER COLUMN email  
ADD MASKED WITH (FUNCTION = 'email()');
```

Mask salary (numeric)

```
ALTER TABLE dbo.customers  
ALTER COLUMN salary  
ADD MASKED WITH (FUNCTION = 'random(1000, 9999)');
```

Step 3 — Test as normal user

```
SELECT customer_id, email, salary FROM dbo.customers;
```

Result:

```
1 | aXXX@XXXX.com | 4721
```

7) Allow users to see unmasked data

Grant UNMASK permission

```
GRANT UNMASK TO [finance_user@company.com];
```

Now:

- Finance user sees real email & salary
 - Others see masked data
-

8) Removing masking

```
ALTER TABLE dbo.customers  
ALTER COLUMN email  
DROP MASKED;
```

9) DDM vs RLS (important difference)

Feature	DDM	RLS
Protects	Columns	Rows
Hides data	Partially	Completely
Changes query result size	✗	✓
Use case	PII masking	Data segregation

They are **complementary**, not alternatives.

10) Best practices (enterprise-grade)

- Use DDM for:
 - Email
 - Phone
 - Salary
 - PAN / Aadhaar (masked)
 - Combine with:
 - RLS for row restriction
 - Roles for access control
 - Never rely on DDM alone for compliance
-

11) Common mistakes (avoid)

- Thinking DDM encrypts data
 - Granting UNMASK too broadly
 - Using DDM instead of RLS
 - Applying masking to primary keys
-

Mental model (lock this)

- **Data stored** → real
 - **Query result** → masked (if unauthorized)
-

One-line final memory lock

Dynamic Data Masking in Fabric Warehouse hides sensitive column values at query time based on user permissions, without changing the stored data.

Column-Level Security in Fabric Warehouse — DENY-based approach (your exact pattern)

This is the **classic SQL authorization model** using:

1. `CREATE ROLE ... AUTHORIZATION schema`
2. `ALTER ROLE ... ADD MEMBER`
3. `DENY SELECT` on specific columns

No masking. No views. **Hard column blocking**.

1 What this approach does (lock this)

- Role **inherits schema ownership**
- User added to role
- Specific columns are **explicitly denied**
- User can query table **except denied columns**

This is **true column-level security**, not masking.

2 Create role with schema authorization

```
CREATE ROLE sales_read_role  
AUTHORIZATION gold;
```

What this means

- `gold` schema owns the role
 - Role automatically aligns with schema security
 - Cleaner governance for schema-scoped access
-

3 Add user to role (Entra ID user)

```
ALTER ROLE sales_read_role  
ADD MEMBER [user1@company.com];
```

Now the user:

- Exists in warehouse
 - Inherits permissions of `sales_read_role`
-

4 Grant base SELECT access on table

Before denying columns, **grant table access**:

```
GRANT SELECT  
ON gold.customer  
TO sales_read_role;
```

Without this, `DENY` is meaningless.

5 Deny access to specific column (CORE STEP)

```
DENY SELECT  
ON gold.customer (unique_id)  
TO sales_read_role;
```

Result

- Table is accessible
 - Column `unique_id` is completely blocked
-

6 Query behavior (VERY IMPORTANT)

User runs:

```
SELECT name, city  
FROM gold.customer;
```

✓ Works

User runs:

```
SELECT unique_id  
FROM gold.customer;
```

✗ Permission denied

User runs:

```
SELECT *  
FROM gold.customer;
```

✗ Fails

Reason: `SELECT *` expands to `unique_id`

7 Why DENY is powerful (and dangerous)

SQL permission precedence rule

`DENY > GRANT`

Even if:

- User has GRANT elsewhere
- User is member of another role

DENY **always wins**.

8 Remove column restriction (rollback)

```
REVOKE DENY
ON gold.customer (unique_id)
FROM sales_read_role;
```

Access restored immediately.

9 How this differs from GRANT-only CLS

Method	Behavior
GRANT SELECT (col1, col2)	Whitelist approach
DENY SELECT (colX)	Blacklist approach

You are using the **blacklist model**.

10 When this DENY pattern is best

Use this when:

- Most columns are allowed
- Only 1–2 sensitive columns need blocking
- You want schema-centric governance
- You want strict enforcement (no masking)

Avoid when:

- Many columns are restricted
- You want partial visibility (use DDM instead)

11 Full example (copy-paste ready)

```
-- Role creation
CREATE ROLE sales_read_role AUTHORIZATION gold;

-- Add user
ALTER ROLE sales_read_role ADD MEMBER [user1@company.com];

-- Base access
GRANT SELECT ON gold.customer TO sales_read_role;

-- Column restriction
DENY SELECT ON gold.customer (unique_id) TO sales_read_role;
```

Mental model (lock this)

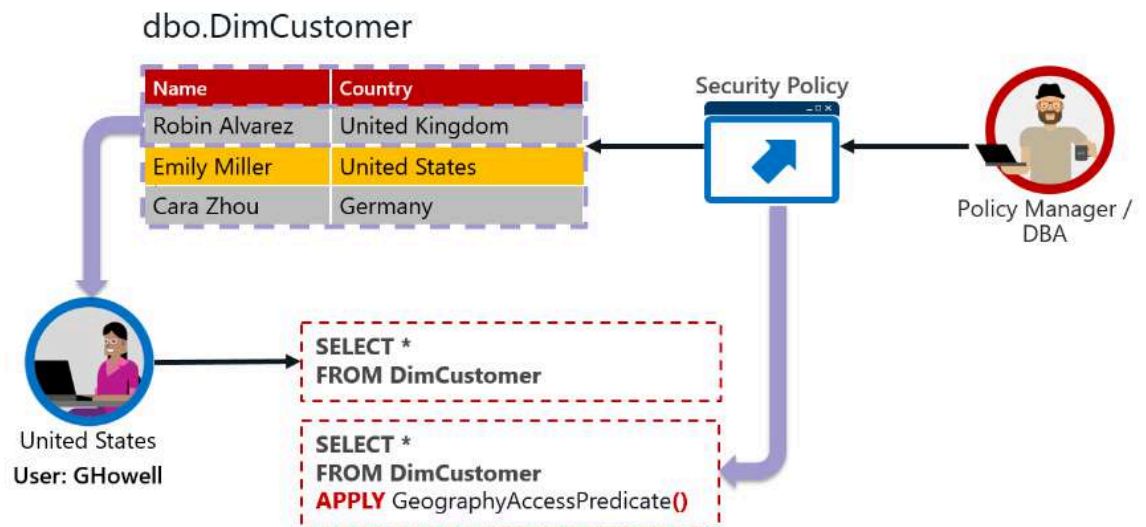
- **GRANT** opens doors
 - **DENY** locks doors permanently
 - **SELECT *** breaks if any column is denied
-

One-line memory lock

In Fabric Warehouse, column-level security can be enforced by granting table access to a role and explicitly DENYing SELECT on sensitive columns—DENY always overrides GRANT.

**Row-Level Security (RLS) in Fabric Warehouse —
complete, clear explanation**

SQL Server 2016 - Row-Level Security



Microsoft Fabric Warehouse and SQL Endpoint

Row Level Security



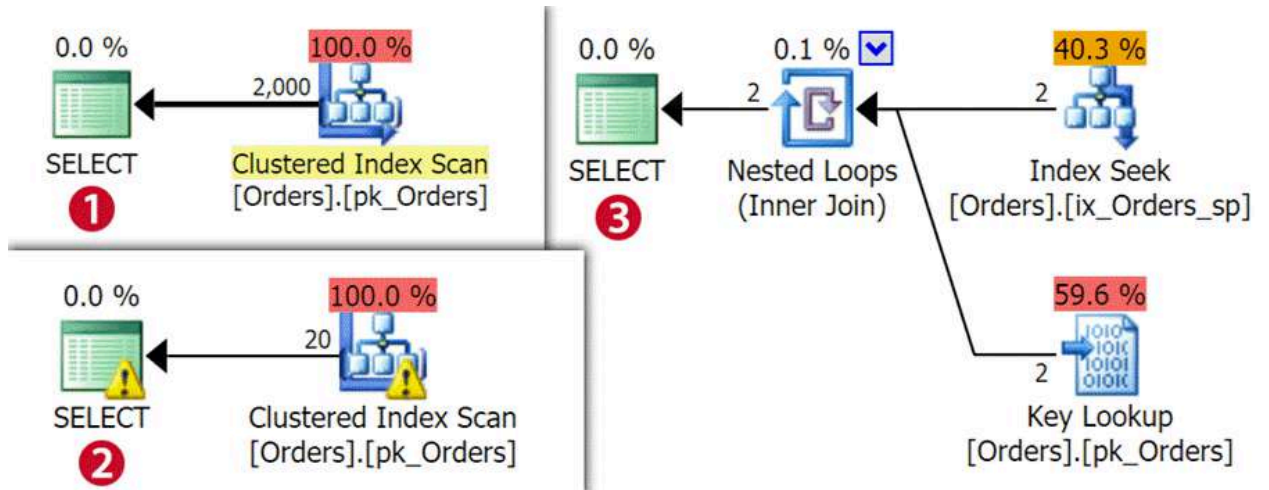

```
CREATE SECURITY POLICY [policy_name]  
ADD FILTER PREDICATE [sch].[ivtf_fn_name](...)  
ON [schema].[tablename] WITH (STATE = ON)
```

Column-Level security



```
GRANT SELECT ON schema.TABLE(col1,col2) TO User
```





1) What is Row-Level Security (RLS)

Row-Level Security restricts which rows of a table a user can see, based on who the user is or what attributes they have.

Simple definition:

RLS filters rows at query time so different users see different subsets of the same table.

- Same table
- Same query
- Different results per user

2) Why RLS is needed

Without RLS:

- Separate tables per region/team
- Complex WHERE clauses in every query
- Easy to bypass security logic
- High maintenance

With RLS:

- Centralized enforcement

- Automatic filtering
 - No query changes
 - Secure by default
-

3) How RLS works internally

User runs `SELECT`

↓

SQL engine rewrites query

↓

Security predicate added automatically

↓

Filtered rows returned

Users never see rows they're not allowed to see.

4) Core components of RLS (important)

RLS is built using three objects:

1. Security predicate function
 2. Security policy
 3. User identity (Entra ID)
-

5) Step-by-step RLS implementation

Step 1 — Example table

```
CREATE TABLE gold.sales (
```



```
    order_id INT,  
    region VARCHAR(50),  
    amount DECIMAL(10,2)  
);
```

Step 2 — Create predicate function

This function decides row visibility.

```
CREATE FUNCTION fn_rls_region(@region AS VARCHAR(50))  
  
RETURNS TABLE  
  
WITH SCHEMABINDING  
  
AS  
  
RETURN  
  
    SELECT 1 AS fn_access  
  
    WHERE @region = USER_NAME();
```

Logic:

- User name must match region value
-

Step 3 — Create security policy

Attach the function to the table.

```
CREATE SECURITY POLICY region_policy  
  
ADD FILTER PREDICATE dbo.fn_rls_region(region)
```

```
ON gold.sales  
  
WITH (STATE = ON);
```

Now RLS is active.

6) Query behavior (what users see)

User `north@company.com`

```
SELECT * FROM gold.sales;
```

Returns only rows where:

```
region = 'north@company.com'
```

Admin user

Admins bypass RLS automatically.

7) Common RLS patterns

Pattern 1 — Region-based access

```
WHERE region = USER_NAME()
```

Pattern 2 — Role-based access

```

WHERE EXISTS (

    SELECT 1

    FROM user_region_map

    WHERE user = USER_NAME()

    AND region = @region

)

```

Pattern 3 — Department-based access

Use lookup table:

```
user_department_map
```

8) Filter vs Block predicates

P r e d i c a t e	Purpose
---	---------

F I L T	Filters SELECT results
------------------	------------------------

E
R

B Blocks
L INSERT/UPDATE/DEL
O ETE
C
K

Fabric Warehouse mainly uses FILTER predicates.

10) Important limitations

- RLS applies only in Warehouse
 - Spark bypasses Warehouse RLS
 - Requires careful function logic
 - Performance depends on predicate complexity
-

11) Best practices

- Keep predicate functions simple
 - Use mapping tables for flexibility
 - Avoid hardcoding logic
 - Test with different users
 - Combine with CLS/DDM
-

Mental model (lock this)

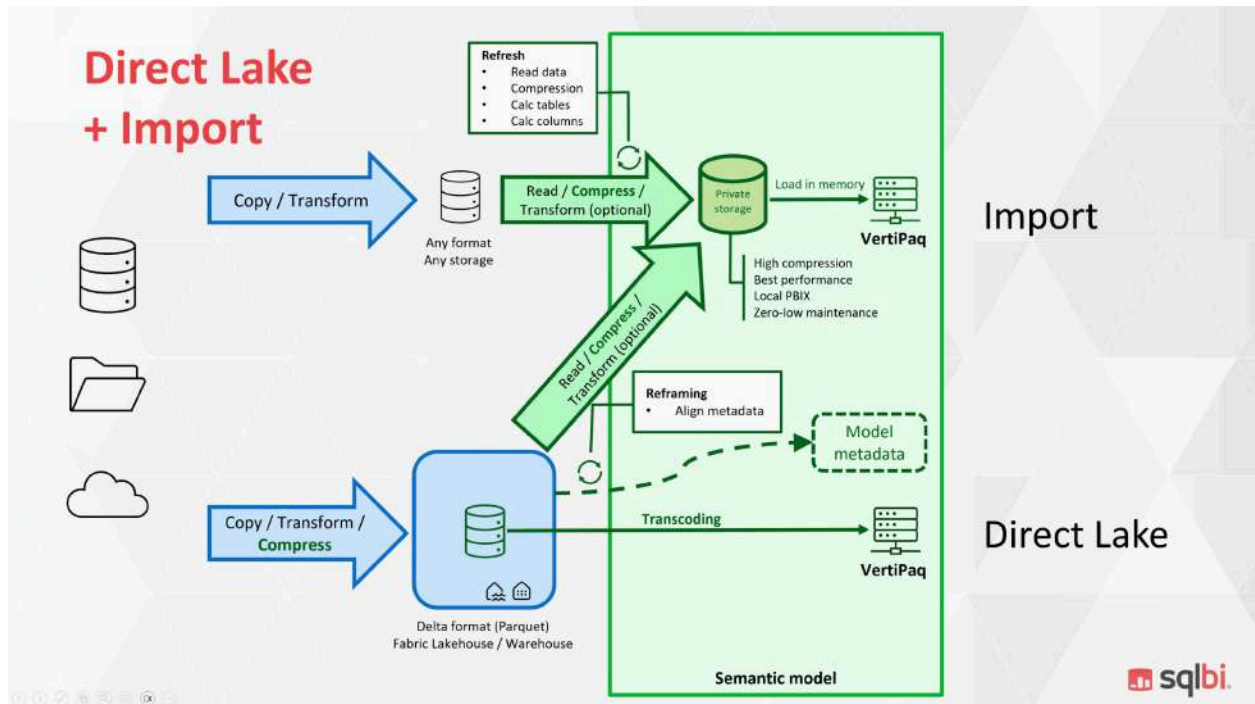
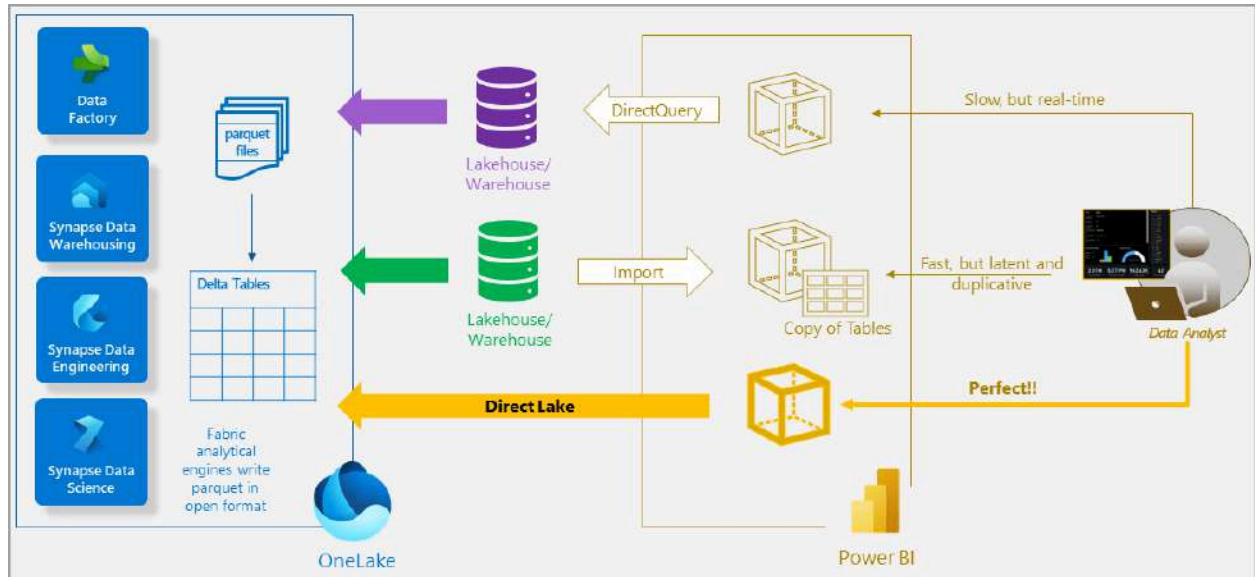
- CLS → *Which columns*
- RLS → *Which rows*
- DDM → *How values look*

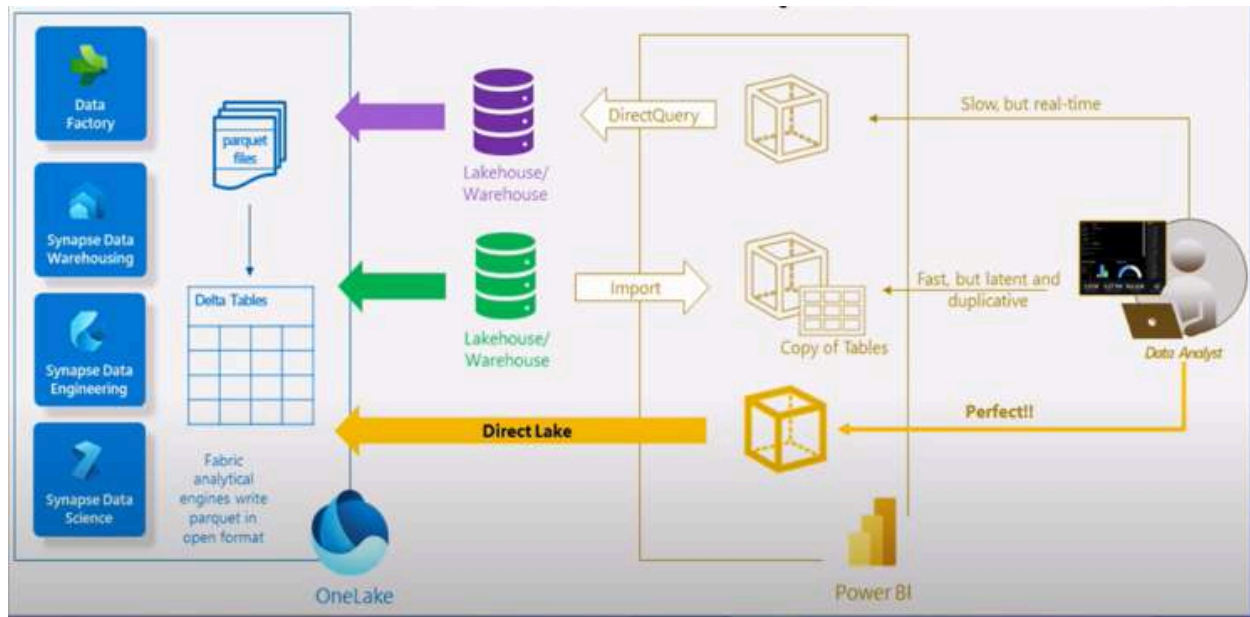
One-line memory lock

Row-level security in Fabric Warehouse enforces user-specific row filtering automatically at query time using security policies and predicate functions.



Direct Lake in Microsoft Fabric — what it is, how it works, and why it's better than Import & DirectQuery





1) What is Direct Lake

Direct Lake is a **Power BI connectivity mode** in **Microsoft Fabric** that lets Power BI **read Delta tables directly from OneLake** without importing data or sending live SQL queries.

Simple definition:

Direct Lake reads Delta files directly from OneLake into the Power BI engine at query time.

No data copy.

No SQL query pushdown.

No refresh dependency.

2) Why Direct Lake was created (the real problem)

Before Fabric, Power BI had only two options:

Import mode

- Copy data into Power BI
- Fast queries
- Slow refresh

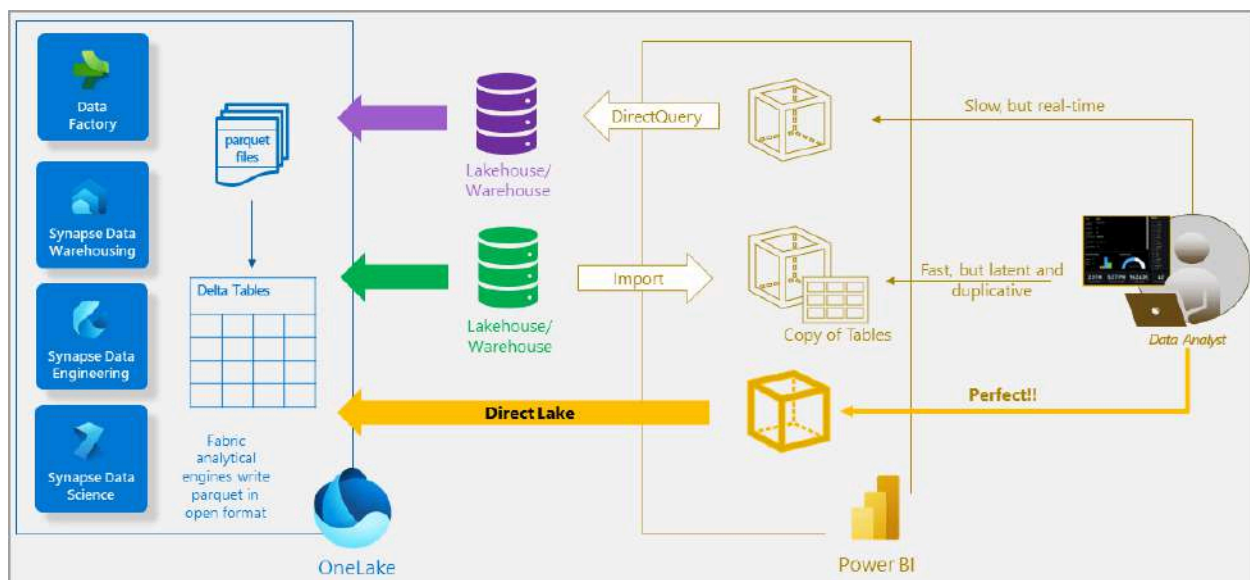
- Data duplication

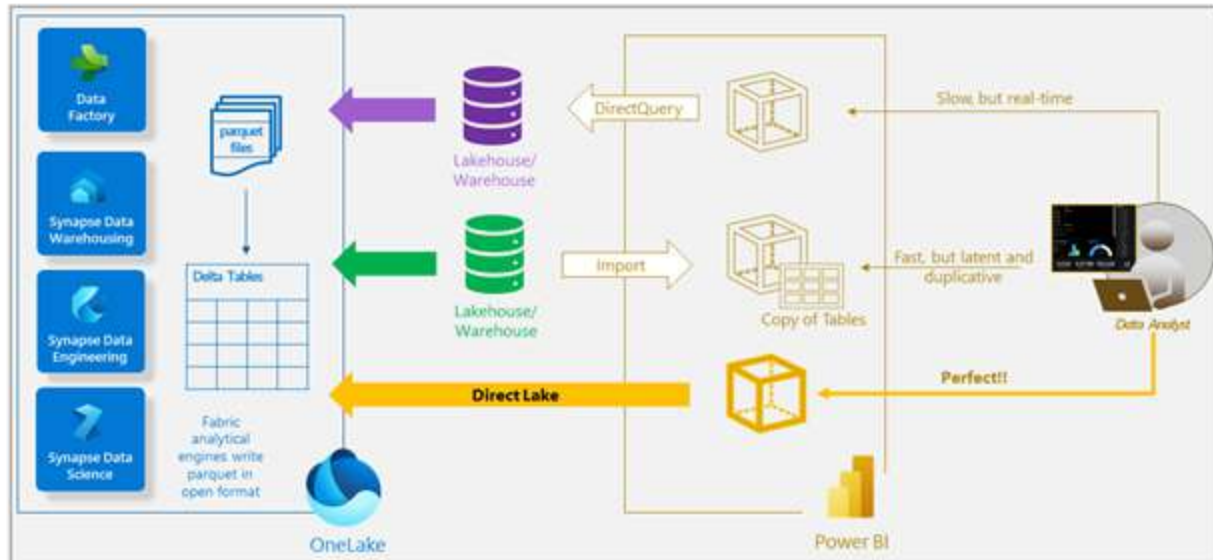
DirectQuery mode

- Query source every time
- No duplication
- Slow dashboards
- High load on source

Direct Lake was created to remove both problems at once.

3) How Direct Lake works (internals, step-by-step)





Step-by-step flow

1. Data is stored as **Delta tables** in OneLake (Parquet + `_delta_log`)
2. Power BI semantic model is created in **Direct Lake mode**
3. When a report is queried:
 - Power BI engine reads **Delta metadata**
 - Reads **Parquet files directly**
 - Loads only required columns into memory
4. Query executes in-memory (VertiPaq-like speed)

No scheduled refresh.

No SQL engine involved.

4) What Direct Lake reads from OneLake

Direct Lake works **only with Delta tables**.

Storage	Supported
CSV	✗
Parquet (plain)	✗

Delta (Parquet + log) 

That's why Lakehouse/Warehouse tables matter.

5) Direct Lake vs Import vs DirectQuery (core comparison)

Feature	Import	DirectQuery	Direct Lake
Data copy	Yes	No	No
Query speed	Very fast	Slow	Very fast
Refresh needed	Yes	No	No
Load on source	Low	High	None
Data freshness	Stale until refresh	Real-time	Near real-time
Source type	Any	SQL only	Delta in OneLake
Scalability	Limited by model	Limited by source	Very high

6) Why Direct Lake is faster than DirectQuery

DirectQuery problem

Power BI → SQL → Storage → SQL → Power BI

- Network latency
 - SQL engine overhead
 - Concurrency limits
-

Direct Lake execution

Power BI → Parquet files (OneLake)

- No SQL translation
- Columnar reads
- Vectorized execution
- Minimal overhead

Result: **Import-level speed without import.**

7) Why Direct Lake is better than Import

Import problems:

- Refresh windows
- Large model sizes
- Duplicate storage
- Long refresh failures

Direct Lake fixes this:

- No refresh
 - No duplication
 - Always up-to-date
 - Reads only required data
-

8) What happens when Direct Lake can't be used (fallback)

If Power BI:

- Needs unsupported feature
- Hits complex calculation
- Cannot read file layout efficiently

It **automatically falls back** to:

- **DirectQuery to SQL endpoint**

This is called **Direct Lake fallback**.

Transparent to user.

9) Security with Direct Lake

Direct Lake respects:

- OneLake permissions
- Lakehouse/Warehouse security
- Row-Level Security (via SQL endpoint fallback if needed)
- Column-Level Security

No security bypass.

10) When to use Direct Lake (clear rule)

Use Direct Lake when:

- Data is in Fabric (Lakehouse/Warehouse)
- Tables are Delta
- Large datasets
- High-concurrency BI
- Near real-time analytics

Avoid when:

- Source is external SQL only
 - Heavy Power BI-only transformations required
-

11) Typical Fabric analytics flow

Source

- Lakehouse (Delta)
- Warehouse (optional)
- Power BI (Direct Lake)

One copy of data.

Multiple engines.

Maximum performance.

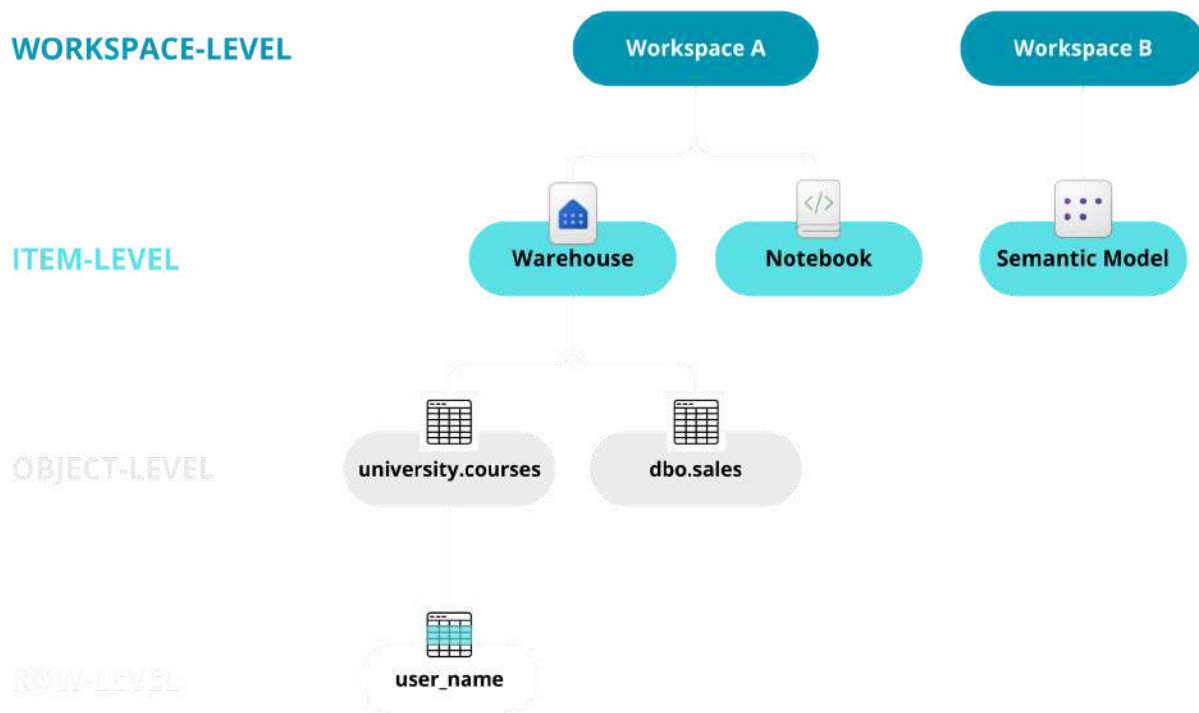
Mental model (lock this)

- **Import** → copy data
 - **DirectQuery** → ask source
 - **Direct Lake** → read lake directly
-

One-line memory lock

Direct Lake lets Power BI read Delta tables directly from OneLake, delivering Import-level performance without data duplication or refresh.

Fabric Access Control — what it is, why it matters, and what workspace-level access means



Manage access



Sample workspace1

+ Add people or groups

Search within workspace



Sample User ⓘ

Admin ▾



Sample user2 ⓘ

Contributor ▾



Admin



Member



Viewer



Remove



1) What is Fabric Access Control

Fabric Access Control is the **security model** that decides:

- **Who can see** Fabric resources
- **What they can do** with them
- **At which layer** the permission applies

In **Microsoft Fabric**, access control is **layered**, not flat.

2) Why Fabric Access Control is important

Without proper access control:

- Users see data they shouldn't
- Sensitive data leaks
- Accidental deletes happen
- Compliance fails
- Audits become impossible

Fabric access control ensures:

- Least-privilege access
 - Clear separation of duties
 - Secure collaboration
 - Enterprise compliance
-

3) Fabric security layers (critical concept)

Fabric security works at **multiple layers**, each answering a different question:

Tenant / Capacity

↓

Workspace

↓

Item (Lakehouse, Warehouse, Dataset)

↓

Data (rows, columns, values)

Each layer **adds restrictions**, never removes them.

4) What is Workspace-Level Access

Workspace-level access controls:

Who can enter a workspace and see the items inside it

It is the **first and mandatory gate**.

If a user does NOT have workspace access:

- They cannot see the workspace
 - They cannot access Warehouse, Lakehouse, BI, anything
 - SQL permissions alone are useless
-

5) Workspace roles (exact meaning)

Role	What the user can do
Admin	Full control (users, settings, items)
Member	Create, modify, delete items
Contributor	Edit existing items
Viewer	Read-only access

Key rule:

Workspace role = visibility + entry permission

6) What workspace access does NOT control

Workspace access **does NOT decide**:

- Which rows a user sees
- Which columns a user sees

- Which values are masked
- Fine-grained data access

Those are handled at **data-level security**.

7) Relationship between workspace access and data access

This is where many people get confused.

Example scenario

- User has **SELECT permission** on a Warehouse table
- But user is **not added to workspace**

Result:

- ✗ Cannot connect
- ✗ Cannot query
- ✗ Cannot even see the Warehouse

Workspace access is **mandatory first**.

8) How Fabric access control works together (full flow)

User

- Workspace Role (Viewer/Contributor/etc.)
- Item Permissions (Warehouse, Lakehouse)
- SQL Permissions (GRANT/DENY)
- RLS / CLS / DDM
- Final data returned

All layers must allow access.

9) Why workspace-level access is intentionally coarse

Fabric is designed for:

- Team collaboration
- Product-based analytics
- Simplified governance

Workspace-level access answers:

“Should this user even be here?”

Fine-grained control is intentionally pushed **inside items**.

10) Common mistakes (avoid)

- Granting SQL permissions without workspace access
 - Making everyone Workspace Admin
 - Using workspace roles instead of RLS/CLS
 - Treating workspace access as data security
-

11) Best-practice model (enterprise-ready)

- Workspace:
 - Small number of Admins
 - Members for engineers
 - Viewers for business users
- Warehouse/Lakehouse:
 - SQL roles
 - RLS / CLS / DDM

This separation scales cleanly.

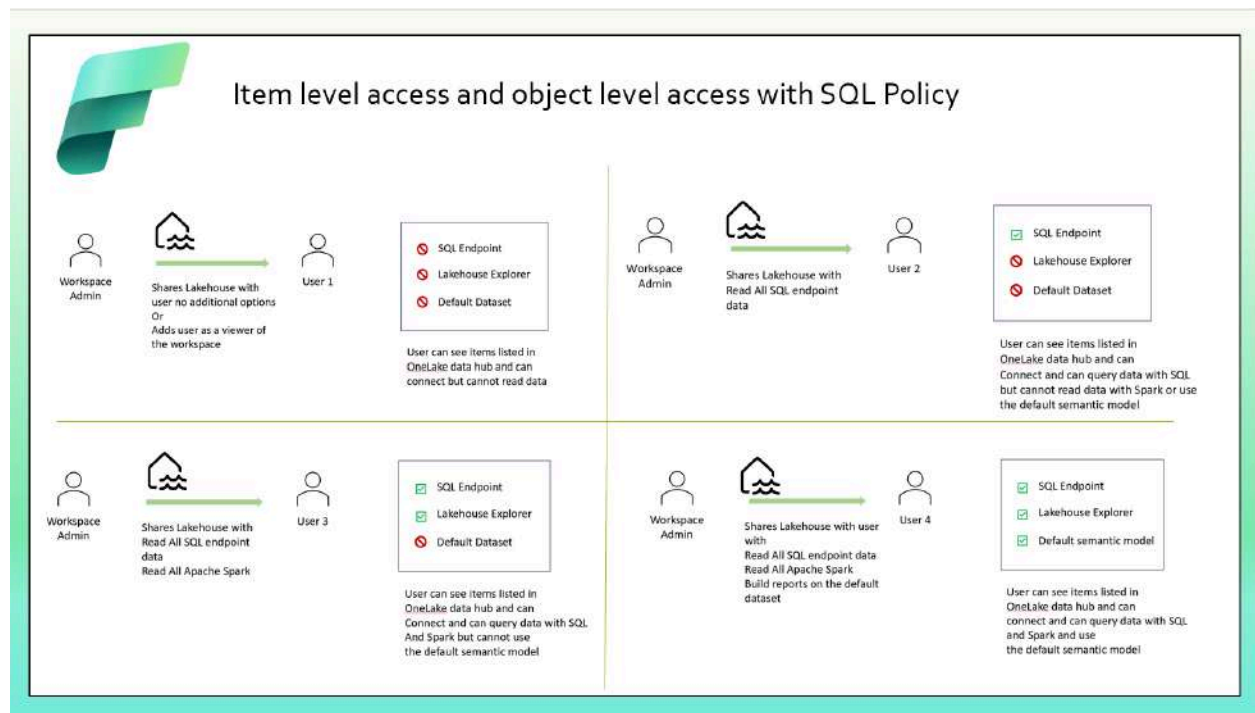
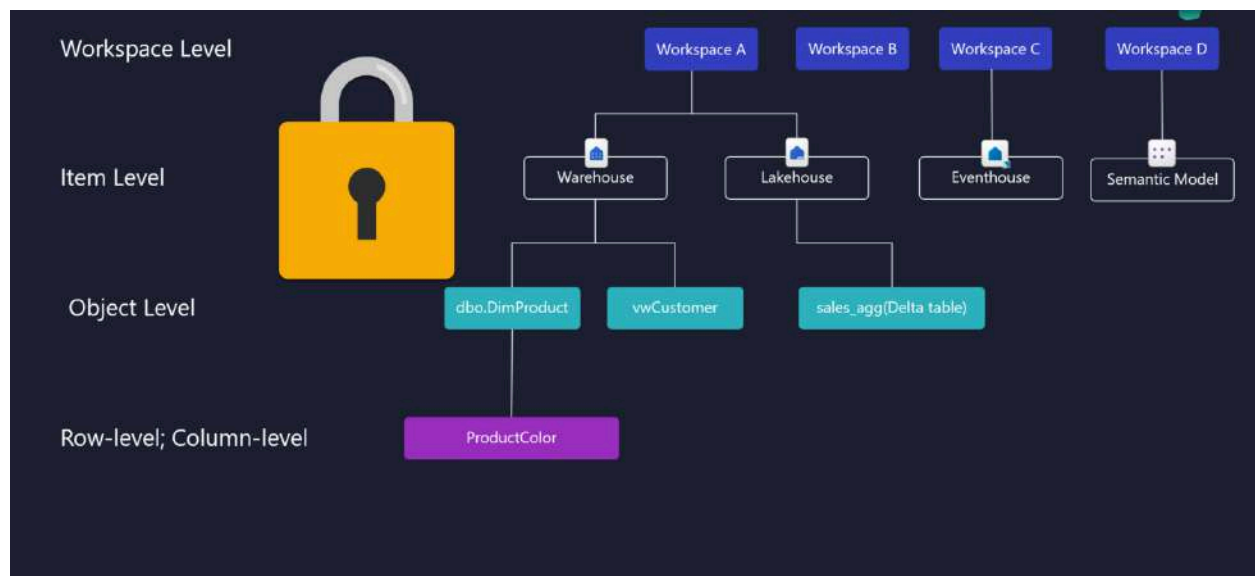
Mental model (lock this)

- **Workspace access** → *Can you enter the building?*
 - **Data security** → *Which rooms can you enter and what can you see inside?*
-

One-line memory lock

Fabric access control is layered security where workspace-level access controls visibility and entry, while item and data-level permissions control what users can actually do and see.

Item-level access vs OneLake-level access in Microsoft Fabric — clear, layered explanation





First, lock the hierarchy

Security in **Microsoft Fabric** works in layers:

Workspace access



Item-level access



OneLake (storage) access



Data-level security (RLS / CLS / DDM)

Each lower layer is **meaningless without the upper layer**.

1 What is Item-Level Access

Definition

Item-level access controls **what a user can do with a specific Fabric item** inside a workspace.

Item =

- Lakehouse
- Warehouse
- Dataset (semantic model)
- Notebook
- Pipeline
- Dataflow Gen2

Simple definition:

Item-level access decides **which Fabric objects a user can use and how.**

Why item-level access exists

Workspace access is **too broad**.

Example problem:

- User should read a Warehouse
- But should NOT edit Notebooks or Pipelines

Item-level access solves this.

What item-level access controls

Depending on item type, it controls:

- Read
- Write
- Execute
- Share
- Manage

Example (Warehouse):

- Read data
 - Create views
 - Run procedures
 - Modify tables
-

Example scenario (Warehouse)

- User has **Workspace Viewer**
- Warehouse item permission = **Read**

Result:

- User can query Warehouse
 - User cannot edit notebooks or pipelines
 - User cannot modify warehouse structure
-

Important rule

Item-level access refines workspace access; it never overrides it.

No workspace access → no item access.

2 What is OneLake-Level Access

Definition

OneLake-level access controls **who can access the physical data stored in OneLake**.

Simple definition:

OneLake access controls **file-level access to data**, not SQL objects.

OneLake is the **single storage layer** for all Fabric items.

What OneLake access protects

- Files
- Folders
- Delta tables (as files)
- Shortcuts to ADLS / S3

It applies when data is accessed via:

- Files experience

- Spark
 - Shortcuts
 - APIs
-

What OneLake access does NOT control

- SQL permissions
- Warehouse views
- Stored procedures
- BI queries

Those are item-level or data-level concerns.

3 How OneLake access is enforced

OneLake access is enforced via:

- Workspace membership
- Item ownership
- Inherited permissions
- Shortcut permissions

If user lacks OneLake access:

- Spark read fails
- File browsing fails
- Shortcut access fails

Even if SQL permissions exist.

4 Item-level vs OneLake-level — direct comparison

Aspect	Item-Level Access	OneLake-Level Access
Scope	Fabric object	Storage (files/folders)
Controls	What you can do	What data you can read/write

Applies to	Warehouse, Lakehouse, etc.	Files, Delta, shortcuts
Used by	SQL, BI, UI	Spark, Files, APIs
Granularity	Object-level	Path/file-level

5 Example (end-to-end clarity)

Scenario

- User should:
 - Query Warehouse
 - NOT read raw files
 - NOT run Spark notebooks

Configuration

1. Workspace role → Viewer
2. Item-level → Read access on Warehouse
3. OneLake-level → No access to Files
4. Data-level → RLS / CLS (optional)

Result

- Warehouse queries work
 - Spark/file access fails
 - Raw data hidden
-

6 Common confusion (important)

✗ “Warehouse permission gives file access”

False.

Warehouse reads files **on behalf of the user**.

✗ “OneLake access gives SQL access”

False.

OneLake access only allows **file-level access**, not SQL.

They are **independent layers**.

7 Why Fabric separates these two

Because different users need different access paths:

- Analysts → SQL access only
- Engineers → Spark + files
- BI users → semantic models only

Single storage. Multiple access paths. Separate controls.

Mental model (lock this)

- **Workspace** → can you enter?
 - **Item** → which tools can you use?
 - **OneLake** → can you touch the files?
 - **Data rules** → what exactly can you see?
-

One-line memory lock

Item-level access controls what you can do with Fabric objects, while OneLake-level access controls whether you can access the underlying files and storage paths.

Endorsement and Lineage in Microsoft Fabric — what they are and why they matter

Customer feedback
AppAccessSettings

Master data
Promoted

Owner: Lela Weeks
Details

Owner: Details

Filter by keyword Filter

Location	Endorsement	Sensitivity
Contoso workspace	Master data	Highly Confidential, Contoso
Contoso workspace	Master data	—
Contoso workspace	Master data	Confidential/Contoso FTE
Contoso workspace	—	Public
Contoso workspace	Certified	Non-Business
Contoso workspace	Promoted	—
Contoso workspace	Certified	Public

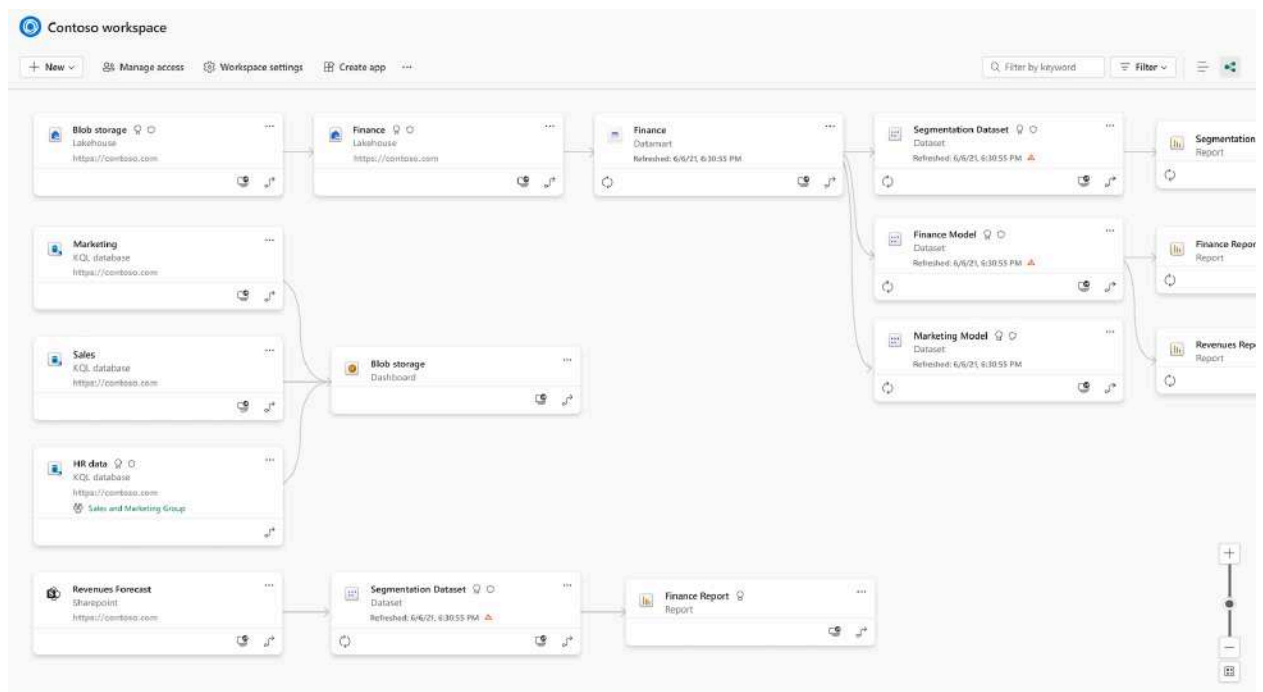
Contoso Daily Sales Master data Data updated 12/12/20

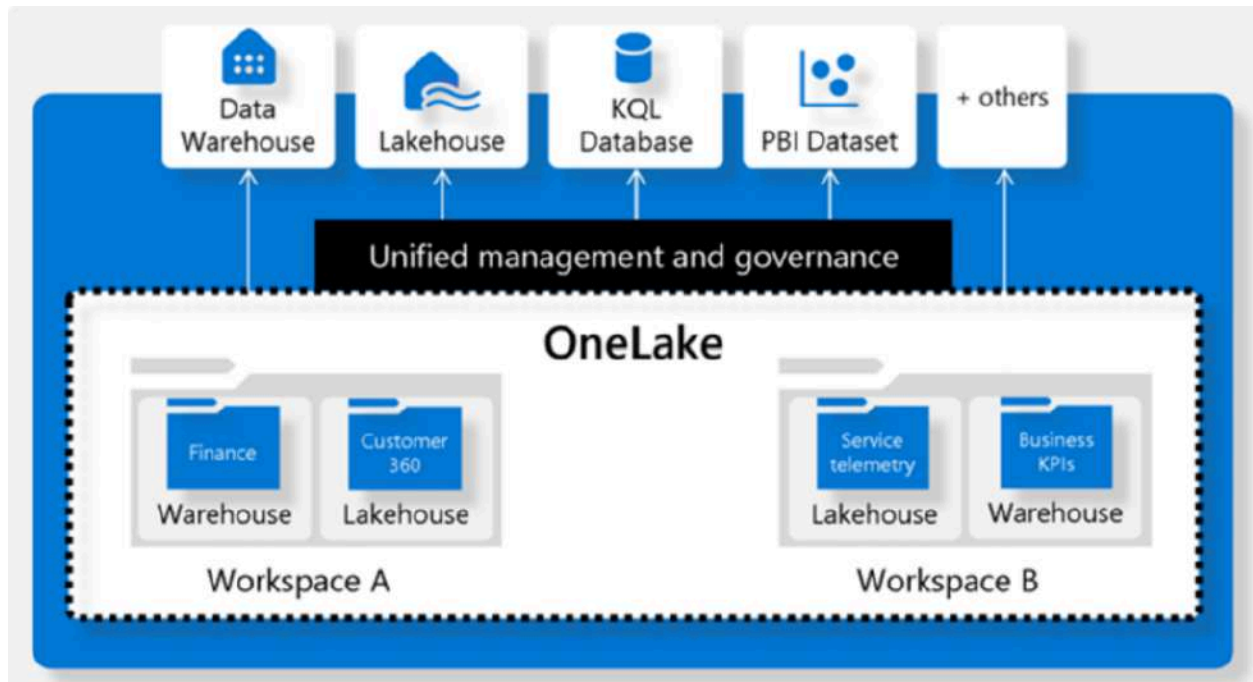
File Refresh Share Create a report Analyze in Excel

Details for "Artifact's name"

Modified by Debra Berger Location Linkdomain Endorsed as master data by Debra Berger

See what already exists
These objects were created using data from Daily Sales.





1) What is Endorsement in Fabric(go to setting and get this)

Definition

Endorsement is a **trust label** applied to Fabric items to indicate **data quality, reliability, and ownership**.

Simple definition:

Endorsement tells users whether an item is trustworthy and approved for use.

It answers the question:

“Can I trust this data for business decisions?”

Types of Endorsement

1 Promoted

- Marked as **recommended**

- Created/maintained by a known team
- Not fully governed

Meaning for users:

“This looks good and is commonly used.”

2 Certified (highest level)

- Fully reviewed and approved
- Owned by data/BI governance team
- Represents **official source of truth**

Meaning for users:

“Use this for reporting. This is the final truth.”

What items can be endorsed

- Lakehouse
- Warehouse
- Semantic models (Power BI datasets)
- Reports
- Dataflows

Endorsement is **metadata only** — no data changes.

Why endorsement is important

Without endorsement:

- Multiple datasets for same metric
- Conflicting reports
- No trust in numbers
- Shadow analytics

With endorsement:

- Clear source of truth

- Faster adoption
 - Reduced confusion
 - Strong governance
-

What endorsement does NOT do

- **✗** Does not control access
- **✗** Does not enforce security
- **✗** Does not improve performance
- **✗** Does not validate correctness automatically

It is **trust signaling**, not enforcement.

2) What is Lineage in Fabric

Definition

Lineage shows **how data flows and transforms** across Fabric items — from source to consumption.

Simple definition:

Lineage answers “Where did this data come from and where is it used?”

What lineage shows

- Data sources
- Transformations
- Dependencies
- Downstream usage

Example:

Files

- Lakehouse
- Warehouse
- Semantic Model

→ Power BI Report

All visible visually.

How lineage works internally

Fabric automatically captures:

- Data movement
- Table dependencies
- SQL references
- Spark writes
- BI consumption

No manual documentation required.

Why lineage is critical

1 Impact analysis

Before changing a table, you know:

- Which reports will break
 - Which teams are affected
-

2 Debugging

If a report is wrong:

- Trace back to source
 - Identify broken transformation
-

3 Governance & audit

- Who created what

- How data flows
 - Compliance readiness
-

Lineage vs Endorsement — clear difference

Aspect	Endorsement	Lineage
Purpose	Trust	Transparency
Question answered	Can I trust this?	Where did this come from?
Type	Label	Graph
Enforcement	✗	✗
Audience	Business users	Engineers, BI, governance

How they work together (important)

- **Lineage** shows *relationships*
- **Endorsement** shows *trust level*

Example:

- Lineage shows a report uses a dataset
- Endorsement shows whether that dataset is certified

Together they answer:

“This report comes from **this** certified dataset.”

Typical enterprise pattern

Raw data

- Lakehouse (not endorsed)
- Warehouse (Certified)
- Semantic Model (Certified)
- Reports (Promoted)

Trust increases as data moves closer to business.

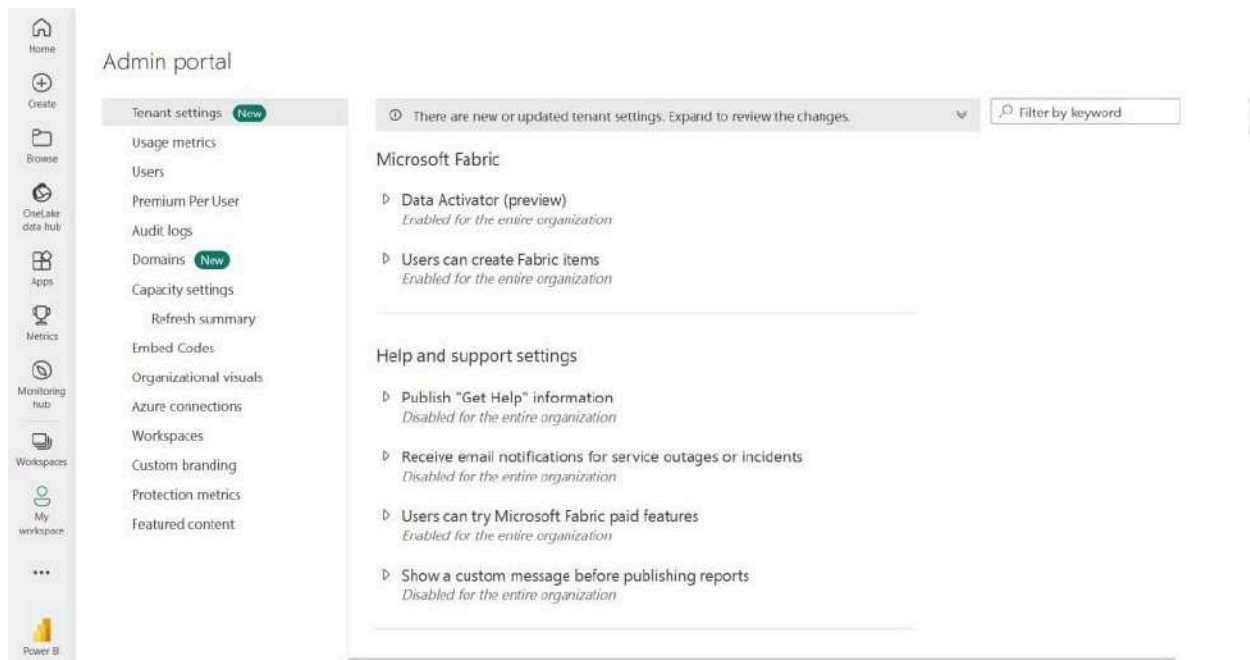
Mental model (lock this)

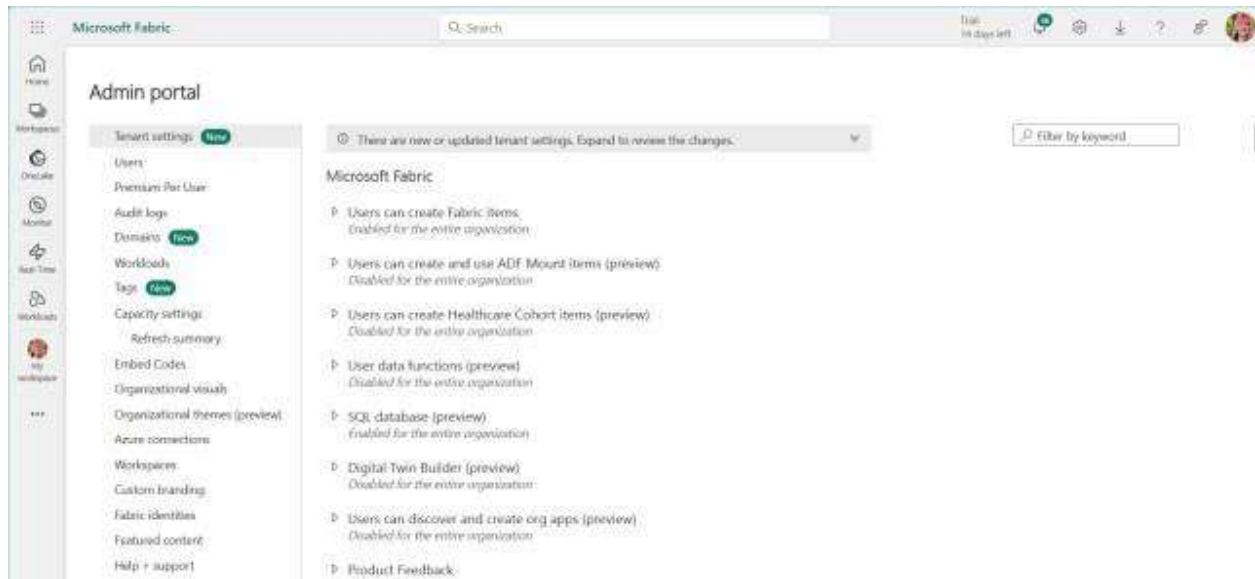
- **Endorsement** = trust badge
 - **Lineage** = data flow map
-

One-line memory lock

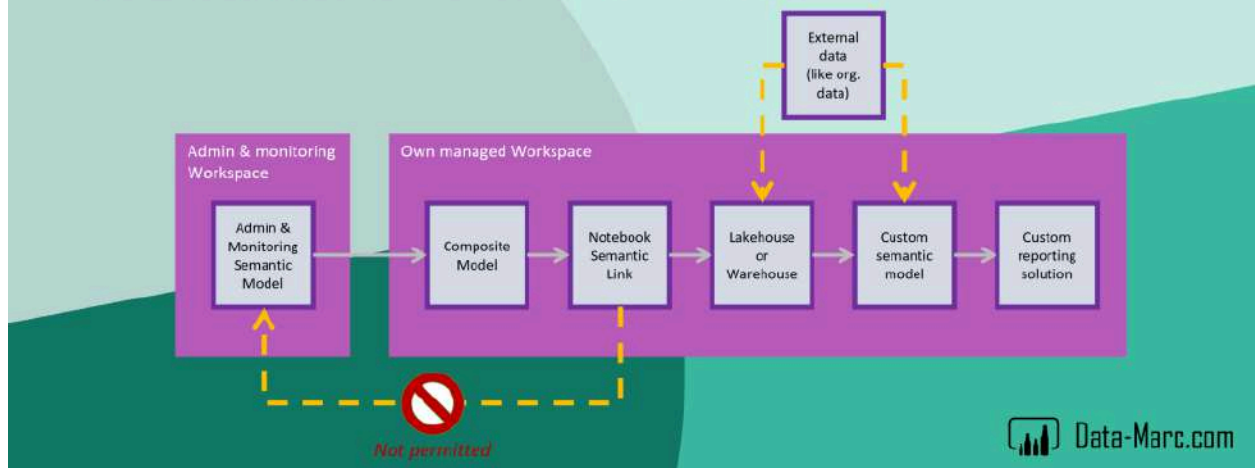
Endorsement indicates trust and approval, while lineage shows how data flows across Fabric from source to consumption.

Fabric Admin Capabilities — what admins control and why it matters





Your Metrics, Your Rules: Extracting and Storing Fabric Admin and Capacity Metrics Data in Fabric



Definition

Fabric Admin capabilities are **tenant-level controls** that govern **who can use Fabric**, **how resources are consumed**, **how data is governed**, and **how the platform is secured**.

Admins operate **above workspaces**. They control the **rules of the system**, not day-to-day analytics.

Why Fabric Admin capabilities exist

Without admin controls:

- Anyone can create resources
- Costs spiral
- Sensitive data leaks
- Governance collapses
- No audit trail

Fabric Admin capabilities enforce:

- Cost control
 - Security boundaries
 - Governance standards
 - Platform stability
-

Where Fabric Admin operates

Admins use the **Fabric Admin Portal** (Tenant scope).

Hierarchy:

Tenant (Admin)

- Capacity
- Workspace
- Item
- Data

Admin decisions affect **all workspaces and users**.

Core Fabric Admin capability areas

1 Tenant-level settings (most powerful)

Admins decide **who can use what across the tenant**.

Examples:

- Enable / disable Fabric workloads (Lakehouse, Warehouse, Data Factory)
- Control who can create workspaces
- Control who can create Lakehouses / Warehouses
- Control external sharing
- Control trial access

These are **global switches**.

2 Capacity management (cost + performance control)

Admins manage **Fabric capacity**, which drives:

- Performance
- Concurrency
- Cost

Admin controls:

- Assign workspaces to capacities
- Monitor capacity utilization
- Throttle or reassign workloads
- Prevent noisy neighbors

Admins answer:

“Which teams get how much compute?”

3 Workspace governance controls

Admins can enforce:

- Workspace naming standards
- Workspace creation restrictions
- Default workspace settings
- Workspace deletion policies

Admins do **not** manage data inside workspaces daily, but they control **who can create and own them**.

4 Security & identity controls

Admins control:

- Entra ID integration
- Who can share externally
- Who can publish content
- Cross-tenant access

Admins define:

“Who is allowed inside Fabric at all.”

5 Data governance & compliance

Admins oversee:

- Endorsement governance (who can certify)
- Sensitivity labels
- Auditing and logs
- Lineage visibility

Admins don't certify data themselves, but they define **who is allowed to certify**.

6 Monitoring & auditing

Admins can:

- View tenant-wide activity logs
- Track user actions
- Monitor query activity (high level)
- Support compliance and audits

This is **oversight**, not tuning individual queries.

7 Feature rollout & control

Admins can:

- Enable preview features
- Disable unstable features
- Control platform evolution

This prevents:

- Breaking changes
 - Uncontrolled experimentation in production
-

What Fabric Admins do NOT do

Admins do **not**:

- Write Spark code
- Build reports
- Design schemas
- Tune individual queries
- Manage row/column security directly

Those belong to **workspace owners and data engineers**.

Fabric Admin vs Workspace Admin (critical distinction)

Aspect	Fabric Admin	Workspace Admin
Scope	Entire tenant	Single workspace
Controls	Platform rules	Content management
Cost control	Yes	No
Security baseline	Yes	Limited

Data modeling No Yes

Fabric Admin = **platform governor**

Workspace Admin = **project owner**

Typical enterprise setup

- Few **Fabric Admins** (central IT / platform team)
- Many **Workspace Admins** (domain teams)
- Clear separation of duties

This prevents chaos.

Mental model (lock this)

- **Fabric Admin** → sets laws
 - **Workspace Admin** → runs cities
 - **Users** → use services
-

One-line memory lock

Fabric Admin capabilities provide tenant-wide control over security, cost, governance, and feature usage, ensuring Microsoft Fabric runs safely and predictably at scale.

Fabric Gateways — what they are, why they exist, and how they work



On-premises data gateway

One gateway for multiple cloud services and experiences

Cloud services



Gateway Cloud Service
Data source connection credentials are encrypted



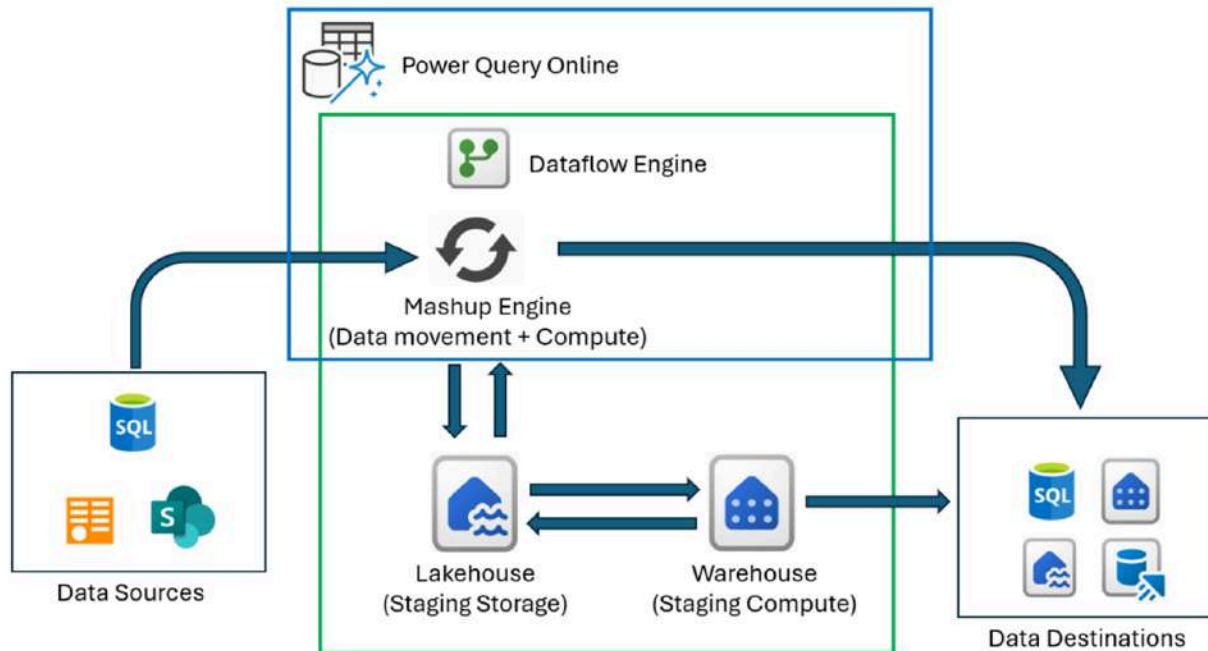
Azure Service Bus



Application Gateway
Data source connection credentials can only be decrypted by the gateway

On-premises data sources





1) What is a Fabric Gateway

A **Fabric Gateway** is a **secure bridge** that allows **Microsoft Fabric** to access **data sources that are not directly reachable from the cloud**.

Simple definition:

Fabric Gateway enables secure connectivity between Fabric and on-premises or private network data sources.

No inbound firewall opening.

No data stored in the gateway.

2) Why Fabric Gateways are needed

Microsoft Fabric is a **cloud SaaS platform**.

Many enterprise data sources are:

- On-premises databases
- Inside private networks
- Behind firewalls
- Not exposed to the internet

Without a gateway:

- Fabric **cannot reach** these systems

Gateway solves this safely.

3) What problems gateways solve

Without gateway:

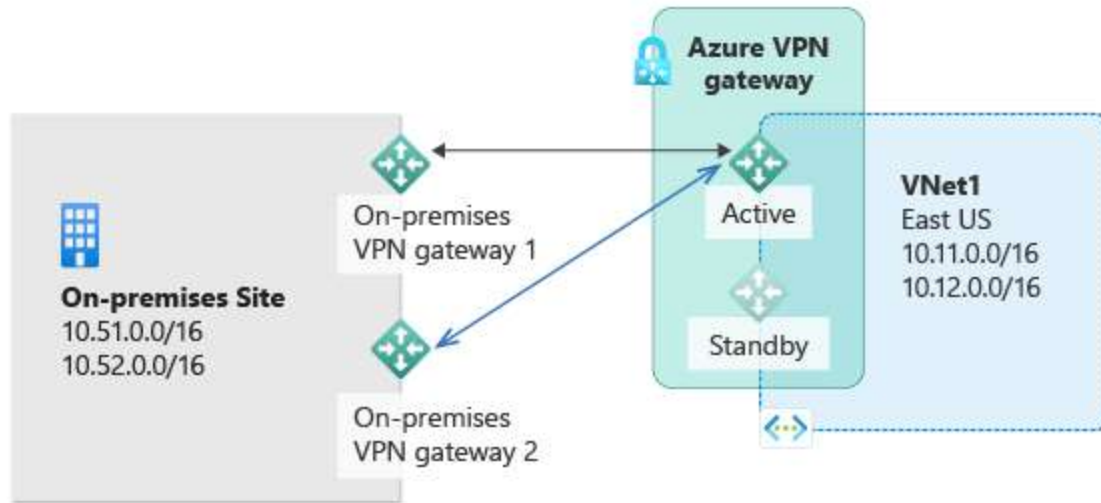
-  Cannot read on-prem SQL Server
-  Cannot access private Oracle / SAP
-  Cannot schedule refresh from on-prem
-  Security team blocks inbound access

With gateway:

-  Secure outbound connection
 -  No firewall inbound rules
 -  Centralized connectivity
 -  Enterprise-approved model
-

4) How Fabric Gateway works (internal flow)





On-prem Data Source



Fabric Gateway (outbound HTTPS)



Microsoft Fabric Service

Key points:

- Gateway initiates **outbound** connection
- Uses **HTTPS (443)**
- Encrypted end-to-end
- Fabric never directly connects inward

5) What Fabric Gateways are used for

Gateways are used by:

- Fabric Data Factory pipelines
- Dataflow Gen2
- Power BI semantic models
- Warehouse COPY / ingestion
- Scheduled refresh

Anything that needs **private data access**.

6) Types of Gateways (important)

1 On-premises Data Gateway (Standard)

- Most common
- Installed on a VM/server
- Supports many users and workloads
- Recommended for production

Use when:

- Enterprise environment
 - Shared access
 - High availability needed
-

2 On-premises Data Gateway (Personal)

- Single user
 - Desktop-based
 - Limited scenarios
 - Not recommended for Fabric production
-

7) What data sources are supported

Typical supported sources:

- SQL Server (on-prem)
- Oracle
- Teradata
- SAP
- File shares
- Private REST APIs

Cloud-public sources (Azure SQL, ADLS) **✗** don't need gateway.

8) What Fabric Gateway does NOT do

- **✗** Does not store data

-  Does not cache data
-  Does not transform data
-  Does not improve query speed

It is **connectivity only**.

9) Gateway vs OneLake (important distinction)

Aspect	Gateway	OneLake
Purpose	Connectivity	Storage
Location	On-prem / VM	Cloud
Stores data		
Needed for private sources		

Gateway gets data **into** Fabric.

OneLake stores data **inside** Fabric.

10) Security model (why enterprises accept it)

- Outbound-only traffic
- Uses Azure Service Bus
- Entra ID authentication
- Central admin control
- Auditable activity

This satisfies:

- Network security teams
 - Compliance requirements
-

11) When you need a Fabric Gateway (clear rule)

You **need a gateway** if:

- Source is on-prem
- Source is in private VNet
- Source is not publicly accessible

You **do not need a gateway** if:

- Source is Azure SQL (public)
 - Source is ADLS / OneLake
 - Source is SaaS (public API)
-

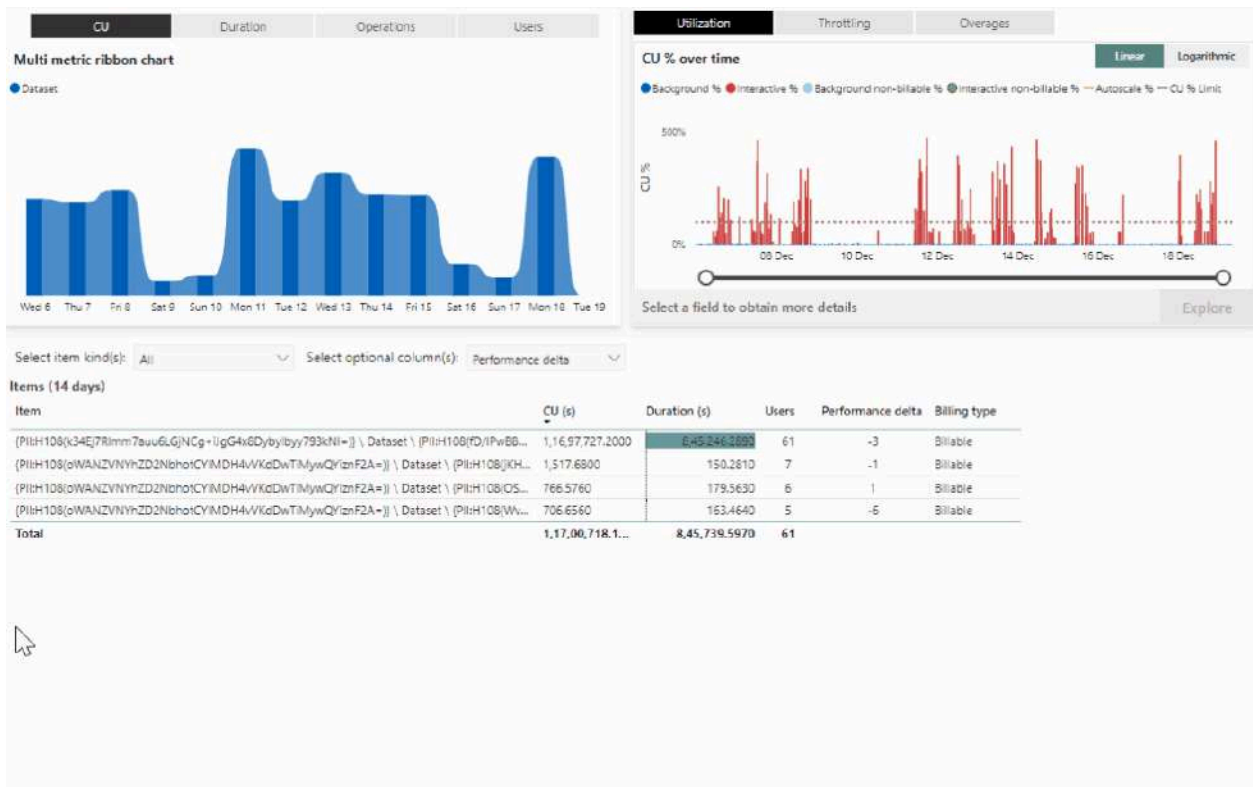
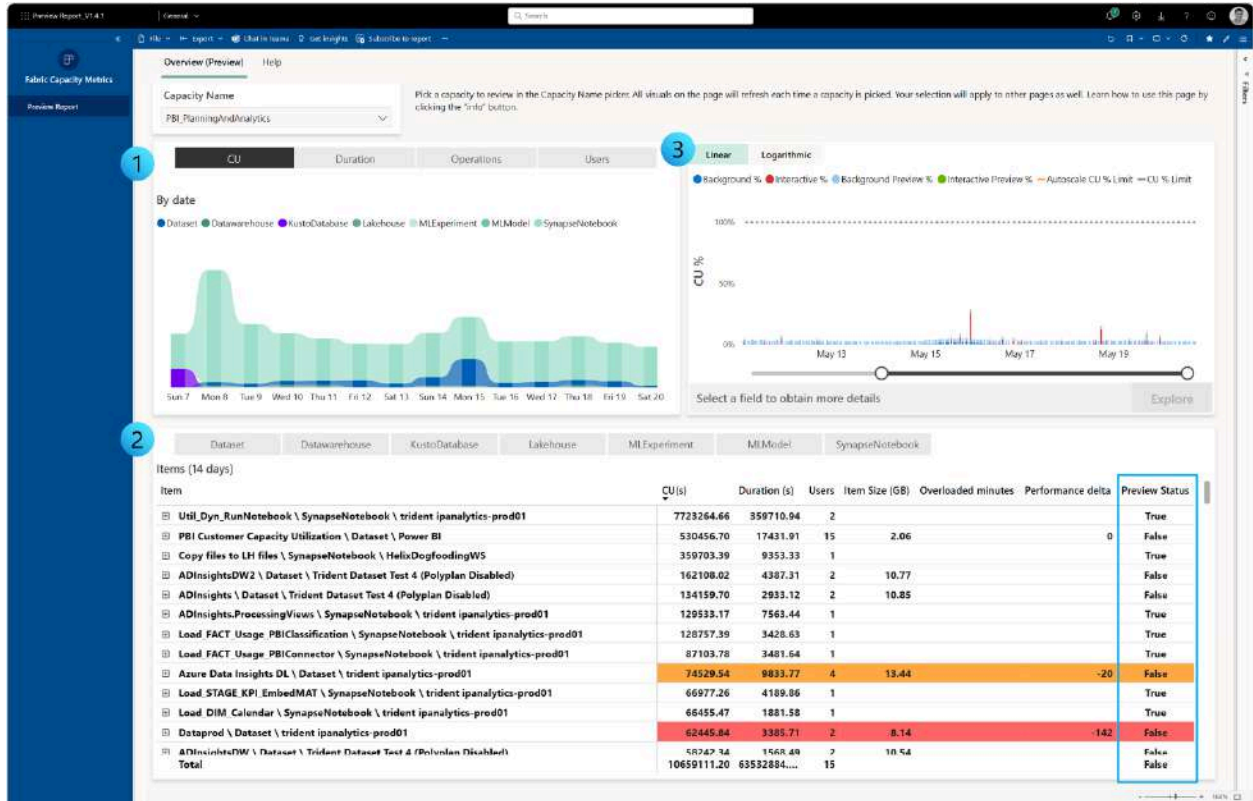
Mental model (lock this)

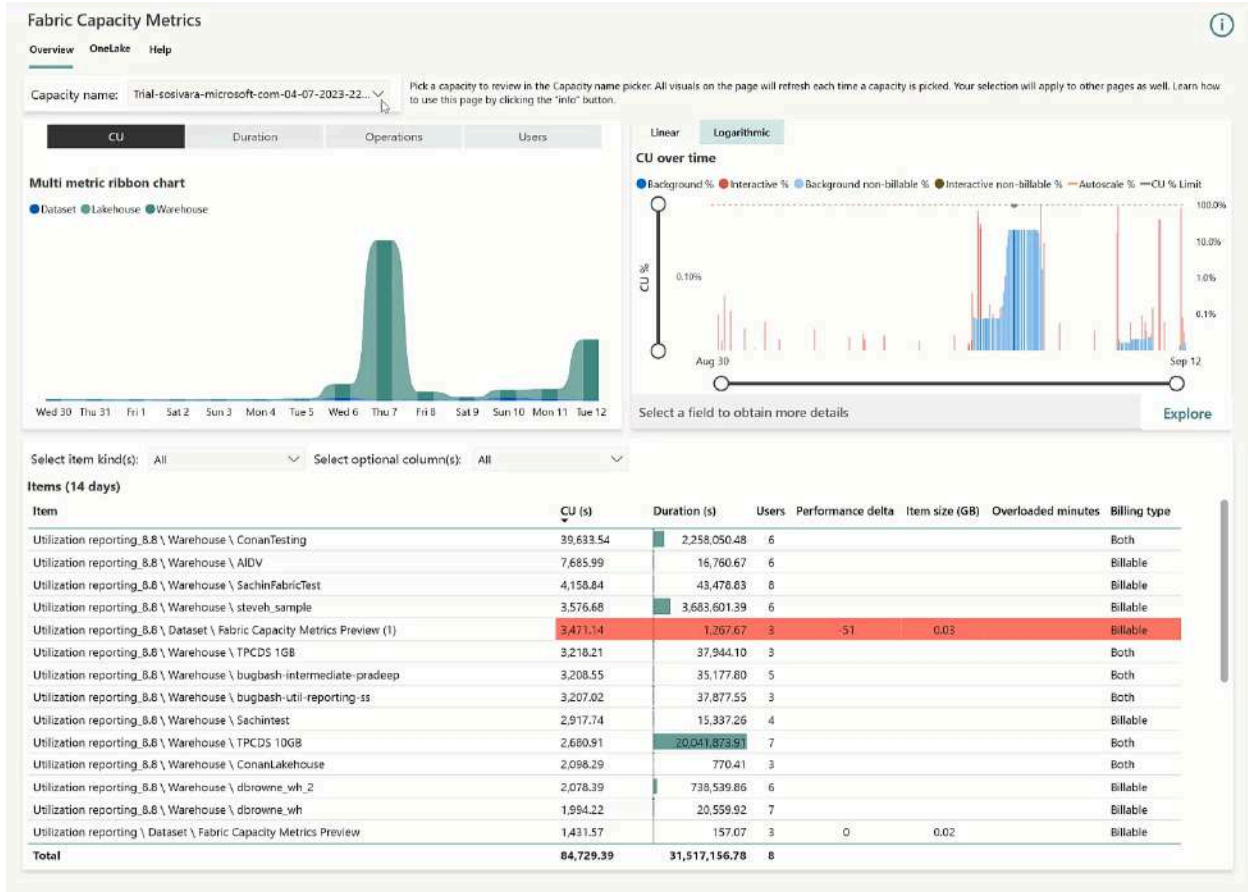
- **Fabric** lives in the cloud
 - **Gateway** reaches into private networks
 - **OneLake** stores the data
-

One-line memory lock

Fabric Gateways provide secure, outbound-only connectivity that allows Microsoft Fabric to access on-premises and private data sources without exposing them to the internet.

Fabric Capacity App — what it is, why it exists, and how it's used





1) What is the Fabric Capacity App

The **Fabric Capacity App** is a built-in monitoring and governance application that shows how Fabric capacity (compute) is being consumed across workloads, workspaces, and users.

Simple definition:

The Fabric Capacity App tells you where your Fabric compute is going, who is using it, and whether you are overloading it.

It is primarily used by **Fabric Admins** and **Capacity Admins**.

2) Why the Fabric Capacity App exists

Fabric runs on **capacity-based compute**, not per-service billing.

Without visibility:

- Teams overuse Spark or BI
- One workspace starves others
- Costs spike
- Performance degrades
- No accountability

The Capacity App solves:

- Cost transparency
 - Performance troubleshooting
 - Capacity planning
 - Governance enforcement
-

3) What problem it specifically answers

It answers **four critical questions**:

1. **Which workload is consuming capacity?**
(Spark, Warehouse, Power BI, Data Factory)
 2. **Which workspace is consuming capacity?**
 3. **Which users/jobs are causing spikes?**
 4. **Is capacity throttling or overloaded?**
-

4) What the Capacity App shows (core sections)

1 Capacity utilization

- CU (Capacity Units) usage
 - Peak vs sustained usage
 - Time-based trends
-

2 Workload breakdown

- Spark

- Warehouse
- Power BI
- Data Factory

Shows **who is expensive**.

3) Workspace-level consumption

- Capacity used per workspace
 - Helps identify noisy neighbors
-

4) Operation-level details

- Spark jobs
- SQL queries
- Refresh operations
- Pipelines

This is where root-cause analysis happens.

5) How Fabric capacity works (important context)

In **Microsoft Fabric**:

- Capacity = shared compute pool
- All workloads draw from same pool
- Heavy Spark job can impact BI
- Warehouse queries and Spark jobs compete

Capacity App visualizes this contention.

6) Who uses the Fabric Capacity App

Role	Why they use it
------	-----------------

Fabric Admin Governance & cost

Capacity Admin Performance tuning

Platform team Planning & scaling

Finance Chargeback/showback

Regular users usually don't need it.

7) What the Capacity App does NOT do

-  Does not optimize queries
-  Does not stop workloads automatically
-  Does not change capacity size
-  Does not enforce security

It is **observability**, not enforcement.

8) How it is used in real life (example)

Scenario

- Power BI reports are slow
- Users complain during business hours

Using Capacity App

- Check timeline → spike at 10 AM
- Drill down → Spark notebook running
- Workspace identified
- Job rescheduled to off-hours

Problem solved **without adding capacity**.

9) Capacity App vs Admin Portal (clarity)

Tool	Purpose
Fabric Admin Portal	Feature & security control
Fabric Capacity App	Compute usage & performance

They complement each other.

10) Best practices using Capacity App

- Monitor daily usage trends
 - Identify long-running Spark jobs
 - Separate heavy workloads into different capacities
 - Schedule batch jobs off-peak
 - Use results for cost allocation
-

Mental model (lock this)

- **Capacity** = engine
 - **Workloads** = cars
 - **Capacity App** = traffic camera
-

One-line memory lock

The Fabric Capacity App provides visibility into how compute capacity is consumed across Fabric workloads, enabling performance tuning, cost control, and governance.

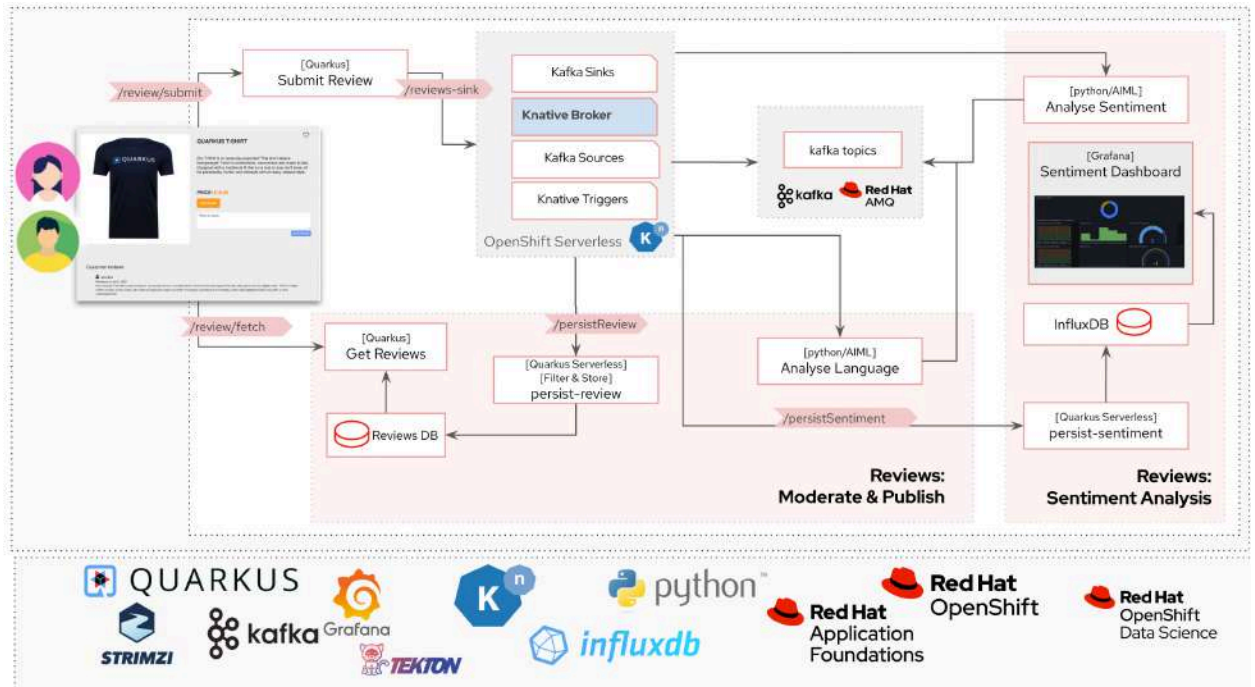
Gg

Gg

Gg

Real-Time Analytics — what it is and its core properties





1) What is Real-Time Analytics

Real-time analytics is the capability to ingest, process, and analyze data immediately as it is generated, with very low latency, so insights are available within seconds or milliseconds, not hours.

Simple definition:

Real-time analytics means analyzing data while it is still “in motion,” not after it is stored.

Examples:

- Fraud detection while a transaction is happening
- Monitoring server metrics live
- Tracking orders as they are placed
- IoT sensor alerts instantly

2) Why real-time analytics exists

Batch analytics answers:

“What happened yesterday?”

Real-time analytics answers:

“What is happening right now, and what should we do immediately?”

Without real-time analytics:

- **Decisions are delayed**
 - **Issues detected too late**
 - **Business reacts instead of acting**
-

3) How real-time analytics works (conceptual flow)

Events (clicks, sensors, logs)



Streaming ingestion



In-memory / low-latency processing



Real-time queries / dashboards / alerts

Key point: data is processed before it is written to long-term storage.

4) Core properties of Real-Time Analytics (very important)

1 Low latency

- **Data is processed in milliseconds to seconds**
- **No waiting for batch windows**

This is the defining property.

2 Continuous ingestion

- Data arrives continuously
 - System never “stops and starts”
 - Handles unbounded data streams
-

3 Incremental processing

- Processes only new events
 - Does not reprocess entire datasets
 - State is maintained in memory
-

4 Time-aware processing

- Uses event time, not just arrival time
 - Handles late-arriving events
 - Supports windows (last 5 minutes, last hour)
-

5 In-memory or near-memory execution

- Avoids disk I/O for speed
 - Uses caches and state stores
 - Optimized for fast aggregation
-

6 High throughput

- Can handle:
 - Millions of events per second
 - Scales horizontally
-

7 Fault tolerance

- System recovers automatically
 - No data loss on failures
 - Checkpoints and replay mechanisms
-

8 Exactly-once or at-least-once processing

- Guarantees how many times an event is processed
 - Critical for correctness (e.g., billing, fraud)
-

5) Real-Time Analytics vs Batch Analytics

A s p e c t	Real-Time	Batch
L a t e n c y	Milliseconds–seconds	Minutes–hours
D a t a t y p e	Streaming	Historical

P r o c e s s i n g	Continu ous	Peri odic
S t o r a g e - f i r s t	✗	✓
U s e c a s e	Alerts, monitori ng	Rep orti ng, tren ds

They are complementary, not competitors.

6) What real-time analytics is NOT

- **✗** Not just fast batch
- **✗** Not replacing warehouses
- **✗** Not always required
- **✗** Not cheap by default

Use it only when immediate insight matters.

7) Typical use cases

- Fraud detection
 - Live dashboards
 - Anomaly detection
 - IoT monitoring
 - Clickstream analytics
 - Log and security monitoring
-

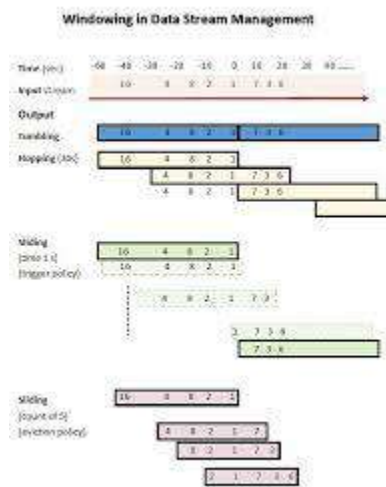
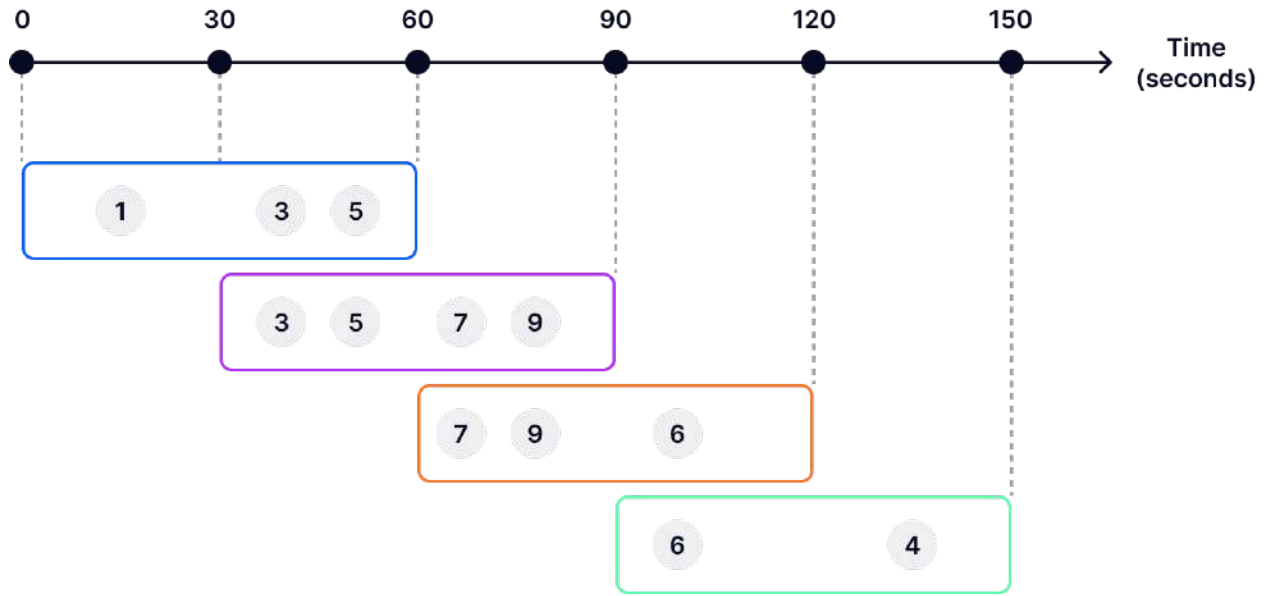
Mental model (lock this)

- Batch → analyze data *at rest*
 - Real-time → analyze data *in motion*
-

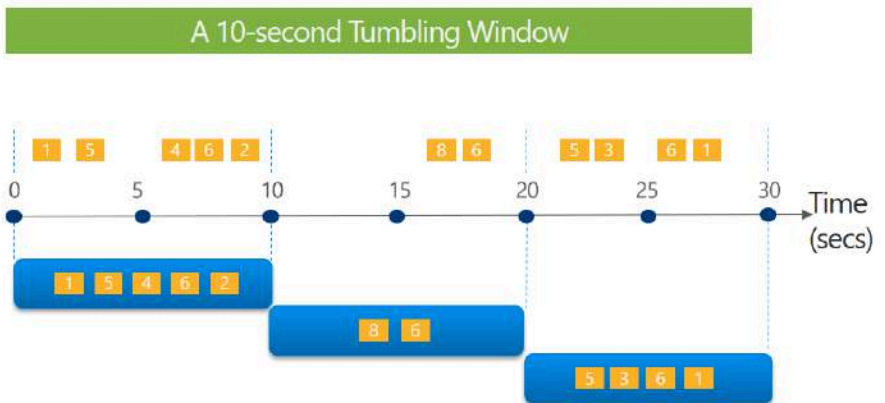
One-line memory lock

Real-time analytics processes continuously arriving data with very low latency to deliver immediate insights and actions while events are still happening.

Windows in Real-Time Analytics — what they are and the different types (with easy examples)



Tell me the count of tweets per time zone every 10 seconds



```
SELECT TimeZone, COUNT(*) AS Count
FROM TwitterStream TIMESTAMP BY CreatedAt
GROUP BY TimeZone, TumblingWindow(second,10)
```

First, lock the concept

In real-time analytics, data never ends (infinite stream).
To analyze it, we cut time into windows.

A window = a fixed or logical time boundary used to group streaming events.

Without windows:

- No SUM
- No COUNT
- No AVG

1 Tumbling Window (fixed, non-overlapping)

What it is

- Fixed-size window

- Does not overlap
- Each event belongs to exactly one window

|-----5 min-----|-----5 min-----|-----5 min-----|

Easy example

Count orders every 5 minutes.

T i m e	O r d e r s
------------------	----------------------------

1 0 : 0 0 - 1 0 : 0 5	1 2 0
---	-------------

1 0 : 0 5 - 1 0	9 5
--------------------------------------	--------

:
1
0

1	1
0	4
:	0
1	
0	
—	
1	
0	
:	
1	
5	

Each 5-minute block is independent.

When to use

- Periodic reporting
 - Billing per interval
 - KPI snapshots
-

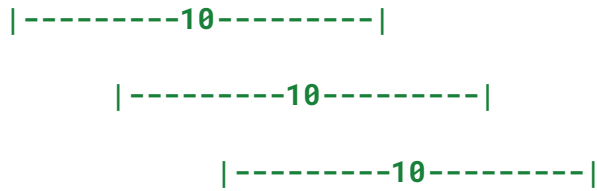
2 Sliding Window (fixed, overlapping)

What it is

- Fixed-size window
- Overlaps
- Moves forward by a smaller step

Window size = 10 min

Slide = 5 min



Easy example

Calculate last 10 minutes sales, updated every 5 minutes.

At 10:10:

- Data from 10:00–10:10

At 10:15:

- Data from 10:05–10:15

Same events appear in multiple windows.

When to use

- Trend analysis
- Moving averages
- Real-time dashboards

③ Session Window (activity-based, dynamic)

What it is

- Window size is not fixed
- Defined by user activity
- Ends after inactivity gap

User active → window open

No activity for X min → window closes

Easy example

User browsing session:

- User clicks at 10:00
- Next click at 10:03
- Next click at 10:06
- No activity till 10:20

If session gap = 5 minutes:

- One session: 10:00–10:06
 - Session closes at 10:11
-

When to use

- User behavior analysis
 - Session analytics
 - Clickstream analysis
-

4 Hopping Window (controlled overlap)

What it is

- Fixed-size window
- Overlaps in a controlled way
- Defined by:
 - Window size
 - Hop interval

Size = 10 min

Hop = 2 min

More overlap than sliding windows.

Easy example

Check errors in:

- 10-minute window
- Updated every 2 minutes

Provides high-resolution monitoring.

When to use

- Fine-grained monitoring
 - Alerting systems
 - High-sensitivity detection
-

5 Global Window (all-time window)

What it is

- One infinite window
 - Never closes
 - Aggregates from start of stream
-

Easy example

Total orders since system started.

```
Total_orders = running_sum(events)
```

When to use

- Running totals
- System-wide counters

⚠ Rarely used alone (memory risk).

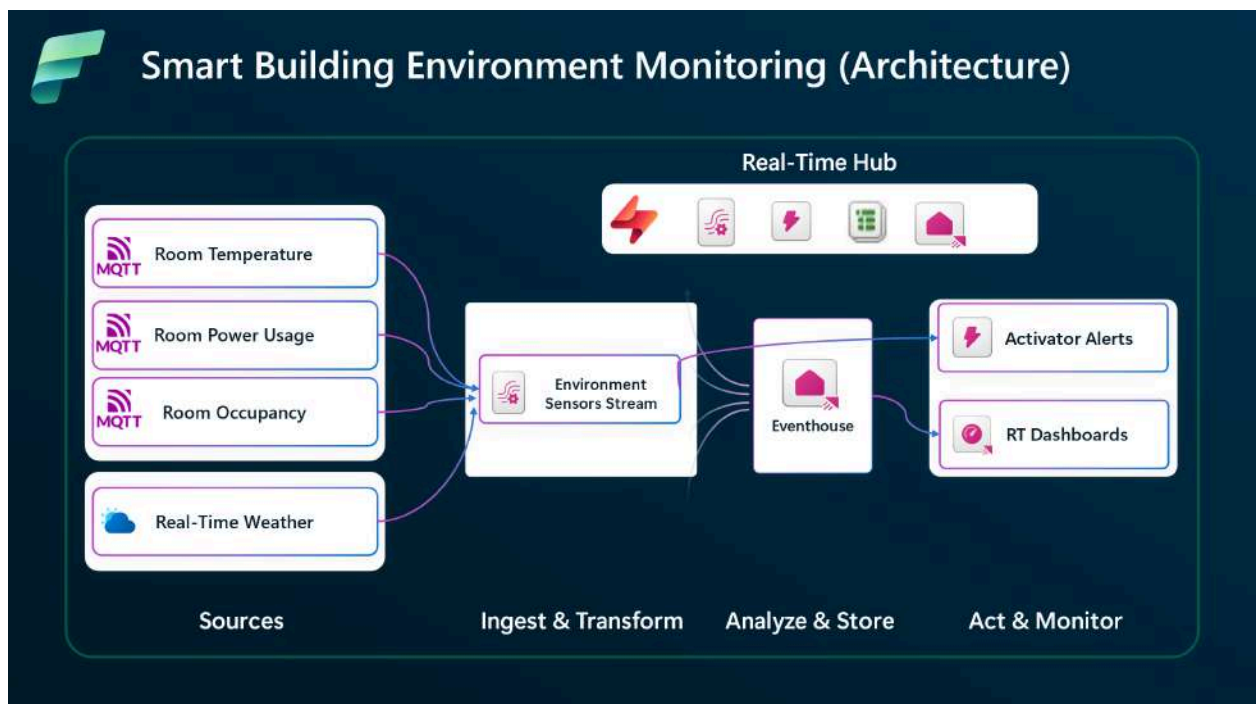
One mental shortcut (very important)

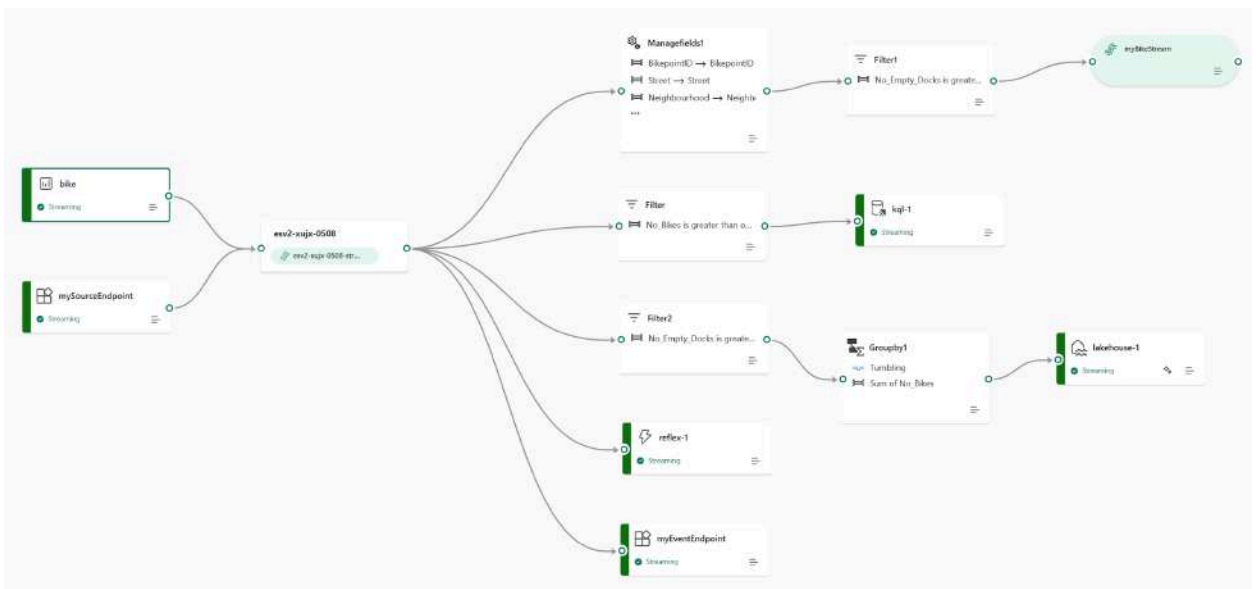
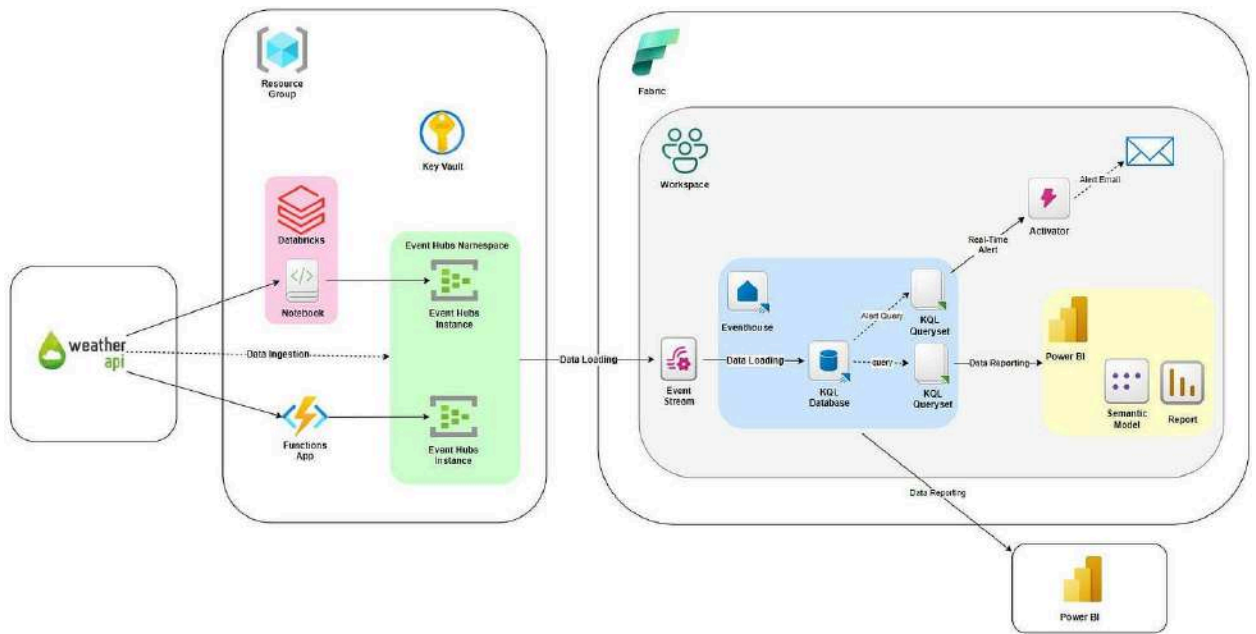
- Tumbling → calendar-based
 - Sliding → moving average
 - Session → user behavior
 - Hopping → alert precision
 - Global → lifetime metric
-

One-line memory lock

Windows divide infinite streaming data into time-based or activity-based groups so real-time analytics can aggregate and analyze events correctly.

Eventstream — meaning, purpose, and mechanics (Microsoft Fabric context)





Definition

An Eventstream is a real-time ingestion and routing layer that continuously captures events as they occur and forwards them to real-time analytics, storage, or downstream consumers.

Short form:

Eventstream moves data while it is still in motion.

Why Eventstream exists

Batch systems fail when:

- Data arrives continuously
- Decisions must be immediate
- Latency matters

Eventstream solves:

- Continuous ingestion
 - Sub-second to seconds latency
 - Decoupling producers from consumers
 - Fan-out to multiple targets
-

What an Eventstream handles

Events, not files.

Examples:

- Clicks
- Transactions
- Sensor readings
- Logs
- Metrics

Each event is a small message with:

- Timestamp
 - Key
 - Payload
-

How Eventstream works (pipeline logic)

Event Source



Eventstream (ingest + route)



Destinations (analytics / storage / alerts)

Core behavior:

- Always on
 - Append-only
 - Ordered by time (per key)
 - Scales horizontally
-

Components of an Eventstream

1) Sources (where events come from)

- Applications
 - IoT devices
 - Message brokers
 - Event hubs
 - REST producers
-

2) Stream processing (lightweight)

- Filtering
- Mapping
- Enrichment
- Routing

Not heavy ETL. Minimal latency.

3) Destinations (where events go)

- Real-Time Analytics (KQL)

- Lakehouse (Delta)
- Warehouse
- Dashboards
- Alerts

Same event can go to multiple destinations.

Eventstream vs Batch ingestion

A s p e c t	Eventstr eam	B at c h
D a t a a r r i v a l	Continu ous	P er io di c
L a t e n c y	Milliseco nds–sec onds	Mi n ut es – h o ur s

D a t a t y p e	Events	Fi le s
P r o c e s s i n g	Increme ntal	F ul l re lo a d
U s e c a s e	Monitori ng, alerts	R e p or ti n g

Eventstream vs Data Factory

A s p e	E v e n t s	D a t a F a
------------------	----------------------------	----------------------------

c
t

t
r
e
a
m

c
t
o
r
y

P
a
t
t
e
r
n

S
t
r
e
a
m
i
n
g

O
r
c
h
e
s
t
r
a
t
i
o
n

L
a
t
e
n
c
y

R
e
a
l
-
t
i
m
e

B
a
t
c
h

D
a
t
a

E
v
e
n
t
s

F
i
l
e
s
/
t
a
b
l

		e s
S t a t e	C o n t i n u o u s	S t a t e l e s s r u n s

They are complementary, not substitutes.

Why Eventstream is critical for Real-Time Analytics

Real-time analytics requires:

- Continuous ingestion
- Time-aware processing
- Immediate availability

Eventstream provides the front door for real-time data.

Typical Fabric architecture

Apps / Devices



Eventstream



Real-Time Analytics (KQL)



Dashboards / Alerts

Optional fan-out:

- Parallel write to Lakehouse for history

What Eventstream is NOT

- Not a data warehouse
- Not a file ingestion tool
- Not batch ETL
- Not long-term storage

It is movement, not storage.

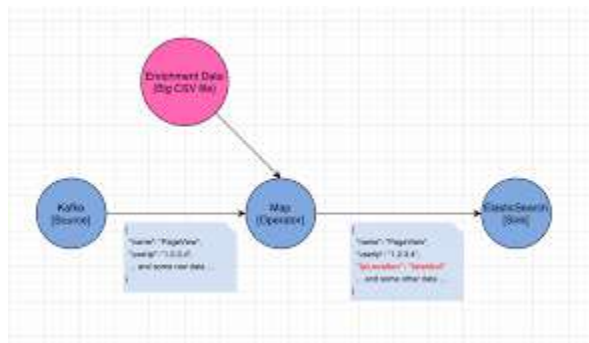
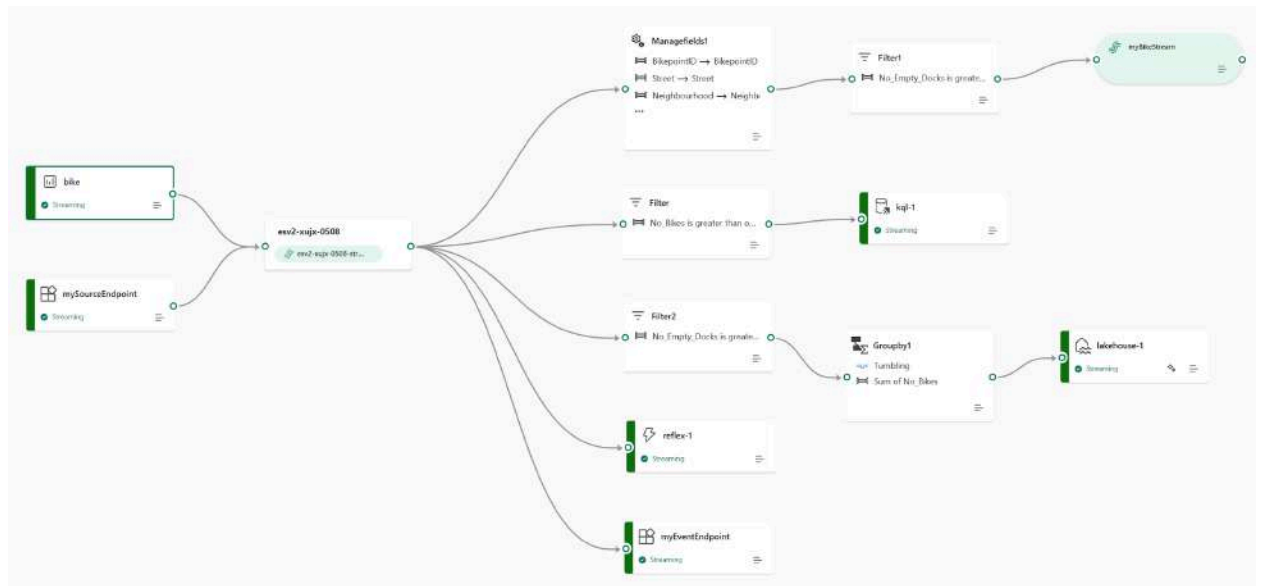
Mental model (lock this)

- Batch → events after they settle
- Eventstream → events while they happen

One-line memory lock

An Eventstream continuously ingests and routes live events with low latency, enabling real-time analytics before data is stored or batched.

Transformations in Eventstream — what they are, why they exist, how they work





Definition

Eventstream transformations are lightweight, real-time operations applied to events while they are flowing, before they reach destinations.

They change shape, content, or routing of events with near-zero latency.

Why transformations are needed

Raw events are noisy and unstructured.

Problems without transformations:

- Irrelevant events flood analytics
- Fields are inconsistent
- Downstream systems do extra work
- Latency increases

Transformations fix data before storage or analytics.

Design rule (critical)

Eventstream transformations are:

- Simple
- Stateless
- Fast

They are not ETL and not Spark.

Where transformations sit

Event Source

↓

Eventstream

└ Transform (filter/map/enrich)

↓

Destinations (KQL / Lakehouse / Alerts)

Core transformation types (with easy examples)

1) Filter — keep or drop events

Purpose: Remove noise.

Example

- Keep only `order_status = 'FAILED'`

Incoming:

```
{ order_id: 1, status: "SUCCESS" }
```

```
{ order_id: 2, status: "FAILED" }
```

After filter:

```
{ order_id: 2, status: "FAILED" }
```

Use when:

- Monitoring
 - Alerts
 - Error tracking
-

2) Projection / Map — select or rename fields

Purpose: Reduce payload size and standardize schema.

Example

Incoming:

```
{ id: 101, customerName: "A", amount: 500, currency: "INR" }
```

After projection:

```
{ order_id: 101, amount: 500 }
```

Use when:

- Only few fields are needed
 - Schema normalization
-

3) Derivation — create new fields

Purpose: Add computed values.

Example

Incoming:

```
{ amount: 1000 }
```

Derived:

```
{ amount: 1000, tax: 180 }
```

Use when:

- Simple calculations
 - Flags and indicators
-

4) Enrichment — add reference data

Purpose: Add context.

Example

- Add **country** using **store_id**

Incoming:

```
{ store_id: 12, sales: 200 }
```

Enriched:

```
{ store_id: 12, sales: 200, country: "India" }
```

Use when:

- Joining with small lookup
- Real-time context needed

5) Routing — send events to different destinations

Purpose: Fan-out or conditional delivery.

Example

- Errors → Alerts
- All events → Lakehouse

```
IF status = 'FAILED' → Alert system
```

```
ELSE → Storage
```

Use when:

- Multiple consumers
- Different SLAs

6) Flattening — handle nested JSON

Purpose: Make data queryable.

Example

```
Incoming:
```

```
{ order: { id: 1, value: 300 } }
```

```
Flattened:
```

```
{ order_id: 1, order_value: 300 }
```

Use when:

- Downstream expects flat schema

What transformations do NOT do

- No heavy joins
- No windowing aggregations
- No stateful processing
- No historical lookups
- No schema enforcement

Those belong to real-time analytics engines or Lakehouse/Spark.

Correct usage pattern

Eventstream

- Filter + Map + Enrich
- Real-Time Analytics (windows, aggregations)
- Lakehouse (history)

Do minimum work early, heavy work later.

Failure behavior

- Transformation failure drops or reroutes event
- Stream continues
- No full pipeline crash

Designed for resilience.

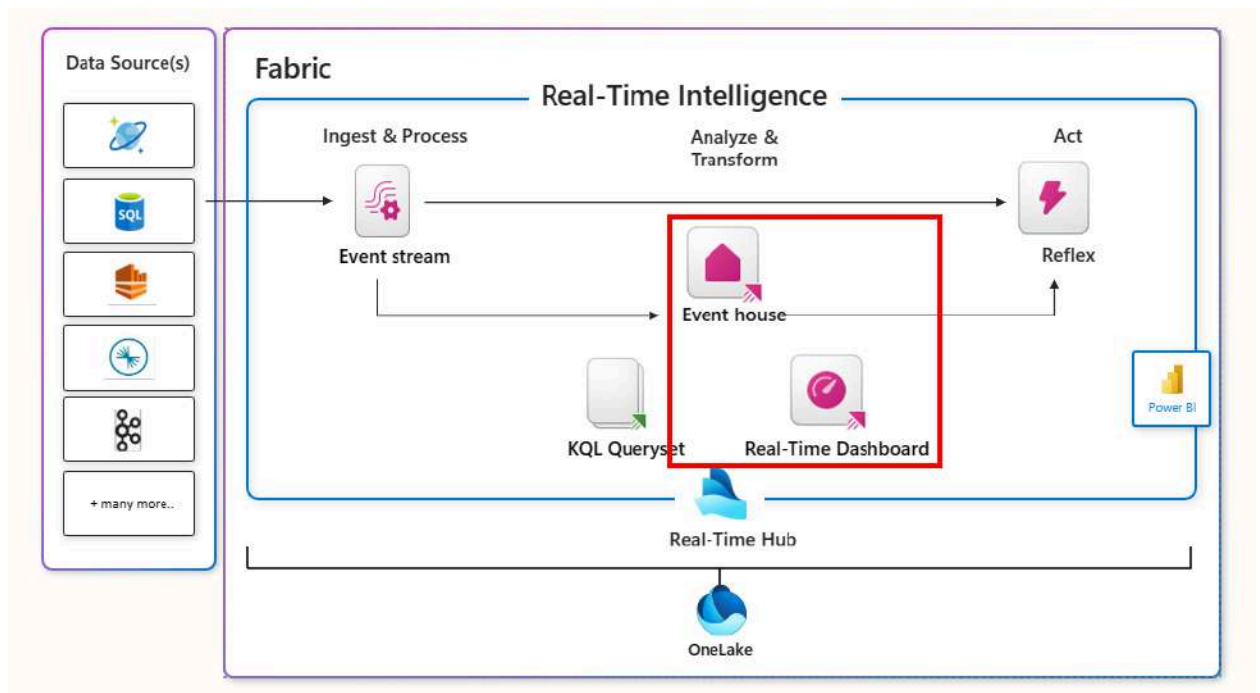
Mental model (lock this)

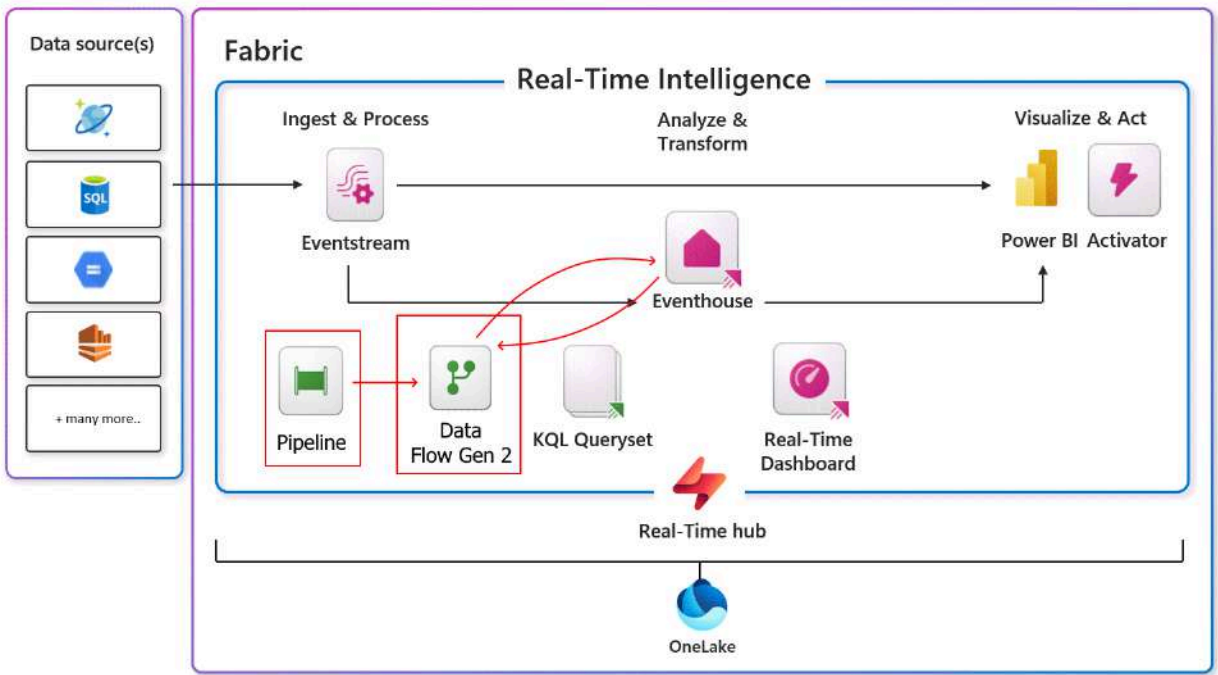
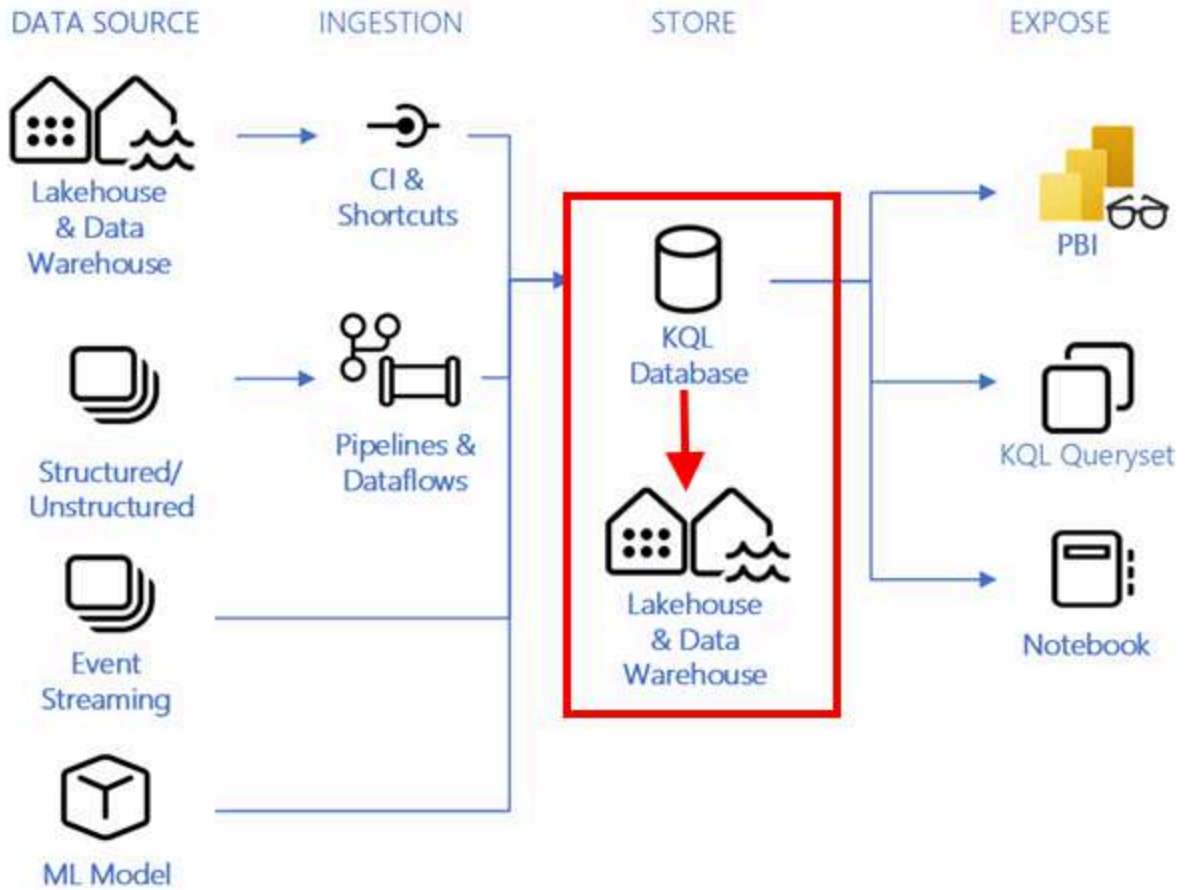
- Eventstream transforms clean events
- Analytics engines analyze events
- Storage systems store events

One-line memory lock

Transformations in Eventstream are lightweight, stateless, real-time operations used to filter, reshape, enrich, and route events before they reach analytics or storage.

Eventhouse and KQL Database — deep, structured explanation (Fabric Real-Time Analytics)





1) What is Eventhouse

Eventhouse is the real-time analytics container in Microsoft Fabric designed to ingest, store, and analyze streaming data at very low latency.

Precise definition:

Eventhouse is a logical workspace that hosts one or more KQL Databases and manages real-time ingestion, retention, and querying of event data.

Think of Eventhouse as the real-time analytics environment, not the database itself.

Why Eventhouse exists

Traditional systems fail for:

- Millions of events per second
- Sub-second queries
- Log-style append-only data
- High-cardinality filtering

Eventhouse solves:

- Massive ingestion throughput
 - Low-latency queries
 - Time-series analytics
 - Operational analytics
-

What Eventhouse contains

An Eventhouse includes:

- One or more KQL Databases
- Ingestion pipelines (Eventstream, Event Hub, APIs)
- Retention and caching policies
- Query and monitoring endpoints

Eventhouse = container + control plane

2) What is a KQL Database (KQL DB)

A KQL Database is the actual data store and query engine inside Eventhouse.

Precise definition:

A KQL Database stores event data in a columnar, compressed, time-indexed format and is queried using KQL (Kusto Query Language).

If Eventhouse is the building, KQL DB is the engine room.

Core characteristics of KQL DB

- Append-only (event-style)
- Columnar storage
- Time-ordered ingestion
- Automatic indexing
- Optimized for reads, not updates

Designed for:

- Logs
 - Metrics
 - Clickstreams
 - Telemetry
 - IoT events
-

3) Eventhouse vs KQL DB (critical distinction)

As
pe
ct

Eventhouse

KQL
Database

What it is	Container / environment	Actual database
Stores data	✗	✓
Query engine	✗	✓
Can exist alone	✗	✗ (needs Eventhouse)
Scope	Logical	Physical
Purpose	Management	Storage + analytics

4) How data flows (end-to-end)

Event Source



Eventstream



Eventhouse



KQL Database



KQL Queries / Dashboards / Alerts

Eventhouse manages.
KQL DB executes.

5) How ingestion works internally

Step-by-step

1. Events arrive (JSON, CSV, structured)
2. Ingestion service batches events
3. Data written to KQL DB
4. Columns compressed and indexed
5. Data becomes queryable in seconds

No Spark.
No Delta.
No files exposed.

6) How querying works (why it's fast)

KQL DB uses:

- Column pruning
- Time-based partitioning
- In-memory caching
- Vectorized execution

Typical query:

Events

```
| where Timestamp > ago(5m)

| summarize count() by EventType
```

This scans:

- Only required columns
- Only last 5 minutes of data

Result: sub-second response.

7) Storage model (important)

KQL DB storage is:

- Columnar
- Compressed
- Time-segmented
- Append-only

Operations supported:

- Insert (append)
- Read (query)

Operations NOT supported:

- Row-by-row updates

- Frequent deletes
- Transactions

This is by design.

8) Retention and hot/cold data

KQL DB supports:

- Hot cache (fast, recent data)
- Warm data (slower)
- Retention policies (auto-delete old data)

Example:

- Keep 30 days of logs
- Automatically purge older data

This keeps cost and performance controlled.

9) Eventhouse vs Lakehouse vs Warehouse (real clarity)

As pe ct	E v e n t h o u s e	Lak eho use	W a r e h o u s e
Da ta	E v e	File s +	T a b

type	n t s	tabl es	l e s
Lat en cy	M i l i s e c o n d s	Sec ond s-m inut es	S e c o n d s
Qu ery lan gu age e	K Q L	Spar k SQL	T - S Q L
Up dat es	A p p e n d - o n l y	Full CRU D	F u i i C R U D

Use case	Monitoring	Processing	Reporting
-------------	------------	------------	-----------

Eventhouse is not a replacement for Lakehouse or Warehouse.

10) When to use Eventhouse + KQL DB

Use when:

- Real-time monitoring
- Log analytics
- IoT telemetry
- Fraud signals
- Live dashboards

Avoid when:

- Heavy joins across many datasets
 - Complex ETL
 - BI dimensional modeling
-

11) Typical enterprise pattern

Live events

→ Eventstream

→ Eventhouse (KQL DB)

→ Alerts / Dashboards

Same events

→ Lakehouse (Delta)

→ Warehouse

→ BI reports

Eventhouse = now

Lakehouse/Warehouse = history

12) Mental model (lock this permanently)

- Eventhouse → real-time analytics workspace
- KQL Database → high-speed event store
- KQL → query language optimized for time + filters

One-line memory lock

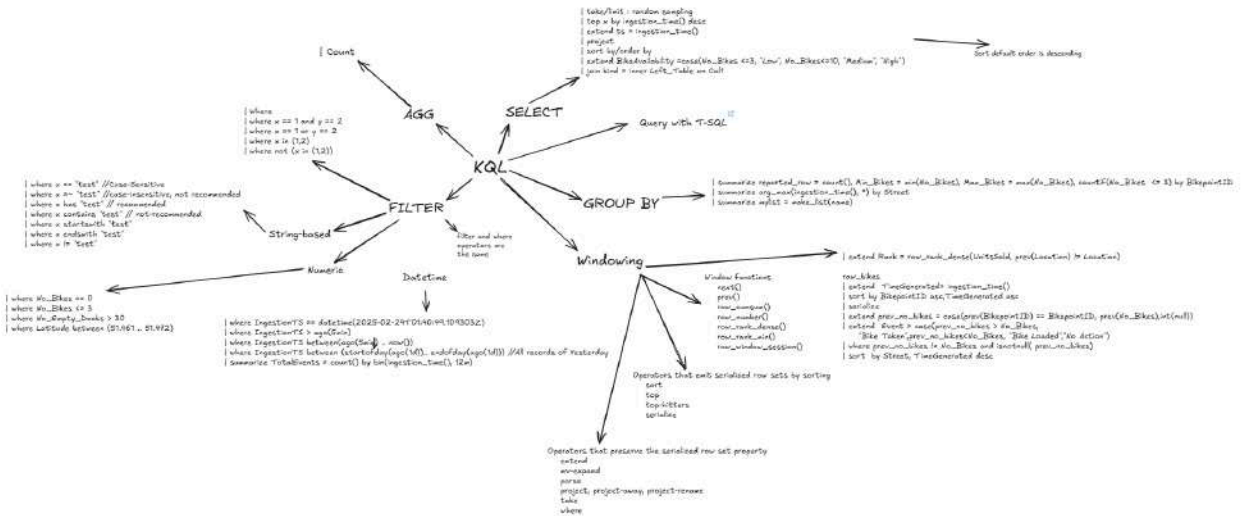
Eventhouse is the real-time analytics container in Fabric, and the KQL Database inside it stores and queries high-volume event data with ultra-low latency using KQL.

KQL from scratch — SELECT and TOP, explained simply with examples

Run	Time range: Last 24 hours	Save	Share	New alert rule	Export	Pin to	Format query
6	// Will search all columns in the Perf table for the value "Memory"						
7	Perf						
8	search "Memory"						
9							
10							
11							

TimeGenerated [UTC]	Computer	ObjectName	CounterName	InstanceName	CounterValue	CounterPath	Type
> 4/7/2022, 7:06:16.810 PM	DC01.na.cont...	Memory	Available MBytes		5729	\\DC01.na.contosohotels.com\Memory\Available MBytes	Pe
> 4/7/2022, 7:08:29.813 PM	SQL12.na.con...	Memory	Available MBytes		11,862	\\SQL12.na.contosohotels.com\Memory\Available MBytes	Pe
> 4/7/2022, 7:07:52.970 PM	AppFE0000FSU	Memory	Available MBytes		6,613	\\AppFE0000FSU\Memory\Available MBytes	Pe
> 4/7/2022, 7:08:56.700 PM	CH1-UBNTVM	Memory	Available MBytes ...	Memory	897	\\CH1-UBNTVM\Memory\Memory\Available MBytes Mem...	Pe
> 4/7/2022, 7:08:48.017 PM	CH1-AVSMG...	Memory	Available MBytes		4,870	\\CH1-AVSMGMTVM\Memory\Available MBytes	Pe
> 4/7/2022, 7:09:16.923 PM	SQL01.na.con...	Memory	Available MBytes		12,003	\\SQL01.na.contosohotels.com\Memory\Available MBytes	Pe
> 4/7/2022, 7:09:16.867 PM	DC01.na.cont...	Memory	Available MBytes		5,604	\\DC01.na.contosohotels.com\Memory\Available MBytes	Pe
> 4/7/2022, 6:56:10.603 PM	App8E01.nac...	Memory	Available MBytes		5,365	\\App8E01.na.contosohotels.com\Memory\Available MBytes	Pe
> 4/7/2022, 7:10:06.553 PM	DC10.na.cont...	Memory	Available MBytes		5,484	\\DC10.na.contosohotels.com\Memory\Available MBytes	Pe
> 4/7/2022, 7:10:48.140 PM	CH1-AVSMG...	Memory	Available MBytes		4,812	\\CH1-AVSMGMTVM\Memory\Available MBytes	Pe
> 4/7/2022, 7:14:56.523 PM	RETAILVM01	Memory	Available MBytes		5,570	\\RETAILVM01\Memory\Available MBytes	Pe
> 4/7/2022, 6:58:29.303 PM	SQL12.na.con...	Memory	Available MBytes		11,880	\\SQL12.na.contosohotels.com\Memory\Available MBytes	Pe
> 4/7/2022, 7:19:31.263 PM	AppFE0000000	Memory	Available MBytes		6,082	\\AppFE0000000\Memory\Available MBytes	Pe
> 4/7/2022, 7:20:06.727 PM	DC10.na.cont...	Memory	Available MBytes		5,401	\\DC10.na.contosohotels.com\Memory\Available MBytes	Pe
> 4/7/2022, 6:55:41.593 PM	AppFE0000003	Memory	Available MBytes		5,913	\\AppFE0000003\Memory\Available MBytes	Pe
> 4/7/2022, 6:58:55.457 PM	JBCX00	Memory	Available MBytes		5,875	\\JBCX00\Memory\Available MBytes	Pe
> 4/7/2022, 6:57:04.210 PM	DC00.na.cont...	Memory	Available MBytes		5,293	\\DC00.na.contosohotels.com\Memory\Available MBytes	Pe
> 4/7/2022, 7:22:29.620 PM	SQL12.na.con...	Memory	Available MBytes		11,883	\\SQL12.na.contosohotels.com\Memory\Available MBytes	Pe
> 4/7/2022, 7:00:16.690 PM	DC01.na.cont...	Memory	Available MBytes		5,735	\\DC01.na.contosohote's.com\Memory\Available MBytes	Pe

2s 524ms | Display time (UTC+00:00) | Query details | 1 - 20 of 24739



Timestamp	ExperimentType	DurationSeconds	EnergyLevel	SuccessRate
2025-08-01T10:00:00Z	WarpDriveTest	120	85	0.92
2025-08-01T11:15:00Z	PhotonBlast	45	60	0.75
2025-08-01T12:30:00Z	WarpDriveTest	180	95	0.88
2025-08-01T13:45:00Z	TeleportTrial	90	70	0.96
2025-08-01T15:00:00Z	PhotonBlast	60	55	0.82
2025-08-02T09:00:00Z	WarpDriveTest	150	90	0.91
2025-08-02T10:30:00Z	TeleportTrial	75	65	0.89

1) What is KQL (one sentence)

KQL (Kusto Query Language) is a read-only, pipe-based query language used to filter, shape, and analyze event data.

Key rule: KQL does not use SELECT like SQL.

2) The single rule you must remember

KQL = pipeline

Table

| operator

| operator

| operator

Read it as:

“Take the table → do this → then this → then this”

3) Your first KQL query (equivalent of SELECT *)

SQL mindset

```
SELECT * FROM Orders;
```

KQL way

```
Orders
```

That's it.

Just writing the table name returns all rows and columns.

4) Selecting specific columns (KQL **project**)

In SQL:

```
SELECT order_id, amount FROM Orders;
```

In KQL:

```
Orders
```

```
| project order_id, amount
```

What **project** does

- Picks columns
 - Drops everything else
-

5) Limiting rows — **take** and **top**

5.1 **take** (any rows, no order guarantee)

Orders

```
| take 5
```

- Returns any 5 rows
- Fast
- No sorting

Use for:

- Quick inspection
 - Debugging
-

5.2 **top** (ordered rows)

Orders

```
| top 5 by amount
```

- Returns top 5 rows by amount
 - Sorted descending by default
-

Ascending order

Orders

```
| top 5 by amount asc
```

6) **top** vs **take** (critical difference)

O p e r a t o r	What it does
t a k e	Any N rows
t o p	Best N rows by a column

Rule:

- Exploration → **take**
- Ranking → **top**

7) SELECT + WHERE equivalent (basic filter)

SQL:

```
SELECT * FROM Orders WHERE amount > 1000;
```

KQL:

```
Orders
| where amount > 1000
```

Pipeline meaning:

- Start with Orders
 - Keep rows where amount > 1000
-

8) Combine **where**, **project**, and **top**

Orders

```
| where amount > 1000  
  
| project order_id, customer, amount  
  
| top 5 by amount
```

Read it slowly:

1. Orders table
 2. Filter expensive orders
 3. Keep only needed columns
 4. Show top 5 by amount
-

9) Sorting without limiting — **sort by**

Orders

```
| sort by amount desc
```

- Sorts all rows
 - Does not limit count
-

10) Common beginner mistakes (avoid)

- ✗ Using **SELECT**
- ✗ Writing SQL-style commas
- ✗ Expecting updates/deletes
- ✗ Forgetting **|** between operators

KQL is query-only.

11) Mental translation (SQL → KQL)

SQL	KQL
SELE CT *	Table name
SELE CT col1, col2	proje ct col1, col2
WHER E	where
TOP 5	top 5 by col
ORDE R BY	sort by

12) Tiny practice dataset (imagine this table)

Query

Orders

| where amount > 700

| top 2 by amount

Final lock-in rules

- KQL starts with table
- **project** = select columns
- **where** = filter rows
- **take** = quick sample
- **top** = ranked results
- Pipes (|) connect steps

One-line memory lock

In KQL, you start with a table and use pipe operators like **project**, **where**, and **top** to shape results—there is no SELECT keyword.

KQL Date & Time Functions + Other Essential KQL Functions — complete notes

```

182
183 Perf
184 | where CounterName == "Avg. Disk sec/Read"
185 | where CounterValue > 0
186 | take 100 // done just to give us a small dataset to demo
187 | extend HowLongAgo=( now() - TimeGenerated )
188 | , RightNow = now()
189 | project TimeGenerated
190 | , RightNow
191 | , HowLongAgo
192 | , Computer
193 | , CounterName
194 | , CounterValue
195
196

```

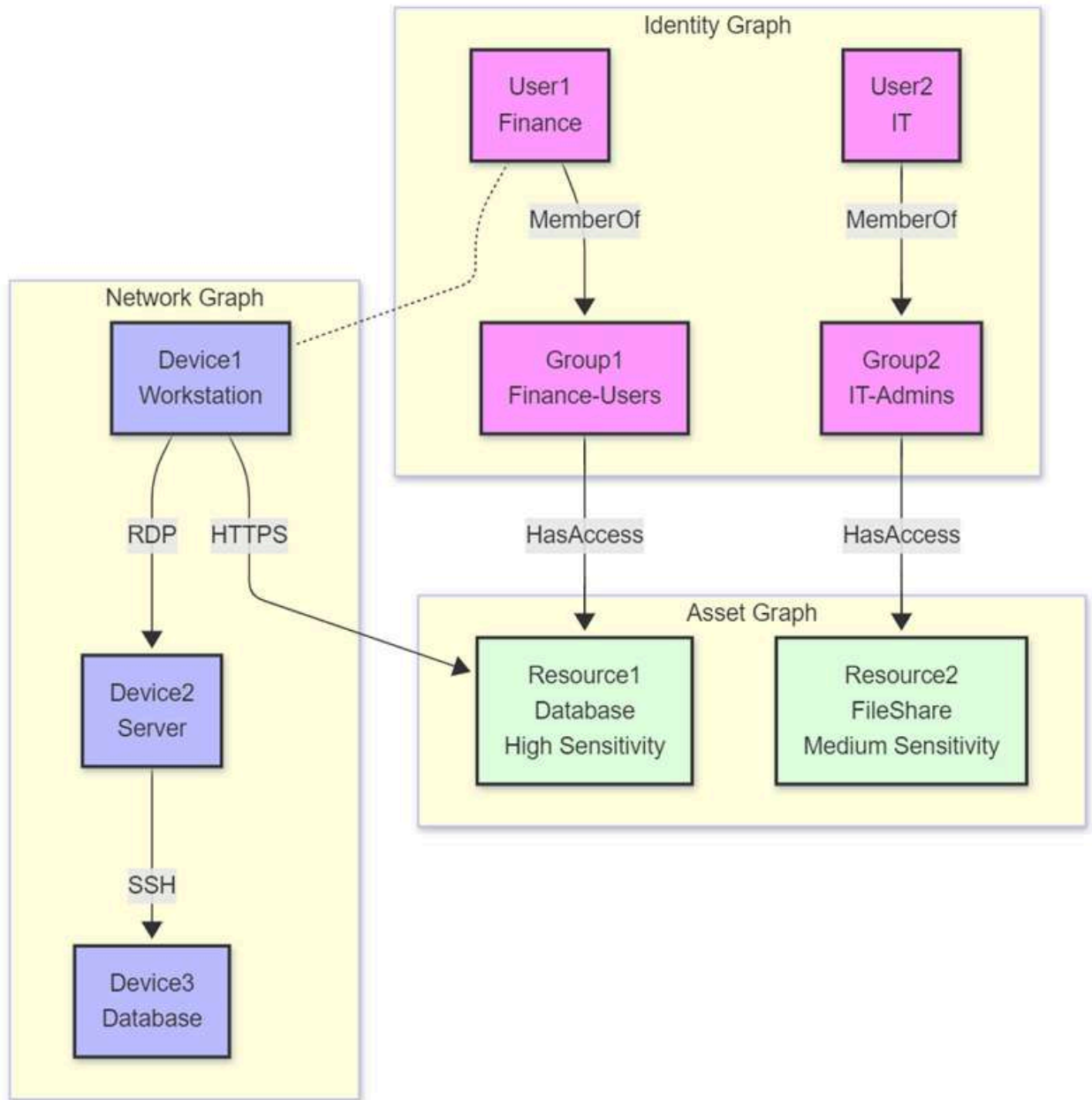
Results		Chart				
TimeGenerated [UTC]	RightNow [UTC]	HowLongAgo	Computer	CounterName	CounterValue	
> 4/14/2022, 5:12:15.713 PM	4/14/2022, 8:34:36.514 PM	03:22:20.8013036	JBOX00	Avg. Disk sec/Read	0	
> 4/14/2022, 5:12:15.713 PM	4/14/2022, 8:34:36.514 PM	03:22:20.8013036	JBOX00	Avg. Disk sec/Read	0	
> 4/14/2022, 4:55:40.470 PM	4/14/2022, 8:34:36.514 PM	03:38:56.0443036	JBOX10	Avg. Disk sec/Read	0	
> 4/14/2022, 4:55:40.470 PM	4/14/2022, 8:34:36.514 PM	03:38:56.0443036	JBOX10	Avg. Disk sec/Read	0	
> 4/14/2022, 4:55:40.470 PM	4/14/2022, 8:34:36.514 PM	03:38:56.0443036	JBOX10	Avg. Disk sec/Read	0	
> 4/14/2022, 4:55:55.107 PM	4/14/2022, 8:34:36.514 PM	03:38:41.4073036	CH1-AVSMGMTVM	Avg. Disk sec/Read	0	
> 4/14/2022, 4:55:55.107 PM	4/14/2022, 8:34:36.514 PM	03:38:41.4073036	CH1-AVSMGMTVM	Avg. Disk sec/Read	0	
> 4/14/2022, 4:56:02.477 PM	4/14/2022, 8:34:36.514 PM	03:38:34.0373036	DC11.na.contosohotels.com	Avg. Disk sec/Read	0.001	
> 4/14/2022, 4:56:02.477 PM	4/14/2022, 8:34:36.514 PM	03:38:34.0373036	DC11.na.contosohotels.com	Avg. Disk sec/Read	0.001	
> 4/14/2022, 4:52:02.397 PM	4/14/2022, 8:34:36.514 PM	03:42:34.1173036	DC11.na.contosohotels.com	Avg. Disk sec/Read	0	
> 4/14/2022, 4:52:02.397 PM	4/14/2022, 8:34:36.514 PM	03:42:34.1173036	DC11.na.contosohotels.com	Avg. Disk sec/Read	0	
> 4/14/2022, 4:52:05.690 PM	4/14/2022, 8:34:36.514 PM	03:42:30.8243036	DC10.na.contosohotels.com	Avg. Disk sec/Read	0	
> 4/14/2022, 4:52:05.690 PM	4/14/2022, 8:34:36.514 PM	03:42:30.8243036	DC10.na.contosohotels.com	Avg. Disk sec/Read	0	
> 4/14/2022, 4:52:19.507 PM	4/14/2022, 8:34:36.514 PM	03:42:17.0073036	RETAILVM01	Avg. Disk sec/Read	0	

```

43 // Bin allows you to summarize into logical groups, like days
44 Perf
45 | summarize NumberOfEntries=count()
46 | by bin(TimeGenerated, 1d)
47
48
49
50

```

Results		Chart	
TimeGenerated [UTC]	NumberOfEntries		
> 4/10/2022, 12:00:00.000 AM	306,425		
> 4/9/2022, 12:00:00.000 AM	3,613,342		



Core rule (fix this first)

KQL is time-first.

Most queries filter, bucket, or aggregate by datetime.

All datetime values are UTC unless converted.

PART A — KQL Date & Time functions (most important)

Assume a column: **Timestamp** (datetime)

1 **now()**

Returns current UTC time.

```
print now()
```

Use: reference current moment.

2 **ago()**

Returns time relative to now.

```
Orders  
| where Timestamp > ago(5m)
```

Meaning: last 5 minutes.

Supported units:

- **s** seconds
 - **m** minutes
 - **h** hours
 - **d** days
-

3 **between**

Range filtering (inclusive).

Orders

```
| where Timestamp between (datetime(2024-01-01) ..  
datetime(2024-01-31))
```

Use when:

- Fixed date ranges
 - Reports
-

4 startofday(), endofday()

Normalize to day boundaries.

Orders

```
| extend day = startofday(Timestamp)
```

Used for:

- Daily aggregation
 - Calendar alignment
-

5 startofweek(), startofmonth(), startofyear()

Orders

```
| extend month = startofmonth(Timestamp)
```

Used for:

- Monthly KPIs
 - Fiscal reporting
-

6 `datetime_diff()`

Difference between two times.

Orders

```
| extend diff_min = datetime_diff('minute', now(), Timestamp)
```

Use:

- Latency calculation
 - SLA checks
-

7 `bin()` — windowing foundation

Buckets time into fixed intervals.

Orders

```
| summarize count() by bin(Timestamp, 5m)
```

This is how tumbling windows are implemented in KQL.

8 `format_datetime()`

Convert datetime to string.

Orders

```
| extend date_str = format_datetime(Timestamp, 'yyyy-MM-dd')
```

Use only for presentation.

9 `to_date()`, `to_timespan()`

Type conversion.

```
| extend d = todatetime("2024-01-01")
```

PART B — Filtering & selection functions

10 `where`

Row filtering.

`Orders`

```
| where amount > 1000
```

11 `project`

Column selection (SQL SELECT).

`Orders`

```
| project order_id, amount
```

12 extend

Create calculated columns.

Orders

```
| extend tax = amount * 0.18
```

13 parse_json()

Extract JSON fields.

Logs

```
| extend payload = parse_json(message)
| project payload.userId
```

PART C — Aggregation & summarization

14 summarize

Aggregation engine.

Orders

```
| summarize total_sales = sum(amount)
```

Common aggregations

- `count()`
 - `sum()`
 - `avg()`
 - `min()`
 - `max()`
 - `dcount()` (distinct count)
-

15 Grouping

Orders

```
| summarize total = sum(amount) by city
```

16 Time-based aggregation (critical)

Orders

```
| summarize count() by bin(Timestamp, 10m)
```

This is the default real-time pattern.

PART D — Sorting & limiting

17 sort by

Orders

| sort by amount desc

18 take vs top

Orders | take 5

Orders | top 5 by amount

- take → any rows
 - top → ordered rows
-

PART E — String functions (frequently used)

19 startswith(), endswith(), contains()

Logs

| where message contains "error"

20 tolower(), toupper()

```
| extend city = tolower(city)
```

21 substring()

```
| extend prefix = substring(order_id, 0, 3)
```

PART F — Conditional logic

22 iff() (if-else)

```
| extend category = iff(amount > 1000, "High", "Low")
```

23 case() (multiple conditions)

```
| extend bucket = case(  
    amount > 5000, "Platinum",  
    amount > 2000, "Gold",  
    "Silver"  
)
```

PART G — Joins (basic awareness)

24 join

Orders

```
| join Customers on customer_id
```

- Avoid heavy joins in real-time
 - Prefer small lookup joins
-

PART H — Performance rules (non-negotiable)

1. Filter early

```
| where Timestamp > ago(10m)
```

2. Project late
 3. Avoid string operations before filtering
 4. Always use time filter
-

Mental execution model (lock this)

Table

```
→ where (time)
```

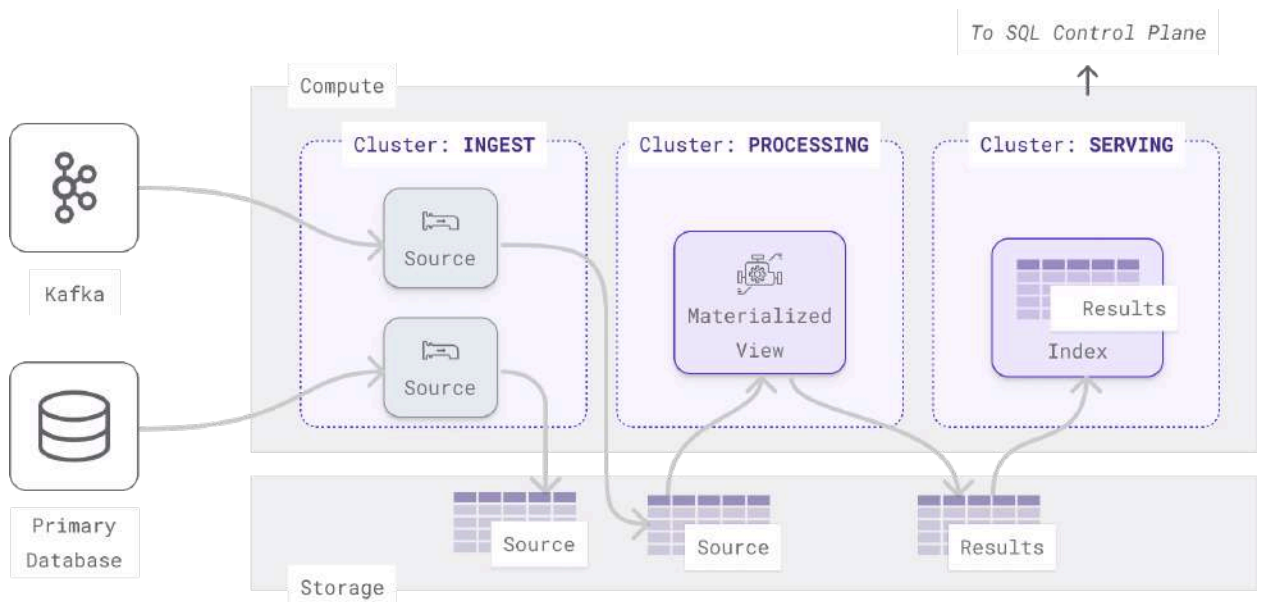
```
→ where (filter)
```

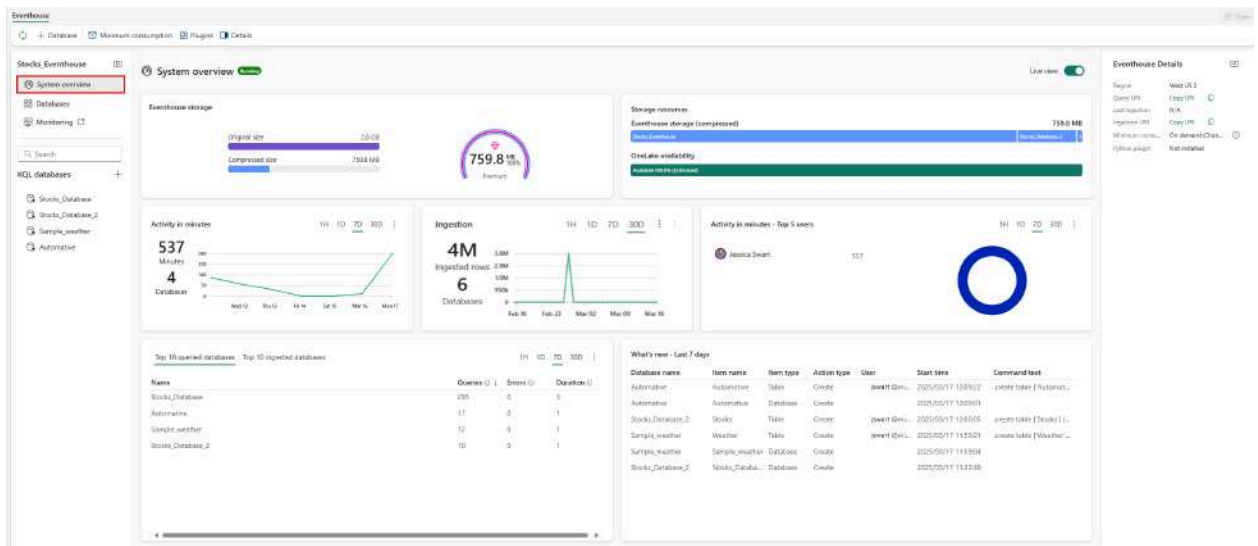
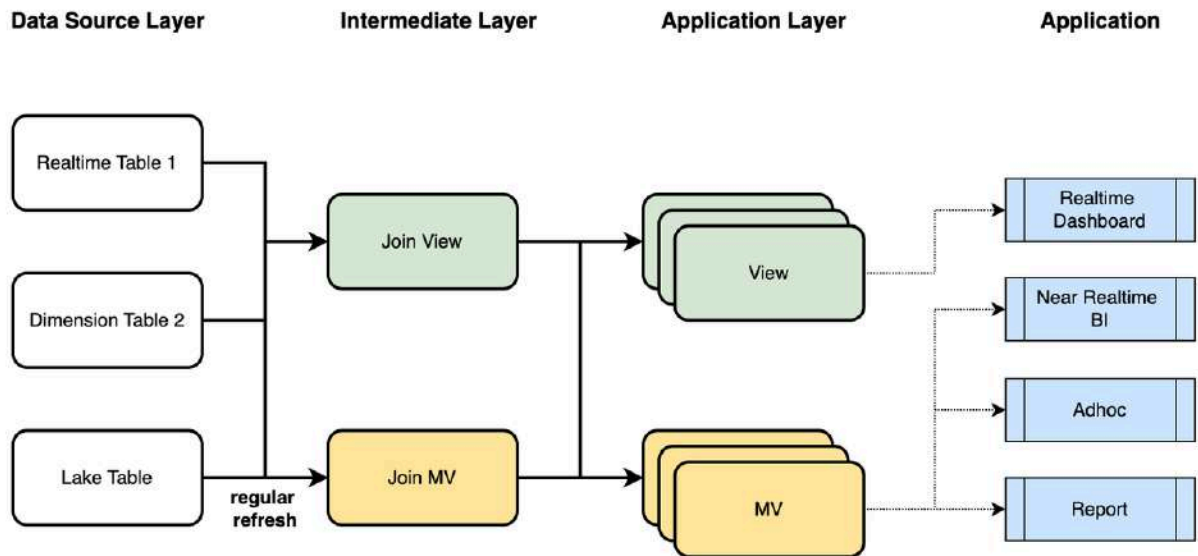
- **extend**
- **summarize**
- **sort / top**

One-line memory lock

KQL date functions control time filtering and windowing, while operators like **where**, **project**, **extend**, and **summarize** form the core pipeline for real-time analytics.

Materialized Views in KQL — creation, behavior, and Async Materialized Views





1) What is a Materialized View (MV) in KQL

A Materialized View is a precomputed, continuously maintained query result stored inside a KQL Database.

Definition:

A KQL materialized view stores the result of a query, not the query itself, and keeps it updated as new data arrives.

Key point:

- Normal KQL query → computed at read time
 - Materialized view → computed at ingest time
-

2) Why Materialized Views are needed

Real-time tables:

- Very large
- Append-only
- Frequently queried with the same aggregations

Problems without MV:

- Recomputing aggregates every query
- High CPU usage
- Slow dashboards

Materialized views solve:

- Repeated aggregations
 - Low-latency dashboards
 - High concurrency
-

3) What MV actually stores

If base table has:

1 billion raw events

Materialized view might store:

Daily aggregates (365 rows)

It stores aggregated / transformed results, not raw events.

4) How Materialized Views work internally

Event ingestion

↓

Base table (raw events)

↓

Materialized View engine

↓

Precomputed result table

- MV is updated incrementally
- Only new data is processed
- No full recomputation

5) How to create a Materialized View (basic)

Example base table

Orders

```
(  
  Timestamp: datetime,  
  city: string,  
  amount: double  
)
```

Create Materialized View

```
.create materialized-view mv_sales_by_city
on table Orders
{
    Orders
    | summarize total_sales = sum(amount) by city
}
```

What happens:

- MV named `mv_sales_by_city` is created
- It stores `city` + `total_sales`
- Updates automatically as Orders ingests data

6) Querying a Materialized View

```
mv_sales_by_city
```

This is extremely fast compared to running `summarize` on Orders.

7) Supported operations inside MV (important)

Allowed:

- `where`
- `project`
- `extend`

- `summarize`
- `bin()`
- Simple joins (small dimension tables)

Not allowed:

- Complex joins across large tables
- Non-deterministic functions
- Heavy string manipulation

Rule:

MV logic must be deterministic and incremental.

8) When NOT to use Materialized Views

- One-time ad-hoc analysis
- Frequently changing query logic
- Low query repetition
- Very small datasets

MV trades storage + ingest cost for query speed.

9) What is an Async Materialized View

This is the part most people miss.

Definition

An Async Materialized View updates asynchronously, not inline with ingestion.

Meaning:

Data ingestion is NOT blocked waiting for the MV to update.

10) Sync vs Async Materialized Views

Aspect	Syn c MV	Async MV
Update tim ing	Duri ng inge stio n	After ingestio n
Ing est lat en cy	High er	Lower
Qu ery fre sh ne ss	Near real- time	Slightly delayed
Thr ou gh put	Low er	Higher
Us e cas e	Criti cal fres hne ss	High-vol ume streams

11) How Async MV works internally

Event ingestion

↓

Base table updated immediately

↓

Ingestion acknowledged

↓

Async MV updater runs later

↓

Materialized view refreshed

There is a lag window (seconds to minutes).

12) How to create an Async Materialized View

```
.create materialized-view mv_sales_by_city_async
on table Orders
with (backfill = true, async = true)
{
    Orders
    | summarize total_sales = sum(amount) by city
}
```

Key flag:

- `async = true`
-

13) When to use Async Materialized Views

Use async MV when:

- Very high ingestion rate
- Ingestion latency is critical
- Dashboards tolerate slight delay
- Log / telemetry scenarios

Avoid async MV when:

- Financial calculations need immediate accuracy
 - SLA requires zero lag
-

14) Consistency model (important clarity)

- Base table is always correct
 - Materialized view is eventually consistent
 - Async MV trades freshness for scale
-

15) Best practice pattern

Raw events table

→ Async MV (high volume)

→ Dashboards / alerts

Curated table

→ Sync MV (low latency)

→ Critical reporting

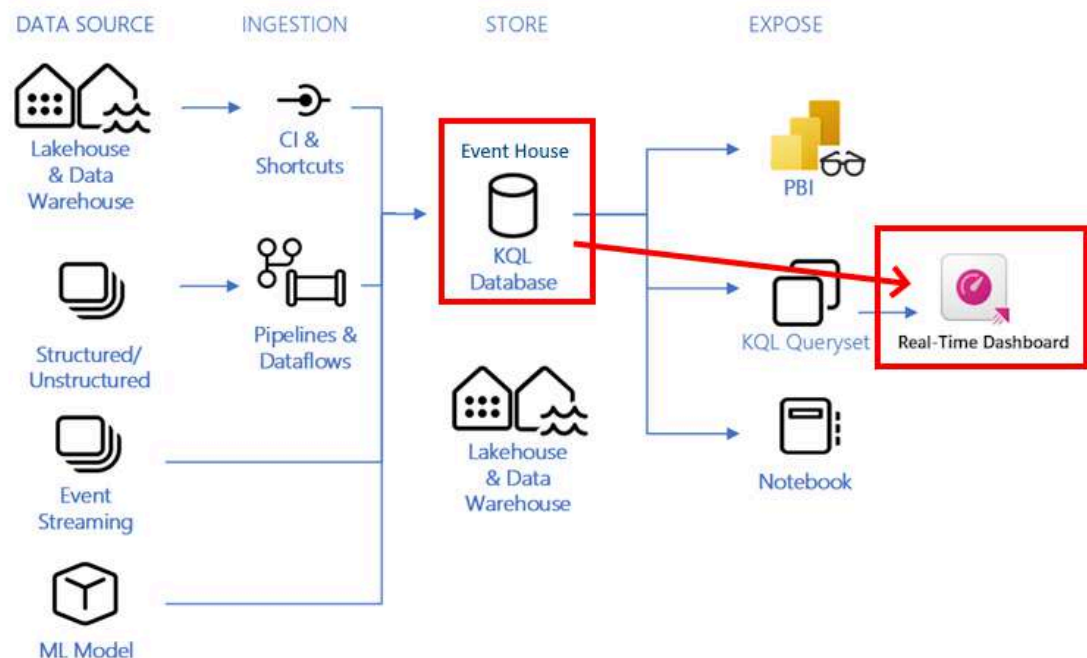
Mental model (lock this)

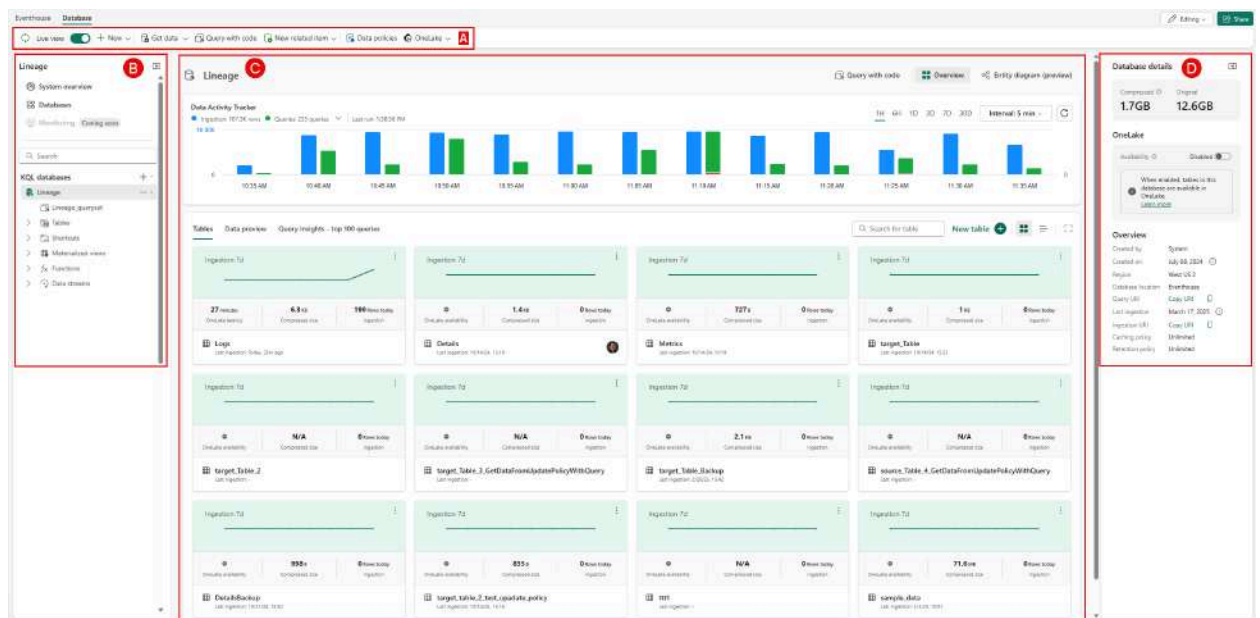
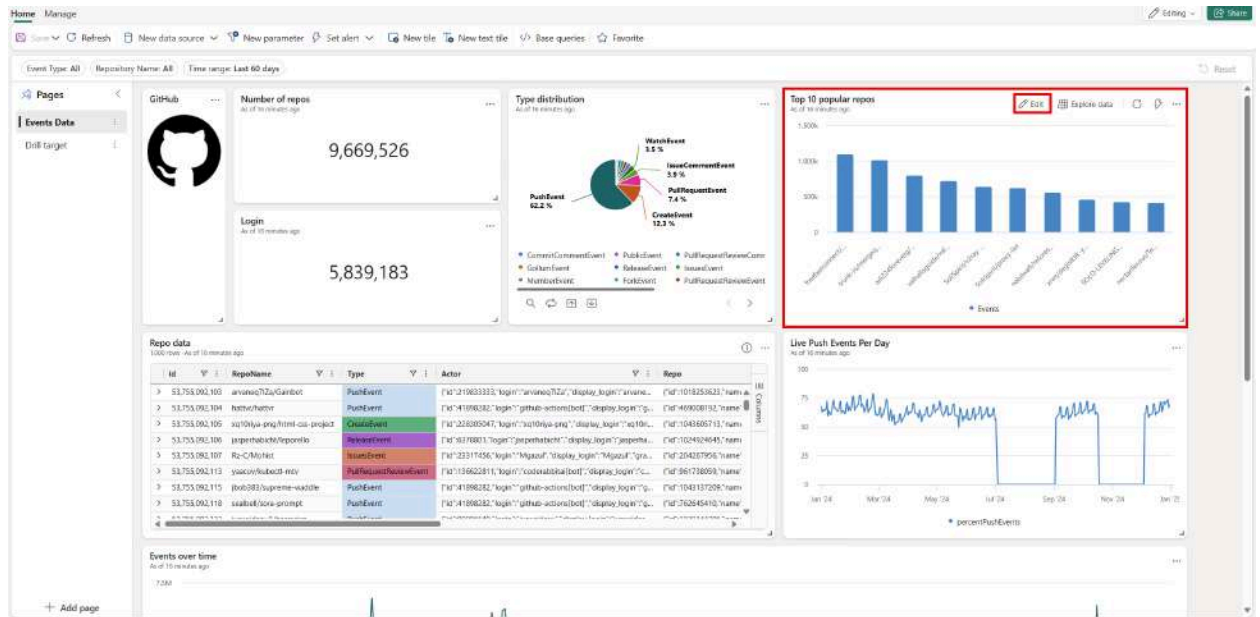
- Materialized View → precomputed result
 - Sync MV → freshest, slower ingest
 - Async MV → faster ingest, slight lag
-

One-line memory lock

A KQL materialized view stores precomputed query results, and an async materialized view updates them asynchronously to reduce ingestion latency at the cost of slight freshness delay.

Real-Time Dashboard & the role of a Base Query (Fabric / KQL context)





1) What is a Real-Time Dashboard

A real-time dashboard is a live visualization surface that auto-updates continuously as new events arrive—typically within seconds—without manual refresh.

Definition:

A real-time dashboard visualizes data in motion, not yesterday's data at rest.

Typical uses

- System health monitoring
- Fraud / anomaly detection
- Live order tracking
- IoT telemetry
- Security and log analytics

What it reads

- KQL tables
 - Materialized views
 - Streaming results inside a KQL Database (Eventhouse)
-

2) How a real-time dashboard works (execution flow)

Events arrive



KQL Database (append-only)



Dashboard query executes repeatedly



Tiles update automatically

No scheduled refresh.
Low latency by design.

3) What is a Base Query

A Base Query is the common, reusable KQL query defined once and shared by all dashboard tiles.

Definition:

The base query establishes the core dataset and filters that every visualization builds on.

Think of it as the foundation layer for the dashboard.

4) Why Base Query exists (the real reason)

Without a base query:

- Every tile re-implements time filters
- Inconsistent logic across tiles
- Higher compute cost
- Harder maintenance

With a base query:

- One source of truth
 - Consistent filtering
 - Better performance
 - Easier changes
-

5) What typically goes into a Base Query

Put global, expensive, or shared logic here:

- Time filters (`Timestamp > ago(5m)`)
- Environment filters (prod only)
- Parsing (`parse_json`)
- Normalization (`extend`)
- Basic projections

Example Base Query

Events

```
| where Timestamp > ago(10m)
```

```
| where environment == "prod"
```

```
| extend payload = parse_json(message)

| project Timestamp, service, level, payload
```

All tiles now inherit this dataset.

6) How tiles use the Base Query

Each tile adds only what is unique:

Tile 1 — Error count

```
base

| where level == "Error"

| summarize count()
```

Tile 2 — Errors by service

```
base

| where level == "Error"

| summarize count() by service
```

Tile 3 — Trend

```
base

| summarize count() by bin(Timestamp, 1m)
```

base = the Base Query result.

7) Performance impact (important)

Base Query advantages

- Filters early (less data scanned)
- Reused execution plan
- Lower CPU per tile
- Faster dashboards under load

Anti-pattern

- Heavy logic repeated in every tile
-

8) What NOT to put in a Base Query

Avoid:

- Tile-specific aggregations
- Sorting (**top**, **sort**)
- Presentation formatting
- Rare filters used by only one tile

Rule:

Base Query = shared preparation, not final answers.

9) Base Query vs Materialized View (clarity)

A	B	Materi
s	a	alized
p	s	View
e	e	
c	Q	
t	u	
	e	

r
y



S
t
o
r
e
d
r
e
s
u
l
t

C
o
m
p
u
t
e
d

A
t
q
u
e
r
y
t
i
m
e

At
ingest
time

S
c
o
p
e

D
a
s
h
b
o
a
r
d

Databa
se

B	S	Heavy
e	h	aggreg
s	a	ations
t	r	
f	e	
o	d	
r	l	
	o	
	g	
	i	
	c	

Use both together:

- MV for expensive aggregates
- Base Query for shared filters

10) When to use a Base Query (always?)

Use a Base Query when:

- Dashboard has multiple tiles
- Same time window applies
- Same parsing/cleanup needed
- Performance matters

Single tile dashboard → optional.

Mental model (lock this)

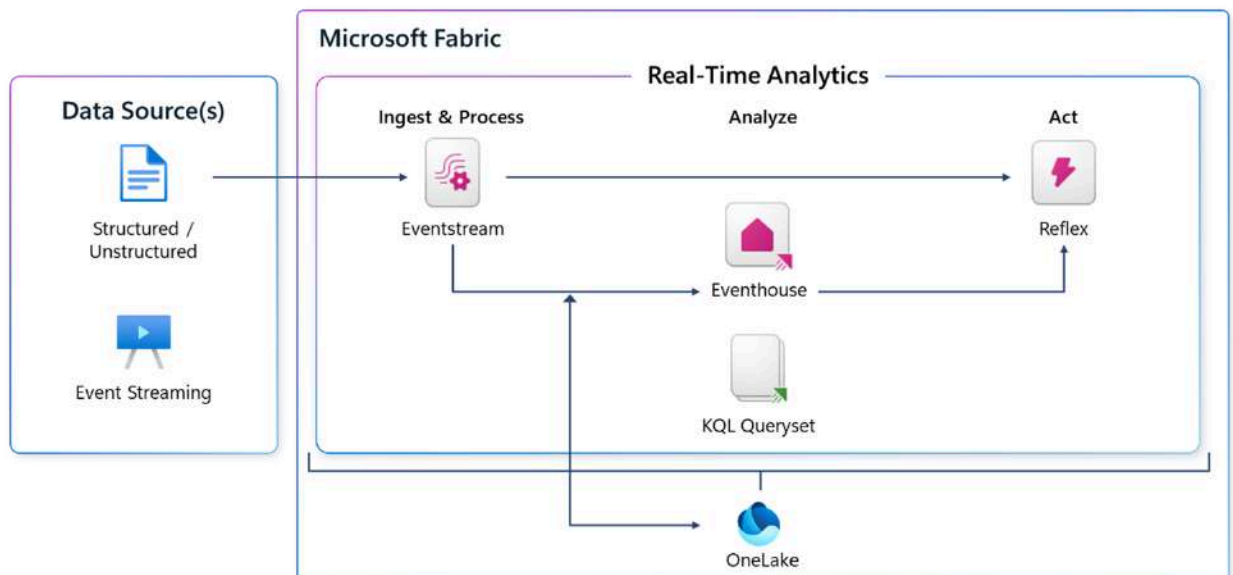
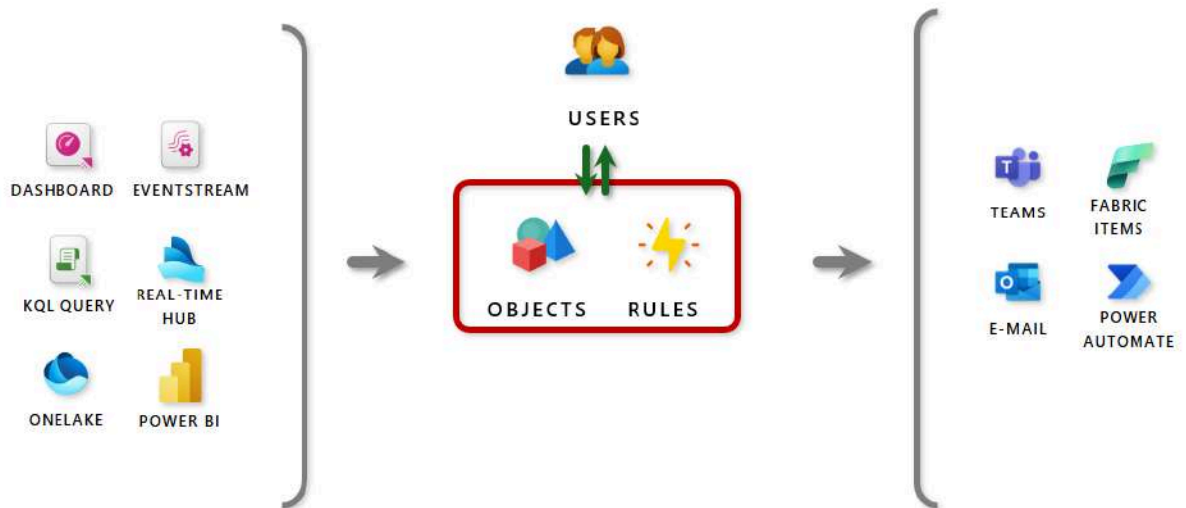
- Real-time dashboard → live window into events
- Base Query → common lens everyone looks through

One-line memory lock

A real-time dashboard shows continuously updating insights, and its base query defines the shared, consistent dataset that all tiles build on for correctness and performance.

Activator in Microsoft Fabric — what it is and how to implement it (end-to-end)

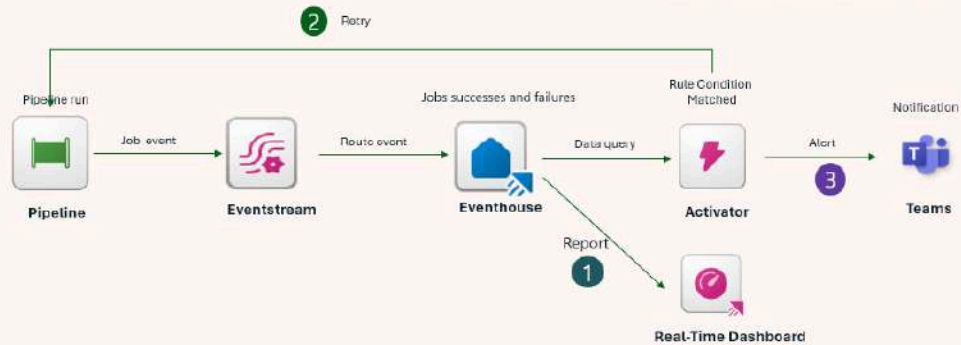
Activator connects data & action systems



Monitoring my jobs and pipelines performance over time

- *How well is my system operating?*

- 1 • I want to **report** on over-time success and failures of my pipeline runs
- 2 • I want to **re-run** failed jobs
- 3 • I want to **get notified** when my pipeline jobs failures exceed the acceptable threshold



1) What is Activator in Fabric

Activator is a real-time rule engine in Microsoft Fabric that listens to live data and automatically triggers actions when conditions are met.

Definition:

Activator watches streaming or near-real-time data and takes action the moment a rule becomes true.

No polling.

No batch checks.

No manual intervention.

2) Why Activator exists (the problem it solves)

Dashboards show problems.

But dashboards do nothing.

Without Activator:

- Someone must notice an issue

- Someone must decide what to do
- Action is delayed

Activator closes the loop:

Data → **Detection** → **Action**

3) What Activator connects to

Activator sits on top of real-time outputs, typically from:

- Eventstream
- Eventhouse (KQL DB)
- Real-time dashboards
- Streaming queries / materialized views

It does not work on batch tables or warehouses.

4) What Activator can trigger (actions)

Common actions:

- Send email
- Send Teams notification
- Call Power Automate flow
- Trigger webhook
- Write to another system

Activator is event-driven automation.

5) How Activator works internally

Live events



Eventhouse / KQL query

↓

Rule evaluation (Activator)

↓

Condition met?

└─ No → do nothing

└─ Yes → trigger action

Rules are evaluated continuously.

6) Core components of Activator

1 Source

The live data stream:

- KQL query
 - Dashboard tile
 - Eventstream output
-

2 Rule

A condition written over fields:

- Thresholds
- Comparisons
- Counts
- States

Example:

`error_count > 100`

3 Action

What happens when rule is true:

- Notify
 - Escalate
 - Automate
-

7) Simple example (conceptual)

Scenario

- Monitor application errors
- Alert team if errors spike

Rule

`If error_count > 50 in last 5 minutes`

Action

- Send Teams alert
-

8) How to implement Activator — step by step

Step 1 — Prepare live data (KQL)

Example base query in Eventhouse:

`AppLogs`

`| where Timestamp > ago(5m)`

`| summarize error_count = countif(level == "Error")`

This query continuously outputs a number.

Step 2 — Create an Activator

- Go to Fabric workspace
- Create Activator
- Choose source:
 - KQL query
 - Dashboard tile
 - Eventstream output

Step 3 — Define the rule

Example rule:

```
error_count > 50
```

Optional:

- Add cooldown
- Add suppression window
- Avoid alert storms

Step 4 — Configure action

Choose action:

- Teams message
- Email
- Power Automate flow

Example message:

```
High error rate detected: {{error_count}}
```

Step 5 — Activate

Once enabled:

- Rule runs continuously
 - Action fires automatically
 - No scheduling required
-

9) Important behavior to understand

- Activator is stateless per evaluation
 - It reacts to current signal
 - It does not store historical state (KQL DB does)
 - Duplicate triggers can occur without cooldown rules
-

10) Activator vs Dashboard vs Materialized View

Fe at ur e	D a s h b o a r d	Materi alized View	A c t i v a t o r
P ur po se	V i s u a l i	Preco mpute	A c t

	z	e	
St or es da ta	✗	✓	✗
Tri gg er s ac tio n	✗	✗	✓
R ea l-ti m e	✓	✓	✓

They are designed to work together.

11) When to use Activator

Use Activator when:

- Immediate response is required
- Threshold-based alerts are needed
- Automation must be event-driven
- Humans should not monitor dashboards

Avoid when:

- Batch reporting

- Historical analysis
 - Complex workflows (use ETL)
-

12) Best-practice pattern (enterprise)

Eventstream

- Eventhouse (KQL)
- Materialized View (optional)
- Real-Time Dashboard
- Activator
- Teams / Automation

Detection + decision + action in seconds.

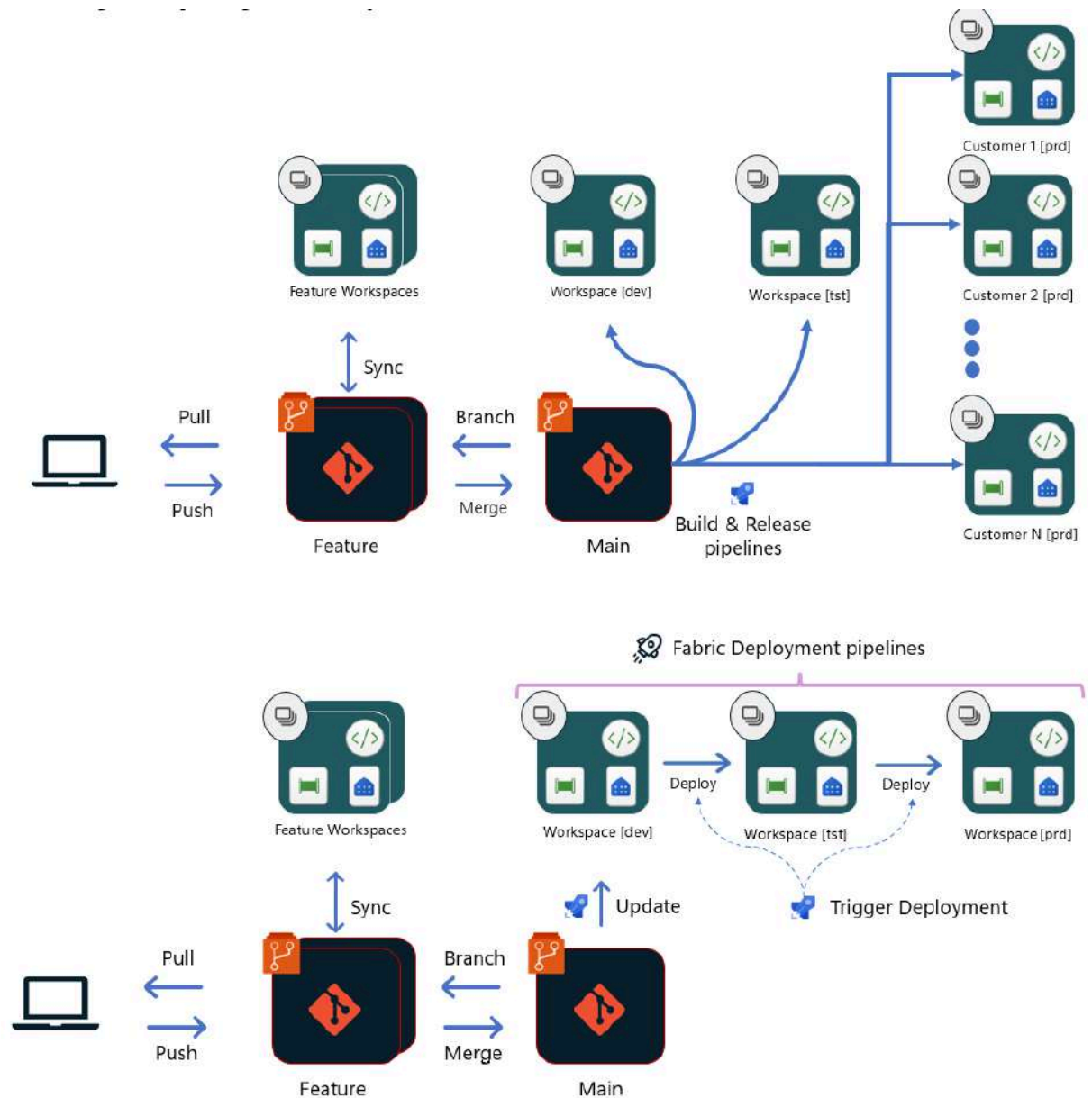
Mental model (lock this)

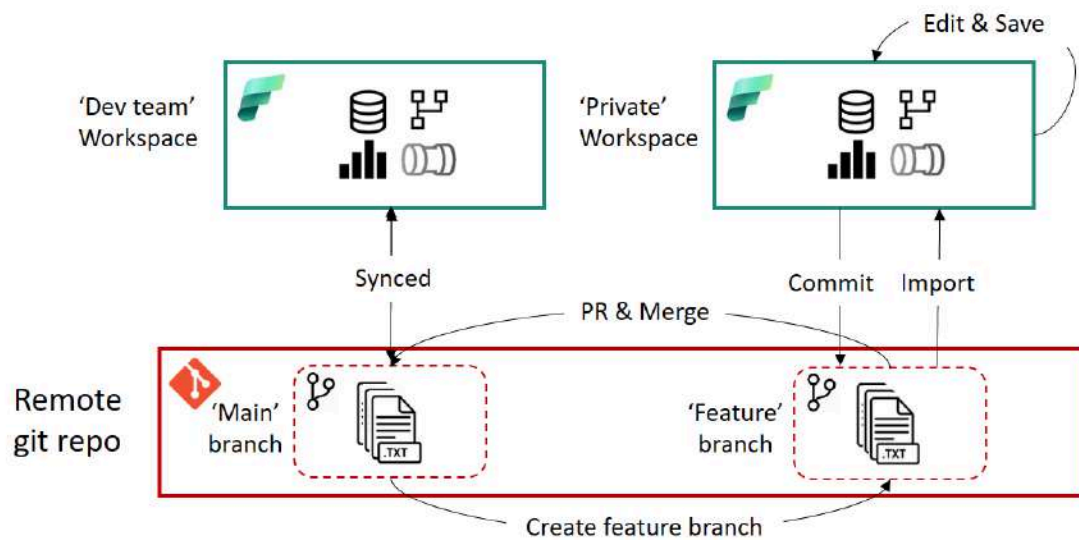
- Eventstream → data in motion
 - Eventhouse → analyze now
 - Dashboard → see now
 - Activator → act now
-

One-line memory lock

Activator in Fabric is a real-time rule engine that continuously evaluates live data and automatically triggers actions the moment defined conditions are met.

CI/CD in Microsoft Fabric — beginner-level, end-to-end explanation





1) What is CI/CD (very simple)

CI/CD = Controlled change movement

- CI (Continuous Integration) → safely bring changes into source control
- CD (Continuous Deployment) → safely move changes between environments

In Fabric, CI/CD is about:

Moving Fabric items (Lakehouse, Warehouse, Pipelines, Notebooks, Reports) from Dev → Test → Prod without manual mistakes.

2) Why CI/CD is needed in Fabric

Without CI/CD:

- Manual copy-paste
- Accidental overwrites
- No rollback
- No history
- Production breaks

With CI/CD:

- Version history
 - Approval flow
 - Repeatable deployments
 - Environment isolation
-

3) Fabric CI/CD building blocks (lock this)

Fabric CI/CD is built using two pillars:

1) **Git integration** → CI

2) **Deployment pipelines** → CD

They are separate but connected.

4) CI in Fabric — what it actually means

CI in Fabric = Workspace ↔ Git synchronization

When CI is enabled:

- Fabric workspace is linked to a Git repo
- Every change becomes versioned
- You get history, diff, rollback

CI answers:

“What changed, when, and by whom?”

5) What can be versioned (important)

Fabric items supported in CI:

- Lakehouse (metadata)
- Warehouse (schema, SQL objects)
- Notebooks

- Data Factory pipelines
- Dataflows Gen2
- Semantic models
- Reports

Not versioned:

- Actual data
 - OneLake files
-

6) Prerequisites for CI (before setup)

You need:

- Azure DevOps or GitHub
 - Fabric workspace (Dev)
 - Workspace Admin or Member role
 - Repo permission (read/write)
-

7) How to set up CI in Fabric (step-by-step)

Step 1 — Create Dev workspace

Create a workspace named:

fabric-dev

All development happens here.

Step 2 — Open Workspace settings

- Go to workspace
 - Click Settings
 - Open Git integration
-

Step 3 — Connect to Git

Choose:

- Azure DevOps or
- GitHub

Provide:

- Organization
 - Repository
 - Branch (usually **main** or **develop**)
 - Folder path
-

Step 4 — Initialize Git connection

Fabric will:

- Export workspace items as JSON/YAML
- Store them in Git
- Create a baseline commit

Now CI is active.

8) What happens after CI is enabled

Developer workflow

1. Edit notebook / pipeline / warehouse
2. Fabric marks workspace as dirty
3. Click Commit
4. Add commit message
5. Changes go to Git

This is Continuous Integration.

9) How Fabric items look in Git (clarity)

In Git you will see:

/Warehouse

/Tables

/Views

/Procedures

/Lakehouse

/Notebooks

/Pipelines

These are definitions, not data.

10) What CI does NOT do

CI does NOT:

- Deploy to prod
- Run tests automatically
- Move data
- Control approvals

CI only:

Keeps source of truth in Git

11) CD in Fabric — what it actually means

CD in Fabric = Deployment Pipelines

Deployment Pipelines move content between:

Dev → Test → Prod

Controlled. Repeatable. Safe.

12) How to set up CD in Fabric (overview)

1. Create Deployment Pipeline
 2. Assign workspaces:
 - Dev
 - Test
 - Prod
 3. Deploy content stage by stage
-

13) CI + CD combined (full picture)

Developer

↓

Dev Workspace

↓ (Commit)

Git Repository ← CI

↓

Deployment Pipeline

↓

Test Workspace

↓

Prod Workspace ← CD

Git = truth
Pipelines = movement

14) Beginner mental model (lock this)

- **Git → history + control**
- **Dev workspace → experimentation**
- **Test workspace → validation**
- **Prod workspace → consumption**

Never build directly in Prod.

15) Common beginner mistakes (avoid)

- **Editing in Prod workspace**
 - **Skipping Git**
 - **Manual copy between workspaces**
 - **Mixing data with definitions**
 - **No environment separation**
-

16) What CI/CD in Fabric is NOT

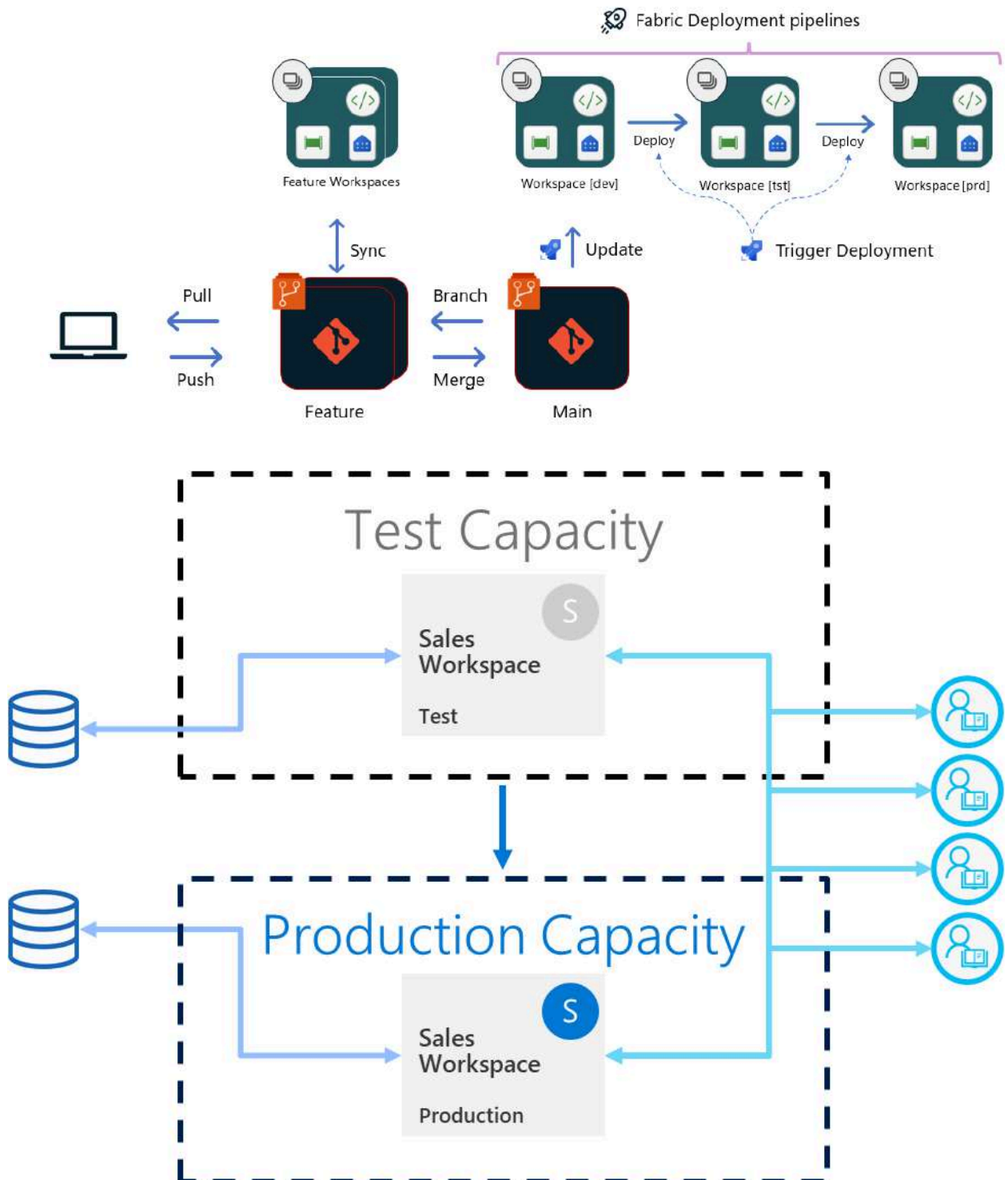
- **Not Spark job CI**
- **Not unit testing framework**
- **Not code compilation**
- **Not data migration tool**

It is analytics lifecycle management.

One-line memory lock

In Fabric, CI is Git integration for versioning workspace items, and CD is Deployment Pipelines that move those items safely from Dev to Test to Prod.

CD (Continuous Deployment) in Microsoft Fabric — what it is and how to set it up (beginner, exact)



1) What CD means in Fabric

CD in Fabric = Deployment Pipelines

Definition:

CD is the controlled movement of Fabric items from Dev → Test → Prod using Deployment Pipelines.

No code execution.

No data movement.

Only item definitions move.

Used by Microsoft Fabric to avoid manual copy and production mistakes.

2) What CD actually deploys

Deployed:

- Lakehouse metadata
- Warehouse schema (tables, views, procedures)
- Notebooks
- Pipelines
- Dataflows Gen2
- Semantic models
- Reports

NOT deployed:

- Data
 - OneLake files
 - Runtime results
-

3) Why CD is mandatory in Fabric

Without CD:

- Manual copy-paste

- Prod overwritten accidentally
- No rollback
- No traceability

With CD:

- Environment isolation
 - Safe promotion
 - Predictable releases
 - Governance-ready deployments
-

4) Fabric CD architecture (lock this)

Dev Workspace



Deployment Pipeline



Test Workspace



Deployment Pipeline



Prod Workspace

Only the pipeline moves content.

5) Prerequisites (must exist)

- 3 workspaces:
 - Dev
 - Test

- Prod
 - Same capacity (recommended)
 - Workspace Admin access
 - CI (Git) optional but strongly recommended
-

6) Step-by-step: set up CD in Fabric

Step 1 — Create workspaces

Create:

fabric-dev

fabric-test

fabric-prod

Do not develop in Test/Prod.

Step 2 — Create a Deployment Pipeline

- Go to Fabric
 - Create Deployment Pipeline
 - Name it clearly (e.g., **analytics-cd-pipeline**)
-

Step 3 — Assign workspaces to stages

Map stages:

- Development → **fabric-dev**
- Test → **fabric-test**
- Production → **fabric-prod**

This mapping is fixed per pipeline.

Step 4 — Validate stage differences

Pipeline shows:

- New items
- Changed items
- Missing items

This is your pre-deploy diff.

Step 5 — Deploy Dev → Test

- Click Deploy
- Fabric copies item definitions
- No data touched

Deployment is atomic per item.

Step 6 — Validate in Test

- Run queries
- Run notebooks
- Validate reports

Fix issues only in Dev, then redeploy.

Step 7 — Deploy Test → Prod

- Same deploy action
- No editing in Prod

Prod becomes stable and controlled.

7) What happens internally during deployment

1. Fabric reads Dev metadata
2. Compares with target stage

3. Applies create/update/delete
4. Keeps IDs aligned
5. Preserves security where possible

No data rewrite.

8) Handling environment differences (important)

Fabric supports deployment rules:

Use rules to change:

- Connection strings
- Lakehouse references
- Warehouse names

Example:

Dev → dev_lakehouse

Prod → prod_lakehouse

Rules prevent hardcoding.

9) CD with and without CI (clarity)

With CI (recommended)

Dev Workspace

→ Commit to Git (CI)

→ Deployment Pipeline (CD)

Without CI

Dev Workspace

→ Deployment Pipeline (CD)

CD works alone, but no history or rollback without CI.

10) What CD does NOT do

- No automated testing
- No approvals engine
- No Spark job execution
- No data refresh

Those belong to other tools/processes.

11) Common beginner mistakes (avoid)

- Editing in Prod
 - Using one workspace for all
 - Moving data via CD
 - Skipping Test stage
 - Hardcoding environment paths
-

Mental model (lock this)

- CI → version control
 - CD → environment promotion
 - Dev → change
 - Test → verify
 - Prod → trust
-

One-line memory lock

CD in Fabric uses Deployment Pipelines to safely promote item definitions from Dev to Test to Prod without moving data.